目录

C O N T E N T S

第1章 认识Vue

1.1 Vue简述 …… 003
1.2 Vue的安装方式 …… 004
1.2.1 CDN引入 ……004
1.2.2 使用npm安装 ……004
1.2.3 使用Vue CLI创建项目 ……005
1.3 Vue开发工具 …… 005
1.3.1 Vue Devtools ……005
1.3.2 Vue CLI ……005
1.4 Vue基础语法 …… 005
1.4.1 模板语法 ……006
1.4.2 模板渲染 ……025
1.4.3 事件绑定 ……030
1.5 自定义指令 …… 033
1.5.1 指令的注册 ……033
1.5.2 指令的定义对象 ……035
1.5.3 指令实例属性 ……037
1.6 组件 …… 039
1.6.1 什么是组件 ……039
1.6.2 组件的基本使用 ……039
1.6.3 组件选项 ……043
1.6.4 组件通信 ……050
1.6.5 动态组件 ……056
本章小结…… 064
练习与实践…… 065

第2章 前端工程化

2.1 Vue cli …… 069
2.1.1 Node.js ……070
2.1.2 NPM ……070
2.1.3 基本使用 ……070
2.2 项目打包与发布 …… 072
2.2.1 静态服务器 ……073
2.2.2 动态服务器 ……074
2.3 Vue Devtools…… 075
2.3.1 Vue Devtools使用 ……075
2.3.2 Vue Devtools案例 ……077
本章小结…… 082
练习与实践…… 082

第3章 UI组件库基础应用

3.1 Element Plus安装及引入…… 085
3.2 Element Plus概述…… 085

3.2.1 Element Plus安装 ……085
3.2.2 Element Plus引入 ……086
3.3 基础组件 …… 087
3.3.1 Button按钮……088
3.3.2 Border边框 ……091
3.3.3 Icon图标 ……092
3.3.4 Containe容器布局 ……094
3.3.5 link链接……095
3.3.6 Text文本 ……096
3.3.7 Space间距 ……100
3.3.8 Typography排版……103
3.4 表单组件 …… 106
3.4.1 Input输入框 ……106
3.4.2 Input Number数字输入框 ……108
3.4.3 InputTag 标签输入框 ……111
3.4.4 Radio单选框 ……113
3.4.5 Checkbox 多选框 ……115
3.4.6 Select选择器 ……118
3.4.7 Transfer穿梭框 ……119
3.4.8 TreeSelect 树形选择 ……121
3.5 Data数据展示…… 127
3.5.1 Table表格 ……127
3.5.2 Image图片 ……132
3.5.3 Pagination分页 ……134
本章小结…… 139
练习与实践…… 139

第4章 UI组件库高级应用

4.1 Vue Router路由 …… 143
4.1.1 Vue Router概述……143
4.1.2 Vue Router安装……143
4.1.3 Vue Router入门……143
4.1.4 Vue Router使用……148
4.1.5 动态路由 ……152
4.1.6 嵌套路由 ……153
4.2 Axios网络请求 …… 158
4.2.1 Axios概述 ……158
4.2.2 Axios安装 ……159
4.2.3 Axios局部网络请求 ……159
4.2.4 全局引入Axios网络请求 ……166
4.2.5 Axios的HTTP请求封装模块 ……171
4.2.6 Axios网络请求跨域解决方案 ……176
4.3 Vuex状态管理 …… 179
4.3.1 Vuex状态管理概述 ……179
4.3.2 Vuex状态管理的核心概念 ……180

4.3.3 Vuex状态管理的使用场景 …… 180
4.3.4 Vuex状态管理的用法 …… 181
4.3.5 Vuex状态管理的模块组 …… 192
本章小结 …… 201
练习与实践 …… 202

第5章 关系数据库和非关系数据库

5.1 关系型数据库MySQL …… 205
5.1.1 数据库的概述 …… 205
5.1.2 MySQL数据库的概述 …… 205
5.1.3 MySQL数据库与表的操作 …… 206
5.1.4 SQL语言的使用 …… 212
5.2 非关系型数据库Redis …… 217
5.2.1 NOSQL概述 …… 217
5.2.2 Redis概述 …… 217
5.2.3 Redis命令 …… 218
5.2.4 Redis应用 …… 220
本章小结 …… 228
练习与实践 …… 229

第6章 Spring Boot开发

6.1 Spring Boot概述 …… 233
6.2 Spring Boot快速入门 …… 233
6.2.1 Spring Boot目录结构 …… 233
6.2.2 Spring Boot项目的pom.xml文件 …… 234
6.2.3 Spring Boot项目配置文件 …… 234
6.2.4 Spring Boot启动类 …… 235
6.2.5 Spring Boot在Controller中常用注解 …… 235
6.3 Spring Boot整合Web应用 …… 237
6.3.1 Spring Boot访问静态资源 …… 237
6.3.2 Spring Boot整合Thymeleaf …… 239
6.3.3 RESTful API的应用 …… 244
6.4 SpringBoot整合持久层 …… 252
6.4.1 依赖与准备 …… 252
6.4.2 数据源与JPA配置 …… 252
6.4.3 实体与仓储接口 …… 253
6.4.4 Service层解耦 …… 254
6.4.5 单元测试 …… 255
本章小结 …… 256
练习与实践 …… 256

第7章 项目总体设计

7.1 总体设计 …… 259
7.1.1 项目需求 ……259
7.1.2 项目需求分析 ……259
7.2 项目详细功能设计 …… 262
7.2.1 功能模块详细设计 ……262
7.2.2 项目概要设计 ……265
7.2.3 数据库设计原则 ……268
7.2.4 数据库详细设计 ……269
7.2.5 接口设计规范 ……272
7.2.6 接口详细设计 ……273
本章小结…… 274
练习与实践…… 274

第8章 系统功能的实现

8.1 PC端的功能实现 …… 277
8.1.1 首页功能实现 ……277
8.1.2 用户管理功能实现 ……280
8.1.3 详情页功能实现 ……285
8.1.4 购物车管理功能实现 ……287
8.1.5 订单管理功能实现 ……298
8.1.6 支付管理功能实现 ……306
8.2 服务端的功能实现 …… 309
8.2.1 用户管理功能实现 ……309
8.2.2 产品管理功能实现 ……312
8.2.3 会员管理功能实现 ……316
8.2.4 订单管理功能实现 ……318
本章小结…… 321
练习与实践…… 322

第9章 项目构建与部署

9.1 环境准备 …… 325
9.2 JAR部署 …… 325
9.2.1 获取JAR包 ……325
9.2.2 本地运行JAR包 ……327



服装智能系统软件开发项目教程

基于 Vue +Spring Boot

第 1 章

认识 Vue



本章导读

在典型的Web应用架构当中，前端主要承担着与用户进行交互以及将数据进行有效呈现的重要职责，而后端则聚焦于对业务逻辑的处理以及对数据的加工和管理，前端和后端二者之间借助API来开展通信。随着Web应用复杂程度的不断提高，传统的基于jQuery的DOM操作开发模式已经越来越难以满足实际的需求，这种状况促使了以Vue为代表的现代前端框架的革新应运而生。在本章中，我们将深入学习Vue.js技术，了解其原理、特性以及在实际项目中的应用方法。

学习目标

1.需要全面且深入地掌握VS Code工具的使用，包括其各种功能、快捷操作以及在实际项目中的高效运用等方面。

2.掌握Vue核心机制，理解其背后的原理和逻辑，在开发中能够灵活运用。

3.熟练使用Vue的模板语法与指令系统的程度，能够在不同的场景下准确、流畅地运用这些语法和指令。

4.掌握组件生命周期与通信机制，清楚每个阶段的特点和作用以及组件之间有效的通信方式。

思政目标及设计建议

根据新时代软件工程师应该具备的基本素养，挖掘课程思政元素，有机融入教学中。本章思政目标及设计建议如表1-1所示。

表1-1　第1章思政目标及设计建议

思政目标	思政元素及融入
培养责任担当与安全忧患意识	讲解如何防止注入式攻击时，类比到实际开发过程中要有责任担当与安全忧患意识等
培养精益求精的精神	在介绍Vue时，重点介绍尤雨溪的个人事迹，激励同学们努力学习，将来能为中国前端技术贡献一份力量
培养自主探索、敬业、专注的工匠精神	通过课前自主学习，培养自主探索、敬业、专注的工匠精神

1.1 Vue简述



Vue.js是一款在当今前端开发领域极具影响力的开源的渐进式JavaScript框架。它是由开发者Evan You（尤雨溪）创建的，并且在2014年的时候首次向广大开发者发布。在后续的内容当中，将详细地介绍什么是Vue，也就是深入探究Vue.js框架的具体概念、特点和功能等方面。同时，还会阐述为什么要选择Vue，分析选择Vue.js框架相较于其他框架在开发效率、性能表现、学习成本等诸多方面所具备的优势。

Vue.js是一个构建用户界面的渐进式框架。与其他大型框架不同的是，Vue被设计为可以自底向上逐层应用。Vue的核心库只关注视图层，这使得它非常容易学习，也便于与其他库或已有项目整合。另一方面，当与现代工具链以及各种支持类库结合使用时，Vue也完全能够为复杂的单页应用提供驱动。

Vue的核心特点包括：

声明式数据绑定：Vue采用了一种声明式的模板语法，允许开发者以直观的方式将DOM元素与应用程序的状态连接起来。例如，使用{{message}}这样的双大括号语法可以轻松地在页面上显示变量message的值。

组件化系统：Vue鼓励使用组件来构建应用，每个组件都是一个独立的可复用单元，拥有自己的HTML模板、CSS样式和JavaScript逻辑。这不仅提高了代码的组织性和维护性，还促进了代码的重用。

响应式系统：Vue实现了细粒度的依赖追踪机制，当底层数据变化时，只有受该变化影响的部分视图会被更新，而不是整个页面重新渲染。这种高效的更新策略确保了良好的性能表现。

指令系统：Vue提供了一系列内置指令，如v-if用于条件渲染、v-for用于列表渲染、v-bind用于属性绑定、v-on用于事件监听等，这些指令可以帮助开发者更加方便地操作DOM。

插件功能：Vue支持插件扩展，允许开发者通过官方或社区提供的插件快速添加新功能，如路由管理、状态管理、表单验证等。

工具链：Vue拥有丰富的官方和社区提供的工具链，包括CLI脚手架工具、调试工具等，帮助提升开发效率和用户体验。

生态系统：Vue背后有一个活跃的社区和强大的生态系统，提供了大量的学习资源、第三方库和支持服务。

Vue之所以成为许多开发者的选择，主要归功于以下几个方面：

1.易于上手：Vue的学习曲线相对平缓，文档清晰详尽，适合初学者快速入门。它的API设计简洁直观，对于有其他JavaScript框架经验的开发者来说也非常容易适应。

2.灵活性：Vue不强制任何特定的项目结构或编码风格，而是让开发者自由选择最适合项目的架构方式。它可以作为小型库集成到现有网页中，也可以作为完整的SPA框架使用。

3.性能优异：Vue采用了虚拟DOM技术，能够智能地最小化实际DOM操作，从而提高渲染效率。此外，其响应式系统的优化也保证了即使在复杂的应用场景下也能保持流畅的用户体验。

4.社区支持：Vue拥有庞大的开发者社区，这意味着你可以找到大量的教程、示例和解决方案。同时，Vue团队积极维护和发展框架，定期发布更新并修复bug。

5.广泛的适用性：无论是构建大型企业级应用还是小型个人项目，Vue都能很好地胜任。它不仅适用于浏览器环境下的Web开发，还可以通过相关工具链用于移动端开发（如Weex）、桌面端应用开发（Electron）以及服务器端渲染（Nuxt.js）等领域。

综上所述，Vue.js以其简单易学、灵活高效的特点，在前端开发领域占据了重要地位，成为众多开发者构建现代Web应用的理想选择。



1.2 Vue的安装方式

Vue.js提供了三种主要的安装方式，每种方式都有其适用场景和特点。以下是详细的介绍：

1.2.1 CDN引入

这种方式非常适合初学者或用于小型项目，因为它不需要额外的构建工具或依赖管理。直接在HTML文件中通过<script>标签引入：可以在HTML文档的<head>或<body>部分添加一个<script>标签来加载Vue。例如：

```
<script src="https://cdn.jsdelivr.net/npm/vue@next"></script>
```

注意事项：当从CDN获取资源时，可能会受到网络速度的影响，因此建议在生产环境中考虑其他更稳定的部署方案。

1.2.2 使用npm安装

适合大型项目和团队开发。通过npm安装可以方便地管理项目依赖。在项目目录下打开Command运行以下命令：

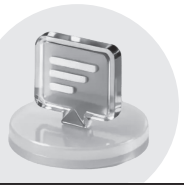
```
npm install vue
```

1.2.3 使用Vue CLI创建项目

Vue CLI是一个命令行工具，可以帮助开发者快速创建和管理Vue项目。安装Vue CLI的命令如下：

```
npm install -g @vue/cli
```

1.3 Vue开发工具



Vue.js的生态系统非常丰富，提供了多种开发工具来帮助开发者更高效地构建和调试应用程序。以下是详细的介绍，涵盖了主要的开发工具及其使用方法。

1.3.1 Vue Devtools

Vue Devtools是一个浏览器扩展，专为Vue应用的调试而设计。它允许开发者在浏览器中直接查看和操作Vue实例的状态、组件树结构等信息。对于Chrome浏览器，可以在Chrome Web Store找到并安装。Firefox用户可以从Firefox Add-ons商店获取。Edge和其他基于Chromium的浏览器也支持该扩展。

1.3.2 Vue CLI

Vue CLI作为官方专门提供的命令行工具，其主要目的在于极大地简化Vue项目的创建过程以及日常管理工作。它可不单单只是提供了一个能够让开发者快速搭建项目的基础架构框架，更为重要的是，它还巧妙地集成了众多现代化前端开发领域里被广泛认可的最佳实践经验。至于关于这个功能强大的工具的安装步骤和相关细节，可查阅之前学习过的Vue教程。

1.4 Vue基础语法



在后续的内容当中，将详细地介绍Vue的基本语法的具体使用方法。主要包括：模板语法、计算属性和侦听器、条件渲染、列表渲染、事件处理以及组件相关等内容。通过这些语法的灵活运用，能够高效地构建出功能丰富且交互性强的前端页面。

1.4.1 模板语法

1.插值

插值指的是在模板中使用特定的语法来插入表达式的值，使得这些值能够随着数据的变化而自动更新。在Vue中，插值通常通过双大括号{{ }}来实现。

（1）基本用法

假设有一个Vue实例，并且在这个实例的数据对象中定义了一个消息：

```
new Vue({
  el: '#app',
  data: {
    message: 'Hello Vue!'
  }
});
```

在HTML模板中，可以这样写：

```
<div id="app">
  <p>{{ message }}</p>
</div>
```

当页面加载时，{{message}}将被替换为data对象中的message属性的值，即"Hello Vue!"。如果message的值发生变化，视图也会相应地更新。

（2）插值表达式

除了简单的属性名外，双大括号内还可以包含任何有效的JavaScript表达式，比如：

①字符串拼接。

```
<p>{{ 'Welcome, ' + user.name }}</p>
```

②三元运算符。

```
<p>{{ isLoggedIn ? 'Logout' : 'Login' }}</p>
```

③访问对象属性或数组元素。

```
<p>{{ greetUser() }}</p>
```

④访问对象属性或数组元素。

```
<p>{{ user.address.city }}</p>
```

（3）注意事项

①避免复杂的逻辑：虽然可以在插值中编写表达式，但应尽量保持简单。过于复杂的逻辑应该放在计算属性或方法中。

②不允许多语句：Vue的插值表达式不支持多条语句，例如不能使用分号分割多个表达式。

③特殊字符转义：对于某些特殊字符（如<，>，&），Vue会自动进行HTML实体编码以防止潜在的安全问题。

总之，插值是Vue中用于数据绑定的基本机制，它让开发者可以轻松地将模型层的数据反映到视图层。结合其他指令和特性，插值为构建动态、响应式的用户界面提供了坚实的基础。

2.指令

Vue的模板指令是其核心功能之一，它们提供了在DOM中嵌入动态行为的方法。通过使用前缀为v-的特殊属性，开发者可以轻松地实现条件渲染、列表渲染、事件监听等功能。以下是Vue模板中常用指令的详细介绍：

（1）条件渲染

①v-if：用于根据表达式的真假值来决定是否渲染元素。

```
<div v-if="seen">现在你看到我了</div>
```

当seen为true时，该 将被渲染到页面上；否则，它将不会出现在DOM中。

②v-else：配合v-if使用，提供一个“否则”的分支。

```
<div v-if="Math.random() > 0.5">
  随机数大于0.5
</div>
<div v-else>
  随机数小于等于0.5
</div>
```

③v-else-if：添加额外的条件分支。

```
<div v-if="type === 'A'">A</div>
<div v-else-if="type === 'B'">B</div>
<div v-else-if="type === 'C'">C</div>
<div v-else>Not A/B/C</div>
```

④v-show：与v-if类似，但始终会渲染并保留在DOM中，只是简单地切换CSS的display属性。

```
<h1 v-show="ok">Hello!</h1>
```

（2）列表渲染

v-for：用于遍历数组或对象，并为每个成员创建一个元素。

```
<ul id="example-1">
  <li v-for="(item, index) in items" :key="index">
    {{ item.message }}
  </li>
```

```
</ul>
```

对于对象：

```
<ul id="v-for-object" class="demo">
  <li v-for="value in object">
    {{ value }}
  </li>
</ul>
```

注意：使用 v-for 时建议总是提供一个唯一的 key 属性，以帮助 Vue 更高效地追踪节点的身份。

（3）属性绑定

v-bind：动态地绑定一个或多个属性，或组件 props 到表达式。

```
<a v-bind:href="url">...</a>
<img v-bind:src="imageSrc">
```

简写形式（:attr）：

```
<a :href="url">...</a>
<img :src="imageSrc">
```

（4）事件处理

v-on：监听 DOM 事件，并执行相应的 JavaScript 表达式或方法。

```
<button v-on:click="counter += 1">增加 1</button>
```

简写形式（`@event`）：

```
<button @click="incrementCounter">增加 1</button>
```

可以传递参数给事件处理器，例如：

```
<button @click="greet('hello')">Say hello</button>
```

（5）表单输入绑定

v-model：创建双向数据绑定，使得用户输入能够自动更新数据模型，反之亦然。

```
<input v-model="message" placeholder="edit me">
<p>Message is: {{ message }}</p>
```

支持多种类型的输入字段，如文本框、复选框、单选按钮等。

Vue 的指令系统为开发者提供了强大的工具来操作 DOM 和管理状态。每种指令都有其特定用途，了解如何正确使用这些指令可以使应用程序更加灵活和易于维护。

3. 属性选项

属性选项（Attribute Bindings）是模板语法中的一个重要组成部分，主要用于将数据绑定到 HTML 元素的属性上。以下是对 Vue 模板语法中属性选项的详细介绍。

v-bind指令用于动态地绑定一个或多个属性，或一个组件prop到表达式的值。它可以根据表达式的值更新绑定的属性。

（1）基本用法

①绑定单个属性：

```
<!-- 绑定id属性 -->
<div v-bind:id="dynamicId"></div>
```

②绑定多个属性：

```
<!-- 绑定多个属性 -->
<div v-bind="{ id: dynamicId, title: dynamicTitle }"></div>
```

（2）缩写

v-bind可以缩写为 :attr，这使得模板更简洁：

```
<!-- 绑定id属性（缩写）-->
<div :id="dynamicId"></div>
```

（3）动态绑定

v-bind可以用于动态绑定属性值，这些值可以是数据对象中的任何类型，包括字符串、数字、布尔值、数组和对象。

①绑定字符串

```
<!-- 绑定字符串 -->
<img :src="imageSrc">
```

②绑定数字

```
<!-- 绑定数字 -->
<input :maxlength="maxLength">
```

③绑定布尔值

```
<!-- 绑定布尔值 -->
<input :disabled="isDisabled">
```

④绑定数组

```
<!-- 绑定数组 -->
<div :class="classArray"></div>
```

⑤绑定对象

```
<!-- 绑定对象 -->
<div :style="styleObject"></div>
```

（4）类和样式绑定

v-bind可以用于绑定类（class）和样式（style）属性，这在动态控制元素的外观时非常有用。

①类绑定

绑定单个类：

```
<!-- 绑定单个类 -->
<div :class="{ active: isActive }"></div>
```

绑定多个类：

```
<!-- 绑定多个类 -->
<div :class="[activeClass, errorClass]"></div>
```

绑定对象：

```
<!-- 绑定对象 -->
<div :class="classObject"></div>
```

②样式绑定

绑定内联样式：

```
<!-- 绑定内联样式 -->
<div :style="{ color: activeColor, fontSize: fontSize + 'px' }"></div>
```

绑定样式对象：

```
<!-- 绑定样式对象 -->
<div :style="styleObject"></div>
```

绑定样式数组：

```
<!-- 绑定样式数组 -->
<div :style="[baseStyles, overridingStyles]"></div>
```

【例1-1】展示了Vue.js框架中v-bind指令的多种常见用法与高级应用场景。

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Vue v-bind 示例</title>
    <style>
        .active {
            color: red;
```

```
        }
        .text-danger {
            color: blue;
        }
    </style>
</head>
<body>
    <div id="app">
        <div :id="dynamicId">动态 ID</div>
        <img :src="imageSrc" alt="动态图片 ">
        <input :maxlength="maxLength" placeholder="输入内容 ">
        <input :disabled="isDisabled" placeholder="禁用输入 ">
        <div :class="{ active: isActive, 'text-danger': hasError }">类绑定 </div>
        <div :class="[activeClass, errorClass]">多个类 </div>
        <div :class="classObject">类对象 </div>
        <div :style="{ color: activeColor, fontSize: fontSize + 'px' }">样式绑定 </div>
        <div :style="styleObject">样式对象 </div>
        <div :style="[baseStyles, overridingStyles]">样式数组 </div>
    </div>
    <script src="https://cdn.jsdelivr.net/npm/vue@2.6.14/dist/vue.js"></script>
    <script>
        new Vue({
            el: '#app',
            data: {
                dynamicId: 'unique-id',
                imageSrc: 'https://via.placeholder.com/150',
                maxLength: 10,
                isDisabled: true,
                isActive: true,
                hasError: false,
                activeClass: 'active',
                errorClass: 'text-danger',
```

```
            classObject: {
                active: true,
                'text-danger': false
            },
            activeColor: 'red',
            fontSize: 30,
            styleObject: {
                color: 'blue',
                fontSize: '16px'
            },
            baseStyles: {
                color: 'green'
            },
            overridingStyles: {
                fontSize: '20px'
            }
        }
    });
  </script>
</body>
</html>
```

在上述代码中，使用v-bind绑定了多种类型的属性，包括字符串、数字、布尔值、数组和对象，使用v-bind绑定了类和样式属性，展示了如何动态控制元素的外观，使用了v-bind的缩写，使模板更简洁。

4. 方法选项

在Vue 3中，方法选项（methods）是组件实例上可用的方法的集合。这些方法可以在模板中通过事件处理器调用，也可以在组件的其他方法中调用。methods中的方法可以访问组件的data、props、computed等属性，这使得它们非常适合处理用户交互和数据操作。

（1）基本用法

在组件的methods选项中定义方法，这些方法可以在模板中通过事件指令（如v-on或@）调用。

【例1-2】在Vue组件的methods选项中定义方法，编写一个简单的计数器功能。

```
<template>
```

```
  <div>
    <button @click="incrementCount">点击增加计数</button>
    <p>当前计数: {{ count }}</p>
  </div>
</template>

<script>
export default {
  data() {
    return {
      count: 0
    };
  },
  methods: {
    incrementCount() {
      this.count += 1;
    }
  }
};
</script>
```

在上述代码中，incrementCount 方法定义在 methods 选项中，当按钮被点击时，incrementCount 方法会被调用，从而增加 count 的值。

（2）传递参数

方法可以接收参数，这些参数可以是静态值，也可以是动态数据。

【例 1-3】在 Vue 项目实践中，创建一个参数传递的案例，确保当用户点击按钮时，将 hello 字符串传递给名为 message 的变量。

示例：

```
<template>
  <div>
    <button @click="greet('Hello')">打招呼</button>
    <p>{{ message }}</p>
  </div>
```

```
</template>

<script>
export default {
  data() {
    return {
      message: ''
    };
  },
  methods: {
    greet(msg) {
      this.message = msg;
    }
  }
};
</script>
```

在上述代码中，greet方法接收一个参数msg，当按钮被点击时，greet方法会被调用，并将传递的字符串’Hello’赋值给message数据属性。

（3）访问事件对象

方法可以访问原生DOM事件对象，这在处理表单输入、键盘事件等场景中非常有用。

【例1-4】在Vue项目实践中，构建一个响应式输入框，实时展示用户输入内容。

示例：

```
<template>
  <div>
    <input @input="handleInput" placeholder="输入内容">
    <p>输入内容：{{ userInput }}</p>
  </div>
</template>

<script>
export default {
  data() {
```

```
    return {
      userInput: ''
    };
  },
  methods: {
    handleInput(event) {
      this.userInput = event.target.value;
    }
  }
};
</script>
```

在上述代码中，handleInput 方法接收一个事件对象 event，通过 event.target.value 获取输入框的当前值，并将其赋值给 userInput 数据属性。

（4）在方法中调用其他方法

方法可以在内部调用其他方法，这在处理复杂的逻辑时非常有用。

【例 1-5】在 Vue 项目实践中，构建一个完整的表单提交与验证机制，展示按钮点击事件处理、表单验证及状态反馈的完整操作流程。

```
<template>
  <div>
    <button @click="submitForm">提交表单</button>
    <p>表单提交状态：{{ formStatus }}</p>
  </div>
</template>

<script>
export default {
  data() {
    return {
      formStatus: ''
    };
  },
  methods: {
```

```
    validateForm() {
      // 这里可以添加表单验证逻辑
      return true; // 假设表单总是有效的
    },
    submitForm() {
      if (this.validateForm()) {
        this.formStatus = '表单提交成功';
      } else {
        this.formStatus = '表单验证失败';
      }
    }
  }
};
</script>
```

在上述代码中，submitForm方法调用了validateForm方法来验证表单。如果表单验证通过，则更新formStatus数据属性。

（5）使用方法返回值

方法可以返回值，这些返回值可以在模板中直接使用。

【例1-6】在Vue项目实践中，通过计算属性computedValue获取一个返回值，并在模板中加以应用。

```
<template>
  <div>
    <p>计算后的值: {{ computedValue }}</p>
  </div>
</template>

<script>
export default {
  data() {
    return {
      baseValue: 10
    };
```

```
  },
  methods: {
    computeValue() {
      return this.baseValue * 2;
    }
  },
  computed: {
    computedValue() {
      return this.computeValue();
    }
  }
};
</script>
```

在上述代码中，computeValue方法返回baseValue的两倍，这个返回值通过计算属性computedValue在模板中使用。

5.计算属性

计算属性（Computed Properties）是Vue中一个非常实用的功能，它允许基于组件的数据动态计算出新的值。与普通的方法不同，计算属性具有缓存机制，只有在其依赖的数据发生变化时，才会重新计算。这使得计算属性在处理复杂的逻辑时更加高效，因为它们不会在每次组件渲染时都重新执行。

计算属性通过computed选项定义，它是一个对象，对象的每个属性都是一个计算属性。计算属性可以是一个函数，也可以是一个包含get和set方法的对象。

【例1-7】有一个组件，需展示用户的全名，全名由名字和姓氏构成，可通过计算属性来实现。

```
<template>
  <div>
    <p>全名：{{ fullName }}</p>
    <input v-model="firstName" placeholder="输入名字">
    <input v-model="lastName" placeholder="输入姓氏">
  </div>
</template>

<script>
export default {
```

```
  data() {
    return {
      firstName: 'John',
      lastName: 'Doe'
    };
  },
  computed: {
    fullName() {
      return `${this.firstName} ${this.lastName}`;
    }
  }
};
</script>
```

在上述代码中，fullName是一个计算属性，它依赖于firstName和lastName。当firstName或lastName发生变化时，fullName会自动重新计算。如果firstName和lastName没有变化，fullName的结果将直接从缓存中获取，不会重新计算。

【例1-8】有一个组件，需展示消息的反转内容，可通过计算属性来实现。

```
<template>
  <div>
    <p>消息：{{ message }}</p>
    <p>反转消息：{{ reversedMessage }}</p>
    <input v-model="message" placeholder="输入消息">
  </div>
</template>

<script>
export default {
  data() {
    return {
      message: 'Hello Vue!'
    };
  },
```

```
  computed: {
    reversedMessage() {
      return this.message.split('').reverse().join('');
    }
  }
};
</script>
```

在上述代码中，reversedMessage是一个计算属性，它依赖于message。当message发生变化时，reversedMessage会自动重新计算。如果message没有变化，reversedMessage的结果将直接从缓存中获取，不会重新计算。

【例1-9】存在一个组件，需展示用户的全名，并允许用户直接对其进行修改，此功能可通过计算属性的get和set方法予以实现。

```
<template>
  <div>
    <p>全名：{{ fullName }}</p>
    <input v-model="fullName" placeholder="输入全名">
    <p>名字：{{ firstName }}</p>
    <p>姓氏：{{ lastName }}</p>
  </div>
</template>

<script>
export default {
  data() {
    return {
      firstName: 'John',
      lastName: 'Doe'
    };
  },
  computed: {
    fullName: {
      get() {
```

```
      return `${this.firstName} ${this.lastName}`;
    },
    set(newValue) {
      const names = newValue.split(' ');
      this.firstName = names[0];
      this.lastName = names[1];
    }
  }
 }
};
</script>
```

在上述代码中，fullName是一个计算属性，它定义了get和set方法。get方法返回firstName和lastName的组合，set方法将新的全名分割为firstName和lastName。这样，用户可以直接在输入框中修改全名，组件会自动更新firstName和lastName。

6.表单控件

在Vue中，表单控件可以通过v-model指令实现双向数据绑定，这意味着表单输入和应用状态之间的同步变得非常简单。v-model在内部会根据不同的输入元素使用不同的属性并更新不同的事件：

对于<input type="text">，<textarea>和<select>元素，v-model会利用value属性和input事件。

对于<input type="checkbox">和<input type="radio">，v-model会利用checked属性和change事件。

【例1-10】在Vue项目中，创建一个文本输入框和多行文本输入框。

```
<template>
 <div>
  <label for="username">用户名:</label>
  <input type="text" id="username" v-model="username">
  <p>输入的用户名是: {{ username }}</p>
 <hr/>
  <label for="description">描述:</label>
  <textarea id="description" v-model="description"></textarea>
  <p>输入的描述是: {{ description }}</p>
 </div>
</template>
```

```
<script>
export default {
  data() {
    return {
      username: '',
       description: ''
    };
  }
};
</script>
```

在上述代码中，username数据属性与文本输入框绑定。当用户在输入框中输入内容时，username的值将自动更新；反之亦然。description数据属性与多行文本输入框绑定，当用户在文本区域中输入内容时，description的值也将自动更新；反之亦然。

【例1-11】在Vue项目中，创建一个单选按钮和复选框。

```
<template>
  <div>
    <label><input type="radio" value="Male" v-model="gender"> 男 </label>
    <label><input type="radio" value="Female" v-model="gender"> 女 </label>
    <p>选择的性别是：{{ gender }}</p>
    <hr/>
    <label><input type="checkbox" v-model="checked"> 是否同意 </label>
    <p>是否同意：{{ checked }}</p>
  </div>
</template>

<script>
export default {
  data() {
    return {
      gender: '',
      checked: false
```

```
    };
  }
};
</script>
```

在上述代码中，gender数据属性与单选按钮绑定。当用户选择一个单选按钮时，gender的值会自动更新为该单选按钮的值，checked数据属性与复选框绑定，当用户勾选或取消勾选复选框时，checked的值会自动更新为true或false。

【例1-12】在Vue项目中，创建一个下拉选择框。

```
<template>
  <div>
    <label for="city">城市:</label>
    <select id="city" v-model="selectedCity">
      <option value="Beijing">北京</option>
      <option value="Shanghai">上海</option>
      <option value="Guangzhou">广州</option>
      <option value="Shenzhen">深圳</option>
    </select>
    <p>选择的城市是: {{ selectedCity }}</p>
  </div>
</template>

<script>
export default {
  data() {
    return {
      selectedCity: ''
    };
  }
};
</script>
```

在上述代码中，selectedCity数据属性与下拉选择框绑定。当用户选择一个选项时，selectedCity的值会自动更新为该选项的值。

7.生命周期

Vue实例有一个完整的生命周期钩子，可以对不同阶段进行响应，生命周期钩子为实例提供了一个在不同阶段执行代码的机会。从实例创建到销毁，Vue实例会经历一系列的生命周期阶段，包括创建、挂载、更新和销毁。

Vue实例的生命周期钩子包括：

beforeCreate：在实例初始化之后，数据观测(data observer)和事件配置之前被调用。

created：在实例创建完成后被调用。此时实例已完成数据观测，属性和方法的运算，$el属性还未显示出来。

beforeMount：在挂载开始之前被调用，相关的render函数首次被调用。

mounted：在实例挂载完成后被调用，此时实例的$el已替换。

beforeUpdate：在数据更新时调用，发生在虚拟DOM重新渲染和打补丁之前。

updated：在数据更新后，虚拟DOM重新渲染和打补丁之后调用。

beforeDestroy：在实例销毁之前调用。此时实例仍然完全可用。

destroyed：在实例销毁后调用。此时Vue实例指示的所有东西都会解绑，所有的事件监听器会被移除，所有的子实例也会被销毁。

【例1-13】在Vue组件中，展示了如何有效利用生命周期钩子。

```
<template>
  <div>
    <h1>{{ message }}</h1>
    <button @click="updateMessage">更新消息</button>
  </div>
</template>

<script>
export default {
  data() {
    return {
      message: '欢迎使用Vue!'
    };
  },
  beforeCreate() {
    console.log('beforeCreate: 实例创建之前');
```

```
  },
  created() {
    console.log('created: 实例创建完成');
  },
  beforeMount() {
    console.log('beforeMount: 挂载开始之前');
  },
  mounted() {
    console.log('mounted: 挂载完成');
  },
  beforeUpdate() {
    console.log('beforeUpdate: 数据更新之前');
  },
  updated() {
    console.log('updated: 数据更新之后');
  },
  beforeDestroy() {
    console.log('beforeDestroy: 实例销毁之前');
  },
  destroyed() {
    console.log('destroyed: 实例销毁之后');
  },
  methods: {
    updateMessage() {
      this.message = '消息已更新!';
    }
  }
};
</script>
```

在上述代码中，组件定义了多个生命周期钩子函数，每个钩子函数在对应的生命周期阶段被调用。通过在控制台中输出信息，可以观察到每个生命周期阶段的执行顺序。

1.4.2 模板渲染

1.条件渲染

在Vue中，条件渲染允许根据数据的值动态地渲染DOM元素。Vue提供了多种指令来实现条件渲染，包括v-if、v-else-if、v-else和v-show。这些指令可以根据表达式的真假值来决定是否渲染对应的DOM元素。

v-if：根据表达式的真假值，条件性地渲染元素。如果表达式为假，元素不会被渲染。

v-else-if：作为v-if的补充，提供“else if块”，必须跟在v-if或v-else-if之后。

v-else：作为v-if的补充，提供“else块”，必须跟在v-if或v-else-if之后。

v-show：根据表达式的真假值，条件性地显示元素。无论表达式真假，元素始终会被渲染，只是通过CSS的display属性来控制显示或隐藏。

【例1-14】在Vue.js开发中，v-if和v-else是一对常用的条件渲染指令，能够根据表达式的真假值动态控制元素的显示与隐藏。

```
<template>
  <div>
    <h1 v-if="isVisible">欢迎使用Vue 3!</h1>
    <h1 v-else>内容不可见</h1>
    <button @click="toggleVisibility">切换可见性</button>
  </div>
</template>

<script>
export default {
  data() {
    return {
      isVisible: true
    };
  },
  methods: {
    toggleVisibility() {
      this.isVisible = !this.isVisible;
    }
  }
```

```
};
</script>
```

在上述代码中，h1元素根据isVisible的值条件性地渲染，点击按钮会切换isVisible的值，从而切换显示不同的h1元素。

【例1-15】创建一个Vue项目，当用户类型为admin时，显示管理员，当用户类型为editor时，显示编辑，当用户类型为guest或其它值时，显示访客。

```
<template>
 <div>
  <h1 v-if="type === 'admin'">管理员</h1>
  <h1 v-else-if="type === 'editor'">编辑</h1>
  <h1 v-else>访客</h1>
  <button @click="changeType('admin')">设置为管理员</button>
  <button @click="changeType('editor')">设置为编辑</button>
  <button @click="changeType('guest')">设置为访客</button>
 </div>
</template>
<script>
export default {
 data() {
  return {
   type: 'guest'
  };
 },
 methods: {
  changeType(newType) {
   this.type = newType;
  }
 }
};
</script>
```

在上述代码中，根据type的值，条件性地渲染不同的h1元素。点击按钮会改变type的值，从而显示不同的内容。

【例1-16】开发一个Vue项目，通过按钮切换文本标题的显示与隐藏状态。

```
<template>
  <div>
    <h1 v-show="isVisible">欢迎使用Vue 3!</h1>
    <button @click="toggleVisibility">切换可见性</button>
  </div>
</template>

<script>
export default {
  data() {
    return {
      isVisible: true
    };
  },
  methods: {
    toggleVisibility() {
      this.isVisible = !this.isVisible;
    }
  }
};
</script>
```

在上述代码中，h1元素始终会被渲染，但会根据isVisible的值通过CSS的display属性来控制显示或隐藏。点击按钮会切换isVisible的值，从而切换h1元素的显示状态。

2.列表渲染

在Vue中，列表渲染允许根据数组或对象的值动态地渲染DOM元素。Vue提供了v-for指令来实现列表渲染，v-for可以遍历数组或对象的每个元素，并为每个元素生成一个DOM节点。

v-for指令用于基于数组或对象渲染列表。它需要一个特殊语法，即item in items，其中item是数组或对象的当前项，items是数据源。

数组渲染：可以遍历数组的每个元素，并为每个元素生成一个DOM节点。

对象渲染：可以遍历对象的每个属性，并为每个属性生成一个DOM节点。

索引访问：可以获取当前项的索引，方便在渲染时使用。

【例1-17】创建一个基于Vue.js框架的项目，通过编写组件和逻辑代码，展示如何对数组数据进行遍历。

```
<template>
  <div>
    <ul>
      <li v-for="(item, index) in items" :key="item.id">
        {{ index + 1 }}. {{ item.name }}
      </li>
    </ul>
  </div>
</template>

<script>
export default {
  data() {
    return {
      items: [
        { id: 1, name: '苹果' },
        { id: 2, name: '香蕉' },
        { id: 3, name: '橙子' }
      ]
    };
  }
};
</script>
```

在上述代码中，items数组中的每个对象都会生成一个<li>元素。v-for指令遍历items数组，item是当前项，index是当前项的索引。:key属性用于为每个列表项提供一个唯一的键值，这有助于Vue高效地更新DOM。

【例1-18】展示如何在界面中渲染对象。

```
<template>
  <div>
    <ul>
```

```
      <li v-for="(value, key) in info" :key="key">
        {{ key }}: {{ value }}
      </li>
    </ul>
  </div>
</template>

<script>
export default {
  data() {
    return {
      info: {
        name: '张三',
        age: 25,
        city: '北京'
      }
    };
  }
};
</script>
```

在上述代码中，info对象的每个属性都会生成一个<li>元素。v-for指令遍历info对象，value是当前属性的值，key是当前属性的键。:key属性用于为每个列表项提供一个唯一的键值。

【例1-19】展示如何在界面如何渲染数字

```
<template>
  <div>
    <ul>
      <li v-for="n in 5" :key="n">
        {{ n }}
      </li>
    </ul>
  </div>
</template>
```

在上述代码中，v-for指令遍历数字5，生成5个<li>元素，每个元素的文本内容为当前的数字。

3.template标签用法

在Vue中，<template>标签是一个非常有用的工具，用于控制渲染逻辑，而不会向最终的DOM添加额外的元素。<template>标签本身不会被渲染，但它可以包含其他元素，并且可以与Vue的指令（如v-if、v-for、v-slot等）结合使用，以实现复杂的渲染逻辑。在前面的多个示例中，已经使用了template标签，这里不再举例。

1.4.3 事件绑定

在Vue中，事件绑定是通过v-on指令实现的，它允许你监听DOM事件，并在触发时执行指定的方法。v-on指令可以用于绑定各种事件，如点击事件、输入事件、键盘事件等。事件绑定使得用户交互能够触发组件的方法，从而实现动态的用户界面。

1.基本用法

在Vue中，事件绑定通过v-on指令实现，用于监听DOM事件并在触发时执行指定的方法。v-on可以缩写为@，使模板更简洁。事件绑定支持多种事件，如点击、输入、键盘事件等，允许组件方法响应用户交互。

v-on指令的基本语法是v-on:eventName=”methodName”，其中eventName是事件名称，methodName是组件方法的名称。v-on可以缩写为@。

【例1-20】基于Vue.js的交互式点击计数器组件，通过点击按钮来增加计数。

```
<template>
  <div>
    <button @click="handleClick">点击我</button>
    <p>点击次数: {{ clickCount }}</p>
  </div>
</template>

<script>
export default {
  data() {
    return {
      clickCount: 0
    };
  },
```

```
  methods: {
    handleClick() {
      this.clickCount += 1;
    }
  }
};
</script>
```

在上述代码中，@click 绑定了一个点击事件，当按钮被点击时，handleClick 方法会被调用，clickCount 的值会增加。

【例 1-21】创建一个简易的交互式组件，当用户在输入框中键入内容时，输入框的值将实时更新。

```
<template>
  <div>z
    <input @input="handleInput" placeholder="输入内容">
    <p>输入内容：{{ userInput }}</p>
  </div>
</template>

<script>
export default {
  data() {
    return {
      userInput: ''
    };
  },
  methods: {
    handleInput(event) {
      this.userInput = event.target.value;
    }
  }
};
</script>
```

在上述代码中，@input 绑定了一个输入事件，当输入框的内容发生变化时，handleInput 方法会被调

用，userInput的值会被更新为输入框的当前值。

【例1-22】提供一个文本输入框，当用户在输入框中按下Enter键时，记录按键次数。

```
<template>
  <div>
    <input @keyup.enter="handleEnter" placeholder="按Enter键">
    <p>按Enter键的次数: {{ enterCount }}</p>
  </div>
</template>

<script>
export default {
  data() {
    return {
      enterCount: 0
    };
  },
  methods: {
    handleEnter() {
      this.enterCount += 1;
    }
  }
};
</script>
```

在上述代码中，@keyup.enter绑定了一个键盘事件，当用户按下Enter键时，handleEnter方法会被调用，enterCount的值会增加。

2.修饰符

Vue提供了多种事件修饰符，用于更精细地控制事件行为。这些修饰符可以添加到v-on指令或其缩写@之后，以实现常见的事件处理需求，如阻止事件冒泡、阻止默认行为、捕获事件等。

（1）.stop：阻止事件冒泡。

（2）.prevent：阻止事件的默认行为。

（3）.capture：在捕获阶段触发事件处理函数。

（4）.self：只有当事件是从当前元素本身触发时才触发处理函数。

（5）.once：事件处理函数只触发一次。

（6）.passive：事件处理函数以被动模式触发，常用于滚动事件等，以提高性能。

1.5 自定义指令



Vue的自定义指令是Vue.js框架中用于扩展HTML元素行为的一种机制。它允许开发者通过自定义指令直接操作DOM元素，以实现特定的功能或行为。自定义指令可以用于处理复杂的DOM操作、添加交互逻辑或封装重复的DOM操作逻辑，从而增强组件的功能性和复用性。自定义指令可以全局注册或局部注册，并且可以通过钩子函数来定义指令在不同生命周期阶段的行为。接下来，我们来学习怎么实现自定义指令。

1.5.1 指令的注册

1.全局指令的注册。

全局注册的自定义指令可以在任何组件中使用，通过app.directive()方法完成注册。

【例1-23】实现一个Vue 3自定义指令，用于在页面加载时自动聚焦到指定输入框。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vue 3 自定义指令示例</title>
  <script src="https://unpkg.com/vue@3"></script>
</head>
<body>
  <div id="app">
    <input v-focus placeholder="聚焦输入框">
  </div>

  <script>
    const { createApp } = Vue;
```

```
    const app = createApp({});

    // 全局注册自定义指令 v-focus
    app.directive('focus', {
      mounted(el) {
        el.focus(); // 在元素挂载完成后聚焦
      }
    });

    app.mount('#app');
  </script>
</body>
</html>
```

在上述代码中，通过app.directive('focus', ...)全局注册了一个名为v-focus的自定义指令。该指令在元素挂载完成后调用el.focus()方法，使输入框自动聚焦。

2.局部指令的注册

局部注册的自定义指令仅在定义它的组件内有效，通过在组件的directives选项中定义指令实现。

【例1-24】在Vue应用实例的directives选项中局部注册highlight指令。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vue 3 自定义指令示例</title>
  <script src="https://unpkg.com/vue@3"></script>
</head>
<body>
  <div id="app">
    <p v-highlight="'yellow'">这是一段高亮的文本</p>
  </div>
  <script>
```

```
    const { createApp } = Vue;

    const app = createApp({
      directives: {
        highlight(el, binding) {
          el.style.backgroundColor = binding.value; // 使用绑定值设置背景颜色
        }
      }
    });

    app.mount('#app');
  </script>
</body>
</html>
```

在上述代码中，通过在组件的directives选项中定义v-highlight指令，实现了局部注册。该指令通过binding.value获取传递的值（如'yellow'），并将其设置为元素的背景颜色。

1.5.2 指令的定义对象

在Vue中，自定义指令的定义对象是一个包含多个生命周期钩子函数的对象。这些钩子函数允许在指令的不同阶段对DOM进行操作。常用的生命周期钩子包括beforeMount、mounted、beforeUpdate、updated和beforeUnmount。通过这些钩子，开发者可以精确控制指令的行为，指令的定义对象可以包含以下生命周期钩子：

1.beforeMount：在元素挂载到DOM之前调用。

2.mounted：在元素挂载到DOM之后调用。

3.beforeUpdate：在元素更新之前调用。

4.updated：在元素更新之后调用。

5.beforeUnmount：在元素卸载之前调用。

【例1-25】Vue 3的自定义指令生命周期钩子与参数传递机制。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vue 3 指令的定义对象示例</title>
  <script src="https://unpkg.com/vue@3"></script>
</head>
<body>
  <div id="app">
    <input v-focus placeholder=" 聚焦输入框 ">
    <p v-highlight="'yellow'">这是一段高亮的文本 </p>
  </div>

  <script>
    const { createApp } = Vue;

    const app = createApp({});

    // 全局注册自定义指令 v-focus
    app.directive('focus', {
      beforeMount(el) {
        el.style.color = 'blue'; // 在挂载前设置文本颜色
      },
      mounted(el) {
        el.focus(); // 在挂载后聚焦到输入框
      }
    });

    // 全局注册自定义指令 v-highlight
    app.directive('highlight', {
      beforeMount(el, binding) {
        el.style.color = 'gray'; // 在挂载前设置文本颜色
      },
      mounted(el, binding) {
        el.style.backgroundColor = binding.value; // 在挂载后设置背景颜色
```

```
    }
  });

  app.mount('#app');
 </script>
</body>
</html>
```

在上述代码中，v-focus指令通过beforeMount钩子在组件挂载前设定输入框的文本颜色，并通过mounted钩子在组件挂载后实现输入框的自动聚焦。而v-highlight指令则利用beforeMount钩子在挂载前配置文本颜色，再借助mounted钩子在挂载后根据绑定的值动态调整背景颜色。

1.5.3 指令实例属性

在Vue 3中，自定义指令的生命周期钩子函数可以访问指令实例的属性，这些属性提供了对指令绑定的详细信息和上下文的访问。常用的指令实例属性包括：

1.el：绑定的DOM元素。

2.binding：包含指令相关信息的对象，如绑定值、表达式、参数和修饰符。

3.vnode：虚拟节点，表示当前绑定的Vue组件节点。

4.prevVNode：更新时的上一个虚拟节点，仅在beforeUpdate和updated钩子中可用。

【例1-26】使用CDN方式引入Vue的示例，展示了如何使用指令实例属性。

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Vue 3 指令实例属性示例</title>
 <script src="https://unpkg.com/vue@3"></script>
</head>
<body>
 <div id="app">
  <input v-focus placeholder=" 聚焦输入框 ">
  <p v-highlight="'yellow'">这是一段高亮的文本</p>
  <button @click="toggleColor">切换颜色</button>
```

```
</div>

<script>
  const { createApp, ref } = Vue;

  const app = createApp({
    setup() {
      const color = ref('yellow');
      const toggleColor = () => {
        color.value = color.value === 'yellow' ? 'lightblue' : 'yellow';
      };
      return { color, toggleColor };
    }
  });

  // 全局注册自定义指令 v-focus
  app.directive('focus', {
    mounted(el) {
      el.focus(); // 使用el访问绑定的DOM元素
    }
  });

  // 全局注册自定义指令 v-highlight
  app.directive('highlight', {
    mounted(el, binding) {
      el.style.backgroundColor = binding.value; // 使用binding访问绑定值
    },
    updated(el, binding) {
      el.style.backgroundColor = binding.value; // 在更新时重新设置背景颜色
    }
  });
```

```
    app.mount('#app');
  </script>
</body>
</html>
```

综上代码所述，呈现如下效果，如图 1-1 所示。

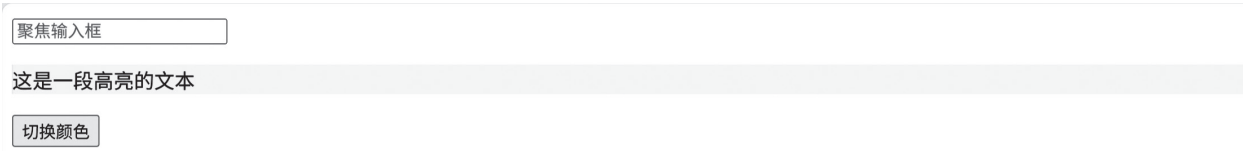


图 1-1　指令实例属性案例

在上述代码中，Vue 的指令实例属性提供了对指令绑定的 DOM 元素、绑定值、虚拟节点等信息的访问。通过这些属性，开发者可以在指令的生命周期钩子中动态操作 DOM，实现复杂的交互逻辑。

1.6 组件



1.6.1 什么是组件

Vue 的组件是 Vue.js 框架中的核心概念之一，它是构成 Vue 应用的基本单元。组件允许开发者将复杂的用户界面拆分为可复用、可维护的独立模块。每个组件都有自己的模板、数据、方法、生命周期钩子以及可选的样式。通过组件化开发，可以提高代码的可维护性和复用性，同时使得大型应用的开发更加高效。

1.6.2 组件的基本使用

组件的注册分为全局注册和局部注册，接下来我们来分别学习全局注册和局部注册该怎么实现。

1. 全局注册

在 Vue 中，全局注册组件允许在任何地方使用该组件，而无需在每个父组件中重复定义。全局组件通过 app.component() 方法注册，注册后可以在应用的任何模板中直接使用。全局注册组件的基本用法如下：

（1）使用 app.component() 方法注册组件，传入组件名称和组件定义。

（2）在模板中通过自定义标签使用该组件。

【例 1-27】创建一个简洁的 Vue 3 应用，展示如何全局注册组件并在多处重复使用。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vue 3 全局组件注册示例</title>
  <script src="https://unpkg.com/vue@3"></script>
</head>
<body>
  <div id="app">
    <!-- 使用全局注册的组件 -->
    <greeting></greeting>
    <greeting></greeting>
  </div>

  <script>
    const { createApp } = Vue;

    // 创建Vue应用实例
    const app = createApp({});

    // 全局注册一个组件
    app.component('greeting', {
      template: `<h1>Hello, {{ name }}!</h1>`,
      data() {
        return {
          name: 'Vue 3'
        };
      }
    });

    // 挂载应用到DOM
```

```
    app.mount('#app');
  </script>
</body>
</html>
```

综上代码所述，呈现如下效果，如图1-2所示。



图1-2　全局组件注册的案例

在上述代码中，全局注册组件：通过app.component(greeting，...)方法注册一个名为greeting的组件，该组件定义包含template和data两部分，其中template定义了组件的HTML结构，而data则定义了组件的响应式数据。

2.局部注册

在Vue中，局部注册组件允许在特定组件内部定义和使用组件，而不对外暴露。局部注册的组件仅在定义它的组件内部有效，这有助于避免命名冲突并保持代码的封装性。局部组件通过在组件的components选项中定义来实现。局部注册组件的基本用法如下：

（1）在组件的components选项中定义局部组件。

（2）在模板中通过自定义标签使用局部注册的组件。

【例1-28】创建一个简洁的Vue 3应用，展示如何局部注册组件。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vue 3 局部组件注册示例</title>
  <script src="https://unpkg.com/vue@3"></script>
</head>
<body>
  <div id="app">
    <!-- 使用局部注册的组件 -->
    <greeting></greeting>
  </div>
```

```
<script>
  const { createApp } = Vue;

  // 定义一个局部组件
  const GreetingComponent = {
    template: `<h1>Hello, {{ name }}!</h1>`,
    data() {
      return {
        name: 'Vue 3'
      };
    }
  };

  // 创建Vue应用实例
  const app = createApp({
    components: {
      // 局部注册组件
      Greeting: GreetingComponent
    }
  });

  // 挂载应用到DOM
  app.mount('#app');
 </script>
</body>
</html>
```

在上述代码中，GreetingComponent定义了模板和数据。其中，template负责定义组件的HTML结构，而data则定义了组件的响应式数据。局部注册方面：在应用的components选项中，通过Greeting: GreetingComponent的方式，将组件局部注册到应用中。

1.6.3 组件选项

1.组件 props

在 Vue 中，props 是组件之间通信的一种方式，允许父组件向子组件传递数据。props 是子组件接收来自父组件的数据的接口，使得子组件可以基于父组件传递的值动态地渲染内容或执行逻辑。props 的基本用法：

（1）定义 props：在子组件中通过 props 选项声明它期望接收的数据。

（2）传递数据：父组件通过子组件的标签属性传递数据。

（3）使用 props：子组件可以在模板或逻辑中使用接收到的 props。

【例 1-29】创建一个 Vue 3 应用，实现父组件向子组件的数据传递。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vue 3 `props` 示例</title>
  <script src="https://unpkg.com/vue@3"></script>
</head>
<body>
  <div id="app">
    <!-- 父组件向子组件传递数据 -->
    <child-component :message="parentMessage"></child-component>
  </div>

  <script>
    const { createApp } = Vue;

    // 定义子组件
    const ChildComponent = {
      props: ['message'], // 声明接收的 props
      template: `<div>子组件接收到的消息：{{ message }}</div>`
    };
```

```
    // 创建Vue应用实例
    createApp({
      components: {
        'child-component': ChildComponent
      },
      data() {
        return {
          parentMessage: 'Hello from Parent!' // 父组件的数据
        };
      }
    }).mount('#app');
  </script>
</body>
</html>
```

综上代码所述，呈现如下效果，如图1-3所示。



图1-3 props的案例

在上述代码中，子组件通过props:[‘message’]声明其期望接收一个名为message的prop。父组件则通过:message=parentMessage将数据传递给子组件。此处采用了绑定语法:，以确保传递的是动态数据，子组件在模板中通过{{message}}访问接收到的prop，并据此动态渲染内容。

2.props验证

在Vue中，props验证允许开发者对父组件传递给子组件的数据进行类型检查和其他验证。通过在子组件中定义props的验证规则，可以确保接收到的数据符合预期，从而提高组件的健壮性和可维护性。props验证支持多种验证方式，包括类型检查、必需性验证、自定义验证函数等。props验证的基本用法：

（1）类型检查：指定props的类型，如String、Number、Boolean、Array、Object等。

（2）必需性验证：通过required:true标记某个prop为必需。

（3）默认值：通过default属性为prop提供默认值。

（4）自定义验证：通过validator函数实现更复杂的验证逻辑。

【例1-30】创建一个Vue 3应用，展示组件props的全面验证机制，涵盖类型检查、必需性验证、默认值设定以及自定义验证功能。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vue 3 `props` 验证示例</title>
  <script src="https://unpkg.com/vue@3"></script>
</head>
<body>
  <div id="app">
    <!-- 父组件传递数据 -->
    <child-component
      :message="parentMessage"
      :count="10"
      :isActive="true"
      :details="{ name: 'Vue', version: 3 }"
    ></child-component>
  </div>

  <script>
    const { createApp } = Vue;

    // 定义子组件
    const ChildComponent = {
      props: {
        // 类型为String，且为必需
        message: {
          type: String,
          required: true
        },
        // 类型为Number，提供默认值
        count: {
```

```
    type: Number,
    default: 0
  },
  // 类型为 Boolean，提供默认值
  isActive: {
    type: Boolean,
    default: false
  },
  // 类型为 Object，提供默认值
  details: {
    type: Object,
    default() {
      return { name: 'Default', version: 0 };
    }
  },
  // 自定义验证函数
  status: {
    validator(value) {
      return ['active', 'inactive', 'pending'].includes(value);
    }
  }
},
template: `
  <div>
    <p>消息：{{ message }}</p>
    <p>计数：{{ count }}</p>
    <p>激活状态：{{ isActive }}</p>
    <p>详情：{{ details.name }} (版本：{{ details.version }})</p>
    <p>状态：{{ status }}</p>
  </div>
`
};
```

```
    // 创建Vue应用实例
    createApp({
      components: {
        'child-component': ChildComponent
      },
      data() {
        return {
          parentMessage: 'Hello from Parent!'
        };
      }
    }).mount('#app');
  </script>
</body>
</html>
```

综上代码所述，呈现如下效果，如图 1-4 所示。



图 1-4　props 验证的案例

在上述代码的类型检查中，用 type 属性指定 prop 类型，如 String、Number、Boolean、Object 等。进行必需性验证时，用 required:true 标记某个 prop 为必需，若父组件未传递该 prop，Vue 会发出警告。设置默认值时，用 default 属性为 prop 提供默认值，对象或数组类型的 prop 默认值需通过函数返回。进行自定义验证时，用 validator 函数实现更复杂的验证逻辑，该函数接收 prop 的值作为参数，返回布尔值表示验证是否通过。

3. 单向数据流

在 Vue 中，单向数据流是组件间数据传递的核心原则。根据单向数据流，数据总是从父组件流向子组件，通过 props 传递。如果子组件需要修改传递的数据，应通过触发事件通知父组件进行更新，而不是直接修改 props。这种模式确保了组件之间的数据流向清晰且易于追踪。单向数据流的基本用法：

（1）父组件向子组件传递数据：通过 props 将数据传递给子组件。

（2）子组件触发事件通知父组件更新数据：子组件通过 $emit 触发自定义事件，父组件监听这些事件

并更新数据。

（3）避免直接修改props：子组件不应直接修改从父组件传递的props，以保持数据流的单向性。

【例1-31】基于当前Vue 3项目，实现单向数据流机制及父子组件间的通信流程。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vue 3 单向数据流示例</title>
  <script src="https://unpkg.com/vue@3"></script>
</head>
<body>
  <div id="app">
    <!-- 父组件传递数据，并监听子组件的事件 -->
      <child-component :message="parentMessage" @update:message="parentMessage = $event"></child-
component>
  </div>

  <script>
    const { createApp } = Vue;

    // 定义子组件
    const ChildComponent = {
      props: ['message'],
      template: `
        <div>
          <p>子组件显示的消息: {{ message }}</p>
          <input v-model="localMessage" @input="updateMessage">
        </div>
      `,
      data() {
        return {
```

```
        localMessage: this.message // 将 props 的值同步到本地数据
      };
    },
    methods: {
      updateMessage() {
        this.$emit('update:message', this.localMessage); // 触发事件通知父组件更新
      }
    }
  };

  // 创建 Vue 应用实例
  createApp({
    components: {
      'child-component': ChildComponent
    },
    data() {
      return {
        parentMessage: 'Hello from Parent!' // 父组件的数据
      };
    }
  }).mount('#app');
</script>
</body>
</html>
```

综上代码所述，呈现如下效果，如图 1-5 所示。



图 1-5　单向数据流的案例

在上述代码中，父组件通过 :message=”parentMessage”向子组件传递数据，同时监听子组件触发的 update:message 事件并更新 parentMessage。而子组件借助 v-model 将输入框的值绑定到本地数据 localMessage，避免直接修改 props。

1.6.4 组件通信

Vue的组件通信是指在Vue应用中，不同组件之间如何传递数据和事件。

1.自定义事件

在Vue中，自定义事件是组件间通信的重要方式之一，尤其适用于父子组件之间的交互。子组件可以通过$emit方法触发自定义事件，父组件则通过监听这些事件来响应子组件的操作。这种通信方式使得父子组件之间的数据传递和行为控制变得灵活且解耦。自定义事件的基本用法：

（1）子组件触发事件：使用this.$emit(eventName,payload)触发自定义事件，其中eventName是事件名称，payload是传递给父组件的数据。

（2）父组件监听事件：在父组件中通过v-on:eventName或@eventName监听子组件触发的事件，并执行相应的处理函数。

【例1-32】创建一个Vue3项目，使用自定义事件实现父子组件通信。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vue 3 自定义事件示例</title>
  <script src="https://unpkg.com/vue@3"></script>
</head>
<body>
  <div id="app">
    <!-- 父组件监听子组件的事件 -->
    <child-component @custom-event="handleCustomEvent"></child-component>
    <p>父组件接收到的消息：{{ message }}</p>
  </div>

  <script>
    const { createApp } = Vue;

    // 定义子组件
    const ChildComponent = {
      template: `
```

```
      <button @click="sendMessage">发送消息到父组件</button>
    `,
    methods: {
      sendMessage() {
        // 触发自定义事件，并传递数据
        this.$emit('custom-event', 'Hello from Child!');
      }
    }
  };

  // 创建Vue应用实例
  createApp({
    components: {
      'child-component': ChildComponent
    },
    data() {
      return {
        message: '' // 用于存储从子组件接收到的消息
      };
    },
    methods: {
      handleCustomEvent(payload) {
        // 接收子组件传递的数据
        this.message = payload;
      }
    }
  }).mount('#app');
</script>
</body>
</html>
```

综上代码所述，呈现如下效果，如图1-6所示。

在上述代码中，子组件通过this.$emit(‘custom-event’，‘Hello from Child!’)触发一个名为



图1-6 自定义事件案例

custom-event的自定义事件，并传递字符串’Hello from Child!’作为数据。父组件通过@custom-event=”handleCustomEvent”监听子组件触发的custom-event事件。

2.编译作用域

在Vue中，编译作用域是指模板中表达式的解析范围。Vue遵循“以组件为作用域”的原则，即模板中的表达式只能访问当前组件的实例属性。这意味着父组件模板中的表达式无法直接访问子组件的属性和方法，反之亦然。这种作用域规则确保了组件之间的解耦和独立性。编译作用域的规则：

（1）父组件模板：只能访问父组件的实例属性（data、methods、computed等）。

（2）子组件模板：只能访问子组件的实例属性。

（3）插值表达式：在父组件模板中使用子组件时，插值表达式（如{{ }}）不会解析子组件的属性。

【例1-33】创建Vue 3项目，实现编译作用域的规则。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vue 3 编译作用域示例</title>
  <script src="https://unpkg.com/vue@3"></script>
</head>
<body>
  <div id="app">
    <!-- 父组件模板 -->
    <p>父组件的数据: {{ parentData }}</p>
    <child-component></child-component>
  </div>

  <script>
    const { createApp } = Vue;

    // 定义子组件
```

```
    const ChildComponent = {
      template: `
        <div>
          <p>子组件的数据: {{ childData }}</p>
          <p>父组件的数据: {{ parentData }} <!-- 这里会报错，因为子组件无法访问父组件的数据 --></p>
        </div>
      `,
      data() {
        return {
          childData: '这是子组件的数据'
        };
      }
    };

    // 创建 Vue 应用实例
    createApp({
      components: {
        'child-component': ChildComponent
      },
      data() {
        return {
          parentData: '这是父组件的数据'
        };
      }
    }).mount('#app');
  </script>
</body>
</html>
```

综上代码所述，呈现如下效果，如图 1-7 所示。

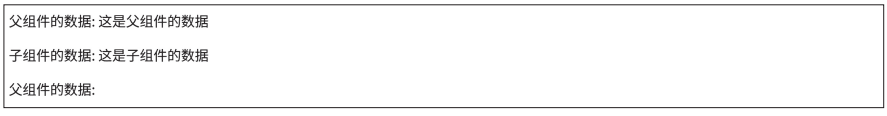


图 1-7　编译作用域规则案例

在上述代码中，父组件可以访问自己的data属性parentData，并将其渲染到模板中。子组件可以访问自己的data属性childData，子组件模板中尝试访问父组件的parentData时会报错，因为子组件无法直接访问父组件的作用域。在父组件模板中使用子组件时，插值表达式（如{{parentData}}）只能解析父组件的作用域，而无法解析子组件的作用域。

3.作用域插槽

作用域插槽是Vue中一种强大的功能，允许父组件在子组件的插槽中访问子组件的数据。与普通插槽不同，作用域插槽允许子组件将数据传递给父组件，父组件可以基于这些数据动态渲染内容。这种方式使得父组件能够更灵活地控制子组件的显示，同时保持子组件的封装性。作用域插槽的基本用法：

（1）子组件定义插槽：子组件通过<slot>标签定义插槽，并使用v-bind将数据传递给父组件。

（2）父组件使用插槽：父组件通过v-slot指令（或缩写#）接收子组件传递的数据，并在插槽中使用这些数据。

【例1-34】通过作用域插槽，我们能够在父组件中定义一个插槽模板，并将子组件传递的数据绑定至该模板内。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vue 3 作用域插槽示例</title>
  <script src="https://unpkg.com/vue@3"></script>
</head>
<body>
  <div id="app">
    <!-- 父组件使用作用域插槽 -->
    <child-component>
      <template v-slot:default="slotProps">
        <p>父组件接收到的数据：{{ slotProps.item.name }}</p>
      </template>
    </child-component>
  </div>

  <script>
```

```
    const { createApp } = Vue;

    // 定义子组件
    const ChildComponent = {
      template: `
        <div>
          <slot :item="item"></slot> <!-- 子组件通过插槽传递数据 -->
        </div>
      `,
      data() {
        return {
          item: { name: 'Vue 3', version: 3 }
        };
      }
    };

    // 创建Vue应用实例
    createApp({
      components: {
        'child-component': ChildComponent
      }
    }).mount('#app');
  </script>
</body>
</html>
```

综上代码所述，呈现如下效果，如图1-8所示。

父组件接收到的数据: Vue 3

图1-8 作用域插槽案例

在上述代码中，子组件通过<slot:item="item"></slot>定义了一个插槽，并使用v-bind将item数据传递给父组件。这里item是一个包含name和version的对象。父组件通过<template v-slot:default="slotProps">接收子组件传递的数据。slotProps是一个对象，包含了子组件传递的所有数据。

1.6.5 动态组件

1.基本用法

Vue的动态组件通过<component>标签和 :is属性实现，允许在同一个挂载点动态切换不同的组件。动态组件常用于实现标签页、步骤条或根据条件渲染不同组件的场景。使用动态组件可以提高代码的复用性和灵活性。动态组件的基本用法：

（1）使用<component>标签：通过<component>标签和 :is属性动态绑定组件。

（2）切换组件：通过改变 :is绑定的值来切换显示的组件。

（3）保持状态：使用keep-alive包裹动态组件，可以缓存组件状态，避免重复渲染。

【例1-35】构建一个基于Vue 3的动态组件应用，通过按钮切换展示不同组件，展示Vue 3的动态组件功能。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vue 3 动态组件示例</title>
  <script src="https://unpkg.com/vue@3"></script>
</head>
<body>
  <div id="app">
    <!-- 动态组件 -->
    <component :is="currentComponent"></component>

    <!-- 切换组件的按钮 -->
    <button @click="currentComponent = 'ComponentA'">显示组件 A</button>
    <button @click="currentComponent = 'ComponentB'">显示组件 B</button>
  </div>

  <script>
    const { createApp } = Vue;

    // 定义组件 A
```

```
    const ComponentA = {
      template: `<div style="background-color: lightblue; padding: 10px;">
        <h3>组件 A</h3>
        <p>这是组件 A 的内容。</p>
      </div>`
    };

    // 定义组件 B
    const ComponentB = {
      template: `<div style="background-color: lightgreen; padding: 10px;">
        <h3>组件 B</h3>
        <p>这是组件 B 的内容。</p>
      </div>`
    };

    // 创建 Vue 应用实例
    createApp({
      components: {
        ComponentA,
        ComponentB
      },
      data() {
        return {
          currentComponent: 'ComponentA' // 默认显示组件 A
        };
      }
    }).mount('#app');
  </script>
</body>
</html>
```

在上述代码中，定义了两个组件 ComponentA 和 ComponentB，每个组件都有自己的模板。使用 <component :is=”currentComponent”></component> 动态绑定组件，通过改变 currentComponent

的值来切换显示的组件。

2.keep alive

keep-alive是Vue中用于缓存动态组件的内置组件。它允许动态组件在切换时保持状态，避免重复渲染，从而提高性能和用户体验。keep-alive可以缓存单个组件或多个组件的状态，并且提供了生命周期钩子来管理缓存行为。keep-alive的基本用法：

（1）包裹动态组件：使用<keep-alive>包裹<component>标签，以缓存动态组件的状态。

（2）生命周期钩子：keep-alive提供了activated和deactivated生命周期钩子，分别在组件被激活和停用时触发。

（3）缓存策略：可以通过include和exclude属性控制哪些组件需要被缓存。

【例1-36】在Vue组件中，使用<keep-alive>标签包裹动态组件，当组件被切换时，其状态会被保留，而不是被销毁。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vue 3 `keep-alive` 示例</title>
  <script src="https://unpkg.com/vue@3"></script>
</head>
<body>
  <div id="app">
    <!-- 使用 keep-alive 包裹动态组件 -->
    <keep-alive>
      <component :is="currentComponent"></component>
    </keep-alive>

    <!-- 切换组件的按钮 -->
    <button @click="currentComponent = 'ComponentA'">显示组件 A</button>
    <button @click="currentComponent = 'ComponentB'">显示组件 B</button>
  </div>

  <script>
```

```
const { createApp } = Vue;

// 定义组件A
const ComponentA = {
  template: `<div style="background-color: lightblue; padding: 10px;">
    <h3>组件A</h3>
    <p>这是组件A的内容。</p>
  </div>`,
  activated() {
    console.log('ComponentA activated');
  },
  deactivated() {
    console.log('ComponentA deactivated');
  }
};

// 定义组件B
const ComponentB = {
  template: `<div style="background-color: lightgreen; padding: 10px;">
    <h3>组件B</h3>
    <p>这是组件B的内容。</p>
  </div>`,
  activated() {
    console.log('ComponentB activated');
  },
  deactivated() {
    console.log('ComponentB deactivated');
  }
};

// 创建Vue应用实例
createApp({
```

```
    components: {
      ComponentA,
      ComponentB
    },
    data() {
      return {
        currentComponent: 'ComponentA' // 默认显示组件 A
      };
    }
  }).mount('#app');
 </script>
</body>
</html>
```

在上述代码中：使用keep-alive包裹动态组件，这样动态切换的组件状态会被缓存。在组件A和组件B中定义了activated和deactivated生命周期钩子，分别在组件被激活和停用时触发。这些钩子可以帮助管理组件的缓存行为。

3.activated钩子函数

activated是Vue中用于动态组件的一个生命周期钩子函数，当使用<keep-alive>包裹动态组件时，该钩子会在组件被激活时触发。这使得开发者可以在组件被重新激活时执行特定的逻辑，例如重新加载数据或更新状态。activated钩子函数的基本用法：

（1）包裹动态组件：使用<keep-alive>包裹动态组件以启用缓存功能。

（2）定义activated钩子：在组件内部定义activated生命周期钩子函数。

（3）执行逻辑：在activated钩子中执行需要在组件激活时运行的逻辑。

【例1-37】展示如何在钩子中调用数据刷新方法。

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Vue 3 `activated` 钩子示例</title>
 <script src="https://unpkg.com/vue@3"></script>
</head>
```

```
<body>
  <div id="app">
    <!-- 使用 keep-alive 包裹动态组件 -->
    <keep-alive>
      <component :is="currentComponent"></component>
    </keep-alive>

    <!-- 切换组件的按钮 -->
    <button @click="currentComponent = 'ComponentA'">显示组件 A</button>
    <button @click="currentComponent = 'ComponentB'">显示组件 B</button>
  </div>

  <script>
    const { createApp } = Vue;

    // 定义组件 A
    const ComponentA = {
      template: `<div style="background-color: lightblue; padding: 10px;">
        <h3>组件 A</h3>
        <p>这是组件 A 的内容。</p>
      </div>`,
      activated() {
        console.log('组件 A 被激活');
        // 在组件被激活时执行逻辑，例如重新加载数据
      }
    };

    // 定义组件 B
    const ComponentB = {
      template: `<div style="background-color: lightgreen; padding: 10px;">
        <h3>组件 B</h3>
        <p>这是组件 B 的内容。</p>
```

```
    </div>`,
    activated() {
      console.log('组件B被激活');
      // 在组件被激活时执行逻辑
    }
  };

  // 创建Vue应用实例
  createApp({
    components: {
      ComponentA,
      ComponentB
    },
    data() {
      return {
        currentComponent: 'ComponentA' // 默认显示组件A
      };
    }
  }).mount('#app');
 </script>
</body>
</html>
```

综上代码所述，呈现如下效果，如图1-9所示。



图1-9 钩子函数案例

在上述代码中，使用<keep-alive>包裹<component :is=”currentComponent”/>，这样动态切换的组件状态会被缓存，在组件A和组件B中定义了activated生命周期钩子函数。

4.异步组件

Vue支持异步组件的定义，允许组件在需要时才加载，从而提高应用的性能和用户体验。异步组件通过返回一个Promise来实现，Vue会在组件真正需要渲染时才加载对应的资源。这种方式特别适合大型应

用，可以减少初始加载时间并按需加载组件。异步组件的基本用法：

（1）定义异步组件：通过返回一个Promise来定义异步组件。

（2）使用动态组件：通过<component>标签和 :is属性动态切换异步组件。

（3）加载状态处理：可以使用loading和error属性来处理加载状态和错误状态。

【例1-38】创建一个Vue 3应用，演示同步和异步组件的加载方式，通过按钮切换展示不同加载机制的效果对比。

```
<!DOCTYPE html>
<html>
<head>
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
<body>
  <div id="app">
    <button @click="currentComponent = 'ComponentA'">加载组件 A</button>
    <button @click="currentComponent = 'ComponentB'">异步加载组件 B</button>
    <component :is="currentComponent"></component>
  </div>

  <script>
    const { createApp, defineAsyncComponent } = Vue;

    // 同步组件
    const ComponentA = {
      template: '<div>同步加载的组件 A</div>'
    };

    // 异步组件
    const ComponentB = defineAsyncComponent(() => {
      return new Promise(resolve => {
        setTimeout(() => {
          resolve({
            template: '<div>延迟 1 秒加载的组件 B</div>'
```

```
        });
      }, 1000);
    });
  });

  createApp({
    components: { ComponentA, ComponentB },
    data() {
      return {
        currentComponent: 'ComponentA'
      }
    }
  }).mount('#app');
 </script>
</body>
</html>
```

综上代码所述，呈现如下效果，如图1-10所示。



图1-10 异步组件的案例

在上述代码中，异步组件通过返回一个对象来定义，使用<component :is=”currentComponent”/>动态切换异步组件，通过按钮点击事件改变currentComponent的值来切换显示的组件。

本章小结

本章首先介绍了Vue的基本概念，接着讲解了集成开发环境VS Code的安装与使用方法，最后通过实际案例详细介绍了Vue中的常用概念及其具体应用方式，帮助读者从理论到实践初步掌握Vue相关知识和开发工具的使用。

练习与实践

简答题

（1）什么是Vue的渐进式框架特征？

（2）简述Vue的响应式系统如何提升渲染性能。

（3）列举并说明Vue中常用的4条内置指令及其作用。

实践题

（1）在1.7.3.1相对时间转换案例基础上，为页面添加以下美化要求：

①使用Flex布局让时间块水平垂直居中；

②添加渐变背景与卡片阴影；

③点击“更新时间”按钮后，出现200ms的透明度过渡动画。

（2）新建一个Vue3单文件组件TodoList.vue，要求：

①通过v-model实现输入框双向绑定；

②用v-for渲染待办事项数组，每项提供“完成/删除”按钮；

③使用自定义指令v-focus让输入框在组件挂载后自动获得焦点；

④数据变化后利用watch将最新列表持久化到localStorage。



服装智能系统软件开发项目教程

基于 Vue +Spring Boot

第 2 章

前端工程化



本章导读

Vue作为一款在前端开发领域广泛应用且备受欢迎的前端框架，它具有简洁易用、响应式设计以及组件化开发等诸多优点。而与之相关的工程化工具和流程能够帮助开发者更好地利用Vue框架的优势，提高开发效率和代码质量。本章将围绕Vue cli、项目打包与发布以及Vue Devtools这几个重要方面展开详细的介绍。通过对这些内容的深入学习，开发者能够更加熟练地运用Vue进行项目开发。

学习目标

1.熟练安装和配置Node.js与NPM环境，理解其在前端工程化中的核心作用，并能解决常见的安装与配置问题。

2.熟练运用Vue cli创建、开发和管理Vue项目，能够根据不同项目需求灵活选择合适的配置选项，搭建高效的项目架构。

3.熟练使用静态服务器工具包（如serve）和动态Web服务器（Tomcat）对Vue项目进行发布，掌握不同发布方式的适用场景及操作细节，确保项目成功上线。

4.熟练掌握Vue Devtools的安装和使用，利用其丰富的调试功能快速定位和解决Vue应用开发中的问题，提升代码质量。

思政目标及设计建议

以新时代软件工程师素养为基础，将课程思政元素自然融入前端工程化教学。本章思政目标及设计建议如表2-1所示。

表2-1　思政目标及设计建议

思政目标	思政元素及融入
培养严谨的职业道德和规范正确的职业操守	在项目开发过程中，强调代码规范和注释的重要性，培养学生严谨、负责的职业道德
增强文化自信	选取具有社会意义的项目案例，如开发一个宣传传统文化的网站、助力社区服务的应用等
培养团队协作能力和社会责任感	组织学生进行小组讨论，探讨前端技术在社会发展中的作用，以及如何利用前端技术解决社会问题

2.1 Vue cli



Vue cli是Vue官方专门提供的、用于快速构建Vue.js项目的一款非常实用的脚手架工具。它在实际的项目开发中具有重要的作用，尤其适用于搭建大型的单页面应用。借助Vue cli工具所提供的开箱即用的构建工具配置，开发者无需进行复杂的手动配置。只需要花费几分钟的时间，就能够快速地初始化一个包含了最佳实践和合理项目结构的Vue.js项目。而且，该工具不仅可以构建单页面应用，还能够构建多页面应用，为开发者提供了更多的项目构建选择。

单页面应用(Single Page Web Application，SPA)：单页面应用指的是只有一个Web页面的应用程序。如图2-1所展示的那样，它是一种从Web服务器加载的客户端应用。在单页面应用中，当进行页面跳转时，仅仅会刷新局部的资源。这意味着公共资源，例如js文件、css文件等，只需要加载一次。这样的设计可以大大提高页面的加载速度和用户体验。

多页面应用(Multi-Page Application,MPA)：在多页面应用中，当进行页面跳转时，会刷新所有的资源。对于每个公共资源，比如js、css等，都需要根据具体情况进行选择性地重新加载。如图2-2所呈现的那样。需要注意的是，在实际操作中，某些功能可能需要在特定的当前目录下才可以使用。

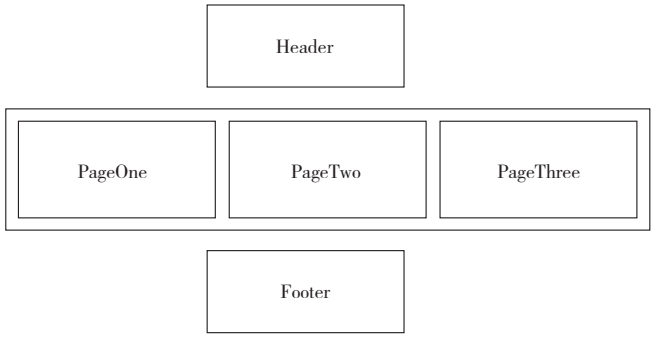


图2-1 单页面应用

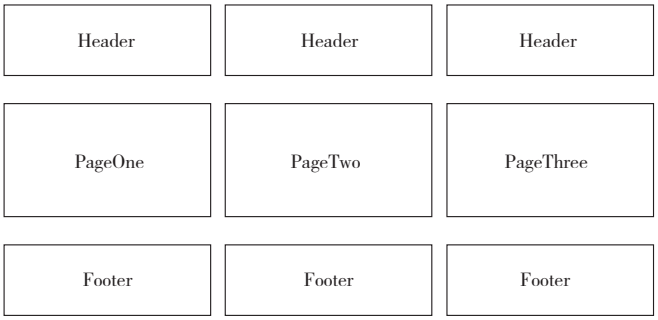


图2-2 多页面应用

单页面和多页面应用详细对比分析，如表2-2所示。

表2-2 单页面应用和多页面应用对比

	单页面应用	多页面应用
组成	一个外壳页面和多个页面片段	多个完整页面
资源共享	共用，只需在外壳部分加载	不共用，每个页面都需要加载
刷新方式	页面局部刷新或更改	整页刷新
URL模式	a.com/#/pageone a.com/#/pagetwo	a.com/pageone.html a.com/pagetwo.html
用户体验	页面片段间的切换快，用户体验良好	页面切换加载缓慢，流畅度不够，用户体验差
搜索引擎优化	页面片段间的切换快，用户体验良好	可利用服务器端渲染(SSR)优化，实现方法简易
维护成本	相对容易	相对复杂
开发成本	较高，常需借助专业框架	较低，但页面重复代码多

2.1.1 Node.js

Node.js是一款基于Chrome V8引擎所构建的JavaScript运行环境。它能够允许JavaScript代码在服务器端进行稳定且高效地运行。通过这种方式，成功地实现了前后端技术的统一，使得前后端的开发在技术层面上更加协调和一致。在前端工程化的领域当中，Node.js常常被用于项目构建、依赖管理以及开发工具链的搭建等重要工作。项目构建方面，Node.js可以帮助开发者更高效地组织和优化代码结构；在依赖管理上，能够精准地管理项目所依赖的各种库和模块；而在开发工具链的搭建中，也发挥着不可或缺的作用，为前端开发提供了强大的支持，详细安装步骤可参考Vue教程。

2.1.2 NPM

NPM（Node Package Manager）是Node.js的默认包管理器，用于安装、管理和共享Node.js模块。它是Vue cli生态系统中不可或缺的一部分，用于管理项目的依赖关系。在安装Node.js时，NPM会被一同安装。验证NPM是否安装成功的方式和Node.js验证方式一样，在命令行输入npm-v并正确查看其版本号，说明安装成功，如图2-3所示。



图2-3 验证npm是否安装成功

2.1.3 基本使用

使用Vue cli创建项目非常简单。可以使用Vue-cli脚手架快速搭建单页面应用，步骤如下：

1.创建一个新的Vue项目，可以使用以下命令：

```
vue create my-project
```

my-project指的是项目名称。执行此命令后，Vue cli会提供一系列的选项，如选择Vue版本、是否安装路由、状态管理等功能。根据项目需求进行选择后，Vue cli会自动创建项目目录结构，并安装项目所需的依赖。

例如：Vue创建项目的交互式界面步骤如下：

（1）选择Vue版本，可以根据需求自行选择vue2或vue3，如图2-4所示。

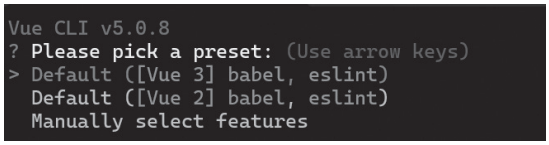


图2-4 Vue版本

（2）出现如图2-5所示界面，说明my-project项目创建成功。

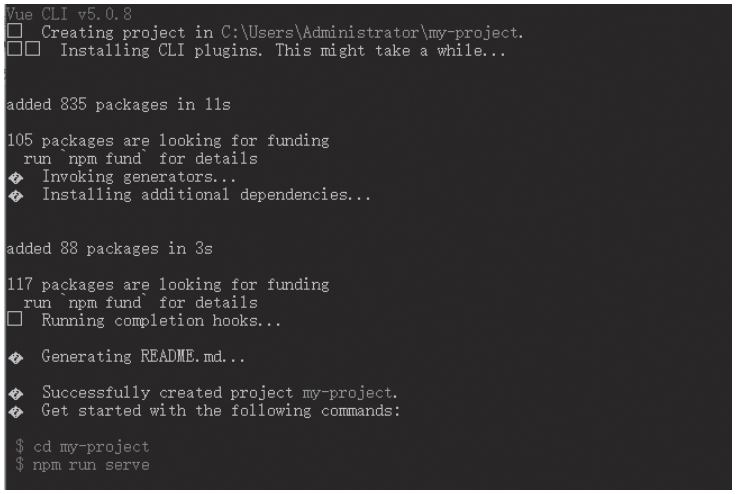


图2-5 my-project项目创建

（3）与此同时，本地路径会生成对应的my-project项目文件夹，如图2-6所示。



图2-6 本地路径下创建my-project项目

2.项目创建和依赖安装完成后，进入项目目录：

```
cd my-project
```

3.启动开发服务器。

```
npm run serve
```

执行此命令，将启动一个本地开发服务器，如图2-7所示。当出现“Local: http://localhost:8080.提示时，说明开发服务器启动成功。根据提示可以将地址复制到浏览器中以打开项目首页，如图2-8所示。

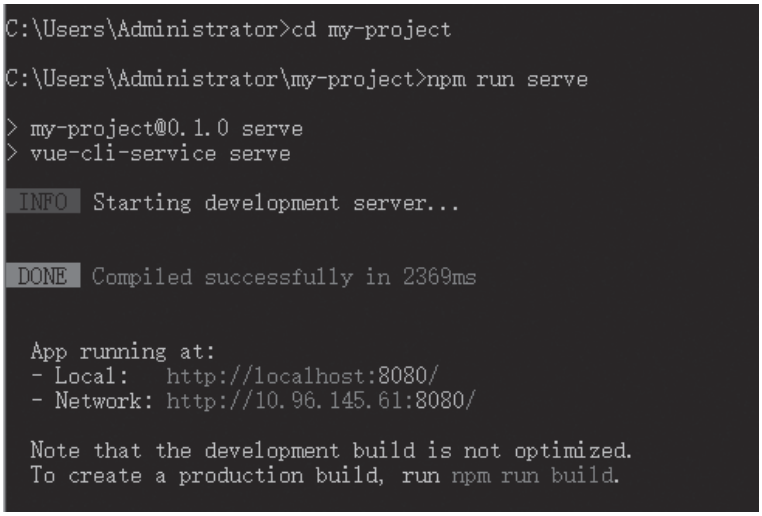


图2-7　启动开发服务器

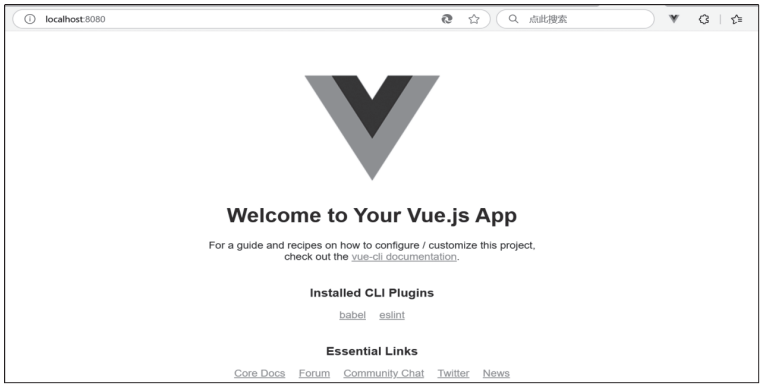


图2-8　项目启动界面

2.2 项目打包与发布



在前端工程化中，项目打包是将项目中的所有资源（如HTML、CSS、JavaScript、图片等）合并、

压缩并优化成一个或多个文件的过程。本节将介绍如何使用Vue cli提供的打包命令对项目进行打包，并探讨两种常见的发布方式：使用静态服务器工具包发布打包和使用动态Web服务器（如Tomcat）发布打包。

2.2.1 静态服务器

静态服务器工具包（如Nginx、Apache等）是一种轻量级的Web服务器，它主要用于提供静态文件（如HTML、CSS、JavaScript等）的访问服务。以serve为例，它是一个简单易用的静态服务器工具。

首先，在项目目录下执行打包命令，如图2-9所示：

```
npm run build
```

执行此命令后，项目目录下会生成一个dist文件夹，其中包含了打包后的静态文件，如图2-10所示。

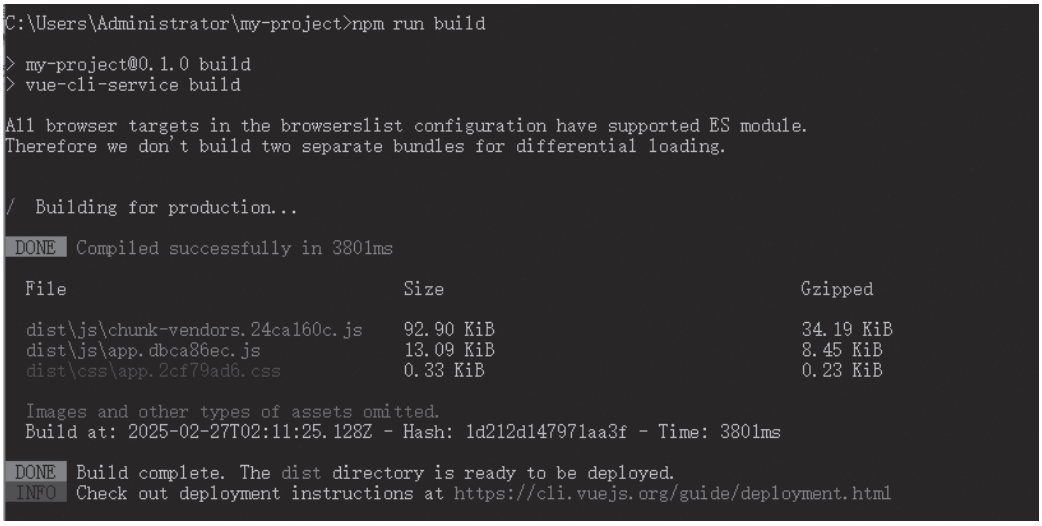


图2-9 打包my-project项目

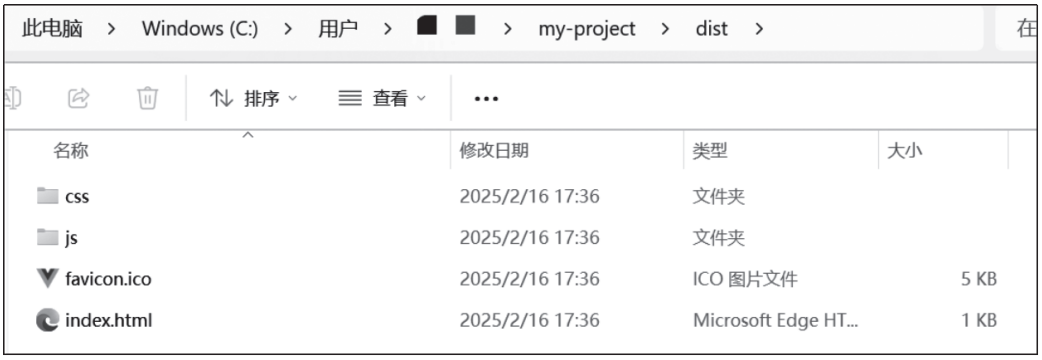


图2-10 打包成功后生成的文件

接下来，全局安装serve，如图2-11所示：

```
npm install -g serve
```

```
C:\Users\Administrator\my-project>npm install -g serve

added 88 packages in 2s

24 packages are looking for funding
  run `npm fund` for details
```

图2-11 全局安装serve

安装完成后，在dist目录下执行以下命令启动静态服务器，如图2-12所示。

```
serve
```

默认情况下，serve会在http://localhost:8080启动服务器，用户可以通过该地址访问项目。如果需要指定端口，可以使用-l参数，例如serve-l 8081会在http://localhost:8081启动服务器，如图2-13所示。

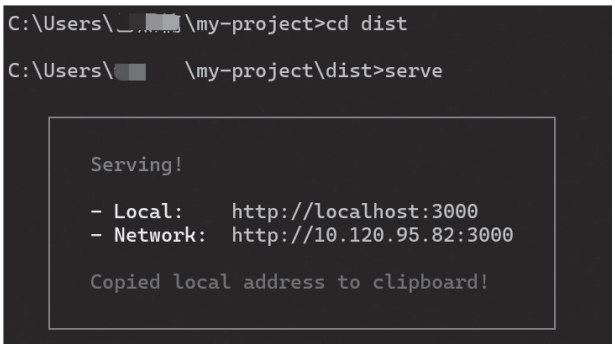


图2-12 启动静态服务器

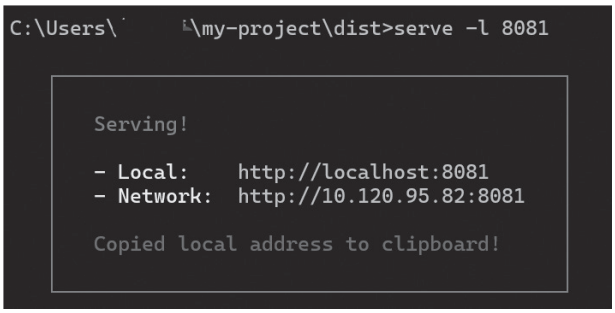


图2-13 指定端口

这种方式适用于纯静态项目，不需要服务器端进行复杂的动态处理。

2.2.2 动态服务器

Tomcat是一个广泛使用的开源Java Servlet容器，可以作为动态Web服务器使用。如果项目中存在与后端交互等动态功能，可能需要使用Tomcat来发布项目。

首先，同样需要在项目目录下执行打包命令：

```
npm run build
```

生成 dist 文件夹。然后，将 dist 文件夹中的所有文件复制到 Tomcat 的 webapps 目录下，如图 2-14 所示。假设 Tomcat 的安装目录为 /opt/tomcat，则将文件复制到 /opt/tomcat/webapps/myapp（myapp 为自定义的应用名称）。

启动 Tomcat 服务器，在浏览器中输入 http://localhost:8080/myapp/（假设 Tomcat 默认端口为 8080），即可访问项目，如图 2-15 所示。



图 2-14　webapps 目录下



图 2-15　Tomcat 方式访问项目

在使用 Tomcat 发布时，需要确保服务器上已经安装并正确配置了 Tomcat，并且项目中的动态请求能够正确地与后端进行交互。

2.3 Vue Devtools



在开发 Vue 项目时，常常需要调试程序。调试环境推荐使用 Chrome 浏览器或者火狐（Firefox）浏览器配合 Vue Devtools 插件。

Vue Devtools 是 Vue.js 官方提供的一款浏览器开发者工具插件，用于调试 Vue.js 应用程序。它可以帮助开发者查看组件层次结构、状态、事件、性能等信息，从而更高效地开发和调试 Vue 应用。本节将介绍如何安装和使用 Vue Devtools，以及如何利用它来提高开发效率和代码质量。

2.3.1 Vue Devtools 使用

Vue Devtools 是 Vue.js 官方提供的浏览器开发者工具插件，专为 Vue.js 应用调试而设计。它能够帮助开发者深入理解应用的运行机制，快速定位问题。通过 Vue Devtools，开发者可以直观地查看组件的层次结构、状态（如 data、props、computed）、事件触发记录以及 Vuex 状态管理的变化。此外，它还提供了性能分析工具，帮助优化应用的渲染性能。无论是调试组件逻辑、追踪事件流，还是分析全局状态，Vue Devtools 都能显著提升开发效率。接下来，将通过具体案例，详细介绍如何安装和使用 Vue

Devtools工具。

首先，启动Vue Devtools。打开火狐浏览器，访问Vue项目。按下F12或Ctrl+Shift+I打开开发者工具，如图2-16所示。



图2-16 启动Vue Devtools

其次，使用Vue Devtools进行测试，Vue Devtools面板包含以下几个部分：

1.组件树(Compoents Tree)：在Vue Devtools界面中，可以看到应用的组件树结构。通过点击组件节点，可以查看每个组件的属性、数据、方法等信息。可以检查组件是否正确渲染，以及组件之间的嵌套关系是否符合预期。如图2-17所示。



图2-17 Compoents功能

2.状态观察（Vuex Tab）：如果应用使用了Vuex管理状态，在Vue Devtools中可以方便地观察Vuex的状态变化。可以查看各个模块的状态值，以及状态是如何随着用户操作或其他事件而改变的，有助于调试与状态相关的问题。如图2-18所示。



图2-18 Vuex功能

3.Timeline功能，聚焦于记录与分析Vue应用运行事件。它记录组件生命周期，帮助开发者判断组件运行状态；记录各类事件触发，可查看时间与相关组件。在性能分析上，展示组件渲染时间，方便找出性能瓶颈组件并优化。还能跟踪响应式数据变化，明晰数据对组件更新的影响。此外，支持事件筛选与搜索，便于开发者快速定位问题，提升调试效率。如图2-19所示。

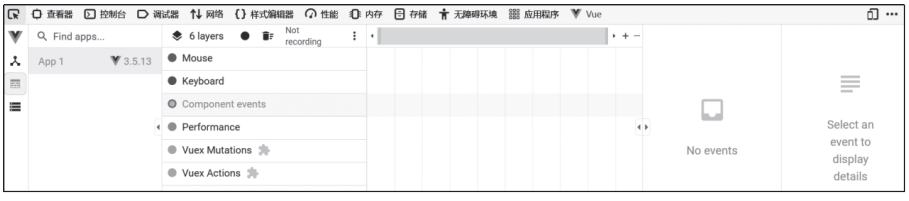


图2-19 Timeline功能

2.3.2 Vue Devtools案例

在这个案例中，将创建一个简单的登录界面，并使用Vue Devtools调试组件的状态、事件和Vuex状态管理。以下是详细的项目源码和使用方法。

1.执行“vue create login-app”命令创建login-app项目，并使用“cd login-app”进入项目目录，如图2-20所示。

```
C:\Users\Administrator>vue create login-app


Vue CLI v5.0.8
? Please pick a preset: Default ([Vue 3] babel, eslint)


Vue CLI v5.0.8
✨  Creating project in C:\Users\Administrator\login-app.
⚙  Installing CLI plugins. This might take a while...


added 835 packages in 25s

105 packages are looking for funding
  run `npm fund` for details
Invoking generators...
Installing additional dependencies...


added 88 packages in 5s

117 packages are looking for funding
  run `npm fund` for details
⚓  Running completion hooks...

Generating README.md...

Successfully created project login-app.
Get started with the following commands:

 $ cd login-app
 $ npm run serve


C:\Users\Administrator>cd login-app
```

图2-20 创建login-demo项目

2.login-app项目目录结构如下：

```
login-app/
├── public/
│   └── index.html
├── src/
│   ├── assets/
│   ├── components/
│   │   └── LoginForm.vue
```

```
|   ├── store/
|   |   └── index.js
|   ├── App.vue
|   └── main.js
├── package.json
└── vue.config.js
```

3.修改main.js-入口文件

```
import { createApp } from 'vue';
import App from './App.vue';
import store from './store';
const app = createApp(App);// 创建应用实例
app.use(store);// 使用 Vuex store
app.mount('#app');// 挂载应用
```

4.创建store/index.js文件，使用Vuex进行状态管理。

```
import { createApp } from "vue";
import Vuex from "vuex";
createApp().use(Vuex);
export default new Vuex.Store({
  state: {
    user: null,
    isLoggedIn: false,
  },
  mutations: {
    login(state, user) {
      state.user = user;
      state.isLoggedIn = true;
    },
    logout(state) {
      state.user = null;
      state.isLoggedIn = false;
    },
  },
```

```
  actions: {
    login({ commit }, user) {
      commit("login", user);
    },
    logout({ commit }) {
      commit("logout");
    },
  },
});
```

5.编写登录界面代码，在src/components目录下创建LoginForm.vue文件，代码如下：

```
<template>
  <div>
    <form @submit.prevent="handleSubmit">
      <div>
        <label for="username">用户名:</label>
        <input id="username" v-model="username" type="text" required />
      </div>
      <div>
        <label for="password">密码:</label>
        <input id="password" v-model="password" type="password" required />
      </div>
      <button type="submit">登录</button>
    </form>
  </div>
</template>
<script>
export default {
  data() {
    return {
      username: "",
      password: "",
    };
```

```
  },
  methods: {
    handleSubmit() {
      const user = { name: this.username };
      this.$store.dispatch("login", user);
     // 触发自定义事件
     // this.$emit('formSubmitted', user);
     console.log(' 表单提交了 ');
      this.$emit('submit', { username: this.username, password: this.password });
    },
  },
};
</script>
```

6. 在 App.vue 中使用 LoginForm 组件

```
<template>
  <div id="app">
    <h1>登录界面案例</h1>
    <LoginForm v-if="!isLoggedIn" />
    <div v-else>
      <p>欢迎回来, {{ user.name }}!</p>
      <button @click="logout">退出登录 </button>
    </div>
  </div>
</template>
<script>
import LoginForm from "./components/LoginForm.vue";
export default {
  components: {
    LoginForm,
  },
  computed: {
    isLoggedIn() {
```

```
      return this.$store.state.isLoggedIn;
    },
    user() {
      return this.$store.state.user;
    },
  },
  methods: {
    logout() {
      this.$store.dispatch("logout");
    },
  },
};
</script>
```

7.Vue Devtools的具体使用方法

（1）在项目根目录下运行以下命令启动开发服务器。

```
npm run serve
```

（2）在浏览器中打开http://localhost:8080即可看到登录界面，如图2-21所示。



图2-21 启动login-app项目

（3）打开Vue DevTools工具：以火狐浏览器为例，在浏览器中打开登录界面后，右键点击页面，选择“检查”（或使用快捷键Ctrl+Shift+I）打开开发者工具，在开发者工具面板中找到“Vue”标签，如图2-22所示。



图2-22 打开Vue DevTools工具

本章小结

本章探讨Vue前端工程化实践，从开发环境搭建到项目部署上线，解析提升开发效率与质量的技术链。开发环境配置中，Node.js与NPM构成Vue项目基础支撑，前者提供运行能力，后者管理依赖包与驱动构建流程。为解决NPM官方镜像下载慢问题，可配置淘宝NPM镜像提升获取效率。

项目工程化构建时，Vue CLI提供标准化初始化方案，开发者可选择或自定义项目特性生成项目骨架。项目创建后，用npm run serve启动开发服务器，其热重载机制可实时更新页面，缩短开发周期。

项目部署与发布阶段，针对不同应用场景提供方案：纯静态Vue项目用npm install -g serve安装服务器工具，执行serve dist命令快速部署；动态项目需将静态资源整合至Java后端项目Web目录，通过后端服务器处理API请求，确保数据交互安全稳定。

调试与性能优化工具方面，Vue Devtools是重要调试工具，功能覆盖组件可视化、状态追踪、事件监听、性能分析等场景。在实际开发中，可通过它定位表单验证错误、追踪异步请求状态、分析页面加载瓶颈，保障项目全流程高效推进与质量可控。

练习与实践

简答题

（1）前端工程化的定义是什么？它主要解决哪些开发痛点？

（2）Node.js在Vue前端工程化链条中承担了哪些核心职责？

（3）NPM中“全局安装”与“局部安装”的区别是什么？分别给出一条典型命令。

实践题

（1）使用Element-Plus或Ant Design Vue替换原生控件；

（2）添加“全部标记完成”按钮与“清除已完成”按钮；

（3）为不同状态（全部/未完成/已完成）切换时加入过渡动画；

（4）将最终dist包通过serve发布，截图提交运行效果。

第 3 章

UI 组件库基础应用



本章导读

在当今开发领域中，Element UI、Element Plus、Vue Router、Axios和Vuex共同构成了Vue.js技术栈中一套完整且高效的企业级前端解决方案，其中，Element UI及其基于Vue 3的升级版Element Plus提供了丰富、美观且易于使用的UI组件库，是构建用户界面的基石，为开发者节省了大量基础组件的开发成本，确保了视觉和交互的一致性，本章将围绕UI组件库。

学习目标

1.能够熟练使用Element UI或Element Plus提供的常用组件，

2.快速搭建美观且符合规范的用户界面，

3.理解两者基于Vue 2与Vue 3的不同技术背景及主要API差异，能在项目中正确选用和迁移。

思政目标及设计建议

将UI技术学习与服装类专业相结合，并融入思政教育，是一个非常有创意且符合“课程思政”理念的方向，本章思政目标及设计建议如表3-1所示。

表3-1　第3章思政目标及设计建议

思政目标	思政元素及融入
培养文化自信与民族自豪感	项目开发前的调研阶段，要求学生深入研究一项中国服饰文化（如汉服形制、苏绣技艺），理解其历史、工艺和美学价值。项目因中国强大的互联网技术而成功，展示内容为博大精深的中国文化，以此激发学生的双重自豪感
锤炼严谨规范、精益求精的工匠精神	强调代码规范性如同服装的“行业标准”与“尺寸号型”，随意破坏规则会导致系统（生产线）混乱，应引导学生像对待高级定制服装般打磨代码细节
强化团队协作与集体主义精神	引导学生反思，复杂系统（如成衣制作）需设计师、版师、工艺师协同完成，个人代码只是整体一部分，须遵循共同接口规范（如工艺单），才能集成成功产品



3.1 Element Plus概述

Element Plus是一套面向开发者、设计师和产品经理推出的，基于Vue 3的桌面端组件库。它提供了丰富多样、品质优良且可复用的组件，助力开发者迅速搭建企业级中后台产品。其前身是在Vue 2生态中广受欢迎的组件库Element UI。随着Vue 3的正式发布，Element Plus作为官方升级版本，完全兼容Vue 3的Composition API，并且具备更出色的性能、更多样的特性以及更具现代感的设计。

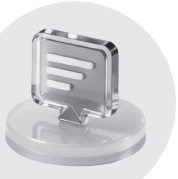
Element Plus的主要特点在于它具备多个显著的特性，这些特性使得它在众多前端框架中脱颖而出：

1.简洁美观：Element-Plus始终坚持简洁的设计理念，这一理念贯穿于其整个开发过程中。它不仅注重界面的简洁性，还提供了多样化的主题和样式定制功能。开发者可以根据自己的需求和喜好，轻松地对主题和样式进行调整和修改，以打造出独具特色的应用界面。

2.高效易用：凭借其丰富的组件库，Element-Plus为开发者提供了极大的便利。这些组件涵盖了各种常见的UI元素，如按钮、表单、菜单等。开发者无需从头开始编写代码，只需选择合适的组件并进行简单的配置，就能够迅速搭建出应用界面。这种方式显著提升了开发效率，节省了大量的时间和精力。

3.响应式布局：Element-Plus兼容响应式设计，这意味着它能够根据不同设备的屏幕尺寸自动调整布局。无论是在电脑、平板还是手机上，应用都能够无缝适配屏幕，为用户提供一致的视觉体验。这种兼容性确保了应用在各种设备上都能够正常显示和使用。

4.国际化支持：支持多语言切换是Element-Plus的一个重要特性。在全球化的背景下，越来越多的应用需要满足不同地区用户的需求。Element-Plus通过提供多语言支持，充分满足了国际化应用的需求。开发者可以轻松地实现应用的多语言切换功能，让用户能够以自己熟悉的语言使用应用。



3.2 Element Plus安装及引入

3.2.1 Element Plus安装

Element-Plus可通过npm或yarn进行安装，以下为具体安装步骤：

1.使用npm安装

```
npm install element-plus --save
```

2.使用yarn安装

```
yarn add element-plus
```

3.2.2 Element Plus引入

在Vue3项目中引入Element Plus时，可以通过多种方式引入样式，以下是常见的2种方法及其实现步骤：

1.全局完整引入

此方式将全面加载所有相关组件及其配套样式文件，适用于需要完整功能支持的项目，采用该方式，项目可获得全面的组件功能，并确保所有样式得到统一应用，进而满足项目在功能性与界面展示方面的高标准要求，对于追求功能全面性和界面一致性的项目而言，这种加载方式无疑是最优选择。

（1）创建第一个Element-Plus项目，以下为具体创建Element-Plus项目的步骤。

①创建一个新的Vue3项目。

```
D:\teachingMaterial\engineering\code>vue create jf-app
```

②进入当前目录。

```
D:\teachingMaterial\engineering\code>cd jf-app
```

③将Vue3项目安装在当前目录。

```
D:\teachingMaterial\engineering\code\jf-app>npm install element-plus
```

（2）将项目中的src/main.js文件进行引入Element-Plus

```
import ElementPlus from 'element-plus';
import 'element-plus/dist/index.css';
const app = createApp(App);
app.use(ElementPlus);
app.mount('#app');
```

2.按需引入

按需引入才是建议使用，在真实的应用场景中并不是每个组件都会用到的，这会造成不小的内存浪费。

（1）自动按需引入。

①安装unplugin-vue-components插件和unplugin-auto-import插件。

```
D:\teachingMaterial\engineering\code\jf_app>npm install -D unplugin-vue-components unplugin-auto-import
```

②修改vite.config.js文件。

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
```

```
import AutoImport from 'unplugin-auto-import/vite'
import Components from 'unplugin-vue-components/vite'
import { ElementPlusResolver } from 'unplugin-vue-components/resolvers'
// https://vitejs.dev/config/
export default defineConfig({
  plugins: [
        vue(),
        // ...
        AutoImport({
          resolvers: [ElementPlusResolver()],
        }),
        Components({
          resolvers: [ElementPlusResolver()],
        })
  ],
  //修改启动的
    server: {
      host: '0.0.0.0', // 这个用于启动
      port: '8080', // 指定启动端口
      open: true ,//启动后是否自动打开浏览器
        }
})
```

③启动服务器

```
D:\teachingMaterial\engineering\code\jf_app>npm run serve
```

3.3 基础组件



基础组件是构建界面不可或缺的元素，其中涵盖了多种实用且常用的控件。具体而言，基础组件包括Button按钮，这是一种用于触发特定操作的交互元素，用户可以通过点击按钮来执行各种命令或功能；Icon图标，它以简洁的图形形式呈现，用于直观地传达信息或指示操作，提升界面的美观性和易用性；以

及Link链接，这是一种用于跳转至其他页面或资源的文本元素，通过点击链接，用户可以方便地导航到不同的内容或功能模块。

3.3.1 Button按钮

1.按钮属性：

（1）type：使用type属性，该属性一共有5种类型。

（2）plain：使用Plain属性，代表是朴素按钮。

（3）round：使用round属性，代表是圆角按钮。

（4）circle：使用circle属性，代表是圆形按钮。

【例3-1】编写一个Vue3项目，实现默认Button按钮的显示，代码如下：

```
<template>
    <el-button>默认的按钮</el-button>
    <el-button type="primary">Primary</el-button>
    <el-button type="success">Success</el-button>
    <el-button type="info">Info</el-button>
    <el-button type="warning">Warning</el-button>
    <el-button type="danger">Danger</el-button>
</template>
```

综上代码所述，呈现如下效果，如图3-1所示。



图3-1 默认Button按钮显示

在上述代码里，编写了默认的按钮显示方式，采用的是type属性，该属性共有5种，分别为primary、success、info、warning和danger。

【例3-2】编写一个Vue3项目，实现朴素Button按钮的显示，代码如下：

```
<template>
    <div class="button-row">
    <el-button plain>朴素按钮</el-button>
    <el-button type="primary" plain>Primary</el-button>
    <el-button type="success" plain>Success</el-button>
    <el-button type="info" plain>Info</el-button>
    <el-button type="warning" plain>Warning</el-button>
```

```
    <el-button type="danger" plain>Danger</el-button>
</template>
```

综上代码所述，呈现如下效果，如图 3-2 所示。



图 3-2　朴素按钮显示

在上述代码中，编写了朴素按钮的显示方式，采用的是 plain 属性，与默认按钮不同之处是颜色和透明度不同。

【例 3-3】编写一个 Vue3 项目，实现圆角 Button 按钮的显示，代码如下：

```
<template>
    <el-button round>圆角按钮</el-button>
    <el-button type="primary" round>Primary 主要按钮</el-button>
    <el-button type="success" round>Success 成功按钮</el-button>
    <el-button type="info" round>Info 信息按钮</el-button>
    <el-button type="warning" round>Warning 警告按钮</el-button>
    <el-button type="danger" round>Danger 危险按钮</el-button>
</template>
```

综上代码所述，呈现如下效果，如图 3-3 所示。



图 3-3　圆角按钮显示

在上述代码中，编写了圆角按钮的显示方式，采用的是 round 属性圆角按钮呈现为圆角形态，默认按钮则为直角样式。

【例 3-4】编写一个 Vue3 项目，实现圆形 Button 按钮的显示，代码如下：

```
<template>
    <el-button circle>C</el-button>
    <el-button type="primary" circle>P</el-button>
    <el-button type="success" circle>S</el-button>
    <el-button type="info" circle>I</el-button>
    <el-button type="warning" circle>W</el-button>
    <el-button type="danger" circle>D</el-button>
</template>
```

综上代码所述，呈现如下效果，如图3-4所示。



图3-4 圆形按钮显示

在上述代码中，编写了圆形按钮的显示方式，采用的是circle属性。

2.按钮状态：disabled 属性来定义按钮是否被禁用，使用 disabled 属性来控制按钮是否为禁用状态。该属性接受一个 Boolean 类型的值。

【例3-5】编写一个Vue3项目，实现默认按钮的禁用状态按钮显示，代码如下：

```
<template>
    <el-button disabled>默认的按钮的禁用状态</el-button>
    <el-button type="primary" disabled>Primary 主要按钮</el-button>
    <el-button type="success" disabled>Success 成功按钮</el-button>
    <el-button type="info" disabled>Info 信息按钮</el-button>
    <el-button type="warning" disabled>Warning 警告按钮</el-button>
    <el-button type="danger" disabled>Danger 危险</el-button>
</template>
```

综上代码所述，呈现如下效果，如图3-5所示。



图3-5 默认按钮禁用状态按钮显示

在上述代码中，编写了默认按钮禁用状态的按钮的显示方式，采用的是disabled属性。

【例3-6】编写一个Vue3项目，实现朴素按钮的禁用状态按钮显示，代码如下：

```
<template>
    <el-button plain  disabled>朴素的按钮禁用状态</el-button>
    <el-button type="primary" plain  disabled>Primary 主要按钮</el-button>
    <el-button type="success" plain  disabled>Success 成功按钮</el-button>
    <el-button type="info" plain  disabled>Info 信息按钮</el-button>
    <el-button type="warning" plain  disabled>Warning 警告按钮</el-button>
    <el-button type="danger" plain disabled>Danger 危险</el-button>
</template>
```

综上代码所述，呈现如下效果，如图3-6所示。



图3-6　朴素按钮禁用状态按钮显示

在上述代码中，编写了朴素按钮禁用状态的按钮的显示方式，采用的是disabled属性。

3.3.2 Border边框

对边框进行统一规范处理，使其可以应用于按钮、卡片、弹窗等组件之中。

1.边框样式：实线、虚线

2.边框圆角样式：No Radius、Small Radius、Large Radius、Round Radius

【例3-7】编写服装选择在线调查问卷，实现Border边框的使用，代码如下：

```
<template>
  <div>
    <table  cellspacing="0px" cellpadding="0px" width="300px">
    <tr id="borderDemo">
      <td colspan="3">
        服装选择在线调查问卷
      </td>
    </tr>
    <tr>
      <td>服装风格</td>
      <td>修身</td>
      <td>宽松</td>
    </tr>
    <tr>
      <td>服装颜色</td>
      <td>红色</td>
      <td>灰色</td>
    </tr>
    <tr>
      <td>服装大小</td>
      <td>S码</td>
      <td>M码</td>
```

```
            </tr>
        </table>
        </div>
</template>
<style scoped>
    table{
        border: 2px solid #220808;
        /* border-radius: 0px; No Radius代表的是直角 */
        /* border-radius: 2px;Small Radius代表的是小圆角 */
        /* border-radius: 4px;Large Radius代表的是大圆角 */
        border-radius: 20px;/* Round Radius非常大的圆角 */
    }
    #borderDemo{
        text-align: center;
    }
    td{
        border: 2px dashed hotpink;
    }
</style>
```

综上代码所述，呈现如下效果，如图3-7所示。



图3-7 服装选择在线调查问卷

在上述代码里，达成了表格实线与虚线的效果，所运用的属性为solid和dashed；同时实现了表格的圆角样式，采用的属性是border-radius。

3.3.3 Icon图标

Element Plus提供了一套常用的图标集合，直接使用图标，可以全局注册，才能够直接在项目中使用。

1.安装Icon图标

（1）安装包管理器：npm install@element-plus/icons-vue

（2）注册全局的图标，创建src/plugins/icons.js文件

```
import * as components from '@element-plus/icons-vue'
export default{
    install:(app) =>{
            for (const key in components) {
              const componentConfig=components[key];
              app.component(componentConfig.name,componentConfig);
            }
    },
};
```

【例3-8】在进行Vue项目编写时，需实现Icon图标的显示功能，具体代码如下：

```
<template>
  <el-icon :size="20">
    <Edit style="width: 1em; height: 1em;margin-right: 8px" />
  </el-icon>
  <el-icon color="#409efc" class="no-inherit">
    <Share style="width: 1em; height: 1em;margin-right: 8px"/>
  </el-icon>
  <el-icon>
    <Delete style="width: 1em; height: 1em;margin-right: 8px"/>
  </el-icon>
  <el-icon class="is-loading">
    <Loading style="width: 1em; height: 1em;margin-right: 8px"/>
  </el-icon>
  <el-button type="primary">
    <el-icon style="vertical-align: middle">
      <Search style="width: 1em; height: 1em;margin-right: 8px"/>
    </el-icon>
  </el-button>
</template>
```

综上代码所述，呈现如下效果，如图3-8所示。



图3-8 Icon图标的案例

在上述代码中，使用了Element Plus UI框架的图标和按钮组件，展示了多种图标的使用方式和配置，采用SVG图标而非字体图标，提供更好的清晰度和控制性，通过CSS类名和属性灵活控制图标样式，所有图标保持一致的视觉大小和间距。

3.3.4 Containe容器布局

1.用于进行布局的容器组件，有助于便捷、高效地搭建页面的基本结构：

<el-container>：外层容器，当子元素中包含 <el-header> 或 <el-footer> 时，全部子元素会垂直上下排列，否则会水平左右排列。

（1）<el-header>：顶栏容器

（2）<el-aside>：侧边栏容器

（3）<el-main>：主要区域容器

（4）<el-footer>：底栏容器

2.以上组件采用了flex布局，使用前请确定目标浏览器是否兼容，此外<el-container>的直接子元素必须是后四个组件中的一个或多个，后四个组件的父元素必须是一个<el-container>。

【例3-9】在进行Vue项目编写时，需实现Container容器布局，具体代码如下：

```
<template>
  <div class="common-layout" style="background-color: beige;width: 1000px;height: 500px;">
    <el-container>
      <el-header style="background-color: azure;width: 1000px;height: 100px;display: block;">Header</el-header>
      <el-container>
        <el-aside style="background-color: silver;display: block;width: 200px;height: 400px;">Aside</el-aside>
        <el-container>
          <el-main style="background-color: khaki;width: 800px;height: 300px;display: block;margin-left: 200px;margin-top: -400px;">Main</el-main>
          <el-footer style="background-color: thistle;width: 800px;display: block;margin-left: 200px;margin-top: 1px;height: 100px;">Footer</el-footer>
```

```
        </el-container>
      </el-container>
    </el-container>
  </div>
</template>
```

综上代码所述，呈现如下效果，如图3-9所示。

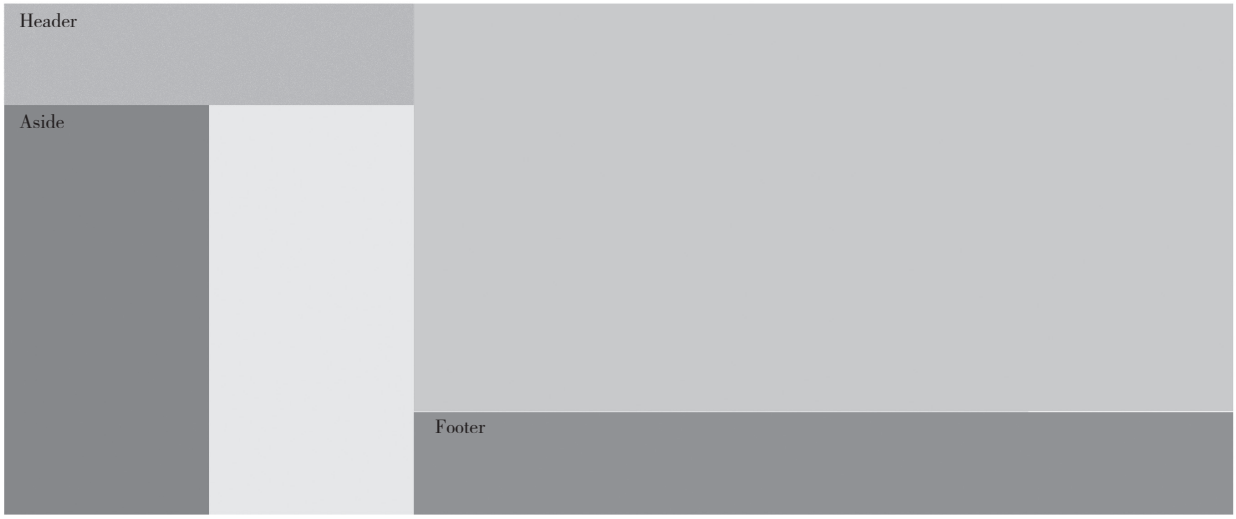


图3-9 Container容器布局案例

在上述代码中，借助Element Plus的布局组件，构建了一个布局结构。

3.3.5 link链接

1.基础用法：在网页设计和内容编辑中，基础的文字链接用法是指通过简单的文本形式创建超链接，使得用户可以通过点击这些文本链接跳转到其他页面或特定位置。

2.基本属性：link链接包含type类型、href属性、underline鼠标滑过时是否显示下划线的属性以及disabled是否处于禁用状态的属性。

3.禁用状态：在此状态下，文字链接处于不可用的状态，用户无法通过点击该链接进行任何操作或跳转至其他页面，链接文本通常会以灰色或其他提示不可用的颜色显示，以明确告知用户当前链接的功能已被禁用。

4.下划线设置功能：该功能主要用于精确控制文档中下划线的显示与否，用户可以通过简单的操作来决定是否在文本下方添加下划线，使得文档的视觉效果更加丰富和直观。

【例3-10】在进行Vue项目编写时，需要实现link链接的功能，具体代码如下：

```
<template>
    <div>
```

```
        <h1>基础用法应用</h1>
        <el-link type="default">默认链接</el-link><br/>
        <el-link type="primary" >主要链接</el-link><br/>
        <el-link type="success">成功链接</el-link><br/>
        <el-link type="warning">警告链接</el-link><br/>
        <el-link type="danger">危险链接</el-link><br/>
        <el-link type="info">信息链接</el-link>
        <h1>禁用状态应用</h1>
        <el-link type="default" disabled>默认链接</el-link><br/>
        <el-link type="primary" disabled>主要链接</el-link><br/>
        <el-link type="success" disabled>成功链接</el-link><br/>
        <el-link type="warning" disabled>警告链接</el-link><br/>
        <el-link type="danger" disabled>危险链接</el-link><br/>
        <el-link type="info" disabled>信息链接</el-link>
        <h1>下划线应用</h1>
        <el-link :underline="false" type="primary">当鼠标滑过无下划线</el-link><br/>
        <el-link type="success">当鼠标滑过有下划线</el-link><br/>
      </div>
</template>
<style scoped>
      .el-link{
        margin-right: 10px;
      }
</style>
```

综上代码所述，呈现如下效果，如图3-10所示。

3.3.6 Text文本

1.基础用法：在程序设计和文档处理的过程中，通过使用type属性这一关键功能，开发者或用户可以灵活地选择并指定Text组件的具体类型。

2.尺寸属性：可运用size属性对尺寸进行配置，可供选择的尺寸规格包括：large、default或small。

3.省略：通过truncated属性，当文本超出视图或最大宽度设置时，将展示省略符，利用line-clamp属性，可灵活控制多行文本的显示样式。



图3-10　link链接案例

4.覆盖：运用属性“tag”对元素进行覆盖。

5.混合使用：混合使用Text组件。

【例3-11】在进行Vue项目编写时，需要实现Text文本操作的功能，具体代码如下：

```
<template>
  <div>
    <h1>基础用法</h1>
        <el-text class="mx-1">默认文本</el-text>
        <el-text class="mx-1" type="primary">主要文本</el-text>
        <el-text class="mx-1" type="success">成功文本</el-text>
        <el-text class="mx-1" type="info">信息文本</el-text>
        <el-text class="mx-1" type="warning">警告文本</el-text>
        <el-text class="mx-1" type="danger">危险文本</el-text>
    <h1>尺寸属性</h1>
```

```
        <el-text class="mx-2" type="warning" size="large">警告文本</el-text>
        <el-text class="mx-2" type="danger">危险文本</el-text>
        <el-text class="mx-2" type="info" size="small">信息文本</el-text>
    <h1>省略</h1>
        <el-text class="w-150px mb-2" truncated>
        Self element set width 100px
        </el-text>
        <el-row class="w-150px mb-2">
        <el-text truncated>Squeezed by parent element</el-text>
        </el-row>
        <el-text line-clamp="2">
            The -webkit-line-clamp CSS property<br />
            allows limiting of the contents of<br />
            a block to the specified number of lines.
        </el-text>
    <h1>覆盖</h1>
      <el-space direction="vertical">
        <el-text>span</el-text>
        <el-text tag="p">This is a paragraph.</el-text>
        <el-text tag="b">Bold</el-text>
        <el-text tag="i">Italic</el-text>
        <el-text>
        This is
        <el-text tag="sub" size="small">subscript</el-text>
        </el-text>
        <el-text>
        This is
        <el-text tag="sup" size="small">superscript</el-text>
        </el-text>
        <el-text tag="ins">Inserted</el-text>
        <el-text tag="del">Deleted</el-text>
        <el-text tag="mark">Marked</el-text>
```

```
        </el-space>
    </div>
</template>
<style scoped>
        .mx-1{
          font-size: 20px;
          margin-right: 20px;
        }
        .mx-2{
          margin-right: 20px;
        }
</style>
```

综上代码所述，呈现如下效果，如图 3-11 所示。



图 3-11　Text 文本案例

在上述代码中，呈现了 Element Plus 文本组件的四种主要功能，分别为基础用法、尺寸控制、文本省略以及标签覆盖，凭借这些功能，能够便捷地创建契合各类设计需求的文本元素，同时维持统一的视觉风格。

3.3.7 Space间距

虽有Divider组件，但很多时候不需要其分割的页面结构，于是会重复使用该组件，这影响了开发效率，间距组件正是为解决此困扰而诞生。

1.基础用法：通过这个组件统一设置组件间的间距，并通过间距组件为多个组件之间提供一致的间距。

2.垂直布局：通过使用direction来控制布局方式，其背后实际上是借助flex-direction属性来实现这一功能。

3.控制间距的大小：通过调整size的值来精确控制间距大小，可使用内置的small、default和large选项来设置间距，分别对应8px、12px和16px的间距，系统默认的间距大小为small，即8px。

【例3-12】在进行Vue项目编写时，实现Space间距基本用法的功能，具体代码如下：

```
<template>
    <el-space wrap>
  <el-card v-for="i in 3" :key="i" class="box-card" style="width: 250px">
    <template #header>
     <div class="card-header">
      <span>Space间距</span>
      <el-button class="button" text>组件按钮</el-button>
     </div>
    </template>
    <div v-for="o in 4" :key="o" class="space item">
     {{ 'space item ' + o }}
    </div>
  </el-card>
 </el-space>
</template>
```

综上代码所述，呈现如下效果，如图3-12所示。

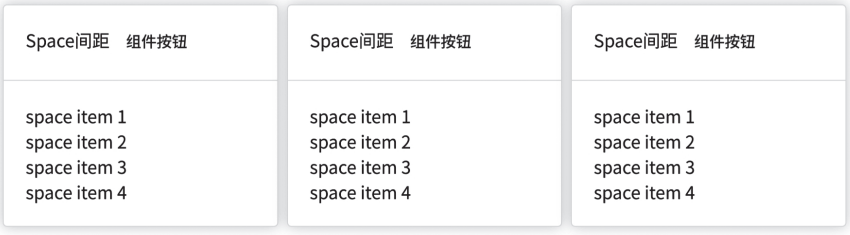


图 3-12 Space间距基本用法案例

在上述代码示例中，展示了Element Plus框架中Space和Card组件联合应用，构建含多个卡片的布局，卡片样式和结构统一，适合展示产品、功能项等需卡片式布局的场景，外观整洁统一，响应式特性出色。

【例3-13】在进行Vue项目编写时，实现Space间距垂直布局的功能，具体代码如下：

```
<template>
    <el-space direction="vertical">
  <el-card v-for="i in 2" :key="i" class="box-card" style="width: 250px">
    <template #header>
      <div class="card-header">
        <span>Space间距</span>
        <el-button class="button" text>垂直布局</el-button>
      </div>
    </template>
    <div v-for="o in 4" :key="o" class="text item">
      {{ 'Space item ' + o }}
    </div>
  </el-card>
 </el-space>
</template>
```

综上代码所述，呈现如下效果，如图3-13所示。

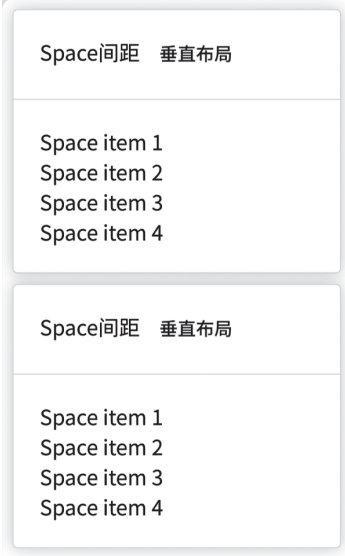


图3-13 Space间距垂直布局案例

在上述代码中展示了Element Plus中的Space组件与Card组件在垂直布局中的使用方法，构建了一个简洁明了的垂直排列卡片界面，这种布局方式尤其适用于需要垂直排列内容的场景，例如表单、设置项列表或移动端界面设计，能够提供清晰的信息层级和出色的可读性。

【例3-14】在进行Vue项目编写时，实现Space间距控制间距的大小的功能，具体代码如下：

```
<template>
    <el-space direction="vertical" alignment="start" :size="30">
  <el-radio-group v-model="size">
    <el-radio value="large">大</el-radio>
    <el-radio value="default">默认</el-radio>
    <el-radio value="small">小</el-radio>
  </el-radio-group>
  <el-space wrap :size="size">
    <el-card v-for="i in 3" :key="i" class="box-card" style="width: 250px">
      <template #header>
        <div class="card-header">
          <span>Card name</span>
          <el-button class="button" text>组件按钮</el-button>
        </div>
      </template>
      <div v-for="o in 4" :key="o" class="text item">
        {{ 'space item ' + o }}
      </div>
    </el-card>
  </el-space>
 </el-space>
</template>
<script setup>
 import { ref } from 'vue'
 const size = ref('default')
</script>
```

综上代码所述，呈现如下效果，如图3-14所示。

在上述代码中，构建了一个响应式布局组件，采用Element Plus的Space和Card组件设计了一个间

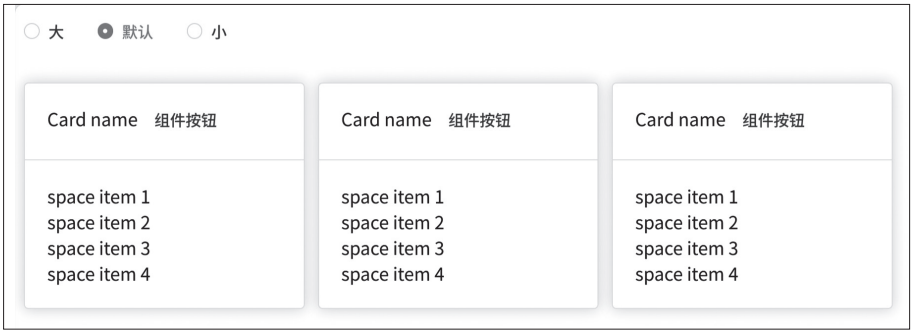


图3-14 Space间距控制大小的案例

距可调的卡片展示界面，充分展现了Vue 3的响应式数据绑定特性以及Element Plus组件库在灵活布局方面的强大能力。

3.3.8 Typography排版

Element Plus提供了强大的Typography模块，用于统一规范字体样式，确保在不同操作系统下的最佳展示效果。

1.字号：通过 --el-font-size-* 变量设置字体大小，例如 extra-large、large、medium、small和extra-small等。

2.行高：使用无单位值（如1.5），以便字体大小变化时自动调整行高。

3.字体族：默认使用以下字体。

【例3-15】在Vue项目中开发一个Typography排版组件，并实现其字号控制功能，具体代码如下：

```
<template>
    <div>
        <p style="font-size: var(--el-font-size-extra-large);">江西服装学院作为主标题</p>
        <p style="font-size: var(--el-font-size-large);">大数据学院作为副标题</p>
        <p style="font-size: var(--el-font-size-medium);">区块链专业作为小标题</p>
        <p style="font-size: var(--el-font-size-small);">正文</p>
        <p style="font-size: var(--el-font-size-extra-small);">辅助文字</p>
    </div>
</template>
<style>
    div{
        margin-left: 250px;
    }
</style>
```

综上代码所述，呈现如下效果，如图3-15所示。

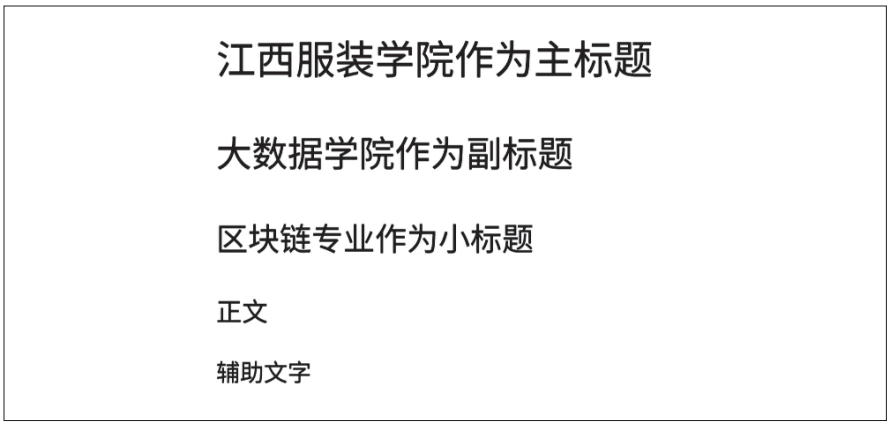


图3-15 Typography排版组件设置字体大小的案例

在上述代码中，基于Element Plus设计体系的文字排版层级演示，通过五行文本分别展示了五级字号样式，第一行采用Extra Large超大字号作为主标题；第二行使用Large大字号作为副标题；第三行选用Medium中等字号作为小标题；第四行应用Small小字号作为正文内容；第五行则使用Extra Small超小字号作为辅助说明文字。

【例3-16】在Vue项目中开发一个Typography排版组件，并实现其行高控制功能，具体代码如下：

```
<template>
  <div>
    <!-- 使用Element Plus的CSS变量设置字体大小和设置文本居中 -->
      <p style="font-size: var(--el-font-size-extra-large);text-align: center;">江西服装学院大数据学院的介绍</p>
      <!--使用Element Plus的CSS变量设置字体大小和设置文本行高 -->
      <p style="font-size: var(--el-font-size-medium);line-height:100px;">
        大数据学院成立于2020年5月，现开设有软件工程、物联网工程、数据科学与大数据技术、区块链工程四个本科专业，普通在校本科生1620人。主要为新一代信息技术行业、纺织服装产业等培养数智化应用型人才，助力企事业单位数字化、信息化和智能化。
        学院有自由专任教师75人，其中博士教师14人，教授7人，副高19人，副高职称以上教师占比34.7%。有省级教学名师1人、江西省高校金牌教授1人、江西省中青年骨干教师1人、江西省中青年学科（专业）带头人1人、江西省高层次D类、E类人才各1人、江西省高水平本科教学团队1个、首批江西省虚拟教研室1个。
      </p>
  </div>
</template>
```

综上代码所述，呈现如下效果，如图3-16所示。

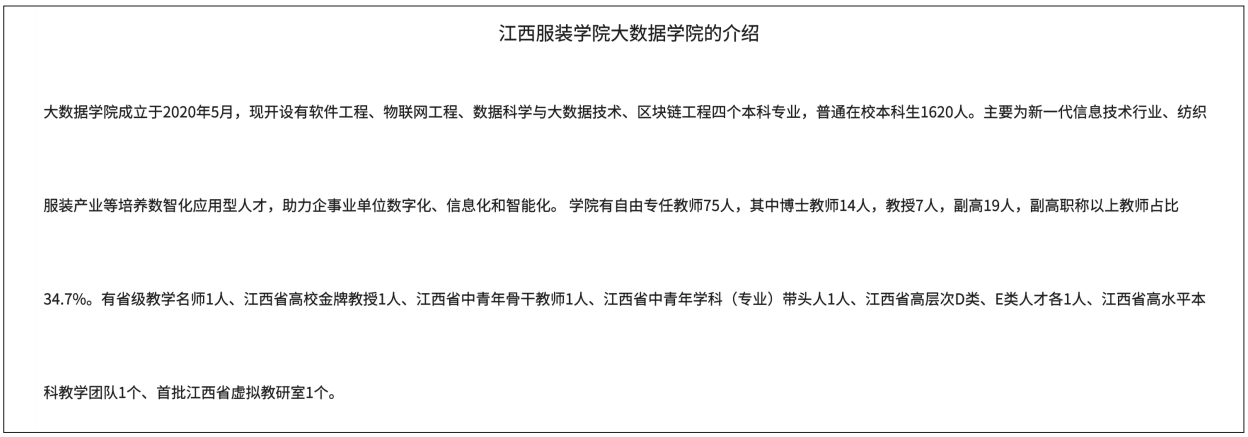


图3-16 Typography排版组件设置行高案例

在上述代码中，创建了一个简洁的学院介绍页面，采用Element Plus的CSS变量进行样式设计，并利用line-height属性设定了100px的行高，然而，这种行高设置在实际应用中可能导致文本间距异常，页面整体结构清晰，通过内联样式直接应用设计系统变量，展示了如何借助Element Plus的设计令牌来维持UI的一致性，内容部分则详尽介绍了大数据学院的成立时间、专业设置、学生规模、人才培养方向及师资力量等关键信息。

【例3-17】在Vue项目中开发一个Typography排版组件，并利用字体族功能，具体代码如下：

```
<template>
    <div>
<!-- 设置字体大小和颜色 -->
        <p style="font-size: var(--el-font-size-extra-large);color: black;">江西服装学院作为主标题</p>
        <p style="font-size: var(--el-font-size-large);color: aquamarine;">大数据学院作为副标题</p>
        <p style="font-size: var(--el-font-size-medium);color:blue;">区块链专业作为小标题</p>
        <p style="font-size: var(--el-font-size-small);color: blueviolet;">正文</p>
        <p style="font-size: var(--el-font-size-extra-small);color: blueviolet;">辅助文字</p>
    </div>
</template>
<style>
    div{
        margin-left: 250px;
 /* 设置字体族 */
font-family: Inter, 'Helvetica Neue', Helvetica, 'PingFang SC',
 'Hiragino Sans GB', 'Microsoft YaHei', '微软雅黑', Arial, sans-serif;
```

```
    }
</style>
```

综上代码所述，呈现如下效果，如图3-17所示。

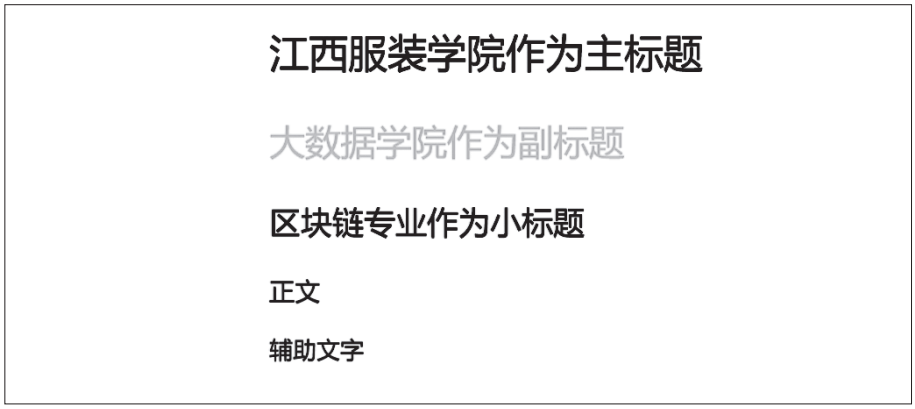


图3-17 Typography排版组件设置字体族案例

在上述代码中，创建了一个层次分明的文本展示界面，使用了Element Plus设计系统的字体大小变量和自定义颜色来构建视觉层次，在样式部分，设置了250px的左外边距使内容区域右移，并定义了全面的字体族栈，确保在不同操作系统和设备上都能获得最佳的文字渲染效果，优先使用Inter字体，并提供了多个备用字体包括中文字体支持。

3.4 表单组件



Form表单组件：表单包含输入框,标签输入框、单选按钮、多选按钮、下拉选择器、穿梭框、树形选择、自动补全输入框和级联选择器等用户输入的组件，使用表单可以收集、验证和提交数据。

3.4.1 Input输入框

Input输入框：通过鼠标或键盘输入字符。

【例3-18】在Vue.js前端开发项目中，为了打造更为智能化的用户交互体验，可以编写Form表单组件，实现Input输入框的基本用法、禁用状态、一键清空功能、密码框功能、带图标输入框功能以及文本域功能。

```
<template>
  <div>
    <!-- 基础用法 -->
```

```
        <h3>基础用法</h3>
        <!-- 基础输入框，使用v-model进行双向数据绑定 -->
        <el-input v-model="inputDemo" style="width: 300px" placeholder="输入框" />
        <hr/>
        <h3>禁用状态</h3>
        <!-- 禁用状态 -->
        <!-- disabled属性使输入框变为禁用状态，用户无法编辑 -->
        <el-input v-model="inputDemo01" style="width: 300px" placeholder="禁用状态" disabled />
        <hr/>
        <h3>一键清空功能</h3>
        <!-- 一键清空功能 -->
        <!-- clearable属性添加清空按钮，点击可清空输入内容 -->
        <el-input v-model="inputDemo02" style="width: 300px" placeholder="一键清空功能" clearable />
        <hr/>
        <h3>密码框的功能</h3>
        <!-- type="password"设置为密码类型，show-password显示密码可见性切换按钮 -->
        <el-input v-model="inputDemo03" style="width: 300px" placeholder="密码框的功能" type="password"
show-password />
        <hr/>
        <h3>带图标输入框功能</h3>
        <!-- prefix-icon属性在输入框前添加图标，这里使用Search图标 -->
        <el-input
            v-model="inputDemo04"
            class="responsive-input"
            placeholder="带图标输入框功能"
            :prefix-icon="Search"
            style="width: 300px"
          />
        <h3>文本域的功能</h3>
        <!-- type="textarea"设置为多行文本域，rows属性定义显示的行数 -->
        <el-input
        v-model="inputDemo05"
```

```
    style="width: 300px"
    :rows="2"
    type="textarea"
    placeholder="文本域的功能"
  />
  </div>
</template>
<script setup>
import {ref} from 'vue'
import {Search } from '@element-plus/icons-vue'
// 基础输入框的值
const inputDemo=ref('')
// 禁用状态输入框的值
const inputDemo01=ref('')
// 可清空输入框的值
const inputDemo02=ref('')
// 密码框的值
const inputDemo03=ref('')
// 带图标输入框的值
const inputDemo04=ref('')
// 文本域的值
const inputDemo05=ref('')
</script>
```

综上代码所述，呈现如下效果，如图3-18所示。

在上述代码中，系统性呈现六种常见输入框使用场景：基础输入框用于常规文本输入；禁用状态输入框用disabled属性实现不可编辑；一键清空功能借clearable属性添加清除按钮；密码框结合type=password与show-password属性切换密码显示；带图标输入框用prefix-icon属性前置搜索图标；文本域通过type=textarea和rows属性创建多行文本输入区域。

3.4.2 Input Number数字输入框

Input Number数字输入框：仅允许输入标准的数字值，可定义范围。

【例3-19】于Vue.js前端开发项目中，为营造更为智能化的用户交互体验，可编写Form表单组件，



图3-18　Input输入框案例

以实现Input数字输入框的基本用法、禁用状态、步进功能以及带前缀与后缀的功能。

```
<template>
    <h3>基本用法</h3>
    <!--
     el-input-number: Element Plus的数字输入框组件
     v-model="number01": 双向数据绑定，关联number01响应式变量
     :min="1": 设置最小值为1
     :max="10": 设置最大值为10
     用户可以通过按钮或直接输入来调整数值（范围限制在1-10之间）
    -->
    <el-input-number v-model="number01" :min="1" :max="10" />
    <hr/>
     <h3>禁用状态</h3>
     <!--
     :disabled="true": 将输入框设置为禁用状态
```

```
    禁用后用户无法通过按钮或输入来修改数值
    外观会变为灰色，表示不可交互状态
    -->
    <el-input-number v-model="number02" :min="1" :max="10" :disabled="true" />
    <hr/>
    <h3>步进功能</h3>
    <!--
     :step="2": 设置每次增加或减少的步长为2
    点击增加按钮时：当前值+2
    点击减少按钮时：当前值-2
    数值变化会遵循min和max的范围限制
    -->
    <el-input-number v-model="number03" :min="1" :max="10" :step="2" />
    <hr/>
    <h3>带前缀与后缀的功能</h3>
     <el-space>
   <el-input-number v-model="number04" :min="1" :max="10">
    <template #prefix>
      <span> ￥</span>
    </template>
   </el-input-number>
  </el-space>
</template>
<script setup>
import { ref } from 'vue'
// 创建三个响应式变量，分别对应三个数字输入框的初始值
const number01 = ref(1)
// 禁用状态输入框的初始值，默认值设置为1
const number02 = ref(1)
// 步进功能输入框的初始值，默认值设置为1
const number03 = ref(1)
// 步进功能输入框的初始值，默认值设置为1
```

```
const number04 = ref(1)
</script>
```

综上代码所述，呈现如下效果，如图3-19所示。

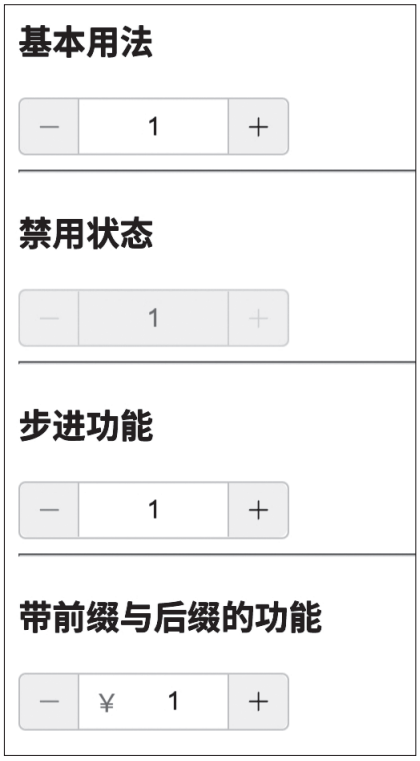


图3-19　Input Number数字输入框案例

在上述代码中，数字输入框组件el-input-number存在四种常见使用方式：借助基本用法对数值范围限制功能予以演示，运用禁用状态呈现不可交互的场景，配置步进值以控制数值的变化幅度，添加前缀符号达成格式化显示。代码借助四个独立的响应式变量分别管理不同输入框的状态，清晰地展现了该组件的核心功能特性。

3.4.3 InputTag 标签输入框

InputTag标签输入框：InputTag组件允许用户添加内容作为标签。

【例3-20】在Vue.js前端开发项目中，为了打造更加智能化的用户交互体验，可以编写Form表单组件，从而实现InputTag标签输入框的基础用法、自定义触发机制和禁用状态功能。

```
<template>
    <h3>基本用法</h3>
    <el-input-tag
    v-model="inputTag01"
```

```
      placeholder="基本用法"
      aria-label="基本用法"
      style="width: 300px;"/>
    <hr/>
    <h3>自定义触发器</h3>
    <!--:trigger="'Space'" 设置空格键为标签生成触发器 -->
    <el-input-tag
    v-model="inputTag02"
    placeholder="按空格键生成标签"
    aria-label="自定义触发键"
    style="width: 300px;"
    :trigger="'Space'"/>
     <hr/>
    <h3>禁用状态</h3>
    <!--disabled 设置输入框为禁用状态 -->
    <el-input-tag
      v-model="inputTag03"
      placeholder="禁用状态"
      aria-label="组件已被禁用"
      style="width: 300px;"
      disabled />
</template>
<script setup>
import { ref } from 'vue'
// 使用ref创建响应式变量，用于双向数据绑定
// 基本用法的标签数据
const inputTag01 = ref([])
// 自定义触发器的标签数据
const inputTag02= ref([])
// 禁用状态的标签数据
const inputTag03= ref([])
</script>
```

综上代码所述，呈现如下效果，如图3-20所示。

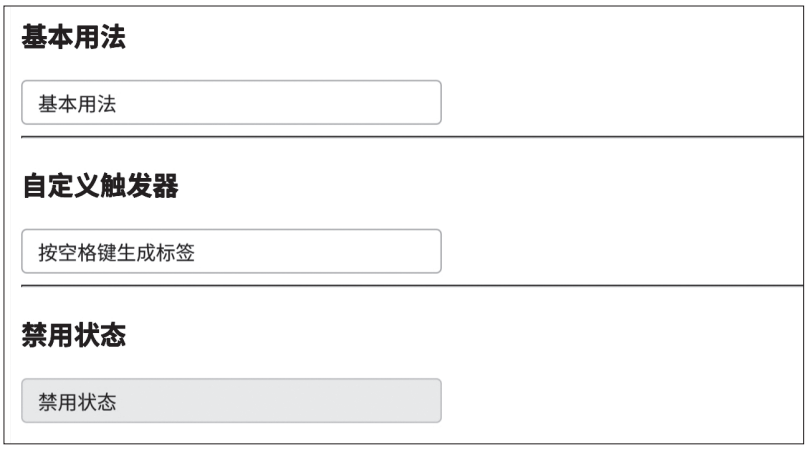


图3-20　InputTag标签输入框案例

在上述代码中，组件的基础功能、自定义交互行为及状态控制。每个输入框均通过v-model实现数据的双向绑定，将用户输入的标签内容实时同步至相应的响应式数组中，充分展现了Vue 3组合式API的简洁性与Element Plus组件的高度可配置性。

3.4.4 Radio单选框

Radio单选框：在一组备选项中进行单选。

【例3-21】在Vue.js前端开发项目中，可编写Form表单组件，进而达成Radio单选框的基础用法、禁用状态功能、单选按钮组和带有边框等功能。

```
<template>
<h3>基础用法</h3>
<!-- 单选按钮组，v-model实现双向绑定 -->
<el-radio-group v-model="radio01">
    <el-radio label="1">单选按钮 1</el-radio>
    <el-radio label="2">单选按钮 2</el-radio>
    <el-radio label="3">单选按钮 3</el-radio>
</el-radio-group>
<hr/>
<h3>禁用状态</h3>
<!-- disabled属性禁用该单选按钮 -->
<el-radio-group v-model="radio02">
    <el-radio label="1" disabled >单选按钮 1</el-radio>
```

```
        <el-radio label="2">单选按钮 2</el-radio>
    </el-radio-group>
    <hr/>
    <h3>单选框组 </h3>
    <!-- 使用 :value 绑定数值类型值 -->
    <el-radio-group v-model="radio03">
        <el-radio :value="3">单选按钮 1</el-radio>
        <el-radio :value="6">单选按钮 2</el-radio>
        <el-radio :value="9">单选按钮 3</el-radio>
     </el-radio-group>
     <hr/>
    <h3>带边框的按钮 </h3>
    <!-- border 属性添加边框样式 -->
    <el-radio-group v-model="radio04">
        <el-radio label="1" border>单选按钮 1</el-radio>
        <el-radio label="2" border>单选按钮 2</el-radio>
    </el-radio-group>
    </template>
    <script setup>
    // 导入 Vue 的响应式 API
    import { ref } from 'vue'
    // 定义响应式变量来存储单选按钮的选中值
    const radio01=ref('1');// 基础用法，默认选中第一个
    const radio02=ref('1');// 禁用状态，默认选中第一个【禁用项】
    const radio03=ref(3);// 数值类型值，默认选中值为 3 的选项
    const radio04=ref('1');// 带边框样式，默认选中第一个
    </script>
```

综上代码所述，呈现如下效果，如图 3-21 所示。

在上述代码中，Radio 单选框不应设置过多可选项，若存在较多可选项，则应使用选择框而非单选框，使用 Radio 组件时，仅需设置 v-model 绑定变量，选中状态意味着变量的值与相应 Radio 的 value 属性值一致，value 属性值可以为字符串、数字或布尔值，disabled 属性可用于控制单选框的禁用状态，单选按钮组适用于从多个互斥选项中进行选择的场景。



图3-21　Radio标签单选按钮案例

3.4.5 Checkbox 多选框

Checkbox多选框：在一组备选项中进行多选。

【例3-22】在Vue.js前端开发项目中，可编写Form表单组件，进而实现Checkbox多选框的基础用法、禁用状态功能、多选按钮组和带有边框等功能。

```
<template>
    <!-- 案例标题 -->
  <h2>请选择服装的颜色和尺码</h2>
 <div>
   <h3>基础用法</h3>
  <el-checkbox v-model="checked1" label="红色" size="large" />
  <el-checkbox v-model="checked2" label="绿色" size="large" />
 </div>
 <div >
  <el-checkbox v-model="checked3" label="蓝色" />
  <el-checkbox v-model="checked4" label="黑色" />
 </div>
 <div>
  <el-checkbox v-model="checked5" label="白色" size="small" />
  <el-checkbox v-model="checked6" label="灰色" size="small" />
```

```
  </div>
  <hr/>
  <h3>禁用状态</h3>
  <div>
   <el-checkbox v-model="checked7" label="灰白色" size="small" />
   <el-checkbox v-model="checked8" label="金色" size="small" disabled />
  </div>
  <hr/>
  <h3>多选按钮组</h3>
  <el-checkbox-group v-model="checkList">
   <el-checkbox label="S码" value="Value A" />
   <el-checkbox label="M码" value="Value B" />
   <el-checkbox label="L码" value="Value C" />
   <el-checkbox label="XL码" value="Value disabled" disabled />
   <el-checkbox
    label="XXL码"
    value="Value selected and disabled"
    disabled
   />
  </el-checkbox-group>
  <hr/>
  <h3>带有边框的按钮</h3>
  <el-checkbox-group v-model="checkListBorder">
      <!-- border:设置多选按钮的边框 -->
   <el-checkbox label="S码" value="Value C" border/>
   <el-checkbox label="M码" value="Value D" border/>
  </el-checkbox-group>
</template>
<script setup>
import { ref } from 'vue'
// 定义各个多选框的响应式状态
// 红色多选框，初始值为选中的状态
```

```
const checked1 = ref(true)
// 绿色多选框，初始值为未选中的状态
const checked2 = ref(false)
// 蓝色多选框，初始值为未选中的状态
const checked3 = ref(false)
// 黑色多选框，初始值为未选中的状态
const checked4 = ref(false)
// 白色多选框，初始值为未选中的状态
const checked5 = ref(false)
// 灰色多选框，初始值为未选中的状态
const checked6 = ref(false)
// 灰白色多选框，初始值为未选中的状态
const checked7 = ref(false)
// 金色多选框，初始值为未选中的状态
const checked8 = ref(false)
//多选按钮组 – 初始选中S码和M码（对应值"Value A"和"Value B"）
const checkList = ref(['Value A', 'Value B'])
//带边框的按钮组 – 初始选中S码（对应值"Value C"）
const checkListBorder=ref(['Value C']);
</script>
```

综上代码所述，呈现如下效果，如图3-22所示。



图3-22　checkbox多选按钮案例

在上述代码中，基于Element Plus构建的服装选择界面，着重展示了Checkbox多选框组件的四种典型应用方式：在基础应用部分，演示了不同尺寸（large、默认、small）的颜色选择，在禁用状态部分，呈现了正常复选框与禁用复选框之间的差异，多选按钮组借助el-checkbox-group对尺码选项进行管理，涵盖正常状态与禁用状态，最后展示的是带边框样式的按钮组，整个案例借助颜色和尺码选择场景，全面呈现了Checkbox组件在实际业务中的应用模式。

3.4.6 Select选择器

Select选择器：当选项过多时，使用下拉菜单展示并选择内容。使用Select选择器时，需注意在2.5.0版本之后，el-select的默认宽度变更为100%。当采用内联形式时，宽度显示会出现异常。为确保显示正常，您需手动配置el-select的宽度，将展示Select选择器的基础用法。

【例3-23】在Vue.js前端开发项目中，可编写Form表单组件，运用select选择器（即下拉列表），实现基本用法，适合单选功能。

```
<template>
    <h3>基本用法和Options属性</h3>
    <el-select v-model="value" placeholder="请选择" style="width: 300px;">
        <!-- v-for:遍历options数组，:key：使用Value作为唯一的key，:label：显示文本，:value：实际值 -->
 <el-option v-for="item in options" :key="item.value" :label="item.label" :value="item.value">
</el-option>
    </el-select>
</template>
<script setup>
// 导入Vue响应式的API
import { ref } from 'vue'
// 创建响应式变量，用于存储选中的值
const value = ref('')
// 定义选择器选项数组，每个选项对象包含value：选项的实际值，label：选项的显示文本
const options = [
    {
        value: ' Element Plus1',
        label:'select组件1'
    },
```

```
        {
            value: ' Element Plus2',
            label:'select 组件 2'
        },
        {
            value: ' Element Plus3',
            label:'select 组件 3'
        }
    ]
    </script>
```

综上代码所述，呈现如下效果，如图 3-23 所示。



图 3-23　Select 下拉选择器基础用法的案例

在上述代码中，使用 Element Plus 组件库的下拉选择器，基础用法适用于单选场景。

3.4.7 Transfer 穿梭框

Transfer 穿梭框：Transfer 的数据通过 data 属性传入，在使用 Element Plus 的 el-transfer 穿梭框组件时，可以通过联动功能实现左右列表的数据同步与交互，穿梭框常用于在两个列表之间转移数据项，如权限分配、数据分类等场景。

【例 3-24】于 Vue.js 前端开发项目中，能够编写 Form 表单组件，运用 Transfer 穿梭框达成基础用法。

```
<template>
    <!--
     Element Plus 穿梭框组件：
        v-model="transferKeys" 双向绑定
```

```
            :data="data" 绑定数据源
            :props="{key:'id',label:'name'}" 配置数据项的字段映射
            filterable 启用搜索过滤功能
            :titles="['待选择','已选择']" 设置左右两侧的标题
            @change="handleChange" 选中发生变化时触发的事件
        -->
    <el-transfer
    v-model="transferKeys"
    :data="data"
    :props="{key:'id',label:'name'}"
    filterable
    :titles="['待选择','已选择']"
    @change="handleChange"/>
</template>
<script setup>
import {ref} from 'vue';
// 定义穿梭框的数据源
// 每项数据项包含：id 作为唯一标识符、name 显示名称、disabled 禁用状态，设置为 true 时该项不可选
择
const data=ref([
    {id:1,name:'选择 1'},
    {id:2,name:'选择 2'},
    {id:3,name:'选择 3',disabled:true}
]);
// 创建响应式变量，存储当前已选中的项的 id 数组
const transferKeys=ref([]);
// 穿梭框变化事件处理函数
// 当选中项发生变化时触发，参数 newKeys 包含当前右侧所有已选项的 id 数组
const handleChange=(newKeys)=>{
    console.log('右侧列表已更新:',newKeys)
};
</script>
```

综上代码所述，呈现如下效果，如图3-24所示。



图3-24 Transfer穿梭框达成基础用法的案例

在上述代码中，使用Element Plus穿梭框（Transfer）组件，借助props指定以id作为键值，以name作为显示标签，然后启用filterable功能后，能够在两侧列表中开展搜索过滤操作，在案例中使用disabled状态，用户无法对其进行选择，当选中项发生变化时，会触发change事件，此时可获取到最新的选中项数组。

3.4.8 TreeSelect 树形选择

在运用Element Plus实现TreeSelect树形选择功能时，可将el-table和el-tree等组件相结合，该实现方式支持多选、回显、筛选等功能，将通过一个案例对这些功能予以展示。

【例3-25】需开发一款服装分类多选组件，此组件应支持三级树形分类结构（即大类-中类-小类），实现多选功能，且能够折叠显示已选中的标签，要求实时展示已选选项的详细信息，提供诸如一键选中特定季节商品以及清空选择等快捷操作，若需查看完整代码，请参阅源码。

```
<template>
  <div class="clothing-tree-select">
    <h1>服装分类选择器</h1>
    <!--
    树形选择区域
    -->
    <div class="select-section">
      <!--
      multiple：启用多选模式
      collapse-tags：多选时折叠标签
```

```
    collapse-tags-tooltip：折叠标签显示提示
    -->
    <el-tree-select
      v-model="selectedValues"
      :data="clothingData"
      multiple
      :props="treeProps"
      collapse-tags
      collapse-tags-tooltip
      placeholder="请选择服装分类"
      style="width: 100%"
    />
  </div>
  <!-- 已选择项显示区域 -->
  <div class="selected-section">
    <h2>已选择的分类：</h2>
    <ul>
      <!-- 遍历显示已选择的分类项 -->
      <!-- 显示分类名称和类型 -->
      <li v-for="item in selectedItems" :key="item.id">
        {{ item.name }} - {{ item.type }}
      </li>
    </ul>
    <!-- 空状态提示 -->
    <p v-if="selectedItems.length === 0" class="empty-tip">暂未选择任何分类</p>
  </div>
  <!-- 操作按钮区域 -->
  <div class="action-buttons">
    <!-- 选中夏季服装按钮 -->
    <el-button type="primary" @click="selectSummerClothing">选中夏季服装</el-button>
    <!-- 清空选择按钮 -->
    <el-button @click="clearSelection">清空选择</el-button>
```

```
    </div>
  </div>
</template>

<script setup>
import { ref, computed } from 'vue';

// TreeSelect 组件配置对象
// 定义数据结构字段映射关系
const treeProps = {
  value: 'id',
  label: 'name',
  children: 'children'
};

// 服装分类数据 – 三级树形结构
const clothingData = ref([
  {
    id: 1,
    name: '上装',
    type: '大类',
    children: [
      {
        id: 2,
        name: 'T恤',
        type: '中类',
        children: [
          { id: 3, name: '短袖T恤', type: '小类' },
          { id: 4, name: '长袖T恤', type: '小类' },
          { id: 5, name: 'POLO衫', type: '小类' },
        ],
      },
```

```
    {
      id: 6,
      name: '衬衫',
      type: '中类',
      children: [
        { id: 7, name: '商务衬衫', type: '小类' },
        { id: 8, name: '休闲衬衫', type: '小类' },
      ],
    },
    {
      id: 9,
      name: '外套',
      type: '中类',
      children: [
        { id: 10, name: '风衣', type: '小类' },
        { id: 11, name: '羽绒服', type: '小类' },
        { id: 12, name: '牛仔外套', type: '小类' },
      ],
    },
  ],
},
{
  id: 13,
  name: '下装',
  type: '大类',
  children: [
    {
      id: 14,
      name: '裤子',
      type: '中类',
      children: [
        { id: 15, name: '牛仔裤', type: '小类' },
```

```
          { id: 16, name: '休闲裤', type: '小类' },
          { id: 17, name: '短裤', type: '小类' },
        ],
      },
      {
        id: 18,
        name: '裙子',
        type: '中类',
        children: [
          { id: 19, name: '连衣裙', type: '小类' },
          { id: 20, name: '半身裙', type: '小类' },
        ],
      },
    ],
  },
  {
    id: 21,
    name: '配饰',
    type: '大类',
    children: [
      { id: 22, name: '帽子', type: '小类' },
      { id: 23, name: '围巾', type: '小类' },
      { id: 24, name: '手套', type: '小类' },
    ],
  },
]);
// 选中的值数组，存储被选中的 id
const selectedValues = ref([]);
// 计算已选择的完整项目信息
const selectedItems = computed(() => {
  const items = [];
//  递归查找函数，遍历树形数据
```

```
  const findItems = (tree) => {
    for (const item of tree) {
      // 如果当前项的id再选中数组中，则添加到结果中
      if (selectedValues.value.includes(item.id)) {
        items.push(item);
      }
//      如果当前项有子节点，递归查找子节点
      if (item.children) {
        findItems(item.children);
      }
    }
  };
//  从根节点开始查找
  findItems(clothingData.value);
//  返回找到的选中项数组
  return items;
});
// 选中夏季服装（短袖T恤、短裤等）
const selectSummerClothing = () => {
  selectedValues.value = [3, 17]; // 短袖T恤和短裤的ID
};
// 清空所有选择的方法
const clearSelection = () => {
  selectedValues.value = [];
};
</script>
```

综上代码所述，呈现如下效果，如图3-25所示。

在上述代码中，开发出一个功能相对完善的服装分类选择器，借助Element Plus的TreeSelect组件，支持多选模式，用户能够选择多个服装分类项目，系统会实时显示已选项目的详细信息，并提供筛选功能，使用户能够快速定位所需分类，同时，该选择器支持一键选中特定季节服装以及清空选择的便捷操作，达成了完整的选择、展示与管理流程。



图3-25　Select下拉选择器多选功能的案例

3.5 Data数据展示

在Data数据展示中，Table表格、Image图片与Pagination分页是三大核心支柱，分别承载着数据的结构化呈现、可视化渲染与海量信息的高效调控。

3.5.1 Table表格

用于展示多条结构类似的数据，可对数据进行排序、筛选、对比或其他自定义操作，除了基础的数据展示，Table表格组件还支持多种增强型表格，包括带边框的表格、带斑马纹的表格以及能够标识行/列状态的表格。

【例3-26】开发服装商品管理表格组件，实现基础商品数据展示，并通过启用斑马纹与边框样式，同时组件需支持根据商品状态进行整行背景色高亮。

```
<template>
  <h3 style="text-align: center;">服装商品数据表格 </h3>
  <!-- 服装商品数据表格 -->
  <el-table
    :data="clothingData"
    stripe
    border
```

```
    style="width: 70%;margin: auto;"
    :row-class-name="tableRowClassName"
  >
    <!-- 商品ID列 -->
    <el-table-column prop="productId" label="商品ID" />
    <!-- 商品名称列 -->
    <el-table-column prop="name" label="商品名称" />
    <!-- 商品分类列 -->
    <el-table-column prop="category" label="分类" />
    <!-- 商品价格列 -->
    <el-table-column prop="price" label="价格" />
    <!-- 库存数量列 -->
    <el-table-column prop="stock" label="库存" />
    <!-- 商品状态列 -->
    <el-table-column prop="status" label="状态">
      <template #default="scope">
        <el-tag
          :type="getStatusType(scope.row.status)"
          effect="dark"
        >
          {{ scope.row.status }}
        </el-tag>
      </template>
    </el-table-column>
  </el-table>
</template>
<script setup>
// 服装商品数据集合
const clothingData = [
  {
    productId: 'CL001', // 商品唯一标识
    name: '女士休闲连衣裙', // 商品名称
```

```
  category: '连衣裙', // 商品分类
  price: '¥299', // 商品价格
  stock: 45, // 库存数量
  status: '在售' // 商品状态：在售/缺货等
 },
 {
  productId: 'CL002',
  name: '男士修身西装',
  category: '西装',
  price: '¥899',
  stock: 23,
  status: '在售'
 },
 {
  productId: 'CL003',
  name: '运动休闲鞋',
  category: '鞋履',
  price: '¥399',
  stock: 0, // 库存为0
  status: '缺货' // 库存为0时显示缺货状态
 },
 {
  productId: 'CL004',
  name: '女士真丝衬衫',
  category: '上装',
  price: '¥199',
  stock: 67,
  status: '在售'
 },
 {
  productId: 'CL005',
  name: '双肩背包',
```

```
    category: '包包',
    price: '¥159',
    stock: 12,
    status: '在售'
  }
]

// 根据状态返回对应的标签类型
const getStatusType = (status) => {
  const typeMap = {
    '在售': 'success',    // 成功状态 – 绿色
    '缺货': 'danger',     // 危险状态 – 红色
    '预售': 'warning',    // 警告状态 – 橙色
    '下架': 'info'        // 信息状态 – 蓝色
  }
  return typeMap[status] || 'info'
}

// 为表格行添加自定义 class
const tableRowClassName = ({ row }) => {
  const classMap = {
    '在售': 'success-row',   // 成功状态 – 浅绿色背景
    '缺货': 'danger-row',    // 危险状态 – 浅红色背景
    '预售': 'warning-row',   // 警告状态 – 浅橙色背景
    '下架': 'info-row'       // 信息状态 – 浅蓝色背景
  }
  return classMap[row.status] || ''
}
</script>

<style scoped>
/* 成功状态行样式 – 浅绿色背景 */
```

```
:deep(.success-row) {
  --el-table-tr-bg-color: #f0f9ff;
}

/* 危险状态行样式 - 浅红色背景 */
:deep(.danger-row) {
  --el-table-tr-bg-color: #fef0f0;
}

/* 警告状态行样式 - 浅橙色背景 */
:deep(.warning-row) {
  --el-table-tr-bg-color: #fdf6ec;
}

/* 信息状态行样式 - 浅蓝色背景 */
:deep(.info-row) {
  --el-table-tr-bg-color: #f4f4f5;
}
</style>
```

综上代码所述，呈现如下效果，如图3-26所示。

服装商品数据表格

商品ID	商品名称	分类	价格	库存	状态
CL001	女士休闲连衣裙	连衣裙	¥299	45	在售
CL002	男士修身西装	西装	¥899	23	在售
CL003	运动休闲鞋	鞋履	¥399	0	缺货
CL004	女士真丝衬衫	上装	¥199	67	在售
CL005	双肩背包	包包	¥159	12	在售

图3-26 Table表格案例

在上述代码中，展示了Element Plus Table组件的基础用法，通过:data属性绑定服装商品数据源，使用el-table-column定义各数据列（商品ID、名称、分类、价格、库存和状态），在进阶功能中，利用stripe和border属性实现斑马纹和边框样式，并通过自定义row-class-name方法为不同状

态（在售、缺货、预售、下架）的行设置差异化背景色。状态列采用作用域插槽结合el-tag组件，配合getStatusType映射函数实现彩色状态标签，形成"标签色彩+行背景"的双重视觉提示体系，完整演示了从基础表格到增强型数据展示的实现路径。

3.5.2 Image图片

图片容器，在保留所有原生img的特性下，支持懒加载，自定义占位、加载失败等，Image图片组件不仅能够实现基础的图片展示功能，还支持进行图片预览操作，该组件可以清晰地呈现各类图片内容，同时提供便捷的预览交互体验，Image组件都能完美兼容并流畅显示，确保图片内容的完整呈现。

【例3-27】创建一个展示Element UI图片组件的演示页面，展示图片预览功能的个性化配置，每张图片设置独立的预览图片列表。

```
<template>
  <!-- 图片展示组件容器 -->
  <div class="demo-image">
    <!-- 标题 -->
    <h3 style="margin-left:300px ;">Image图片案例展示</h3>
    <!-- 图片展示区块 -->
    <div class="block">
      <!-- 图片1：填充模式 – 拉伸图片填满容器，不保持宽高比 -->
      <el-image
        style="width: 100px; height: 100px; margin-right: 70px;"
        src="/images/1.jpg"
        fit="fill"
        :preview-src-list="previewList1" />
        <!-- preview-src-list：设置图片预览功能，点击图片可查看大图并切换浏览所有图片 -->
      <!-- 图片2：包含模式 – 保持宽高比缩放，完整显示图片 -->
      <el-image
        style="width: 100px; height: 100px; margin-right: 70px;"
        src="/images/2.jpg"
        fit="contain"
        :preview-src-list="previewList2" />
        <!-- preview-src-list：预览图片列表，包含所有展示图片的路径 -->
      <!-- 图片3：覆盖模式 – 保持宽高比缩放，填满容器，可能裁剪部分图片 -->
```

```
      <el-image
        style="width: 100px; height: 100px; margin-right: 70px;"
        src="/images/3.jpeg"
        fit="cover"
        :preview-src-list="previewList3" />
        <!-- preview-src-list：启用图片预览模态框，支持左右切换查看所有图片 -->
      <!-- 图片 4：原始模式 – 保持图片原始尺寸，不进行缩放 -->
      <el-image
        style="width: 100px; height: 100px; margin-right: 70px;"
        src="/images/4.jpg"
        fit="none"
        :preview-src-list="previewList4" />
        <!-- preview-src-list：点击图片时打开预览窗口，可浏览图片集合 -->
      <!-- 图片 5：缩放模式 – 取 none 或 contain 中较小的一个 -->
      <el-image
        style="width: 100px; height: 100px; margin-right: 70px;"
        src="/images/5.jpg"
        fit="scale-down"
        :preview-src-list="previewList5" />
        <!-- preview-src-list：预览功能配置，用户可点击查看高清大图并进行图片导航 -->
    </div>
  </div>
</template>
<script>
export default {
  data() {
    return {
      previewList1: ['/images/1.jpg'],
      previewList2:['/images/2.jpg'],
      previewList3:['/images/3.jpeg'],
      previewList4:['/images/4.jpg'],
      previewList5:['/images/5.jpg']
```

```
    }
  }
}
</script>
```

综上代码所述，呈现如下效果，如图3-27所示。



图3-27 Image图片展示案例

在上述代码中，实现了一个图片展示组件，通过Element UI的el-image组件展示了五种不同的图片适配模式（fill、contain、cover、none和scale-down），每张图片尺寸为100x100像素且间距70px，同时为每张图片配置了独立的预览功能，通过preview-src-list属性绑定各自的预览图片数组，用户点击任意图片即可查看对应的大图预览效果。

3.5.3 Pagination分页

当数据量过多时，使用分页分解数据，可以展示基础的分页功能用法，包括常规的分页按钮布局以及翻页操作；同时还可以呈现带有背景色的分页样式，通过不同的颜色设计增强视觉效果；此外，分页组件还支持多种附加功能特性，例如显示总数统计、快速跳转页码、自定义每页显示数量等扩展功能，为用户提供更全面和灵活的分页交互体验。

【例3-28】创建一个功能完善的分页组件，实现分页组件的基本使用方法，提供带有背景颜色的分页样式，达成完整的分页功能（包含总数显示、分页大小选择、跳页功能），并支持简洁模式与禁用状态。

```
<template>
  <h3>1.分页的基础用法</h3>
  <!-- 基础分页示例 - 少量页码 -->
  <div class="example-pagination-block">
    <div class="example-demonstration">当页码较少时</div>
    <el-pagination layout="prev, pager, next" :total="50" />
  </div>
  <!--2.基础分页示例 - 多页码 -->
  <div class="example-pagination-block">
```

```
  <div class="example-demonstration">当页码超过7页时</div>
  <el-pagination layout="prev, pager, next" :total="1000" />
</div>

<h3>3.带有背景色的分页</h3>
<!-- 背景色分页 -->
<div>
   <el-pagination background layout="prev, pager, next" :total="1000" />
</div>

<!-- 分页附加功能 -->
<h3>4.分页附加功能</h3>

<!-- 自定义分页大小和总数显示 -->
<div class="example-pagination-block">
  <div class="example-demonstration">自定义分页配置</div>
  <!--
 总数据条数：total
 可选的分页大小：page-sizes
  默认分页大小:page-size
  当前页码:current-page
  布局组件:layout
  背景色:background
  分页大小改变事件：@size-change
  页码改变事件：@current-change
  -->
  <el-pagination
    :total="365"
    :page-sizes="[5, 10, 15, 20]"
    :page-size="10"
    :current-page="currentPage"
    layout="total, sizes, prev, pager, next, jumper"
```

```
    background
    @size-change="onPageSizeChange"
    @current-change="onPageChange"
  />
</div>

<!-- 简洁分页样式 -->
<div class="example-pagination-block">
  <div class="example-demonstration">简洁分页样式</div>
  <!--
  simple：简洁模式

  -->
  <el-pagination
    :total="200"
    :page-size="20"
    layout="prev, pager, next"
    simple
    @current-change="onPageChange"
  />
</div>
<!--
禁用状态分页
disabled：禁用状态
  -->
<div class="example-pagination-block">
  <div class="example-demonstration">禁用状态分页 </div>
  <el-pagination
    :total="100"
    disabled
    layout="prev, pager, next"
  />
```

```
  </div>
</template>

<script>
export default {
  data() {
    return {
      currentPage: 1  // 当前页码
    }
  },
  methods: {
    // 处理分页大小改变
    onPageSizeChange(newSize) {
      console.log('每页显示数量改为:', newSize)
      // 重新加载数据，重置到第一页
      this.currentPage = 1
      this.loadData()
    },
    // 处理页码改变
    onPageChange(newPage) {
      console.log('切换到第', newPage, '页')
      this.currentPage = newPage
      this.loadData()
    },
    // 模拟数据加载
    loadData() {
      console.log('加载数据...', {
        page: this.currentPage,
        pageSize: 10
      })
      // 这里可以调用API获取数据
    }
```

```
  }
}
</script>

<style scoped>
.example-pagination-block + .example-pagination-block {
  margin-top: 10px;
}
.example-pagination-block .example-demonstration {
  margin-bottom: 16px;
}
</style>
```

综上代码所述，呈现如下效果，如图3-28所示。



图3-28 Pagination 分页案例

在上述代码中，Element UI分页组件的功能，通过案例展示了分页的基础用法、样式变体和高级功能，包括普通分页、背景色分页、自定义分页配置、简洁模式和禁用状态。

本章小结

本章围绕Element Plus组件库，系统梳理了从安装、引入到核心组件实战的完整流程。首先对比Element Plus与Element UI的技术差异，明确Vue3项目优先选用Element Plus。随后以“基础→布局→表单→数据展示”为主线，结合服装类专业场景，深度演示Button、Container、Space、Table、Pagination、Cascader等20余个组件的常用属性与插槽用法，并融入主题定制、响应式、国际化等进阶特性。通过大量可复制、可运行的示例，读者能够快速搭建美观、统一、易维护的中后台页面，为后续路由、状态管理及接口联调奠定坚实基础。

练习与实践

简答题

（1）Element Plus与Element UI在框架依赖与开发语言上有何本质区别？

（2）简述按需引入相比全局完整引入带来的两点主要收益。

（3）说明el-button组件中type、plain、round三个属性分别控制什么视觉效果。

（4）概括el-container布局组件对子元素排列方向的判定规则。

（5）写出el-link组件控制“禁用状态”与“下划线显示”的两个对应属性名。

（6）概述el-text组件实现“超出省略”与“多行截断”时需使用的两个属性。

（7）总结el-space组件中size与alignment各自可接受的合法取值。

（8）概述el-table组件实现“斑马纹”“边框”“行状态色”需要配置的三个属性或插槽。

（9）说明el-pagination组件中layout属性的书写格式及三项常用关键字。

（10）概述el-cascader开启“多选”“懒加载”“搜索”功能时分别需配置的属性或钩子。

实践题

（1）完成“时间转换案例”：

①使用Element Plus的el-card、el-form、el-date-picker、el-button、el-result等组件；

②实现“输入任意时间→显示格式化后的年月日－时分秒、时间戳、相对时间（x分钟前）”三项结果；

③在原功能基础上做页面美化：统一主题色、添加过渡动画、响应式居中布局，并给出完整代码与运行截图。

（2）完成“服装分类穿梭框”：

①基于el-transfer组件，左侧为“待选服装分类”（含男装、女装、童装及其子类，共不少于20项，允许部分置灰），右侧为“已选分类”；

②支持关键字搜索、全选/反选、一键清空、已选数量实时统计；

③将最终选中数据以JSON形式会显示在el-input type=“textarea”中；

④给出完整Vue3+Element Plus代码（含模拟数据）与运行效果截图。

第 4 章

UI 组件库高级应用



本章导读

在前面第3章所讲解内容的基础之上，本章主要研究的是UI组件的高级应用。这里所涉及的高级应用主要包括以下几个重要的部分。首先是Vue Router路由的相关应用，它在整个UI组件的应用体系中起着非常关键的导航作用；其次是Axios网络请求的应用，通过它能够实现与服务器之间高效的数据交互；最后是Vuex状态管理的应用，这对于管理应用程序中的复杂状态有着极大的帮助。

学习目标

1.熟练运用Vue Router配置路由路径（Routes）、实现页面组件间的导航跳转（声明式与编程式）、管理路由参数、嵌套路由以及利用导航守卫控制页面访问权限，构建流畅的单页面应用程序。

2.熟练使用Axios发起GET、POST等常见HTTP请求，并与后端API进行数据交互。

3.熟练运用Vuex的核心概念，并能在中大型项目中合理使用Vuex来集中管理跨组件的共享状态，确保状态变化的可预测性和可维护性。

思政目标及设计建议

将UI技术学习与服装类专业相结合，并融入思政教育，是一个非常有创意且符合“课程思政”理念的方向，本章思政目标及设计建议如表4-1所示。

表4-1　第4章思政目标及设计建议

思政目标	思政元素及融入
树立文化认同与传承意识	项目启动阶段，要求学生调研并选取一项中国传统服饰工艺（如扎染、云锦、苗绣等），梳理其历史渊源、纹样符号与当代价值
培养绿色设计与可持续理念	在组件库搭建环节，引入“低碳前端”概念：统计并优化打包体积、减少冗余依赖、使用轻量化图标字体，模拟服装产业“零浪费裁剪”理念
培育协同共享与集体主义精神	采用“开源裁缝铺”模式：每组开源自己开发的Vue组件，并为他组提交issue与PR。引导学生在贡献与回馈中体会“人人为我，我为人人”的集体主义价值，塑造开放协作、共享共赢的职业氛围。

4.1 Vue Router路由



4.1.1 Vue Router概述

Vue Router是Vue.js官方提供的路由管理库，作为Vue.js生态系统的核心组成部分之一，它与Vue.js框架深度集成，通过Vue Router，开发者可以轻松实现前端路由功能，构建现代化的单页面应用(SPA)，它不仅提供了简洁明了的API设计，还支持动态路由匹配、嵌套路由、路由参数传递等高级功能，大大简化了基于Vue.js开发复杂单页应用的难度。

4.1.2 Vue Router安装

1.对于一个现有的使用JavaScript包管理器的项目，从npm registry中安装Vue Router。

```
npm install vue-router@4
```

2.启动一个新项目，你可能会发现使用create-vue这个脚手架工具更容易，它能创建一个基于Vite的项目，并包含加入Vue Router的选项：

```
npm create vue@latest
```

3.使用包管理器的项目通常会使用ES模块来访问Vue Router

```
Import { createRouter } from 'vue-router'
```

4.1.3 Vue Router入门

Vue Router是Vue官方提供的客户端路由解决方案，客户端路由的主要功能是在单页应用（SPA）中，将浏览器的URL与用户所看到的内容紧密绑定，当用户在应用内浏览不同页面时，URL会相应更新，而页面无需从服务器重新加载，Vue Router基于Vue的组件系统构建，通过配置路由，可以明确指示Vue Router为每个URL路径展示相应的组件。

【例4-1】创建一个简洁的Vue Router示例，演示如何配置路由并实现页面间的切换。

1.创建App.vue页面，编写Vue Router路由入门案例

```
<template>
  <!-- 页面标题 -->
  <h1>Vue Router 入门案例</h1>
  <!-- 显示当前路由的完整路径 -->
```

```
    <!-- $route是Vue Router提供的路由信息对象，fullPath属性包含查询参数和hash的完整路径 -->
    <p>
      当前的路由Path为:{{$route.fullPath}}
    </p>
    <!-- 导航区域 -->
    <nav>
      <!-- RouterLink是Vue Router提供的导航组件，用于声明式导航 -->
      <!-- to属性指定目标路由路径 -->
      <RouterLink to="/">路由到Home页面：Go to Home</RouterLink><br/>
      <RouterLink to="/about">路由到About页面：Go to About</RouterLink>
    </nav>
    <!-- 主要内容区域 -->
    <!-- RouterView是Vue Router提供的路由出口组件 -->
    <!-- 根据当前路由动态渲染对应的组件内容 -->
    <main>
      <RouterView/>
    </main>
  </template>
```

在上述代码中，在这个template中使用了两个由Vue Router提供的组件：RouterLink和RouterView，不同于常规的<a>标签，我们使用组件RouterLink来创建链接，这使得Vue Router能够在不重新加载页面的情况下改变URL，处理URL的生成、编码和其他功能，它不一定要在App.vue中，你可以把它放在任何地方，但它需要在某处被导入，否则Vue Router就不会渲染任何东西。

2.编写main.js页面，引入Vue Router路由

```
// 从Vue框架中导入createApp函数，用于创建Vue应用实例
import { createApp } from 'vue'
// 从router.js文件中导入路由配置实例
// 注意：这里导入的router应该是一个已经配置好的Vue Router实例
import router from './router'
// 导入根组件App.vue
import App from './App.vue'
// 创建Vue应用实例，以App组件作为根组件
// 使用.use(router)安装Vue Router插件
```

```
// 使用 .mount('#app') 将应用挂载到 HTML 中 id 为 "app" 的 DOM 元素上
createApp(App)
    .use(router)  // 注册路由插件，使整个应用支持路由功能
    .mount('#app') // 将 Vue 应用挂载到页面，启动应用
```

3. 编写 HomeView.vue 的页面

```
<script>
// 定义组件的 JavaScript 逻辑部分
export default {
  methods: {
    // 定义名为 goToAbout 的方法，用于处理导航逻辑
    goToAbout() {
      // 使用 Vue Router 的编程式导航
      // this.$router.push() 方法会将用户导航到指定的路由
      // 这里导航到 '/about' 路径对应的页面
      this.$router.push('/about')
    },
  },
}
</script>
<template>
  <!-- 组件的模板部分，定义页面的 HTML 结构 -->

  <!-- 页面标题 -->
  <h2>HomeView</h2>

  <!-- 按钮元素，点击时触发 goToAbout 方法 -->
  <!-- @click 是 Vue 的事件绑定语法，当按钮被点击时调用 goToAbout 方法 -->
  <button @click="goToAbout">Go to About</button>
</template>
```

在上述代码中，实现了一个简单的 Vue.js 首页组件，具有编程式导航功能，组件模板中显示 HomeView 标题和一个导航按钮，当用户点击 Go to About 按钮时，会触发 goToAbout 方法，通过 Vue Router 的 $router.push(“/about”) 实现向关于页面的跳转，这展示了 Vue Router 编程式导航的典型用

法，即在JavaScript方法中控制路由跳转，而不是使用声明式的<RouterLink>组件。

4.编写AboutView.vue的页面

```
<template>
  <!-- 关于页面标题 -->
  <h2>AboutView</h2>

  <!-- 搜索输入框 -->
  <!-- v-model.trim="search" 实现双向数据绑定，并自动去除首尾空格 -->
  <!-- maxlength="20" 限制输入最大长度为20个字符 -->
  <label>
    Search: <input v-model.trim="search" maxlength="20">
  </label>
</template>
<script setup>
// 使用Vue 3的<script setup>语法，这是Composition API的编译时语法糖
import { computed } from 'vue'
import { useRoute, useRouter } from 'vue-router'
// 使用Vue Router的Composition API函数
// useRouter()返回路由实例，用于编程式导航
// useRoute()返回当前路由对象，包含路径、参数、查询等信息
const router = useRouter()
const route = useRoute()
// 创建一个计算属性，用于双向绑定搜索参数
// computed接受一个包含get和set方法的对象
const search = computed({
  // getter：从当前路由的查询参数中获取search值
  // route.query.search获取URL中的search查询参数
  // ?? '' 使用空值合并运算符，如果search为null或undefined则返回空字符串
  get() {
    return route.query.search ?? ''
  },
  // setter：当search值改变时更新URL的查询参数
```

```
  // 使用 router.replace() 更新 URL，不会添加新的历史记录
  // { query: { search } } 设置查询参数，会生成类似 ?search=xxx 的 URL
  set(search) {
    router.replace({ query: { search } })
  }
})
</script>
```

在上述代码中，实现了一个具有搜索功能的关于页面，通过 Vue 3 的 Composition API 将输入框与 URL 查询参数进行双向绑定，当用户在输入框中输入内容时，会自动更新 URL 中的 search 查询参数；反之，当 URL 中包含 search 参数时，输入框也会显示对应的值。

5. 编写 router.js 页面，路由到不同的页面

```
// 从 vue-router 库中导入创建内存历史记录和路由器的函数
import { createMemoryHistory, createRouter } from 'vue-router'

// 导入页面组件
// HomeView 组件作为首页
import HomeView from '.components/helloworld/HomeView.vue'
// AboutView 组件作为关于页面
import AboutView from '.components/helloworld/AboutView.vue'

// 定义路由配置数组
// 每个路由对象包含路径和对应的组件
const routes = [
    { path: '/', component: HomeView },       // 根路径对应 HomeView 组件
    { path: '/about', component: AboutView }, // /about 路径对应 AboutView 组件
]

// 创建路由实例
const router = createRouter({
    // 使用内存历史记录模式
    // createMemoryHistory() 不会操作浏览器的真实 URL，适合测试或非浏览器环境
    history: createMemoryHistory(),
```

```
    // 传入路由配置
    routes,
})

// 导出路由实例，供main.js或其他文件使用
export default router
```

在上述代码中，配置并导出了一个Vue Router实例，该实例使用内存历史记录模式（create Memory History），定义了两个路由规则：根路径”/”对应HomeView首页组件，”/about”路径对应AboutView关于页面组件。这种配置特别适用于测试环境、服务器端渲染或非浏览器环境，因为它不会操作浏览器的真实URL，而是在内存中管理路由状态。

综上代码所述，呈现如下效果，如图4-1所示。



图4-1 Vue router入门案例

4.1.4 Vue Router使用

带参数的动态路由匹配需要将给定匹配模式的路由映射到同一个组件，在Vue Router中，我们可以在路径中使用一个动态字段来实现，称之为路径参数，创建一个路由时，可以选择给路由一个name，叫作路由名称。

【例4-2】创建一个简洁的Vue Router示例，演示如何配置路由并实现页面间的切换。

1.创建App.vue页面，编写Vue Router动态路由的案例。

```
<template>
  <!-- 路由导航链接 -->
  <!-- 首页链接 -->
```

```
  <router-link to="/" style="margin-left: 30px;">首页</router-link>
  <!-- 关于页面链接 -->
  <router-link to="/about" style="margin-left: 30px;">关于页面</router-link>
  <!-- 时尚品牌页面链接 -->
  <router-link to="/news" style="margin-left: 30px;">时尚品牌页面</router-link>

  <!-- 路由视图组件 -->
  <!-- 这是路由内容的显示区域，根据当前路由路径动态渲染对应的组件 -->
  <router-view></router-view>
</template>

<style>
/* 应用根组件的样式 */
#app {
  /* 字体设置 */
  font-family: Avenir, Helvetica, Arial, sans-serif;
  /* 字体抗锯齿渲染 */
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  /* 文本居中 */
  text-align: center;
  /* 文字颜色 */
  color: #2c3e50;
  /* 顶部外边距 */
  margin-top: 60px;
}
</style>
```

编写 main.js 页面，引入 Vue Router 路由。

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
//使用use明确路由
```

```
createApp(App).use(router).mount('#app')
```

2.编写HomeView.vue的页面。

```
<template>
    <h1>首页 HomeView</h1>
</template>
```

3.编写AboutView.vue的页面。

```
<template>
    <h1>关于页面 AboutView</h1>
</template>
```

4.编写brandView.vue的页面。

```
<template>
    <!-- 服装品牌列表 -->
    <ul>
        <!-- 香奈儿品牌链接 -->
        <li><router-link to="/branddetails/香奈儿">香奈儿</router-link></li>
        <!-- 古驰品牌链接 -->
        <li><router-link to="/branddetails/古驰">古驰</router-link></li>
        <!-- 路易威登品牌链接 -->
        <li><router-link to="/branddetails/路易威登">路易威登</router-link></li>
    </ul>
</template>

<script>
// 脚本部分 - 当前为空，可添加品牌相关的业务逻辑
</script>

<style>
    /* 去除列表默认样式 */
    ul{
        list-style-type: none;
    }
</style>
```

在上述代码中，实现了一个品牌导航列表的界面层，通过Vue Router的路由链接机制构建了三个奢侈品品牌的详情页入口。采用语义化的模板结构，每个列表项都配置了指向品牌详情页面的路由路径，并利用动态参数传递品牌标识符。

5.编写BrandDetailsView.vue的页面。

```
<template>
    <h1>关于页面BrandDetailsView：{{$route.params.name }}</h1>
</template>
```

6.编写router.js页面，路由到不同的页面。

```
import { createRouter, createWebHashHistory } from "vue-router";
import HomeView from "../components/routerdemo/HomeView.vue"; // 全局执行，一般只有首页会放在全局执行
// 路由配置数组 - 定义所有页面的路由信息
const routes = [
    {
        path: "/", // 根路径
        name: "home", // 路由名称
        component: HomeView // 首页组件 - 全局导入
    },
    {
        path: "/about", // 关于页面路径
        name: "about", // 路由名称
        component: () => import('../components/routerdemo/AboutView.vue') // 关于页面组件 - 懒加载
    },
    {
        path: "/news", // 新闻/品牌页面路径
        name: "news", // 路由名称
        component: () => import('../components/routerdemo/brandView.vue') // 品牌列表页面组件 - 懒加载
    },
    {
        path: "/branddetails/:name", // 品牌详情页面路径，包含动态参数
        name: "branddetails", // 路由名称
         component: () => import('../components/routerdemo/BrandDetailsView.vue') // 品牌详情页面组件 - 懒
```

```
加载
        }
    ]
    // 创建路由器实例
    const router = createRouter({
        history: createWebHashHistory(), // 使用哈希路由模式
        routes // 路由配置
    })
    // 导出路由器实例
    export default router;
```

在上述代码中，Vue.js路由配置采用哈希路由模式构建了完整的单页面应用导航架构。系统包含四个核心路由节点：根路径指向全局导入的首页组件以确保首屏加载性能；关于页面和品牌列表页面通过动态导入实现按需加载；特别值得注意的是品牌详情页采用参数化路由设计，通过：name占位符实现品牌维度的动态内容渲染。

综上代码所述，呈现如下效果，如图4-2所示。



图4-2 Vue router动态路由匹配案例

4.1.5 动态路由

动态路由的主要语法规则如下：

1.动态路由通过路径参数（如:id）实现，例如，路径/xxx/:id可以匹配/xxx/123，其中id是动态参数，Vue Router的路由匹配机制借助路径规则、动态参数、自定义正则表达式等功能，具备了灵活且强大的URL匹配能力。开发者能够依据自身需求，精准把控路由的匹配逻辑，以满足各类复杂的单页面应用场景。

2.动态参数和正则表达式来匹配路径：例如，carts/:cartId(\\d+)只匹配数字类型的cartId

```
const routes = [
  { path: '/carts/:cartId(\\d+)' }, // 仅匹配数字
  { path: '/products/:productName' } // 匹配任意字符串
];
```

3.可重复参数：使用*或+修饰符标记参数为可重复，例如，chapters/:sections+匹配/one/two/three，并将其解析为数组

```
const routes = [
  { path: '/chapters/:sections+' } // 匹配 /one, /one/two 等
];
```

4.可选参数：使用?修饰符标记参数为可选，例如，users/:userId?匹配/users和/users/123。

```
const routes = [
  { path: '/users/:userId?' } // 匹配 /users 和 /users/123
];
```

5.路由匹配配置选项：

（1）大小写敏感(sensitive)默认情况下，路径匹配不区分大小写，通过设置sensitive:true，可以使匹配区分大小写。

（2）严格模式(strict)默认情况下，路径匹配忽略尾部的斜杠。通过设置strict:true，可以强制要求路径严格匹配尾部斜杠。

```
const router = createRouter({
  routes: [
    { path: '/users/:id', sensitive: true, strict: true }
  ]
});
```

4.1.6 嵌套路由

在Vue.js里，Vue Router具备强大的嵌套路由功能，允许在父路由之下定义多个子路由。这种设计极为适合构建拥有复杂层次结构的单页应用（SPA），并且能够借助<router-view>标签达成组件的动态渲染。在此情形下，URL的片段通常与特定的嵌套组件结构相对应。

若要使用<router-view>渲染子路由，在父组件中，必须包含<router-view>标签，子路由的组件才能够正确渲染。子路由可包含动态参数，比如:id，还能通过$route.params获取参数值。此外，可为父路由设置默认子路由。

【例4-3】创建一个简洁的Vue Router示例，演示如何配置路由并实现页面间的切换。

1.编写HomeView.vue文件，实现主页页面的布局。

```
<template>
    <h1>首页 HomeView</h1>
</template>
```

```
<script>
</script>
<style>
</style>
```

2.编写BrandView.vue文件，实现时尚品牌页面的布局。

```
<template>
    <!-- 时尚品牌列表 -->
    <ul>
        <!-- 香奈儿品牌链接 -->
        <li><router-link to="/branddetails/香奈儿">香奈儿</router-link></li>
        <!-- 古驰品牌链接 -->
        <li><router-link to="/branddetails/古驰 ">古驰 </router-link></li>
        <!-- 路易威登品牌链接 -->
        <li><router-link to="/branddetails/路易威登 ">路易威登 </router-link></li>
    </ul>
</template>

<script>
// 脚本部分 - 当前为空，可添加品牌相关的业务逻辑
</script>

<style>
    /* 去除列表默认样式 */
    ul{
        list-style-type: none;
    }
</style>
```

3.编写BrandDetailsView.vue文件，实现时尚品牌详细页面的布局。

```
<template>
    <h1>关于页面 BrandDetailsView：{{$route.params.name }}</h1>
</template>
```

4.编写AboutView.vue文件，实现关于页面的布局。

```
<template>
    <div class="about">
        <router-link to="/about/us">关于时尚品牌</router-link>|
        <router-link to="/about/info">关于时尚品牌的信息</router-link>
        <router-view></router-view>
    </div>
</template>

<script>
</script>

<style>
</style>
```

编写 AboutInfoView.vue 文件，实现关于时尚品牌信息的页面布局。

```
<template>
    <h3>关于时尚品牌</h3>
</template>

<script>
</script>

<style>
</style>
```

编写 AboutUSView.vue 文件，实现关于时尚品牌的页面布局。

```
<template>
    <h3>关于时尚品牌</h3>
</template>

<script>
</script>

<style>
```

```
</style>
```

5.编写index.js文件作为路由切换。

```
import { createRouter, createWebHashHistory } from "vue-router";
import HomeView from "../components/views/HomeView.vue";

// 路由配置数组
const routes = [
    {
        path: "/",                  // 路径
        name: "home",               // 路由名称
        component: HomeView         // 直接导入的组件（同步加载）
    },
    {
        path: "/about",             // 父级路径
        redirect: "/about/us",      // 重定向到子路由
        name: "about",              // 路由名称
        // 懒加载组件（异步加载，提高首屏加载性能）
        component: () => import('../components/views/AboutView.vue'),
        children: [                 // 嵌套路由（子路由）
            {
                path: "us",         // 二级路径，不要加"/"
                // 懒加载子组件
                component: () => import('../components/views/AboutSon/AboutUSView.vue'),
            },
            {
                path: "info",       // 二级路径
                // 懒加载子组件
                component: () => import('../components/views/AboutSon/AboutInfoView.vue'),
            }
        ]
    },
    {
```

```
        path: "/brand",              // 品牌页路径
        name: "brand",               // 路由名称
        // 懒加载组件（异步执行，按需加载）
        component: () => import('../components/views/BrandView.vue')
    },
    {
        path: "/branddetails/:name", // 动态路由，name为参数
        name: "branddetails",        // 路由名称
        // 懒加载品牌详情组件
        component: () => import('../components/views/BrandDetailsView.vue')
    }
]

// 创建路由实例
const router = createRouter({
    history: createWebHashHistory(), // 使用hash模式的路由历史记录
    routes                           // 传入路由配置
})

// 导出路由实例
export default router;
```

在上述代码中，配置了一个Vue.js应用的路由系统，采用hash路由模式，其中特别需要注意二级路由和子路由的配置方式，关于页面（“/about”）通过children属性定义了两个子路由：“us”和“info”，这些二级路径的配置遵循了Vue Router的规范——不使用前导斜杠，因此实际访问路径为“/about/us”和“/about/info”，这种嵌套路由结构使得AboutView组件可以作为布局容器渲染相应的子组件，同时默认重定向到“/about/us”子路由，确保了更好的用户体验和路由组织的清晰性。

6.编写App.vue文件，实现路由链接到其他页面的布局。

```
<template>
  <!-- 路由链接到页面上 -->
  <router-link to="/">首页</router-link>|
  <router-link to="/about">关于页面</router-link>|
  <router-link to="/brand">时尚品牌页面</router-link>
```

```
  <!-- 路由的显示入口 -->
  <router-view></router-view>
</template>
<style>
#app {
  font-family: Avenir, Helvetica, Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-align: center;
  color: #2c3e50;
  margin-top: 60px;
}
</style>
```

综上代码所述，呈现如下效果，如图4-3所示。

首页| 关于页面| 时尚品牌页面
关于时尚品牌| 关于时尚品牌的信息

关于时尚品牌

图4-3 Vue router嵌套路由的案例

4.2 Axios网络请求



4.2.1 Axios概述

Axios是一个基于promise网络请求库，作用于node.js和浏览器中。它是isomorphic的（即同一套代码可以运行在浏览器和node.js中）。在服务端它使用原生node.js http模块，而在客户端（浏览端）则使用XMLHttpRequest。

Axios具有以下显著特点：首先，它支持从浏览器中创建XMLHttpRequest，同时也支持从node.js中发起http请求，这为前后端开发提供了统一的网络请求方式。其次，它支持Promise API，能够利用Promise的链式调用特性，让异步请求的处理更加简洁和清晰。再者，它可以拦截请求和响应，在请求发出前和响应返回后进行一些自定义操作，比如添加统一的请求头、对响应数据进行预处理等。另外，它还能自动转换JSON数据，省去了手动解析和序列化的繁琐步骤。并且，它提供了取消请求的功能，方便在不需要继续请求时及时终止，避免不必要的资源浪费。

4.2.2 Axios安装

1.使用npm进行安装Axios网络请求

```
$ npm install axios
```

2.使用bower进行安装Axios网络请求

```
$ bower install axios
```

3.使用Yarn进行安装Axios网络请求

```
$ yarn add axios
```

4.使用pnpm进行安装Axios网络请求

```
$ pnpm add axios
```

4.2.3 Axios局部网络请求

局部引用Axios网络请求，即于组件内引入Axios网络请求。网络请求可选用Get请求方式或Post请求方式进行访问操作和传递参数的操作，以下将用两个案例进行演示。

【例4-4】开发服装商品信息展示页面，通过应用程序编程接口（API）获取服装数据，并在页面上呈现商品信息，通过局部引入的Axios进行网络请求，采用GET请求方式执行访问操作。

1.编写AxiosDemo.vue页面

```
<template>
  <div class="hello">
    <!-- 显示服装标题 -->
    <h3>{{ clothing.title }}</h3>
    <!-- 条件渲染：如果有描述信息则显示 -->
    <p v-if="clothing.description">{{ clothing.description }}</p>
    <!-- 条件渲染：如果有价格信息则显示 -->
    <p v-if="clothing.price">价格：{{ clothing.price }}</p>
    <!-- 条件渲染：如果有材质信息则显示 -->
```

```
    <p v-if="clothing.material">材质：{{ clothing.material }}</p>
  </div>
</template>

<script>
// 导入 axios 库用于发送 HTTP 请求
import axios from "axios"

export default {
  // 组件名称
  name: 'AxiosDemo',
  // 组件数据
  data() {
    return {
      // 存储服装信息的对象
      clothing: {}
    }
  },
  // 生命周期钩子：组件挂载后执行
  mounted() {
    // 使用 axios 发送 GET 请求
    axios({
      method: "get", // 请求方法为 GET
      url: "https://fakestoreapi.com/products/1" // 请求的 API 地址
    }).then(res => {
      // 请求成功回调
      if (res.data) {
        // 将 API 返回的数据映射到 clothing 对象
        this.clothing = {
          title: res.data.title || "服装名称", // 标题，如果没有则使用默认值
          description: res.data.description || "服装描述", // 描述，如果没有则使用默认值
          price: res.data.price ? `¥${res.data.price}` : "未知", // 价格，如果有则添加人民币符号
```

```
          material: "未知材质" // 材质信息（API未提供，使用默认值）
        }
      }
      // 在控制台输出服装信息，用于调试
      console.log(this.clothing)
    }).catch(error => {
      // 请求失败回调，输出错误信息
      console.error("获取服装数据失败：", error)
    })
  }
}
</script>

<style scoped>
/* 组件样式区域 */
.hello {
  padding: 20px; /* 内边距 */
  font-family: Arial, sans-serif; /* 字体设置 */
}

/* 标题样式 */
h3 {
  color: #42b983; /* 主题色 */
  margin-bottom: 10px; /* 底部外边距 */
}

/* 段落样式 */
p {
  margin: 5px 0; /* 上下外边距 */
  color: #333; /* 文字颜色 */
}
</style>
```

在上述代码中，基于Vue.js和Axios的服装商品信息展示组件，通过GET请求从Fakestore API获取商品数据，在页面中展示服装标题、描述、价格和材质信息。组件在挂载后自动发起请求，将返回的数据映射到本地clothing对象，并使用条件渲染确保数据不存在时不显示空内容，同时包含完整的错误处理机制和简洁的样式设计。

2.编写App.vue页面

```
<template>
  <!-- 显示 Vue logo 图片 -->
  <img alt="Vue logo" src="./assets/logo.png" />
  <!-- 引入 AxiosDemo 组件并传递 msg 属性 -->
  <AxiosDemo msg="Hello Vue 3 + Vite" />
</template>

<script setup>
  // 使用 Vue 3 的 script setup 语法
  // 导入 AxiosDemo 组件
  import AxiosDemo from './components/AxiosDemo.vue'
</script>

<style>
/* 应用根容器样式 */
#app {
  font-family: Avenir, Helvetica, Arial, sans-serif; /* 字体设置 */
  -webkit-font-smoothing: antialiased; /* Webkit 浏览器 */
  -moz-osx-font-smoothing: grayscale; /* Firefox 浏览器抗锯齿 */
  text-align: center; /* 文本居中 */
  color: #2c3e50; /* 文字颜色 */
  margin-top: 60px; /* 顶部外边距 */
}
</style>
```

综上代码所述，呈现如下效果，如图4-4所示。

【例4-5】开发服装商品信息展示页面，通过应用程序编程接口（API）获取服装数据，并在页面上呈现商品信息，通过局部引入的Axios进行网络请求，采用POST请求方式执行访问操作。



图4-4　Axios网络请求Get请求案例

1.编写AxiosDemo02.vue页面

```
<template>
  <div class="hello">
    <!-- 显示服装标题 -->
    <h3>{{ clothing.title }}</h3>
    <!-- 条件渲染：如果有描述信息则显示 -->
    <p v-if="clothing.description">{{ clothing.description }}</p>
    <!-- 条件渲染：如果有价格信息则显示 -->
    <p v-if="clothing.price">价格: {{ clothing.price }}</p>
    <!-- 条件渲染：如果有材质信息则显示 -->
    <p v-if="clothing.material">材质: {{ clothing.material }}</p>
  </div>
</template>

<script>
// 导入 axios 库用于发送 HTTP 请求
import axios from "axios"

export default {
  // 组件名称
  name: 'AxiosDemo02',
  // 组件数据
  data() {
```

```
    return {
      // 存储服装信息的对象
      clothing: {}
    }
  },
  // 生命周期钩子：组件挂载后执行
  mounted() {
    // 使用 axios 发送 POST 请求
    axios({
      method: "post", // 请求方法为 POST
      url: "https://fakestoreapi.com/products", // 使用支持 POST 的端点
      data: { // POST 请求的数据体
        title: '新款时尚服装',
        price: 199.99,
        description: '高品质时尚服装',
        category: 'clothing'
      }
    }).then(res => {
      // 请求成功回调
      if (res.data) {
        // 将 API 返回的数据映射到 clothing 对象
        this.clothing = {
          title: res.data.title || "服装名称", // 标题，如果没有则使用默认值
          description: res.data.description || "服装描述", // 描述，如果没有则使用默认值
          price: res.data.price ? `¥${res.data.price}` : "未知", // 价格，如果有则添加人民币符号
          material: "未知材质" // 材质信息（API 未提供，使用默认值）
        }
      }
      // 在控制台输出服装信息，用于调试
      console.log(this.clothing)
    }).catch(error => {
      // 请求失败回调，输出错误信息
```

```
      console.error("获取服装数据失败：", error)
    })
  }
}
</script>

<style scoped>
/* 组件样式区域 */
.hello {
  padding: 20px; /* 内边距 */
  font-family: Arial, sans-serif; /* 字体设置 */
}

/* 标题样式 */
h3 {
  color: #42b983; /* 主题色 */
  margin-bottom: 10px; /* 底部外边距 */
}

/* 段落样式 */
p {
  margin: 5px 0; /* 上下外边距 */
  color: #333; /* 文字颜色 */
}
</style>
```

在上述代码中，基于Vue.js和Axios的服装商品展示组件，通过POST请求向Fakestore API提交新款服装数据并获取响应，在页面中展示商品标题、描述、价格和材质信息，组件在挂载时自动发送包含服装详情的POST请求，将返回的数据映射到本地对象并使用条件渲染确保数据完整性，同时包含错误处理和简洁的样式设计。

2.编写App.vue页面

```
<template>
  <!-- 显示Vue logo图片 -->
```

```
  <img alt="Vue logo" src="./assets/logo.png" />
  <!-- 引入 AxiosDemo 组件并传递 msg 属性 -->
  <!-- <AxiosDemo msg="Hello Vue 3 + Vite" /> -->
   <AxiosDemo02 msg="Hello Vue 3 + Vite" />
</template>

<script setup>
  // 使用 Vue 3 的 script setup 语法
  // 导入 AxiosDemo 组件
  // import AxiosDemo from './components/AxiosDemo.vue'
  import AxiosDemo02 from './components/AxiosDemo02.vue'
</script>

<style>
/* 应用根容器样式 */
#app {
  font-family: Avenir, Helvetica, Arial, sans-serif; /* 字体设置 */
  -webkit-font-smoothing: antialiased; /* Webkit 浏览器 */
  -moz-osx-font-smoothing: grayscale; /* Firefox 浏览器 */
  text-align: center; /* 文本居中 */
  color: #2c3e50; /* 文字颜色 */
  margin-top: 60px; /* 顶部外边距 */
}
</style>
```

综上代码所述，呈现如下效果，如图 4-5 所示。

4.2.4 全局引入 Axios 网络请求

如果使用全局引入 Axios 网络请求，就不需要在每一个组件上去引用 Axios 网络请求，如果使用全局引入 Axios 网络请求，必须将局部引入 Axios 网络请求删除掉，否则不可以使用全局的。

【例 4-6】开发服装商品信息展示页面，通过应用程序编程接口（API）获取服装数据，并在页面上呈现商品信息，通过全局引入的 Axios 进行网络请求，采用 GET 请求和 POST 请求方式执行访问操作。



图 4-5　Axios 网络请求 Post 请求案例

1. 编写 AxiosDemo.vue 文件

```
<template>
  <!-- 组件模板结构 -->
  <div class="hello">
    <!-- 显示服装标题 -->
    <h3>{{ clothing.title }}</h3>
    <!-- 条件渲染：当有描述信息时才显示 -->
    <p v-if="clothing.description">{{ clothing.description }}</p>
    <!-- 条件渲染：当有价格信息时才显示 -->
    <p v-if="clothing.price">价格：{{ clothing.price }}</p>
    <!-- 条件渲染：当有材质信息时才显示 -->
    <p v-if="clothing.material">材质：{{ clothing.material }}</p>
  </div>
</template>

<script>
// 导入 axios 库用于 HTTP 请求
```

```
import axios from "axios"
export default {
  name: 'AxiosDemo02', // 组件名称
  // 组件数据定义
  data() {
    return {
      // 存储服装信息的响应式对象
      clothing: {}
    }
  },
  // 生命周期钩子：组件挂载完成后执行
  mounted() {
    // 发送 POST 请求到模拟 API
    axios({
      method: "post", // 请求方法
      url: "https://fakestoreapi.com/products", // API 端点
      data: { // 请求体数据
        title: '新款时尚服装',
        price: 199.99,
        description: '高品质时尚服装',
        category: 'clothing'
      }
    }).then(res => {
      // 请求成功回调
      // 检查响应数据是否存在
      if (res.data) {
        // 处理响应数据，设置默认值
        this.clothing = {
          title: res.data.title || "服装名称", // 标题，若无则使用默认值
          description: res.data.description || "服装描述", // 描述，若无则使用默认值
          price: res.data.price ? `¥${res.data.price}` : "未知", // 价格，格式化显示
          material: "未知材质" // 固定材质值
```

```
        }
      }
      // 在控制台输出处理后的服装数据
      console.log(this.clothing)
    }).catch(error => {
      // 请求失败处理
      // 在控制台输出错误信息
      console.error("获取服装数据失败:", error)
    })
  }
}
</script>
<style scoped>
/* 组件样式区域 – scoped表示样式只作用于当前组件 */
/* 组件容器样式 */
.hello {
  padding: 20px; /* 内边距 */
  font-family: Arial, sans-serif; /* 字体设置 */
}

/* 三级标题样式 */
h3 {
  color: #42b983; /* 主题色 */
  margin-bottom: 10px; /* 底部外边距 */
}

/* 段落通用样式 */
p {
  margin: 5px 0; /* 上下外边距 */
  color: #333; /* 文字颜色 */
}
</style>
```

在上述代码中，Vue组件演示了如何使用局部引入的Axios发送POST请求到模拟API，组件在挂载后向FakeStore API提交新的服装产品数据，然后处理响应结果并在页面中显示服装信息。

2.编写App.vue文件

```
<template>
   <!-- 显示 Vue logo 图片 -->
  <img alt="Vue logo" src="./assets/logo.png" />
  <!-- 引入 AxiosDemo 组件并传递 msg 属性 -->
   <AxiosDemo msg="Hello Vue 3 + Vite" />
</template>
<script setup>
  import AxiosDemo from './components/AxiosDemo.vue'
</script>

<style>
#app {
  font-family: Avenir, Helvetica, Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-align: center;
  color: #2c3e50;
  margin-top: 60px;
}
</style>
```

3.编写main.js文件

```
import { createApp } from 'vue'
import App from './App.vue'
import axios from 'axios'
const app = createApp(App)
//将 axios 挂载到全局
app.config.globalProperties.$axios = axios
app.mount('#app')
```

综上代码所述，呈现如下效果，如图4-5所示。

4.2.5 Axios的HTTP请求封装模块

在现代前端开发中，封装Axios是一种常见的实践，能够提高代码的可维护性、复用性和扩展性。通过封装，可以实现统一的请求管理、错误处理以及更简洁的API调用方式。

【例4-7】开发服装商品信息展示页面，通过应用程序编程接口（API）获取服装数据，利用封装好的Axios实例进行网络请求，实现模块间的调用。

1.编写request.js文件

```
// 导入 axios 和 querystring 模块
import axios from "axios";
import querystring from "querystring"
// 错误处理函数 – 根据HTTP状态码进行错误提示
const errorHandle = (status, info) => {
    switch (status) {
        case 400:
            console.log("语义有误");
            break;
        case 401:
            console.log("服务器认证失败");
            break;
        case 403:
            console.log("服务器拒绝访问");
            break;
        case 404:
            console.log("访问地址错误");
            break;
        case 500:
            console.log("服务器遇到意外了");
            break;
        case 502:
            console.log("服务器无响应");
            break;
        default:
            console.log(info);
```

```
                break;
        }
    }
    // 创建 axios 实例并进行自定义配置
    const instance = axios.create({
        // 网络请求的公共配置
        timeout: 5000 // 请求超时时间设置为5秒
    })
    // 拦截器 – 最常用的功能
    // 请求拦截器 – 在发送请求之前执行
    instance.interceptors.request.use(
        config => {
            // 如果是 POST 请求，将数据转换为 URL 编码格式
            if (config.method === "post") {
                config.data = querystring.stringify(config.data)
            }
            // config 包含了网络请求的所有信息
            return config;
        },
        error => {
            // 对请求错误做处理
            return Promise.reject(error)
        }
    )
    // 响应拦截器 – 在接收到响应之前执行
    instance.interceptors.response.use(
        response => {
            // 如果状态码为200，解析响应数据；否则拒绝
            return response.status === 200 ? Promise.resolve(response) : Promise.reject(response)
        },
        error => {
            // 对响应错误做处理
```

```
            const { response } = error;
            // 调用错误处理函数，传递状态码和错误信息
            if (response) {
                errorHandle(response.status, response.info)
            }
        }
)
// 导出配置好的 axios 实例
export default instance;
```

在上述代码中，创建了一个经过自定义配置的Axios实例，它通过请求拦截器自动将POST请求的数据转换为URL编码格式，并通过响应拦截器实现了统一的错误处理机制，能够根据不同的HTTP状态码提供相应的错误提示，同时设置了5秒的请求超时时间。

2.编写path.js文件

```
// 基础配置对象 – 包含 API 相关配置
const base = {
    // 基础 API 地址 – 使用 Fake Store API 模拟数据
    baseUrl: "https://fakestoreapi.com",
    // 商品详情接口路径 – 获取单个商品信息
    chengpin: "/products/1"
}

// 导出配置对象供其他模块使用
export default base;
```

3.编写index.js文件

```
// 导入 axios 实例和接口路径配置
import axios from "../utils/request.js"
import path from "./path.js"

// API 接口集合对象
const api = {
    // 获取商品详情的方法
    getChengPin() {
```

```
        // 发起 GET 请求，拼接基础URL和商品详情路径
        return axios.get(path.baseUrl + path.chengpin)
    }
}
// 导出 API 接口对象
export default api;
```

在上述代码中，通过封装的方式创建了一个API接口管理模块，它导入预先配置好的Axios实例和路径配置，将获取商品详情的网络请求封装成getChengPin方法，该方法内部自动拼接基础URL和接口路径并返回Promise对象，最终以统一的对象形式导出所有API接口，实现了网络请求的逻辑封装和集中管理。

4.编写HelloWorld.vue文件

```
<template>
    <!-- 模板部分 - 定义组件结构 -->
    <div class="hello">
        <!-- 显示从父组件传递的msg属性 -->
        <h1>{{msg}}</h1>
    </div>
</template>

<script>
    // 导入API接口
    import api from "../api/index"
    // 组件定义
    export default{
        // 组件名称
        name: 'HelloWorld',
        // 定义接收的props属性
        props: {
            // msg属性，类型为字符串
            msg: String
        },
        // 生命周期钩子 - 组件挂载完成后执行
```

```
        mounted() {
            // 调用API获取商品数据
            api.getChengPin().then(res=>{
                // 在控制台输出返回的数据
                console.log(res.data)
            })
        }
    }
</script>
<!-- scoped样式 - 只作用于当前组件 -->
<style scoped>
a {
  color: #42b983;
}
</style>
```

在上述代码中，展示了如何通过封装调用的方式使用 API 接口，在组件挂载完成后调用封装好的 api.getChengPin() 方法来获取商品数据，并将响应结果在控制台输出，体现了前端组件与后端 API 接口的解耦设计，使得网络请求逻辑得以封装复用，组件只需关注数据的使用而不需要处理具体的请求细节。

5. 编写 App.vue 文件

```
<template>
  <!-- 根组件模板 -->
  <!-- Vue logo 图片 -->
  <img alt="Vue logo" src="./assets/logo.png">
  <!-- HelloWorld 子组件，传递 msg 属性 -->
  <HelloWorld msg="Welcome to Your Vue.js App"/>
</template>
<script>
// 导入 HelloWorld 组件
import HelloWorld from './components/HelloWorld.vue'
// 定义并导出根组件
export default {
  // 组件名称
```

```
  name: 'App',
  // 注册子组件
  components: {
    HelloWorld
  }
}
</script>
<style>
/* 应用全局样式 */
#app {
  /* 字体设置 */
  font-family: Avenir, Helvetica, Arial, sans-serif;
  /* 字体抗锯齿优化 */
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  /* 文本居中 */
  text-align: center;
  /* 文字颜色 */
  color: #2c3e50;
  /* 顶部外边距 */
  margin-top: 60px;
}
</style>
```

6.编写main.js文件

```
import { createApp } from 'vue'
import App from './App.vue'
createApp(App).mount('#app')
```

综上代码所述，呈现如下效果，如图4-6所示。

4.2.6 Axios网络请求跨域解决方案

JavaScript采取的是同源策略，同源策略是浏览器的一项安全策略，浏览器只允许JavaScript代码请求和当前所在服务器域名，协议相同的数据接口上数据，这就是同源策略，也就是说，当协议，域



图 4-6　Axios 网络请求 Get 请求案例

名，端口任意一个不相同时，都会产生跨域问题，目前主流的跨域解决方案有两种：第一种是前台解决方案 proxy，第二种是后台解决方案 cors，后台解决方案与后端技术整合的相关内容，将在后续予以详细阐述。

【例 4-8】开发一个用于演示前端跨域问题解决方案的 Vue.js 单页面应用，通过代理配置实现前后端分离开发环境下的跨域请求。

1. 编写 vue.config.js 页面，解决前端开发环境中的跨域问题。

```
const { defineConfig } = require('@vue/cli-service')
module.exports = defineConfig({
 // 转译依赖，将 node_modules 中的依赖包进行 Babel 转译
 transpileDependencies: true,
 // 开发服务器配置
 devServer: {
  // 指定服务器监听的主机地址，'0.0.0.0' 表示监听所有网络接口
  host: '0.0.0.0',
  // 开发服务器端口号
  port: 8080,
  // 启动开发服务器后自动打开浏览器
  open: true,

  // 代理配置，用于解决跨域问题
  proxy: {
   // 代理所有以 '/api' 开头的请求
   '/api': {
    // 目标服务器地址，将请求代理到该地址
    target: 'http://192.168.174.1:8081',
    // 开启跨域，改变请求头中的 Origin 为目标地址
```

```
        changeOrigin: true
        // 例如：前端请求 /api/user 会被代理到 http://192.168.174.1:8081/api/user
      }
    }
  }
})
```

2.编写HelloWorld.vue页面

```
<template>
    <!-- 模板部分：定义组件的 HTML 结构 -->
    <div class="hello">
        <!-- 标题 -->
        <h3>跨域解决方案</h3>
    </div>
</template>
<script>
    // 导入 axios 库，用于发送 HTTP 请求
    import axios from "axios"

    // 导出 Vue 组件定义
    export default{
        // 组件名称，用于在 Vue DevTools 中显示和组件递归调用
        name: 'HelloWorld',

        // 生命周期钩子：组件挂载到 DOM 后执行
        mounted() {
            // 使用 axios 发送 GET 请求
            axios({
                method:"get",          // 请求方法：GET
                url: "/api/hello.html"  // 请求地址，会被开发服务器的代理处理
            }).then(res=>{
                // 请求成功后的回调函数
                // res 是响应对象，res.data 包含服务器返回的数据
```

```
                console.log(res.data)  // 在控制台打印响应数据
            })
            // 注意：这里可以添加 .catch() 来处理请求错误的情况
        }
    }
</script>
<style scoped>
/* 使用 scoped 属性，表示这些样式只作用于当前组件 */
a {
  /* 链接颜色设置为 Vue 的主题绿色 */
  color: #42b983;
}
</style>
```

在上述代码中，组件在页面加载完成后，通过axios库向”/api/hello.html”接口发起GET请求，并利用Vue CLI开发服务器的代理配置解决跨域问题。

综上代码所述，呈现如下效果，如图4-7所示。



图4-7　Axios网络请求跨域解决方案案例

4.3 Vuex状态管理



4.3.1 Vuex状态管理概述

Vuex是一个专为Vue.js应用程序开发的状态管理模式和库。它提供了一个集中式的存储管理所有组件的状态，并确保状态以可预测的方式发生变化，简单来说，状态管理可以理解成为更方便的管理组件之

间的数据交互，提供了一个集中式的管理方案，任何组件都可以按照指定的方式进行读取和改变数据。

Vuex采用单一状态树，用一个对象就包含了全部的应用层级状态，并作为唯一数据源而存在。这意味着，任何一个状态的变化，都是从一个单一的树结构中进行操作的，使得状态的变化更加可追踪和可调试。同时，Vuex也集成到了Vue的官方调试工具devtools extension，提供了诸如零配置的time-travel调试、状态快照导入导出等高级调试功能。

4.3.2 Vuex状态管理的核心概念

1.State：是专门用于存储数据的地方，在整个应用程序里，所有的组件都能够对这些存储的数据进行访问，以此来获取所需的信息。

2.Getter：主要用于从State中派生计算属性，它可以对State中的数据进行进一步的处理和计算，从而得到我们所需要的特定属性值。

3.Mutation：是唯一一种可以同步修改状态的方法，通过它我们能够确保状态的修改是按照预期的顺序和逻辑进行的。

4.Action：主要用于提交Mutation，并且它还支持异步操作，这使得我们在处理一些需要异步处理的任务时能够更加灵活和方便。

以下是一个表示“单向数据流”理念的简单示意，如图4-8所示。

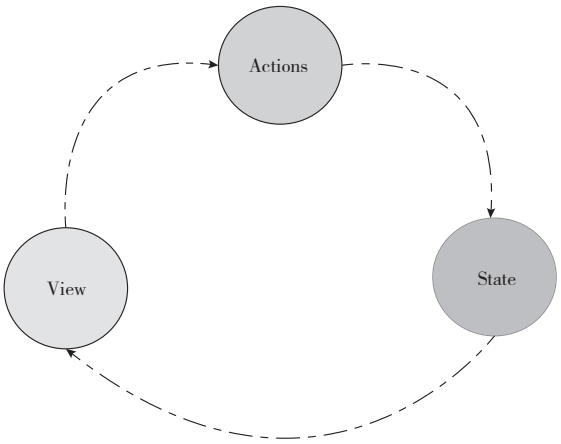


图4-8 单向数据流

4.3.3 Vuex状态管理的使用场景

Vuex可以帮助管理共享状态，并附带了很多的概念和框架，这需要对短期和长期效益进行权衡，如果不打算开发大型单页应用，使用Vuex可能是烦琐冗余的，最好不要使用Vuex，一个简单的store模式就足够所需了。但是，如果需要构建一个中大型单页应用，很可能会考虑如何更好地在组件外部管理状态，Vuex将会成为自然而然的选择。

4.3.4 Vuex状态管理的用法

1.在项目安装vuex，可以通过npm进行安装。

```
npm install vuex
```

2.State读取数据

在Vuex中，state是存储应用状态的核心对象，可以直接通过store.state.xx获取状态值，但需要注意其使用场景和限制。

【例4-9】开发一个简单的单页面应用来演示Vuex在组件状态共享中的应用。

（1）编写index.js页面，创建并导出一个Vuex状态管理仓库。

```
import { createStore } from "vuex";
//Vuex的核心作用是管理组件之间的状态的
export default createStore({
    //所有的状态都放在这里
    state: {
        clothes: "时尚T恤"
    }
})
```

（2）编写HelloWorld.vue页面，在Vue组件中访问Vuex状态的两种不同方式。

```
<template>
  <!-- 组件模板部分 -->
  <h1>服装展示页面</h1>
  <div class="hello">
    <!-- 直接通过 $store 访问 Vuex 状态 -->
    <p>服装名称 = {{$store.state.clothes}}</p>
    <!-- 分隔线 -->
    <hr/>
    <!-- 通过计算属性访问 Vuex 状态 -->
    <p>{{clothes}}</p>
  </div>
</template>
<script>
// 导入 Vuex 的 mapState 辅助函数
import { mapState } from "vuex"
```

```
// 导出组件选项
export default{
 // 组件名称
 name:"HelloWorld",
 // 组件接收的 props
 props:{
  msg:String
 },
 // 计算属性 – 专门用于读取 Vuex 的数据
 computed:{
  // 使用 mapState 将 Vuex 的 state 映射到组件的计算属性
  // 这样可以直接在模板中使用 clothes 而不是 $store.state.clothes
  ...mapState(["clothes"])
 }
}
</script>
```

在上述代码中，演示了从Vuex store中获取和显示共享状态数据，使用$store.state.clothes直接访问全局状态，使用mapState将状态映射为计算属性，展示组件与Vuex store的集成模式。

（3）编写App.vue页面

```
<script setup>
// 使用 Vue 3 的 <script setup> 语法，这是单文件组件的编译时语法糖
// 查看 https://v3.vuejs.org/api/sfc-script-setup.html#sfc-script-setup
import HelloWorld from './components/HelloWorld.vue'
</script>
<template>
 <!-- 页面根模板 -->
 <img alt="Vue logo" src="./assets/logo.png" />
 <!-- 显示 Vuex 状态管理中的服装名称 -->
 <p>服装名称 = {{$store.state.clothes}}</p>
 <!-- 引入 HelloWorld 子组件 -->
 <HelloWorld/>
</template>
```

```
<style>
/* 应用根样式 */
#app {
  /* 字体设置 */
  font-family: Avenir, Helvetica, Arial, sans-serif;
  /* 字体抗锯齿渲染 */
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  /* 文本居中 */
  text-align: center;
  /* 文字颜色 */
  color: #2c3e50;
  /* 顶部外边距 */
  margin-top: 60px;
}
</style>
```

综上代码所述，呈现如下效果，如图4-9所示。



图4-9　Vuex状态管理State案例

3.Getter管理数据

在Vuex中，getter是从state中派生状态的计算属性，可以通过computed属性轻松访问这些值。

【例4-10】开发一个基于Vue 3和Vuex的服装商城库存管理系统，用于展示服装库存状态。

（1）编写index.js页面，创建一个Vuex状态管理仓库，用于管理服装商城应用的状态数据。

```
import { createStore } from "vuex";
// Vuex的核心作用是管理组件之间的状态 – 服装商城状态管理
export default createStore({
    // 所有的状态都放在这里 – 服装商城数据
    state: {
        clothesStock: 0  // 服装库存数量
    },
    getters: {
        getClothesStock(state) {  // 获取服装库存信息
            return state.clothesStock > 0 ? state.clothesStock : "库存不足，请及时补货"
        }
    }
})
```

编写HomeView.vue页面，创建一个服装商城的首页组件，用于显示服装库存状态信息。

```
<template>
  <h1>服装商城首页</h1>  <!-- 页面标题改为服装商城 -->
  <p>{{ getClothesStock }}</p>  <!-- 显示服装库存状态 -->
</template>
<script>
import { mapGetters } from "vuex";  // 从Vuex导入mapGetters辅助函数
export default {
 computed: {
   ...mapGetters(["getClothesStock"]),  // 映射getters中的服装库存getter
 },
};
</script>

<style>
</style>
```

在上述代码中，通过Vuex的辅助函数mapGetters将全局状态管理中的计算属性映射到本地，实现了状态数据的响应式获取和显示。

（2）编写App.vue页面，创建Vue应用的根组件（App.vue），作为整个服装商城应用的入口和布局容器。

```
<script setup>
// 检查 https://v3.vuejs.org/api/sfc-script-setup.html#sfc-script-setup
// import HelloWorld from './components/HelloWorld.vue'
import Home from './components/HomeView.vue'  // 导入服装商城首页组件
</script>
<template>
  <img alt="Vue logo" src="./assets/logo.png" />
   <p>当前服装库存 = {{$store.state.clothesStock}}</p>  <!-- 显示服装库存数量 -->
  <!-- <HelloWorld/> -->
  <Home/>  <!-- 渲染服装商城首页组件 -->
</template>

<style>
#app {
  font-family: Avenir, Helvetica, Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-align: center;
  color: #2c3e50;
  margin-top: 60px;
}
</style>
```

综上代码所述，呈现如下效果，如图4-10所示。



图4-10　Vuex状态管理Getters案例

4.Mutation修改数据

更改vuex的store中的状态的唯一方法是提交mutation，vuex中的Mutation非常类似于事件，每个mutation都有一个字符串的事件类型(type)和一个回调函数(handler)，这个回调函数就是为我们实际进行状态更改的地方，并且它会接受state作为第一个参数。

【例4-11】开发一个基于Vue 3和Vuex的服装商城库存管理系统，使用Vuex进行全局状态管理和实现库存数据的计算逻辑处理。

（1）编写index.js页面，创建一个服装商城的集中式状态管理仓库。

```
import { createStore } from "vuex";
// Vuex的核心作用是管理组件之间的状态 – 用于服装商城的状态管理
export default createStore({
    // 所有的状态都放在这里（响应式数据）
    state: {
        clothesStock: 10 // 服装库存数量，初始值为10
    },
    // 计算属性，用于对state中的数据进行加工处理
    getters: {
        getClothesStock(state) {
            // 获取服装库存信息，如果库存大于0返回实际数量，否则返回提示信息
            return state.clothesStock > 0 ? state.clothesStock : "库存数据异常"
        }
    },
    // 修改状态的方法，必须是同步函数
    mutations: {
        addClothesStock(state, num) {
            // 增加服装库存，接收一个数字参数
            state.clothesStock += num
        }
    }
})
```

（2）编写HomeView.vue页面，创建一个服装商城的库存管理界面组件。

```
<template>
    <!-- 服装商城页面模板 -->
```

```
    <h1>服装商城页面</h1>
    <!-- 显示服装库存信息 -->
    <p>{{getClothesStock}}</p>
    <!-- 增加库存按钮，点击触发 addClickHandle 方法 -->
    <button @click="addClickHandle">增加库存</button>
</template>

<script>
    // 从 Vuex 导入辅助函数
    import { mapGetters,mapMutations } from 'vuex'
    export default{
        // 计算属性 – 用于响应式数据
        computed: {
            // 将 Vuex 中的 getters 映射到组件的计算属性
            ...mapGetters(["getClothesStock"])
        },
        // 方法定义
        methods:{
            // 将 Vuex 中的 mutations 映射到组件的方法
            ...mapMutations(["addClothesStock"]),
            // 按钮点击事件处理函数
            addClickHandle(){
                // 固定调用方式 – 直接调用 mutation
                //this.$store.commit("addClothesStock",15)

                // 通过映射的方法调用 mutation，传入增加的数量 20
                this.addClothesStock(20)
            }
        }
    }
</script>
```

```
<style>
    /* 样式部分 – 可根据需要添加样式 */
</style>
```

在上述代码中，组件利用 mapGetters 将处理后的库存数据映射为计算属性，实时显示库存状态（正常时显示数量，异常时提示信息）；同时通过 mapMutations 将状态修改方法映射到组件，当用户点击”增加库存”按钮时，调用 addClothesStock 方法以每次 20 件的数量增加库存。

（3）编写 App.vue 页面，创建 Vue 应用的根组件（App.vue），作为整个服装商城应用的入口和布局容器。

```
<script setup>
// This starter template is using Vue 3 <script setup> SFCs
// Check out https://v3.vuejs.org/api/sfc-script-setup.html#sfc-script-setup
// import HelloWorld from './components/HelloWorld.vue'
import Home from './components/HomeView.vue'
</script>

<template>
  <img alt="Vue logo" src="./assets/logo.png" />
   <p>服装库存 = {{$store.state.clothesStock}}</p>
  <!-- <HelloWorld/> -->
  <Home/>
</template>

<style>
#app {
  font-family: Avenir, Helvetica, Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-align: center;
  color: #2c3e50;
  margin-top: 60px;
}
</style>
```

综上代码所述，呈现如下效果，如图 4-11 所示。



图 4-11　Vuex 状态管理 Mutation 案例

5.Action 修改数据

Action 类似于 mutations，不用于 Action 提交的是 mutations，而不是直接变更状态，Action 可以包含任意异步操作。Action 作用就是如果有异步操作就使用 Action，如果没有异步操作使用 mutations。

【例 4-12】开发一个基于 Vue 3 的服装库存管理 Web 应用程序，该应用程序具备展示当前服装库存数量的功能，并能够实现同步补货与异步补货功能。

（1）编写 index.js 页面，创建全局状态管理中心，管理服装库存数据，提供同步和异步的状态更新方法。

```
import { createStore } from "vuex";
import axios from "axios";
//Vuex的核心作用是管理组件之间的状态的
export default createStore({
    //所有的状态都放在这里
    state: {
        inventory: 50  // 服装库存数量
    },
    getters: {
        getInventory(state) {
            return state.inventory > 0 ? state.inventory : "库存不足"
        }
    },
    mutations: {
```

```
        addInventory(state, num) {
            state.inventory += num
        }
    },
    actions: {//为异步操作准备的
        asyncAddInventory({ commit }) {
            axios.get("/api/hello.html")
                .then(res => {
                    commit("addInventory", res.data)
                })
        }
    }
})
```

（2）编写HomeView.vue页面，作为主要的业务组件，展示服装库存管理界面，提供用户交互功能来管理库存数据。

```
<template>
    <h1>服装库存管理</h1>
    <p>当前库存：{{getInventory}}</p>
    <button @click="addClickHandle">补货</button>
    <button @click="addAsyncClickHandle">异步补货</button>
</template>
<script>
    import { mapGetters,mapMutations,mapActions } from 'vuex'
    export default{
        computed: {
            ...mapGetters(["getInventory"])
        },
        methods:{
            ...mapMutations(["addInventory"]),
            ...mapActions(["asyncAddInventory"]),
            addClickHandle(){
                //固定调用方式
```

```
                //this.$store.commit("addInventory",15)
                this.addInventory(20)
            },
            addAsyncClickHandle(){
                //this.$store.dispatch("asyncAddInventory")
                this.asyncAddInventory()
            }
        }

    }
</script>

<style>
</style>
```

在上述代码中，通过mapGetters将Vuex中的getInventory计算属性映射到组件，实时显示库存数量，点击按钮触发addClickHandle，通过mapMutations映射的addInventory方法直接提交mutation，库存立即增加20件，点击按钮触发addAsyncClickHandle，通过mapActions映射的asyncAddInventory方法分发action，适用于需要异步处理的补货场景。

（3）编写App.vue页面，作为应用的根组件，负责整体的布局和样式设置，引入并渲染主要的业务组件。

```
<script setup>
// This starter template is using Vue 3 <script setup> SFCs
// Check out https://v3.vuejs.org/api/sfc-script-setup.html#sfc-script-setup
// import HelloWorld from './components/HelloWorld.vue'
import HomeView from './components/HomeView.vue'
</script>
<template>
  <img alt="服装店logo" src="./assets/logo.png" />
  <!-- <HelloWorld msg="Hello Vue 3 + Vite" /> -->
  <HomeView/>
</template>
<style>
```

```
#app {
  font-family: Avenir, Helvetica, Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  text-align: center;
  color: #2c3e50;
  margin-top: 60px;
}
</style>
```

综上代码所述，呈现如下效果，如图4-12所示。



图4-12 Vuex状态管理Action案例

4.3.5 Vuex状态管理的模块组

1.模块组的概述

由于使用单一状态树，应用的所有状态会集中到一个比较大的对象。当应用变得非常复杂时，store对象就有可能变得相当臃肿。

为了解决以上问题，Vuex允许我们将store分割成模块（module），每个模块拥有自己的state、mutation、action、getter，甚至是嵌套子模块——从上至下进行同样方式的分割。

2.模块局部状态

对于模块内部的mutation和getter，接收的第一个参数是模块的局部状态对象，编写Vuex状态管理

的模块组

```
const moduleA = {
  state: () => ({
    count: 0 // 服装库存数量
  }),
  mutations: {
    increment (state) {
      // 这里的 `state` 对象是模块的局部状态
      state.count++ // 增加一件服装库存
    }
  },
  getters: {
    doubleCount (state) {
      return state.count * 2 // 计算服装库存的两倍数量
    }
  }
}
```

同样，对于模块内部的action，局部状态通过context.state暴露出来，根节点状态则为context.rootState：

```
const moduleA = {
  // ...
  actions: {
    incrementIfOddOnRootSum ({ state, commit, rootState }) {
      // 如果当前模块的服装库存数量与根状态的服装库存数量之和为奇数
      if ((state.count + rootState.count) % 2 === 1) {
        commit('increment') // 提交增加服装库存的 mutation
      }
    }
  }
}
```

对于模块内部的getter，根节点状态会作为第三个参数暴露出来：

```
const moduleA = {
```

```
  // ...
  getters: {
    sumWithRootCount (state, getters, rootState) {
      return state.count + rootState.count
    }
  }}
```

3.Vuex状态管理的模块组的使用

【例4-13】开发一个基于Vue 3的服装库存管理Web应用程序，该应用程序具备展示当前服装库存数量的功能，并能够实现同步补货与异步补货功能。

（1）编写modules/clothesModule.js页面，创建了一个状态管理库。

```
// store/modules/clothesModule.js – 创建服装模块
const clothesModule = {
    // 开启命名空间，确保模块的独立性
    namespaced: true,

    // 模块的初始状态
    state: () => ({
        items: [
            { id: 1, name: 'T恤', stock: 10, price: 50 },
            { id: 2, name: '牛仔裤', stock: 5, price: 100 }
        ]
    }),

    // 同步修改状态的方法
    mutations: {
        /**
         * 增加商品库存
         * @param {Object} state – 模块状态
         * @param {Object} payload – 包含id和quantity的参数
         */
        ADD_STOCK(state, payload) {
            const item = state.items.find(item => item.id === payload.id)
```

```
            if (item) item.stock += payload.quantity
        },

        /**
         * 减少商品库存
         * @param {Object} state – 模块状态
         * @param {Object} payload – 包含 id 和 quantity 的参数
         */
        REDUCE_STOCK(state, payload) {
            const item = state.items.find(item => item.id === payload.id)
            // 检查库存是否足够
            if (item && item.stock >= payload.quantity) {
                item.stock -= payload.quantity
            }
        }
    },

    // 处理异步操作和业务逻辑
    actions: {
        /**
         * 异步补货操作
         * @param {Object} context – 包含 commit 等方法的上下文对象
         * @param {Object} payload – 补货信息
         * @returns {Promise} 返回补货结果的 Promise
         */
        async restock({ commit }, payload) {
            return new Promise((resolve) => {
                // 模拟异步操作（如 API 调用）
                setTimeout(() => {
                    commit('ADD_STOCK', payload)
                    resolve(`补货成功：${payload.quantity} 件`)
                }, 300)
```

```
            })
        },

        /**
         * 销售商品操作
         * @param {Object} context – 包含 commit 和 state 的上下文对象
         * @param {Object} payload – 销售信息
         * @returns {Promise} 返回销售结果的 Promise
         */
        sell({ commit, state }, payload) {
            return new Promise((resolve, reject) => {
                const item = state.items.find(item => item.id === payload.id)
                // 检查库存是否足够
                if (!item || item.stock < payload.quantity) {
                    reject('库存不足')
                } else {
                    commit('REDUCE_STOCK', payload)
                    resolve(`销售成功：${payload.quantity}件`)
                }
            })
        }
    },

    // 计算属性，用于从状态中派生数据
    getters: {
        /**
         * 获取所有商品
         * @param {Object} state – 模块状态
         * @returns {Array} 所有商品数组
         */
        allItems: (state) => state.items,
```

```
        /**
         * 获取低库存商品（库存小于10）
         * @param {Object} state – 模块状态
         * @returns {Array} 低库存商品数组
         */
        lowStockItems: (state) => state.items.filter(item => item.stock < 10)
    }
}

export default clothesModule
```

在上述代码中，通过开启命名空间确保模块独立性。模块包含初始的商品数据状态（T恤和牛仔裤），提供了同步修改库存的mutations（ADD_STOCK和REDUCE_STOCK），处理业务逻辑的actions（异步补货restock和销售sell），以及计算派生数据的getters（获取所有商品和低库存商品）。

（2）编写index.js页面，集中管理和组织应用的状态数据，提供可预测的状态管理。

```
// store/index.js – Vuex 主入口文件，注册所有模块
import { createStore } from 'vuex'
import clothesModule from './modules/clothesModule'

/**
 * 创建并配置 Vuex 商店
 * 主要目的：集中管理应用的状态和数据流
 */
export default createStore({
    // 注册模块
    modules: {
        // 注册服装模块，命名为 'clothes'
        // 这样可以在组件中通过 'clothes' 命名空间访问该模块
        clothes: clothesModule
    }
})
```

在上述代码中，Vuex状态管理的主入口文件，主要负责创建和配置整个应用的状态管理商店。通过createStore方法创建Vuex实例，并在modules选项中注册了clothesModule服装管理模块，将其命名为

“clothes”。这样的模块化设计使得服装相关的状态、操作和获取器都被封装在独立的命名空间中，既保持了代码的清晰结构，又避免了不同模块间的命名冲突，为组件提供了统一的状态管理入口。

（3）编写main.js页面

```
// main.js – 挂载到Vue应用
import { createApp } from 'vue'
import App from './App.vue'
import store from './store'
const app = createApp(App)
app.use(store)
app.mount('#app')
```

（4）编写App.vue页面，实现服装库存的展示、销售和补货等交互功能。

```
<!-- App.vue – 使用Vuex模块的服装库存管理组件 -->
<template>
  <div>
    <h2>服装库存管理</h2>
    <!-- 遍历显示所有商品 -->
    <div v-for="item in allItems" :key="item.id" class="item">
      <span>{{ item.name }} – 库存: {{ item.stock }} – 价格: {{ item.price }}元</span>
      <!-- 销售按钮 -->
      <button @click="sellItem(item.id, 1)">销售1件</button>
      <!-- 补货按钮 -->
      <button @click="restockItem(item.id, 5)">补货5件</button>
    </div>

    <!-- 低库存商品展示区域 -->
    <div>
      <h3>低库存商品:</h3>
      <ul>
        <li v-for="item in lowStockItems" :key="item.id">
          {{ item.name }} (库存: {{ item.stock }})
        </li>
      </ul>
```

```
    </div>
  </div>
</template>

<script>
import { mapGetters, mapActions } from 'vuex'

export default {
  computed: {
    // 映射带命名空间的 getters：从 clothes 模块获取状态数据
    ...mapGetters('clothes', ['allItems', 'lowStockItems'])
  },

  methods: {
    // 映射带命名空间的 actions：从 clothes 模块获取操作方法
    ...mapActions('clothes', ['restock', 'sell']),

    /**
     * 补货商品方法
     * @param {number} id – 商品 ID
     * @param {number} quantity – 补货数量
     */
    async restockItem(id, quantity) {
      try {
        const result = await this.restock({ id, quantity })
        alert(result) // 显示补货成功消息
      } catch (error) {
        alert(error) // 显示错误信息
      }
    },

    /**
```

```
    * 销售商品方法
    * @param {number} id – 商品 ID
    * @param {number} quantity – 销售数量
    */
   async sellItem(id, quantity) {
    try {
     const result = await this.sell({ id, quantity })
     alert(result) // 显示销售成功消息
    } catch (error) {
     alert(error) // 显示库存不足等错误信息
    }
   }
  },

  // 组件挂载完成后执行
  mounted() {
   console.log('所有商品:', this.allItems)
   console.log('低库存商品:', this.lowStockItems)
  }
}
</script>

<style>
/* 商品项样式 */
.item {
  margin: 10px 0;
  padding: 10px;
  border: 1px solid #ccc;
}

/* 按钮样式 */
button {
```

```
  margin-left: 10px;
}
</style>
```

在上述代码中，组件通过Vuex的mapGetters和mapActions辅助函数，从clothes模块中映射获取商品数据和操作方法，实现了完整的库存管理功能。

综上代码所述，呈现如下效果，如图4-13所示。



图4-13　Vuex状态管理Module模块组的案例

本章小结

本章在Vue Router、Axios、Vuex三大组件展开，系统梳理了构建中大型单页应用必备的核心技能与工程化思维。Router部分先以“声明式+编程式”导航、动态参数、嵌套路由、导航守卫为线索，结合服装品牌详情、后台权限等场景，演示路径匹配、组件缓存、鉴权拦截的最佳实践，帮助学生建立“URL即功能”的设计观。Axios部分从局部引入到全局挂载，再到封装request模块与跨域代理，层层递进：先通过GET/POST两种请求完成商品数据拉取与提交，再抽离baseURL、拦截器、错误码映射，形成可复用的HTTP层，最后以vue.config.js的devServer.proxy解决本地开发跨域痛点，让“前端独立、后端并行”成为标准流程。Vuex部分围绕state、getter、mutation、action、module五个核心概念，用服装库存管理贯穿：同步改库存、异步补货、模块拆分、命名空间隔离，直观展示单向数据流如何降低组件耦合。本章既提供可直接套用的代码模板，为学生后续应对真实业务复杂度和团队协作奠定扎实基础。

练习与实践

简答题

（1）使用Axios发起POST请求时，如何设置请求体数据？

（2）Vuex的Mutation为什么必须是同步函数？

（3）嵌套路由中，父组件必须提供哪个标签才能渲染子组件？

（4）全局引入Axios后，在组件内应如何调用？

（5）Vuex的Getter与组件内普通computed有何区别？

实践题

（1）使用Vuex模块组实现一个“购物车”模块，支持添加商品、删除商品、实时计算总价，并在Header组件中显示商品件数徽标。

（2）基于嵌套路由实现一个“服装搭配”页面：父路由/match展示搭配列表，子路由/match/:id展示该套搭配的详情与推荐商品，要求切换搭配时URL同步更新并有加载动画。

（3）利用Axios拦截器实现全局请求Loading：在请求发起时显示Element Plus的Loading服务，响应到达后自动关闭，并要求对500状态码统一跳转到/error页面。

第 5 章

关系数据库和非关系数据库



本章导读

在实践中，MySQL与Redis并非相互替代，而是协同互补的黄金搭档，典型的架构模式是利用Redis作为MySQL的前置缓存层，拦截热点数据请求，从而大幅减轻后端数据库的负载压力。这种“Redis缓存+MySQL持久化”的组合方案，共同构建了高性能、高可用的数据解决方案，完美满足了现代应用对速度与稳定性的双重需求。

学习目标

1.掌握数据库、表、行、列等基本概念，熟练编写增删改查（CRUD）语句。

2.了解索引（如B+Tree）的工作原理，知道如何创建高效索引来加速查询。

3.理解事务的ACID特性，以及不同隔离级别（读未提交、读已提交、可重复读、串行化）下可能出现的并发问题（脏读、不可重复读、幻读）。

4.熟练掌握Redis的5种核心数据类型及其适用场景。

5.理解Redis数据如何持久化到磁盘，防止重启后数据丢失。

思政目标及设计建议

将关系型数据库、非关系型数据库与服装类专业深度融合，并融入思想政治教育，是极具创新性且契合“课程思政”理念的研究方向。本章将阐述思政目标及设计建议，如表5-1所示。

表5-1　第5章思政目标及设计建议

思政目标	思政元素及融入
培养文化自信与民族自豪感	通过构建承载中华优秀传统文化的服装数据库，让学生在设计源头就接触、理解并热爱中国传统服饰文化（如汉服、旗袍、少数民族服饰等），激发其传承与创新的使命感
锤炼严谨规范、精益求精的工匠精神	数据库设计需要严谨的逻辑、规范的结构和高度的准确性。这恰好与服装设计中精准的尺寸、严谨的工艺和精益求精的“工匠精神”相契合

5.1 关系型数据库MySQL



5.1.1 数据库的概述

1.数据库的定义

数据库（Database）是按照数据结构来组织、存储和管理数据的仓库。每个数据库都配备了一个或多个不同的API，用于创建、访问、管理、搜索和复制所保存的数据。尽管数据也可以存储在文件中，但文件读写数据的速度相对较慢。因此，当前普遍采用关系型数据库管理系统（RDBMS）来存储和管理大规模数据。所谓关系型数据库，是建立在关系模型基础上的数据库，借助集合代数等数学概念和方法来处理数据库中的数据。

2.数据库的专业术语

（1）数据库：数据库是由多个关联表组成的集合。

（2）数据表：表是数据的矩阵形式，数据库中的表类似于一个简单的电子表格。

（3）列：一列（数据元素）包含相同类型的数据，例如邮政编码数据。

（4）行：一行（元组或记录）是一组相关的数据，例如一条用户订阅信息。

（5）冗余：存储双份数据，尽管冗余会降低性能，但能提升数据安全性。

（6）主键：主键具有唯一性，一个数据表中只能有一个主键，可通过主键查询数据。

（7）外键：外键用于连接两个表。

（8）复合键：复合键（组合键）将多个列作为一个索引键，常用于复合索引。

（9）索引：索引可加快对数据库表中特定信息的访问速度，是对表中一列或多列值进行排序的结构，类似于书籍的目录。

（10）参照完整性：参照完整性要求关系中不得引用不存在的实体，与实体完整性一同构成关系模型必须满足的完整性约束条件，旨在确保数据的一致性。

5.1.2 MySQL数据库的概述

MySQL数据库由瑞典MySQL AB公司开发，目前属于Oracle公司，MySQL作为一款开源的关系型数据库管理系统，其数据库本质上属于文件系统。它借助SQL命令，依据特定的数据结构，对建立于计算机存储设备之上的数据仓库进行组织、存储与管理。数据库是长期存于计算机内部、具备组织性且可共享的数据集合，其中的数据以二维表形式呈现（类似于Excel表格），并且各表之间能够构建关联关系

（例如，“用户表”与“订单表”可通过“用户ID”实现关联）。

5.1.3 MySQL数据库与表的操作

数据库的语法结构涵盖创建数据库、删除数据库、查看数据库以及使用数据库的操作。

1.创建数据库的语法结构

```
CREATE  DATABASE数据库的名字 CHARACTER SET utf8;
```

2.删除数据库的语法结构

```
DROP DATABASE数据库名称;
```

3.查看数据库的语法结构

```
SHOW DATABASES;
```

4.进入数据库，进行使用数据库的语法结构

```
USE 数据库名称;
```

【例5-1】使用Navicat Premium 15客户端，创建一个MySQL数据库，数据库的名称为database_demo，查看数据库以及删除数据库，命令如下：

```
#创建数据库
CREATE  DATABASE database_demo CHARACTER SET utf8;
#查看数据库
SHOW DATABASES;
#删除数据库
DROP DATABASE database_demo;
```

MySQL的数据类型主要分为三大类：数值类型、日期和时间类型、字符串（字符和字节）类型，如表5-2所示。

表5-2 数据类型表

分类	数据类型	说明	备注
整数类型	tinyint	1个字节	非常小的整数
	smallint	2个字节	小整数
	mediumint	3个字节	中等整数
	int	4个字节	标准整数
	bigint	8个字节	大整数
定点数类型	decimal(m,d)	精确小数	m是总位数 d是小数位
	numeric(m,d)	decimal的同义词	
浮点类型	float	4个字节	单精度浮点数
	double	8个字节	双精度浮点数

续表

分类	数据类型	说明	备注
日期和时间类型	date	3个字节	仅日期'YYYY-MM-DD'
	time	3个字节	小数秒，时间 'HH:MM:SS[.fraction]'
	datetime	5个字节	小数秒，日期和时间
	timestamp	4个字节	时间戳，自动时区转换
	year	1个字节	年份值
字符串类型	char(m)	定长字符串	m为字符串数，范围是0-255
	varchar(m)	可变字符串	m为最大字符串数，范围是0-65535
	tinytext	最大长度255个字符	
	text	最大长度65,535个字符（64KB）	
	mediumtext	最大长度16,777,215个字符（16MB）	
	longtext	最大长度4,294,967,295个字符（4GB）	
	enum	枚举，值必须是列表中的一个	
	set	集合，值可以是列表中多个	

对数据库表进行创建表结构、删除表结构、修改表结构的语法规则如下。

1.创建表结构

语法结构如下：

```
CREATE TABLE表名(
字段名称 字段类型,
字段名称 字段类型,
.....
)
```

【例5-2】使用Navicat Premium 15作为客户端，连接至MySQL数据库服务器，并在database_demo数据库中创建名为users的表，命令如下：

```
CREATE TABLE users(
id INT(11),
name VARCHAR(255),
sex CHAR(2)
)
```

2.查看表结构

语法结构如下：

```
DESC表名称;
SHOW表名称;
```

3.删除表结构

语法结构如下：

```
DROP TABLE 表名;
```

【例5-3】使用Navicat Premium 15作为客户端连接至MySQL数据库服务器，并在database_demo数据库中查看users表结构和删除users表。命令如下：

```
#查看所有的表
SHOW TABLES;
#查看users表结构
DESC uses;
#删除表结构
DROP TABLE users
```

4.更新表结构

涵盖修改表的名称、修改表结构的字段名称、修改表结构的字段类型、对表结构进行添加字段和删除字段，语法结构如下：

（1）修改表的名称，语法结构如下：

```
ALTER TABLE 表名 RENAME to 新表名;
```

（2）修改表结构的字段，语法结构如下：

```
ALTER TABLE 表名 CHANGE 字段名称 新字段名称描述;
```

（3）修改表结构的字段的类型，语法结构如下：

```
ALTER TABLE 表名 MODIFY 字段名称 字段类型;
```

（4）进行对表结构进行添加字段，语法结构如下：

```
ALTER TABLE 表名 ADD 字段描述;
```

（5）进行对表结构删除字段，语法结构如下：

```
ALTER TABLE 表名 DROP字段名称;
```

【例5-4】使用Navicat Premium 15作为客户端连接至MySQL数据库服务器，并在database_demo数据库中对users表结构进行更新，命令如下：

```
#修改表的名称
ALTER TABLE users RENAME to user;
#修改字段
ALTER TABLE user CHANGE name username VARCHAR(40);
#修改字段类型
ALTER TABLE user MODIFY id VARCHAR(50);
```

```
#添加字段
ALTER TABLE  user ADD password VARCHAR(20);
#删除字段
ALTER TABLE  user DROP password;
```

5.数据完整性约束

（1）数据完整性约束的定义

数据完整性是指数据库中数据的准确性与一致性。它确保数据库中的数据具备可信性、有效性，并且严格符合业务规则。

（2）为什么使用数据完整性约束

①防范垃圾数据：阻止无效、错误或不符合格式规范的数据流入数据库。

②保障业务规则：于数据库层面强制实施业务逻辑（例如年龄不得为负数，邮箱必须包含“@”符号）。

③维护数据关联：确保表与表之间的关联关系准确且有效。

（3）MySQL数据库约束类型

①主键约束-PRIMARY KEY，唯一标识表中的每一行记录，一个表只能有一个主键。主键列的值必须是唯一且非空的，命令如下：

```
#方式1：列级约束（创建表时）
CREATE  TABLE student(
   id INT PRIMARY KEY,
   name VARCHAR(100)
);
#方式2：表级约束（常用于复合主键）
CREATE  TABLE course_demo(
   student_id INT,
   course_id INT,
   PRIMARY KEY (student_id, course_id) -- 复合主键
);
#方式3：使用 ALTER CREATE 添加
ALTER TABLE student add PRIMARY KEY (id);
```

②外键约束-FOREIGN KEY，建立两个表之间的关联，确保一个表中的数据引用另一个表中存在的值。它实现了参照完整性，命令如下：

```
#创建订单表结构，直接定义主键，设置外键
CREATE  TABLE orders (
```

```
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
     ON DELETE CASCADE   #当父表（customers）中的记录被删除时，同步删除子表（orders）中的相关记录
    ON UPDATE CASCADE   #当父表中的主键更新时，同步更新子表中的外键
);
#使用 ALTER TABLE 添加
ALTER TABLE orders ADD CONSTRAINT fk_orders_customers FOREIGN KEY (customer_id) REFERENCES customers(customer_id);
```

ON DELETE | UPDATE CASCADE：执行级联操作。

ON DELETE | UPDATE SET NULL：将相关字段设置为NULL。

ON DELETE | UPDATE SET DEFAULT：将相关字段设置为默认值。

ON DELETE | UPDATE RESTRICT（或NO ACTION）：拒绝执行操作（默认行为）。

①唯一约束-UNIQUE，确保列中的所有值都是不同的。允许有NULL值（但只能有一个NULL，具体取决于数据库设置）。

```
#列级约束
CREATE  TABLE users (
   id INT PRIMARY KEY,
   email VARCHAR(255) UNIQUE,
   username VARCHAR(50) UNIQUE
);

#表级约束
CREATE  TABLE users (
   id INT,
   email VARCHAR(255),
   UNIQUE (email)
);

#ALTER TABLE 添加
```

```
ALTER TABLE users ADD UNIQUE (email);
```

②非空约束-NOT NULL，强制列不能接受NULL值。

```
CREATE  TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    hire_date DATE NOT NULL
);
```

③检查约束-CHECK（MySQL 8.0.16+开始完整支持），根据指定的条件验证列中的值。注意：MySQL在8.0.16版本之后才完全支持并强制遵守CHECK约束。

```
#创建products表，设置检查约束
CREATE  TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10,2) CHECK (price > 0), -- 确保价格大于0
    stock_quantity INT CHECK (stock_quantity >= 0) -- 确保库存非负
);
CREATE  TABLE persons (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    age INT,
    gender CHAR(1),
    CONSTRAINT chk_age CHECK (age >= 0 AND age <= 150),
    CONSTRAINT chk_gender CHECK (gender IN ('M', 'F', 'O'))
);
#使用 ALTER TABLE 添加
ALTER TABLE products ADD CONSTRAINT chk_price_positive CHECK (price > 0);
```

④默认值约束-DEFAULT，当向表中插入新记录，如果没有为该列指定值，则使用默认值。

```
CREATE  TABLE orders (
    order_id INT PRIMARY KEY,
    order_date DATE DEFAULT (CURRENT_DATE), -- 使用当前日期作为默认值
    status VARCHAR(20) DEFAULT 'Pending'
);
```

5.1.4 SQL语言的使用

SQL全称是Structured Query Language（结构化查询语言），是一种专门用于与关系型数据库进行通信和交互的标准计算机语言，旨在管理、查询、操作及定义关系型数据库中的数据。

根据其功能，SQL语言通常被划分为四大类：DDL（数据定义语言）、DML（数据操作语言）、DCL（数据控制语言）以及DQL（数据查询语言），如表5-3所示。

表5-3 SQL语言的分类

SQL语言分类名称	含义	特点	核心命令
DDL数据定义语言	用于定义和管理数据库中的各类结构，包括数据库本体、数据表、视图、索引等	通常由数据库管理员（DBA）执行，操作会立即生效（自动提交事务）	CREATE、ALTER、DROP
DML数据操作语言	用于对数据库表中的记录（数据）进行增、删、改操作	操作需要显式地提交事务（COMMIT）才会永久生效，或者可以回滚事务（ROLLBACK）撤销操作	INSERT、UPDATE、DELETE
DCL数据控制语言	用于控制对数据库的访问权限和安全性	主要由DBA使用，确保数据库安全	GRANT、REVOKE
DQL数据查询语言	有时会被归类为DML，但由于其使用频率极高，通常会被单独列出。尽管它仅包含一个核心命令，却堪称SQL中最复杂且功能最强大的部分	SELECT关键词可以搭配多种子句，实现复杂的查询，例如：WGERE、FROM、GROUP BY等	SELECT

SQL语言的主要使用方式如下所示。

1.插入数据：INSERT语句，语法结构如下：

```
#插入语法结构1：
#注意：默认插入全部字段,必须保证VALUES内容(类型，顺序)和表中一致，若类型非数值，必须加引号
INSERT INTO 表名 VALUES(字段1,字段2.....,字段n);
#插入语法结构2：
#注意：插入指定的字段，必须保证表名和values内容(类型，顺序)一致
INSERT INTO 表名(字段名,字段名.....,字段名n) VALUES(字段值，字段值，字段值n);
```

2.更新数据：UPDATE语句，语法结构如下：

```
#更新语句结构1：
#注意点：如果没有写WHERE条件，代表的是修改整个表格的数据
UPDATE 表名 SET 字段名=字段值,字段名=字段值,....;
#更新语句结构2：
#注意点：如果写WHERE条件，只修改符合条件的数据
UPDATE 表名 SET 字段名=字段值,字段名=字段值,....where 条件;
```

3.删除数据：DELETE 语句，语法结构如下：

```
#删除语句结构 1：
#注意点：如果没有写 WHERE 条件，代表的是删除整个表格的数据
DELETE FROM 表名；
#删除语句结构 2：
#注意点：如果写 WHERE 条件，只删除符合条件的数据
DELETE FROM 表名 WHERE 条件；
```

4.查询数据：SELECT 语句，语法结构如下：

```
SELECT.... FROM 表名
                    WHERE 条件
                    GROUP BY 分组字段
                    HAVING 条件
                    ORDER BY 排序字段 ASE| DESC
```

【例 5-5】服装电商平台需要建立一个商品管理系统，用于管理服装商品信息，包括：商品基本信息（名称、价格、库存等）、商品分类管理、价格区间统计、库存预警。

1.创建数据库和 fz_products 表，命令如下：

```
#创建数据库（如果不存在）
CREATE DATABASE IF NOT EXISTS database_demo;
USE database_demo;
#创建服装商品表
CREATE TABLE fz_products (
    product_id INT PRIMARY KEY AUTO_INCREMENT COMMENT '商品 ID',
    product_name VARCHAR(100) NOT NULL COMMENT '商品名称',
    product_price DECIMAL(10,2) NOT NULL COMMENT '商品价格',
    product_category VARCHAR(50) COMMENT '商品分类',
    stock_quantity INT NOT NULL DEFAULT 0 COMMENT '库存数量',
    brand VARCHAR(50) COMMENT '品牌',
    size VARCHAR(20) COMMENT '尺码',
    color VARCHAR(30) COMMENT '颜色',
    create_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
     update_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
COMMENT '更新时间'
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='服装商品表';
```

2.初始化数据，命令如下：

```
-- 插入初始服装商品数据
INSERT INTO fz_products (product_name, product_price, product_category, stock_quantity, brand, size, color)
VALUES
('纯棉T恤-白色', 39.90, '上装', 150, '优衣库', 'M', '白色'),
('纯棉T恤-黑色', 39.90, '上装', 120, '优衣库', 'L', '黑色'),
('牛仔裤-经典蓝', 89.90, '下装', 80, '李维斯', '32', '蓝色'),
('牛仔裤-深色', 89.90, '下装', 60, '李维斯', '34', '深蓝'),
('羽绒服-长款', 299.00, '外套', 30, '波司登', 'XL', '黑色'),
('羽绒服-短款', 259.00, '外套', 25, '波司登', 'M', '红色'),
('运动鞋-跑步', 199.00, '鞋类', 50, '耐克', '42', '白色'),
('运动鞋-篮球', 259.00, '鞋类', 40, '耐克', '44', '黑色'),
('衬衫-商务', 68.00, '上装', 100, '海澜之家', 'L', '白色'),
('衬衫-休闲', 58.00, '上装', 90, '海澜之家', 'M', '蓝色'),
('毛衣-针织', 98.00, '上装', 70, 'ZARA', 'S', '灰色'),
('毛衣-羊毛', 158.00, '上装', 45, 'ZARA', 'L', '米色'),
('短裤-运动', 38.00, '下装', 200, '安踏', 'M', '黑色'),
('短裤-休闲', 45.00, '下装', 180, '安踏', 'L', '卡其色'),
('连衣裙-夏季', 128.00, '连衣裙', 60, 'Only', 'S', '碎花'),
('连衣裙-秋季', 168.00, '连衣裙', 40, 'Only', 'M', '纯色');
```

3.查询所有的商品信息，命令如下：

```
SELECT * FROM fz_products;
```

4.查询商品名和商品价格，命令如下：

```
SELECT product_name, product_price FROM fz_products;
```

5.查询所有商品都有哪些价格，命令如下：

```
#DISTINCT：筛选掉重复的值
SELECT DISTINCT product_price FROM fz_products;
```

6.查询所有的商品，按价格进行排序，命令如下：

```
#升序排序（默认）
SELECT * FROM fz_products ORDER BY product_price ASC;
#降序排序
```

```
SELECT * FROM fz_products ORDER BY product_price DESC;
```

7.基本聚合函数统计，命令如下：

```
#商品总数
SELECT COUNT(*) AS total_products FROM fz_products;
#最高价格
SELECT MAX(product_price) AS max_price FROM fz_products;
#最低价格
SELECT MIN(product_price) AS min_price FROM fz_products;
#平均价格
SELECT AVG(product_price) AS avg_price FROM fz_products;
#价格总和
SELECT SUM(product_price) AS total_price FROM fz_products;
```

8.按分类统计商品信息，命令如下：

```
SELECT
    product_category AS '商品分类',
    COUNT(*) AS '商品数量',
    MIN(product_price) AS '最低价格',
    MAX(product_price) AS '最高价格',
    AVG(product_price) AS '平均价格',
    SUM(stock_quantity) AS '总库存'
FROM fz_products
GROUP BY product_category
ORDER BY AVG(product_price) DESC;
```

9.品牌分析，命令如下：

```
SELECT
    brand AS '品牌',
    COUNT(*) AS '商品数量',
    AVG(product_price) AS '平均价格',
    SUM(stock_quantity) AS '总库存量',
    SUM(product_price * stock_quantity) AS '库存总价值'
FROM fz_products
WHERE brand IS NOT NULL
```

```
GROUP BY brand
ORDER BY SUM(product_price * stock_quantity) DESC;
```

10.价格区间分析，命令如下：

```
SELECT
    CASE
        WHEN product_price < 50 THEN '50元以下'
        WHEN product_price BETWEEN 50 AND 100 THEN '50–100元'
        WHEN product_price BETWEEN 100 AND 200 THEN '100–200元'
        ELSE '200元以上'
    END AS '价格区间',
    COUNT(*) AS '商品数量',
    AVG(product_price) AS '平均价格',
    SUM(stock_quantity) AS '总库存'
FROM fz_products
GROUP BY
    CASE
        WHEN product_price < 50 THEN '50元以下'
        WHEN product_price BETWEEN 50 AND 100 THEN '50–100元'
        WHEN product_price BETWEEN 100 AND 200 THEN '100–200元'
        ELSE '200元以上'
    END
ORDER BY MIN(product_price);
```

11.库存预警分析，命令如下：

```
#查询商品库存状态分析
SELECT
    product_name AS '商品名称',        #选择商品名称字段并设置中文别名
    product_price AS '价格',           #选择商品价格字段并设置中文别名
    stock_quantity AS '库存数量',      #选择库存数量字段并设置中文别名
    CASE
        WHEN stock_quantity < 50 THEN '库存不足'      #当库存小于50时显示"库存不足"
        WHEN stock_quantity BETWEEN 50 AND 100 THEN '库存正常' #库存50–100显示"库存正常"
        ELSE '库存充足'                #其他情况（库存大于100）显示"库存充足"
```

```
    END AS '库存状态'              #将CASE语句结果命名为"库存状态"
FROM fz_products                  #从服装商品表中查询数据
ORDER BY stock_quantity ASC;      #按库存数量升序排列，便于查看库存紧张的商品
```

在上述代码中，本案例所涉及的所有SQL语句均可在Navicat Premium 15环境中直接执行，通过该案例的实践，能够系统掌握服装商品数据的管理与分析方法，其内容涵盖数据库与数据表的创建、初始数据的插入、基础查询操作、聚合函数的综合应用以及业务场景的分析等核心环节，从而实现案例的完整性与实用性。

5.2 非关系型数据库Redis



5.2.1 NOSQL概述

NoSQL(Not Only SQL）意为“不仅仅是SQL”，是一项全新的数据库理念，泛指非关系型的数据库。它突破了传统关系型数据库在数据模型、扩展性、性能等方面的限制，以灵活的数据模型、高可扩展性和高性能等特性，满足了现代应用对于数据处理多样化的需求。NoSQL数据库不依赖固定的表结构，能够轻松应对海量数据的存储与处理，在大数据、云计算、移动互联网等新兴领域得到了广泛应用。NOSQL非关系型数据库的分类，如表5-4所示。

表5-4　NoSQL非关系型数据库分类表

数据库分类	相关产品	典型应用	数据模型	优势	劣势
键值存储数据库	Redis	内容缓存	键值对	快速查询	存储的数据缺少结构化
列存储数据库	Hbase, Riak	分布式的文件系统	以列簇式存储，将同一列数据存储在一起	查找速度快，可扩展性强，更容易进行分布式扩展	功能相对局限
文档型数据库	MongDB	Web应用	一系列键值对	数据要求不严格	查询性能不高，而且缺乏统一的查询语法
图形数据库	Ne04j	社交网络	图结构	利用图结构相关算法	需要对整个图做计算才能得出结果，不容易做分布式的集群方案

5.2.2 Redis概述

Redis是一种开源的、基于内存的键值存储数据库，它以其高性能、丰富的数据结构和灵活的扩展性而受到广泛关注。Redis支持多种数据结构，如字符串、哈希、列表、集合和有序集合等，这使得它能够满足各种复杂的应用场景需求。此外，Redis还提供了持久化机制，可以将内存中的数据保存到磁盘中，

以确保数据的安全性和可靠性。同时，Redis还支持主从复制和集群部署，可以轻松地实现高可用性和可扩展性。Redis主要应用于如下场景：

1.首先是缓存场景，利用其高性能的读写能力，Redis可以显著提升系统的响应速度，减轻后端数据库的压力。

2.其次是会话存储，在Web应用中，Redis常被用作会话存储的解决方案，确保用户状态的快速读写和持久化。再者是消息队列，Redis的列表和发布/订阅功能使其成为轻量级消息队列的理想选择，支持异步处理和实时通信。

3.最后是分布式锁，Redis的原子性操作特性使其能够轻松实现分布式锁，解决多进程或多服务器环境下的资源竞争问题。

5.2.3 Redis命令

Redis常用的命令包括：SET用于设置键值对，GET用于获取键对应的值，HSET和HGET用于操作哈希类型的数据，LPUSH和LPOP用于操作列表类型的数据，SADD和SMEMBERS用于操作集合类型的数据，ZADD和ZRANGE用于操作有序集合类型的数据等。这些命令简单易用，大大提高了开发效率。

1.SET 命令

功能：用于设置键值对，若键已存在则覆盖原有值。

举例：SET username "Alice"

执行后，Redis中会存储键username与值"Alice"的映射关系。若再次执行SET username "Bob"，则username的值会从"Alice"更新为"Bob"。

2.GET 命令

功能：用于获取指定键对应的值，若键不存在则返回nil。

举例：GET username

若之前通过SET username "Alice"设置了值，执行此命令会返回"Alice"；若键username未被设置过，则返回nil。

3.HSET 命令

功能：用于操作哈希（Hash）类型数据，为哈希表中的字段设置值，支持一次设置多个字段。

举例：HSET user:100 name "Alice" age 30 email "alice@example.com"

执行后，会创建或更新哈希表user:100，其中包含字段name（值"Alice"）、age（值30）、email（值"alice@example.com"）。若哈希表user:100不存在，会自动创建；若字段已存在，则覆盖原有值。

4.HGET 命令

功能：用于获取哈希表中指定字段的值，若哈希表或字段不存在则返回nil。

举例：HGET user:100 name

若哈希表user:100中存在字段name，则返回其值"Alice"；若字段name不存在或哈希表user:100未创建，则返回nil。

5.LPUSH 命令

功能：用于操作列表（List）类型数据，将一个或多个值插入到列表的头部（左侧）。

举例：LPUSH fruits "apple" "banana"

执行后，列表fruits的头部会依次插入"banana"和"apple"（因LPUSH按参数顺序从左到右插入头部，最终列表顺序为["banana", "apple"]）。若列表fruits不存在，会先创建空列表再执行插入。

6.LPOP 命令

功能：用于移除并返回列表的头部（左侧）第一个元素，若列表不存在或为空则返回nil。

举例：LPOP fruits

若列表fruits为["banana", "apple"]，执行后会移除并返回"banana"，列表变为["apple"]；若列表为空，返回nil。

7.SADD 命令

功能：用于操作集合（Set）类型数据，向集合中添加一个或多个成员，集合中成员具有唯一性，重复成员会被自动忽略。

举例：SADD tags "redis" "database" "cache" "redis"

执行后，集合tags会包含成员"redis"、"database"、"cache"，重复的"redis"会被忽略，最终集合大小为3。若集合tags不存在，会自动创建。

8.SMEMBERS 命令

功能：用于返回集合中的所有成员，结果无序，若集合不存在则返回空列表。

举例：SMEMBERS tags

若集合tags包含成员"redis"、"database"、"cache"，执行后会返回这三个成员（顺序不固定，如["database", "redis", "cache"]）；若集合tags不存在，返回空列表[]。

9.ZADD 命令

功能：用于操作有序集合（Sorted Set）类型数据，向有序集合中添加一个或多个成员，并为每个成员指定一个分数（score），成员按分数从小到大排序（分数可重复，成员唯一）。

举例：ZADD students 90 "Alice" 85 "Bob" 95 "Charlie"

执行后，有序集合students会添加成员"Alice"（分数90）、"Bob"（分数85）、"Charlie"（分数95）。系统会按分数自动排序，成员"Bob"（85）<"Alice"（90）<"Charlie"（95）。若成员已存在，会更新其分数并重新排序。

10.ZRANGE 命令

功能：用于返回有序集合中指定排名范围内的成员，按分数从小到大排序（默认），支持通过WITHSCORES参数同时返回分数。

举例：ZRANGE students 0 2 WITHSCORES

执行后，会返回有序集合students中排名第0到第2的成员（包含首尾）及其分数，结果为["Bob","85","Alice","90","Charlie","95"]（其中0表示最小分数成员，2表示第三个成员）。若不添加WITHSCORES，则仅返回成员["Bob","Alice","Charlie"]。

5.2.4 Redis应用

RedisCli类是一个专门基于Jedis客户端进行封装处理的Redis工具类，它为使用者提供了一系列针对Redis常用数据结构以及相关功能的操作接口。这些操作接口涵盖了Redis多个方面的常用功能，主要包括以下具体的功能内容。

1.连接管理

（1）该类通过JedisPool连接池来实现对Redis连接的有效管理，借助连接池可以更高效地使用和分配Redis连接资源。

（2）其构造函数RedisCli(String host, int port)用于对连接池进行初始化操作，在初始化过程中会根据传入的主机地址和端口信息来构建合适的连接池。

（3）close()方法的作用是关闭连接池资源，当不再需要使用连接池时，调用该方法可以释放相关的系统资源，避免资源的浪费。

2.String 类型操作

（1）set(String key, String value)方法的功能是设置字符串键值对，通过该方法可以将指定的字符串值与对应的键进行关联存储到Redis中。

（2）get(String key)方法用于获取指定键的字符串值，当需要获取某个键所对应的字符串内容时，调用该方法即可从Redis中获取相应的值。

（3）decr(String key)方法的作用是对指定键的数值进行递减操作，若该键对应的值是一个数值类型，调用此方法可以对其进行递减操作。

3.Hash 类型操作

（1）hset(String key, Map<String, String> fieldValues)方法用于设置哈希表字段和值，通过传入一个键和一个包含字段和值的映射，该方法可以将这些信息存储到Redis的哈希表中。

（2）hgetAll(String key)方法用于获取哈希表中所有字段和值，当需要查看某个哈希表的全部内容时，调用该方法可以获取到该哈希表中所有的字段及其对应的值。

4.List 类型操作

（1）lpush(String key, String... values)方法的功能是将一个或多个值插入到列表头部，通过传入一个键和一个或多个值，这些值会被依次插入到对应列表的头部位置。

（2）lrange(String key, long start, long stop)方法用于获取列表指定范围内的元素，根据传入的键、起始位置和结束位置，该方法可以从Redis的列表中获取指定范围的元素。

（3）blpop(int timeout, String key)方法是阻塞式弹出列表头部元素的方法，当列表为空时，该方法会阻塞一段时间，直到列表中有元素可以弹出或者超时。

5.Set 类型操作

（1）sadd(String key, String... members)方法的作用是向集合添加一个或多个成员，通过传入一个键和一个或多个成员，这些成员会被添加到对应的集合中。

（2）smembers(String key)方法用于获取集合中的所有成员，当需要查看某个集合的全部内容时，调用该方法可以获取到该集合中所有的成员。

6.Sorted Set 类型操作

（1）zadd(String key, double score, String member)方法用于向有序集合添加成员和分数，通过传入一个键、一个分数和一个成员，该成员会带着对应的分数被添加到有序集合中。

（2）zrangeWithScores(String key, long start, long stop)方法用于获取有序集合指定范围的成员及分数，根据传入的键、起始位置和结束位置，该方法可以从有序集合中获取指定范围的成员及其对应的分数。

7.Key 管理

（1）expire(String key, int seconds)方法用于设置键的过期时间，通过传入一个键和一个表示秒数的时间，该方法可以为指定的键设置过期时间，当时间到达后，该键会自动从Redis中删除。

8.高级特性

（1）txDemo(String key, int delta)方法用于演示事务操作，该操作包括对指定键对应的值进行递增操作以及设置该键的过期时间，通过该方法可以展示Redis事务操作的功能。

（2）pipeDemo()方法用于演示管道操作，该操作会批量执行1000次set操作，通过该方法可以展示Redis管道操作在批量执行命令时的高效性。

（3）subscribe(String channel)方法用于订阅指定频道的消息，通过传入一个频道名称，调用该方法后可以订阅该频道发布的消息。

（4）publish(String channel, String message)方法用于向指定频道发布消息，通过传入一个频道名称和一条消息，该消息会被发布到指定的频道中。

【例5-6】编写程序实现RedisCli类，实现基于Jedis客户端的主要功能。主要实现步骤如下。

1. 创建项目结构，如图5-1所示。

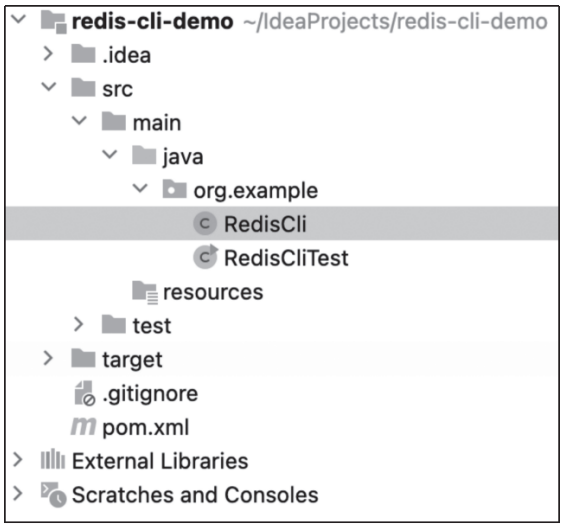


图5-1 目结构图

2. 创建项目依赖，添加jedis相关的依赖包。

```
<?xml version="1.0" encoding="UTF-8"?>
<project>
    <modelVersion>4.0.0</modelVersion>
    <groupId>demo</groupId>
    <artifactId>redis-cli-demo</artifactId>
    <version>1.0</version>

    <dependencies>
        <!-- 官方客户端 -->
        <dependency>
            <groupId>redis.clients</groupId>
            <artifactId>jedis</artifactId>
            <version>5.0.2</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <!-- 打包可执行 jar -->
            <plugin>
```

```
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-jar-plugin</artifactId>
            <version>3.3.0</version>
            <configuration>
              <archive>
                <manifest>
                  <mainClass>Main</mainClass>
                </manifest>
              </archive>
            </configuration>
          </plugin>
          <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <configuration>
              <source>8</source>
              <target>8</target>
            </configuration>
          </plugin>
        </plugins>
    </build>
</project>
```

3. 编写程序源代码，实现常用的功能。

```
package org.example;
import redis.clients.jedis.*;
import redis.clients.jedis.resps.Tuple;

import java.util.List;
import java.util.Map;
import java.util.Set;

/**
```

```
 * 纯 Jedis 封装 Redis 常用命令
 * 不做任何框架依赖，直接 new 即可使用
 */
public class RedisCli {
    private final JedisPool pool;

    public RedisCli(String host, int port) {
        // 连接池默认配置
        pool = new JedisPool(host, port);
    }

    /* ---------- String ---------- */
    public void set(String key, String value) {
        try (Jedis jedis = pool.getResource()) {
            jedis.set(key, value);
        }
    }

    public String get(String key) {
        try (Jedis jedis = pool.getResource()) {
            return jedis.get(key);
        }
    }

    /* 递减库存，返回最新值 */
    public Long decr(String key) {
        try (Jedis jedis = pool.getResource()) {
            return jedis.decr(key);
        }
    }

    /* ---------- Hash ---------- */
```

```
public void hset(String key, Map<String, String> fieldValues) {
    try (Jedis jedis = pool.getResource()) {
        jedis.hset(key, fieldValues);
    }
}

public Map<String, String> hgetAll(String key) {
    try (Jedis jedis = pool.getResource()) {
        return jedis.hgetAll(key);
    }
}

/* ---------- List ---------- */
public void lpush(String key, String... values) {
    try (Jedis jedis = pool.getResource()) {
        jedis.lpush(key, values);
    }
}

public List<String> lrange(String key, long start, long stop) {
    try (Jedis jedis = pool.getResource()) {
        return jedis.lrange(key, start, stop);
    }
}

/* 阻塞弹出，模拟简单队列消费 */
public List<String> blpop(int timeout, String key) {
    try (Jedis jedis = pool.getResource()) {
        return jedis.blpop(timeout, key);
    }
}
```

```
/* ---------- Set ---------- */
public void sadd(String key, String... members) {
    try (Jedis jedis = pool.getResource()) {
        jedis.sadd(key, members);
    }
}

public Set<String> smembers(String key) {
    try (Jedis jedis = pool.getResource()) {
        return jedis.smembers(key);
    }
}

/* ---------- Sorted-Set ---------- */
public void zadd(String key, double score, String member) {
    try (Jedis jedis = pool.getResource()) {
        jedis.zadd(key, score, member);
    }
}

public Set<Tuple> zrangeWithScores(String key, long start, long stop) {
    try (Jedis jedis = pool.getResource()) {
        return (Set<Tuple>) jedis.zrangeWithScores(key, start, stop);
    }
}

/* ---------- Key 过期 ---------- */
public void expire(String key, int seconds) {
    try (Jedis jedis = pool.getResource()) {
        jedis.expire(key, seconds);
    }
}
```

```
/* ---------- 事务 ---------- */
public void txDemo(String key, int delta) {
    try (Jedis jedis = pool.getResource()) {
        Transaction tx = jedis.multi();
        tx.incrBy(key, delta);
        tx.expire(key, 60);
        List<Object> results = tx.exec();
        System.out.println("事务执行结果：" + results);
    }
}

/* ---------- 管道 ---------- */
public void pipeDemo() {
    try (Jedis jedis = pool.getResource()) {
        Pipeline p = jedis.pipelined();
        for (int i = 0; i < 1000; i++) {
            p.set("pipe:" + i, String.valueOf(i));
        }
        p.sync(); // 一次性发送
    }
    System.out.println("管道批量 1000 条写入完成");
}

/* ---------- 发布订阅（简单演示）---------- */
public void subscribe(String channel) {
    new Thread(() -> {
        try (Jedis jedis = pool.getResource()) {
            jedis.subscribe(new JedisPubSub() {
                @Override
                public void onMessage(String ch, String msg) {
                    System.out.println("收到消息 频道=" + ch + " 内容=" + msg);
```

```
                }
            }, channel);
        }
    }).start();
}

public void publish(String channel, String message) {
    try (Jedis jedis = pool.getResource()) {
        jedis.publish(channel, message);
    }
}

/* 关闭连接池 */
public void close() {
    pool.close();
}
}
```

本章小结

本章全面系统地介绍了关系型数据库MySQL与非关系型数据库Redis的核心概念、操作方法及协同应用，强调二者在现代数据管理中的互补性与融合价值。MySQL作为关系型数据库的代表，以表结构、SQL语言和ACID特性保障数据的一致性与完整性，适用于结构化数据的复杂查询与事务处理；Redis则以内存键值存储、丰富的数据结构和持久化机制，在高并发、低延迟场景中发挥缓存与实时计算的优势。通过“Redis缓存+MySQL持久化”的架构模式，可有效提升系统性能与稳定性。通过理论学习与实践案例相结合，学生不仅掌握了CRUD、索引、事务、约束等MySQL核心技能，也熟练运用了Redis的五种数据类型及其在缓存、队列、排行榜等场景中的应用，为后续构建高性能、高可用的数据驱动型应用奠定了坚实基础。

练习与实践

简答题

（1）MySQL 中主键约束与唯一约束有何区别？

（2）简述 ACID 四大特性的含义及其在业务中的意义。

（3）Redis 的 RDB 与 AOF 两种持久化方式有何不同？

（4）列表（List）与集合（Set）在 Redis 中的最大区别是什么？

（5）MySQL 里 FOREIGN KEY 级联选项 CASCADE 与 RESTRICT 的区别？

（6）简述 Redis 中 Sorted Set 的排序机制及应用场景。

实践题

（1）基于 Navicat，新建数据库 cloth_shop，创建顾客表 customers(字段自拟)，要求含主键、邮箱唯一、年龄 0–150 的 CHECK 约束，并插入 5 条测试数据。

（2）使用 Jedis 编写 Java 程序：向 Redis 有序集合 ss:priceRank 写入 16 条服装商品的价格与名称，然后读取并打印价格最低的 3 件商品。

（3）结合 fz_products 表，写一条 SQL 完成“库存不足 50 且价格高于 100 元”的商品列表，并按库存升序、价格降序输出，最后将结果导出为 CSV 文件。



服装智能系统软件开发项目教程

基于 Vue +Spring Boot

第 6 章

Spring Boot 开发



本章导读

Spring Boot作为Spring框架的拓展，借助自动配置与起步依赖，极大程度地简化了基于Spring的应用开发工作，在开启Spring Boot项目之前，需要准备JDK 8及以上版本、Maven工具以及IDE开发环境，项目创建支持三种主要方式：通过Spring Initializr在线生成、利用IDE内置工具创建，或者基于Maven进行手工配置，典型的Spring Boot项目结构遵循Maven的约定，通过整合Web模块，Spring Boot会自动配置内嵌服务器（默认采用Tomcat），并简化JSON序列化操作，开发者仅需聚焦于业务逻辑的实现。

学习目标

1.掌握Spring Boot如何通过自动配置和起步依赖来简化Spring应用的初始搭建和开发过程，理解其“约定大于配置”的核心思想。

2.能够独立准备并配置Spring Boot开发所需的基本环境，包括正确版本的JDK、项目管理工具（Maven/Gradle）以及集成开发环境（IDE）。

3.熟练运用Spring Boot项目的三种创建方式（Spring Initializr、IDE工具、手动配置），并能根据项目需求选择最合适的一种。

4.认识并理解Spring Boot标准项目的目录结构及其约定，明确主启动类、配置文件、控制器、静态资源等各组成部分的位置与作用。

思政目标及设计建议

将Spring Boot开发与服装类专业深度融合，并融入思想政治教育，是极具创新性且契合“课程思政”理念的研究方向。本章将阐述思政目标及设计，建议如表6-1所示。

表6-1 第6章思政目标及设计建议

思政目标	思政元素及融入
培养文化自信与民族自豪感	如同SpringBoot集成了Spring生态的精华并简化了开发流程，使开发者能站在巨人肩膀上创新
锤炼严谨规范、精益求精的工匠精神	SpringBoot项目中精准地注解使用与规范的项目结构，正如高级定制服装中“一丝一线皆需精准”的工匠准则

6.1 Spring Boot概述



Spring Boot显著简化了Spring应用的开发流程，其秉持“约定优于配置”的核心理念，使开发人员摆脱烦琐的配置工作，从而能够更专注于业务逻辑的实现。利用Spring Boot，仅需经过简单的步骤，即可迅速搭建并运行一个独立且可部署的产品级应用，无需进行复杂的配置操作，它内置了大量常用的开发依赖与自动化配置，切实达成了“开箱即用”的高效开发模式，让应用从开发到部署的全流程更为简便和顺畅。

J2EE作为企业级应用开发的主流平台，长期以来存在开发过程烦琐、配置项目众多、开发效率低下、部署流程复杂以及第三方技术集成难度较高等一系列问题。具体来说，开发人员需要处理大量的XML配置文件，这些配置不仅数量庞大，而且内容复杂，导致开发周期显著延长。同时，由于框架本身的复杂性，开发团队在实现业务逻辑时往往需要编写大量重复性代码，严重影响了开发效率。

快速构建独立的Spring项目，并实现与主流框架的无缝集成。采用嵌入式servlet容器，应用无需打包成war格式。starters组件自动处理依赖关系和版本控制，提供大量自动配置功能，显著简化开发流程。同时，支持自定义默认配置，无需编写xml文件，无需代码生成，真正做到开箱即用。此外，提供准生产环境下的运行时应用监控，并天然支持云计算集成。

6.2 Spring Boot快速入门



本节通过以下内容使读者能够快速入门，主要包括：Spring Boot目录结构以及Spring Boot项目的pom.xml文件。首先会详细剖析Spring Boot项目的标准目录结构，让读者对项目的各个组成部分有清晰地认识；接着会深入讲解Spring Boot项目的pom.xml文件，包括其中各个依赖项的配置和作用等。

6.2.1 Spring Boot目录结构

1.src/main/java：这是存放 Java 源代码文件的目录，通常按照包结构进行组织。该目录包含了业务逻辑以及其他 Java 类。

2.src/main/resources：此目录用于存放应用的配置信息，例如应用名称、服务端口、数据库配置等。若应用包含 Web 模块，则会包含 static 和 templates 目录，分别用于存储静态资源和 Web 页面模板文件。

3.src/test/java：该目录包含项目的测试源代码文件，主要用于单元测试和集成测试。

4.src/test/resources：此目录包含测试配置文件和测试资源文件，用于存放测试用例数据及其他与测试相关的文件。

6.2.2 Spring Boot项目的pom.xml文件

Maven项目的核心配置文件，即POM（Project Object Model）文件，用于详细定义和管理项目的依赖关系、插件配置以及整体构建过程。该文件通过XML格式组织内容，不仅明确声明了项目所需的外部库和框架，还指定了构建生命周期中各阶段的具体任务，如编译、测试、打包和部署等。通过配置POM，开发者能够确保项目构建的一致性和可重复性，同时有效管理依赖项的版本和范围，从而提升开发效率和项目的可维护性。

在Spring Boot项目中，pom.xml文件通常包含项目的基本信息，如项目坐标（groupId、artifactId、version），这些信息唯一标识了项目在Maven仓库中的位置。此外，该文件还定义了项目的父POM，通过继承父POM来获取一系列默认配置和依赖管理，这极大地简化了配置工作。在依赖管理部分，pom.xml详细列出了项目所需的各种库和框架，并指定了它们的版本，Maven会自动下载这些依赖项并管理它们之间的依赖关系，避免了版本冲突的问题。同时，pom.xml还支持配置各种插件，这些插件可以在构建生命周期的不同阶段执行特定的任务，如代码生成、资源处理、测试报告生成等，进一步增强了项目的构建能力和灵活性。

6.2.3 Spring Boot项目配置文件

1.application.properties、application.yml这两种类型的文件主要用于存放程序中各类依赖模块的配置信息。在一个Spring Boot项目里，会有众多的依赖模块，这些模块需要特定的配置才能正常运行，而application.properties和application.yml就承担起了存储这些配置信息的重任，它们能够让开发者方便地对各个依赖模块进行参数设置和调整。

2.static目录在Spring Boot项目中有着专门的用途，它是专门用来存放HTML、CSS、JavaScript等静态资源文件的地方。HTML文件是构建网页结构的基础，CSS文件用于美化网页的样式，JavaScript文件则可以为网页添加交互功能。将这些静态资源文件统一存放在static目录下，有助于项目的管理和维护。

3.templates目录的作用是存放各类模板文件，例如JSP、Thymeleaf等。模板文件在项目中用于生成动态内容，JSP是Java Server Pages的缩写，它可以将Java代码和HTML代码结合起来，生成动态的网页。Thymeleaf是一个现代的服务器端Java模板引擎，它可以在Web和非Web环境中使用，能够方便地生成HTML页面。将这些模板文件存放在templates目录下，能够使项目的结构更加清晰。

4.mapper目录是针对使用Mybatis的项目而设置的，在这个项目中，此处用于存放Mybatis的映射文件及配置文件。Mybatis是一个优秀的持久层框架，它通过映射文件将Java对象与数据库表进行映射，

从而实现对数据库的操作。配置文件则用于对 Mybatis 进行一些基本的配置，如数据库连接信息等。将 Mybatis 的映射文件和配置文件存放在 mapper 目录下，有利于对 Mybatis 相关的文件进行集中管理。

6.2.4 Spring Boot启动类

Spring Boot项目的启动类通常位于src/main/java的根目录下，使用@SpringBootApplication注解标记，并包含main方法作为程序入口。

```
// 启动类
@SpringBootApplication
public class DemoApplication {
public static void main(String[] args) {
SpringApplication.run(DemoApplication.class, args);
}
}
```

6.2.5 Spring Boot在Controller中常用注解

1.@Controller注解

在SpringBoot中，@Controller注解是一个非常关键的注解，它标记一个类作为Spring MVC的控制器。这个注解的主要作用是将特定的请求映射到控制器的方法上，并支持视图解析，即可以将方法的返回值解析为视图，并渲染到客户端，需要创建一个Java类，并使用@Controller注解来标注这个类

```
@Controller
public class HelloController {
}
```

2.@RestController注解

在Spring框架中，@RestController注解专门用于构建RESTful Web服务，它融合了@Controller和@ResponseBody的双重功能。当一个类被@RestController注解标注时，其所有方法返回的数据将直接写入HTTP响应体，通常以JSON或XML格式呈现。为此，需创建一个Java类，并使用@RestController注解对其进行标注。

```
@RestController
public class HelloController {
}
```

3.@RequestMapping注解

@RequestMapping是Spring框架中用于将HTTP请求映射到控制器方法的注解。它是Spring MVC

和REST控制器中最常用的注解之一。通过使用@RequestMapping，开发者可以将特定的请求或请求模式映射到相应的控制器，从而指定处理该请求的具体方法。

（1）@RequestMapping注解同样适用于类级别。当应用于类级别时，该注解意味着该类中的所有方法将统一处理映射到该类路径下的请求。为此，需要创建一个Java类，并通过@RequestMapping注解对其进行标注。

```
@RestController
@RequestMapping
public class HelloController {
}
```

（2）@RequestMapping注解不仅适用于类级别，同样也可用于方法级别。在方法级别上，@RequestMapping能够更细致地定义请求映射。为此，需创建一个Java方法，并通过@RequestMapping注解对该方法进行标注。

```
@RestController
public class HelloController {
@RequestMapping
    public void testHello(){
}
}
```

4.@GetMapping注解

@GetMapping注解是Spring Boot中用于处理HTTP GET请求的常用注解。该注解能够协助开发者定义并处理特定URL路径的GET请求。实际上，@GetMapping是@RequestMapping(method=RequestMethod.GET)的一种便捷替代方式。

```
@RestController
public class HelloController {
@GetMapping
    public void testHello(){
}
}
```

5.@PostMapping注解

@PostMapping是Spring MVC框架中用于处理HTTP POST请求的专用注解，常用于实现RESTful API的数据提交功能。作为@RequestMapping注解的特化版本，它简化了POST请求的映射过程，提升了开发效率。

```
@RestController
public class HelloController {
@PostMapping
    public void testHello(){
}
}
```

6.3 Spring Boot整合Web应用



本节主要介绍以下内容，主要包括：Spring Boot访问静态资源的方式，以及如何通过配置优化静态资源的访问性能；同时讲解Spring Boot与Thymeleaf模板引擎的整合步骤，包括依赖引入、配置方法及基础使用示例，帮助开发者快速构建动态Web页面。

6.3.1 Spring Boot访问静态资源

在Spring Boot中，可以通过配置静态资源路径或自定义映射来实现对主机资源的访问，目前访问静态资源有两种方式：

1.第一种方式：默认静态资源访问，Spring Boot 默认支持以下目录作为静态资源路径

（1）classpath:/static/：根目录下创建一个static目录，存放静态资源

（2）classpath:/public/：根目录下创建一个public目录，存放静态资源

（3）classpath:/resources/：根目录下创建一个resources目录，存放静态资源

（4）classpath:/META-INF/resources/：根目录的META-INF目录下创建一个resources目录，存放静态资源

【例6-1】创建一个Spring Boot项目，将静态资源存放在默认静态资源目录下，实现直接访问静态资源，如图6-1所示。

将资源文件放入上述目录后，可以通过以下URL直接访问：

```
http://localhost:8080/asserts/js/bootstrap.min.js
```

综上代码所述，呈现如下效果，如图6-2所示。

2.第二种方式：自定义静态资源路径，如果需要自定义静态资源路径，可以通过修改配置文件或编写Java 配置类。

（1）修改配置文件：application.properties或application.yml。

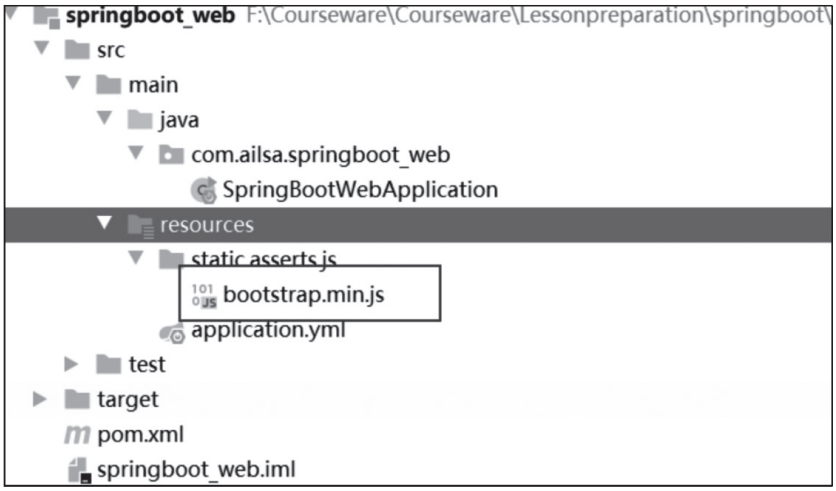


图6-1　静态资源引入项目



```
/*!
  * Bootstrap v4.0.0 (https://getbootstrap.com)
  * Copyright 2011-2018 The Bootstrap Authors (https://github.com/twbs/bootstrap/graphs/contributors)
  * Licensed under MIT (https://github.com/twbs/bootstrap/blob/master/LICENSE)
  */
!function(t,e){"object"==typeof exports&&"undefined"!=typeof module?e(exports,require("jquery"),require("popper.js")):"function"==typeof define&&define.amd?
define(["exports","jquery","popper.js"],e):e(t.bootstrap={},t.jQuery,t.Popper)}(this,function(t,e,n){"use strict";function i(t,e){for(var n=0;n<e.length;n++){var
i=e[n];i.enumerable=i.enumerable||!1,i.configurable=!0,"value"in i&&(i.writable=!0),Object.defineProperty(t,i.key,i)}}function s(t,e,n){return e&&i(t.prototype,e),n&&i(t,n),t}function r()
{return(r=Object.assign||function(t){for(var e=1;e<arguments.length;e++){var n=arguments[e];for(var i in n)Object.prototype.hasOwnProperty.call(n,i)&&(t[i]=n[i])}return
t}).apply(this,arguments)}e=e&&e.hasOwnProperty("default")?e.default:e,n=n&&n.hasOwnProperty("default")?n.default:n;var o,a,l,h,c,u,f,d,_,g,p,m,v,E,T,y,C,I,A,b,D,S,w,N,O,k,P=function(t){var
e=!1;function n(e){var n=this,s=!1;return t(this).one(i.TRANSITION_END,function(){s=!0}),setTimeout(function(){s||i.triggerTransitionEnd(n)},e),this}var i=
{TRANSITION_END:"bsTransitionEnd",getUID:function(t){do{t+=~~(1e6*Math.random())}while(document.getElementById(t));return t},getSelectorFromElement:function(e){var n,i=e.getAttribute("data-
target");i&&"#"!==i||(i=e.getAttribute("href")||""),"#"===i.charAt(0)&&(n=i,i=n="function"==typeof t.escapeSelector?t.escapeSelector(n).substr(1):n.replace(/(:|\.|\
[|\]|,|=|@)/g,"\\$1"));try{return t(document).find(i).length>0?i:null}catch(t){return null}},reflow:function(t){return t.offsetHeight},triggerTransitionEnd:function(n)
{t(n).trigger(e.end)},supportsTransitionEnd:function(){return Boolean(e)},isElement:function(t){return(t[0]||t).nodeType},typeCheckConfig:function(t,e,n){for(var s in
n)if(Object.prototype.hasOwnProperty.call(n,s)){var r=n[s],o=e[s],a=o&&i.isElement(o)?"element":(l=o,{}.toString.call(l).match(/\s([a-zA-Z]+)/)[1].toLowerCase());if(!new
RegExp(r).test(a))throw new Error(t.toUpperCase()+': Option "'+s+'" provided type "'+a+'" but expected type "'+r+'".')}var l}};return e="undefined"==typeof window||!window.QUnit&&
{end:"transitionend"},t.fn.emulateTransitionEnd=n,i.supportsTransitionEnd()&&(t.event.special[i.TRANSITION_END]={bindType:e.end,delegateType:e.end,handle:function(e)
{if(t(e.target).is(this))return e.handleObj.handler.apply(this,arguments)}}),i}(e),L=(a="alert",h="."+(l="bs.alert"),c={o=o(e).fn[a],u=
{CLOSE:"close"+h,CLOSED:"closed"+h,CLICK_DATA_API:"click"+h+".data-api"},f="alert",d="fade",_="show",g=function(){function t(t){this._element=t}var e=t.prototype;return e.close=function(t)
{t=t||this._element;var e=this._getRootElement(t);this._triggerCloseEvent(e).isDefaultPrevented()||this._removeElement(e)},e.dispose=function()
{o.removeData(this._element,l),this._element=null},e._getRootElement=function(t){var e=P.getSelectorFromElement(t),n=!1;return e&&(n=o(e)[0]),n||(n=o(t).closest("."+f)
[0]),n},e._triggerCloseEvent=function(t){var e=o.Event(u.CLOSE);return o(t).trigger(e),e},e._removeElement=function(t){var
e=this;o(t).removeClass(_),P.supportsTransitionEnd()&&o(t).hasClass(d)?o(t).one(P.TRANSITION_END,function(n){return
e._destroyElement(t,n)}).emulateTransitionEnd(150):this._destroyElement(t)},e._destroyElement=function(t){o(t).detach().trigger(u.CLOSED).remove()},t._jQueryInterface=function(e){return
this.each(function(){var n=o(this),i=n.data(l);i||(i=new t(this),n.data(l,i)),"close"===e&&i[e](this)})},t._handleDismiss=function(t){return function(e)
{e&&e.preventDefault(),t.close(this)}},s(t,null,[{key:"VERSION",get:function(){return"4.0.0"}}]),t}(),o(document).on(u.CLICK_DATA_API,'[data-dismiss="alert"]',g._handleDismiss(new
g)),o.fn[a]=g._jQueryInterface,o.fn[a].Constructor=g,o.fn[a].noConflict=function(){return o.fn[a]=c,g._jQueryInterface},g),R=(m="button",E="."+(v="bs.button"),T=".data-api",y=
(p=e).fn[m],C="active",I="btn",A="focus",b='[data-toggle^="button"]',D='[data-toggle="buttons"]',S="input",w=".active",N=".btn",O=
{CLICK_DATA_API:"click"+E+T,FOCUS_BLUR_DATA_API:"focus"+E+T+" blur"+E+T},k=function(){function t(t){this._element=t}var e=t.prototype;return e.toggle=function(){var
t=!0,e=!0,n=p(this._element).closest(D)[0];if(n){var i=p(this._element).find(S)[0];if(i){if("radio"===i.type)if(i.checked&&p(this._element).hasClass(C))t=!1;else{var s=p(n).find(w)
[0];s&&p(s).removeClass(C)}if(t)
{if(i.hasAttribute("disabled")||n.hasAttribute("disabled")||i.classList.contains("disabled")||n.classList.contains("disabled"))return;i.checked=!p(this._element).hasClass(C),p(i).trigger("ch
ange")}i.focus(),e=!1}e&&this._element.setAttribute("aria-pressed",!p(this._element).hasClass(C)),t&&p(this._element).toggleClass(C)},e.dispose=function()
{p.removeData(this._element,v),this._element=null},t._jQueryInterface=function(e){return this.each(function(){var n=p(this).data(v);n||(n=new t(this),p(this).data(v,n)),"toggle"===e&&n[e]
()})},s(t,null,[{key:"VERSION",get:function(){return"4.0.0"}}]),t}(),p(document).on(O.CLICK_DATA_API,b,function(t){t.preventDefault();var e=t.target;p(e).hasClass(I)||
(e=p(e).closest(N)),k._jQueryInterface.call(p(e),"toggle")}).on(O.FOCUS_BLUR_DATA_API,b,function(t){var e=p(t.target).closest(N)[0];p(e).toggleClass(A,/^focus(in)?
$/.test(t.type))}),p.fn[m]=k._jQueryInterface,p.fn[m].Constructor=k,p.fn[m].noConflict=function(){return p.fn[m]=y,k._jQueryInterface},k),j=function(t){var
e="carousel",n="bs.carousel","."+n,o=t.fn[e],a={interval:5e3,keyboard:!0,slide:!1,pause:"hover",wrap:!0},l={interval:"(number|boolean)",keyboard:"boolean",slide:"(boolean|string)",pause:"
(string|boolean)",wrap:"boolean"},h="next",c="prev",u="left",f="right",d=
{SLIDE:"slide"+i,SLID:"slid"+i,KEYDOWN:"keydown"+i,MOUSEENTER:"mouseenter"+i,MOUSELEAVE:"mouseleave"+i,TOUCHEND:"touchend"+i,LOAD_DATA_API:"load"+i+".data-
api",CLICK_DATA_API:"click"+i+".data-api"},_="carousel",g="active",p="slide",m="carousel-item-right",v="carousel-item-left",E="carousel-item-next",T="carousel-item-prev",y=
```

图6-2　查看静态资源内容

```
spring:
  web:
    resources:
      static-locations: classpath:/hello/,classpath:/them # 自定义资源目录
```

（2）编写 Java 配置类：编写一个配置类实现 WebMvcConfigurer 接口，示例代码如下：

```
@Configuration
public class MySpringMvcConfig implements WebMvcConfigurer {
    /**
```

```
     * 添加静态资源，外部可以直接访问地址，
     * @param registry
     */
    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        //第一个设置访问路径，对外暴露访问的路径
        // 第二个设置资源路径，内部文件放置的目录
            registry.addResourceHandler("/").addResourceLocations("/login");     registry.addResourceHandler("/
static/**").addResourceLocations("classpath:/static/");
            WebMvcConfigurer.super.addResourceHandlers(registry);
        }
    }
```

6.3.2 Spring Boot整合Thymeleaf

Thymeleaf是一款面向Web及独立环境的服务器端Java模板引擎，能够高效处理HTML、XML、JavaScript、CSS乃至纯文本数据。其主要目标是提供一种浏览器可正确显示且格式规范的模板创建方法，因此也适用于静态建模。

Thymeleaf作为一个XML/XHTML/HTML5模板引擎，广泛应用于Web及非Web环境中的开发。它是一个基于Apache License 2.0许可的开源Java库，由Daniel Fernández创建，此人亦是Java加密库Jasypt的作者。

Thymeleaf提供了一款可选模块，用于与Spring MVC的整合。在应用开发过程中，开发者可以选用Thymeleaf完全替代JSP或其他模板引擎，如Velocity、FreeMarker等。其主要目标是确保模板在浏览器中正确显示且格式规范，从而也可用于静态建模。通过Thymeleaf，开发者可以创建经过验证的XML与HTML模板。相较于编写逻辑或代码，只需将特定标签属性添加至模板中，这些属性便会在DOM（文档对象模型）上执行预定义的逻辑。Thymeleaf模板引擎的特点是：开箱即用，Thymeleaf允许处理多种模板，每种模板称为模板模式，主要包括：

1.XML（可扩展标记语言）是一种用于编码文档的结构化标记语言，它被设计用来传输和存储数据，同时兼顾人类可读性和机器可处理性。有效的XML文档必须严格遵守XML语法规则，包括正确的标签嵌套、属性引用以及文档结构完整性，这样才能确保数据交换的准确性和一致性。

2.XHTML（可扩展超文本标记语言）是XML和HTML的结合体，它继承了XML的严格语法要求，同时保持了HTML的语义和功能，旨在推动Web内容向更规范、更兼容的方向发展。有效的XHTML文档不仅需要符合XHTML的规范标准，还必须通过相关的验证检查，以保证在不同浏览器和设备上的正确

渲染与交互。

3.HTML5作为最新的超文本标记语言标准，引入了许多新特性和API，极大地丰富了Web应用的功能和用户体验，同时加强了对多媒体和移动设备的支持。旧版HTML5则指的是HTML5标准发展过程中的早期版本或特定实现，它们可能包含一些已被修订或废弃的特性，但在某些历史环境或特定项目中仍有应用。

下面详细说明一下Thymeleaf的使用，主要包括以下步骤。

1.引入Thymeleaf模板引擎的依赖，创建一个Spring Boot项目将依赖引入到pom.xml文件。

```
<!--引入thyemleaf的模板-->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
```

2.Thymeleaf模板引擎使用

在IDEA中，使用CTRL+N快捷键搜索ThymeleafProperties类，查看其源代码。只需将HTML页面放置于classpath:/templates/目录下，thymeleaf目录下静态资源即可进行渲染。

```
/**
 * Thymeleaf 模板引擎配置属性类
 * 用于映射 application.properties/yml 中以 spring.thymeleaf 为前缀的配置项
 *
 * 通过 @ConfigurationProperties 注解将配置文件中的属性绑定到该类的字段上
 * 前缀为 "spring.thymeleaf"，对应配置文件中的 spring.thymeleaf.* 属性
 */
@ConfigurationProperties(prefix = "spring.thymeleaf")
public class ThymeleafProperties {
    /**
     * 默认编码格式 - UTF-8
     * 用于模板文件的编码处理
     */
    private static final Charset DEFAULT_ENCODING = Charset.forName("UTF-8");
      /**
     * 默认内容类型 - text/html
     * 指定渲染后的输出内容类型
```

```
 */
private static final MimeType DEFAULT_CONTENT_TYPE = MimeType.valueOf("text/html");
/**
 * 默认模板文件前缀路径 – classpath:/templates/
 * 表示模板文件存放在类路径的 templates 目录下
 */
public static final String DEFAULT_PREFIX = "classpath:/templates/";

/**
 * 默认模板文件后缀 – .html
 * 表示模板文件使用 .html 作为文件扩展名
 */
public static final String DEFAULT_SUFFIX = ".html";
/**
 * 是否检查模板是否存在
 * 设置为 true 时，在启动阶段会检查模板是否存在
 * 开发环境下建议开启，生产环境下可关闭以提高启动速度
 */
private boolean checkTemplate = true;
/**
 * 是否检查模板位置是否存在
 * 设置为 true 时，在启动阶段会检查模板目录是否存在
 */
private boolean checkTemplateLocation = true;
/**
 * 模板文件前缀路径
 * 默认值："classpath:/templates/"
 * 可自定义配置，如："file:/path/to/templates/"
 */
private String prefix = "classpath:/templates/";
/**
 * 模板文件后缀
```

```
     * 默认值：".html"
     * 可自定义配置，如：".tpl"
     */
    private String suffix = ".html";
    /**
     * Thymeleaf 模板模式
     * 默认值："HTML5"
     * 其他可选值："HTML", "XML", "TEXT", "JAVASCRIPT", "CSS", "RAW"
     */
    private String mode = "HTML5";
    // 注：实际类中通常还会有对应的 getter 和 setter 方法
}
```

3.Thymeleaf模板引擎使用步骤

（1）在Spring Boot项目中编写test.html文件。

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>成功页面</title>
</head>
<body>
    SUCCESS
</body>
</html>
```

（2）修改Spring Boot项目中编写test.html文件，引入Thymeleaf模板引擎的命名空间。

```
<html xmlns:th="http://www.thymeleaf.org">
```

（3）在Spring Boot项目中，默认情况下，templates目录下的模板文件受到保护，不能通过URL直接访问。这些文件需要经过模板引擎渲染后才能展示。如果需要访问templates文件夹中的HTML文件，虽然可以通过修改配置文件来实现直接访问，但这种方法违背了模板引擎的设计原则，并不推荐使用。通常情况下，应该通过Controller控制器来转发访问，这样可以更好地实现MVC架构的分离和业务逻辑处理。

```
spring:
```

```
web:
  resources:
    static-locations: classpath:/hello/,classpath:/them/,classpath:/templates/
```

4.Thymeleaf模板引擎基本语法

Thymeleaf是一个现代的Java模板引擎，主要用于生成动态Web页面。它可以与Spring Boot无缝集成，提供了丰富的功能和灵活的语法。

（1）th:action属性用于指定处理表单提交的后台控制器路径，其功能类似于传统HTML中<form>标签的action属性。通过该属性，开发者可以将用户输入的数据准确传递到指定的后端处理程序，实现表单数据的接收与处理，从而确保Web应用程序的交互功能正常运行。

```
<body>
    <h1>success</h1>
    <!--提交的请求地址是/test/hello-->
    <form id="login-form" th:action="@{/test/hello}">
        <button>提交</button>
    </form>
</body>
```

（2）在Thymeleaf模板引擎中，通过使用`th:text`属性，可以动态地设置HTML元素的文本内容。该属性允许开发者在模板中绑定数据模型中的变量或表达式，从而在页面渲染时将这些值作为元素的文本内容进行显示。通过这种方式，能够实现前后端数据的无缝集成与动态更新，提升页面的交互性和灵活性。

```
<td th:text="${user.id}"></td>
```

（3）th:each是Thymeleaf模板引擎中用于对象遍历的核心属性，其功能类似于JSTL（JSP标准标签库）中的<c:forEach>标签，主要用于在页面模板中对集合或数组进行迭代操作。通过th:each属性，开发者可以方便地遍历数据集合，动态生成重复的HTML结构，从而实现数据的列表展示或其他循环渲染需求。该属性支持多种迭代场景，包括对List、Set、Map等集合类型的遍历，并允许在迭代过程中访问当前元素、索引和状态信息，提供了灵活且强大的数据展示能力。

```
<tr th:each="user,userStat:${list}">
        <td>
            下标：<span th:text="${userStat.index}"></span>,
        </td>
        <td th:text="${user.id}"></td>
        <td th:text="${user.username}"></td>
```

```
        <td th:text="${user.age}"></td>
    </tr>
```

（4）在Java编程中，通过th:map方法可以高效地遍历Map集合。这种方法允许开发人员逐个访问集合中的每个键值对，并对它们执行相应的操作。无论是处理简单的键值数据，还是需要动态操作复杂结构，th:map都提供了一种灵活且直观的方式来实现遍历功能。

```
<div th:each="map,mapStat:${dataMap}">
    <div th:text="${map}"></div>
    key:<span th:text="${mapStat.current.key}"></span><br/>
    value:<span th:text="${mapStat.current.value}"></span><br/>
</div>
```

（5）th:if是Thymeleaf模板引擎中用于条件判断的核心属性，它能够根据指定的布尔表达式的结果来决定是否渲染或显示某个HTML元素。当表达式的结果为true时，对应的元素会被渲染；如果结果为false，则元素不会被渲染或显示。这个属性在处理动态内容时非常有用，特别是在需要根据不同条件展示不同界面元素的情况下。

```
<div>
    <span th:if="${age}>19">
        比标准年龄大
    </span>
</div>
```

6.3.3 RESTful API的应用

REST（Representational State Transfer）是一种广泛应用的软件架构设计风格，而非严格的技术标准。它由Roy Fielding在2000年的博士论文中首次系统性地提出，旨在为分布式系统，特别是互联网服务，提供一组核心的设计原则和约束条件。这些原则包括无状态通信、统一的接口设计、资源导向以及客户端-服务器模型的分离等，共同构成了REST架构的基础。

由于其结构清晰、遵循通用标准、易于开发者理解和实现，同时具备良好的可扩展性和灵活性，REST架构在互联网软件开发领域日益受到青睐。目前，众多网站和Web服务，尤其是API设计，纷纷采用RESTful风格，以提升系统的可维护性、可伸缩性以及跨平台的互操作性，使其成为现代网络应用开发中主流的架构选择之一。

1.RESTful风格的特点

（1）每一个URI都明确地指向并标识出一种特定的资源，每一个资源都通过其URI被赋予了一个唯一的标识符，这种设计确保了资源的唯一性和可识别性。

（2）客户端与服务器之间通过特定的通信协议进行交互，以传递此类资源的表现形式，这种表现形式通常是对资源状态的某种抽象描述或数据封装，其目的在于实现信息的高效传输与解析。

（3）客户端通常通过四种标准的HTTP请求方法，即GET、POST、PUT和DELETE，对服务器端的资源进行创建、读取、更新与删除等基本操作，从而实现对REST架构风格中"表现层状态转化"这一核心设计原则的具体应用。

2.访问方式

（1）传统的访问方式

在这种传统访问方式中，参数直接拼接在URL后面，以问号作为分隔符，多个参数之间使用和号进行连接，例如：http://localhost:8080/xxxx?id=1.这种方式虽然简单直接，但存在一些明显的弊端。一方面，随着参数数量的增多，URL会变得冗长且难以阅读和维护；另一方面，安全性相对较低，因为所有参数都明文显示在URL中，容易被窃取或篡改。

【例6-2】创建一个Spring Boot项目，需要实现一个兼容的传统风格接口，采用@Controller与@ResponseBody组合的方式，通过"/tradition"路径接收id参数并在控制台记录日志，最终返回“这是传统方式访问的”固定文本响应。

编写TraditionController类，体现了传统访问方式下的控制器编写方法。

```
package com.ailsa.springboot.controller;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseBody;
import org.springframework.web.bind.annotation.RestController;
/**
 * @Description: 传统方式访问的控制器 - 使用@Controller注解配合@ResponseBody返回数据
 *               区别于@RestController，该类中的方法默认返回视图名称，需要@ResponseBody注解才能返回数据
 * @Version: 1.0
 */
@Controller
public class TraditionController {

    /**
     * 处理传统方式访问的请求
     * 使用@ResponseBody注解将方法返回值直接写入HTTP响应体，而不是解析为视图名称
```

```
     *
     * @param id 请求参数，从URL查询字符串或表单参数中获取
     * @return String 直接返回字符串数据作为响应内容
     */
    @RequestMapping("/tradition")
    @ResponseBody
    public String tradition(String id){
        // 打印接收到的id参数值到控制台，用于调试和日志记录
        System.out.println("接收到的id参数：" + id);
        // 返回固定的响应字符串
        return "这是传统访问方式";
    }
}
```

综上代码所述，呈现如下效果，如图6-3所示。

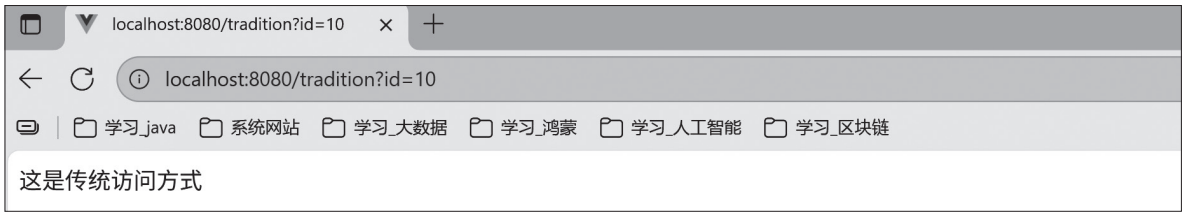


图6-3 传统访问方式的案例

在上述代码中，Spring MVC中传统访问方式的典型实现，它采用@Controller注解标明这是一个控制器类，但与传统页面跳转不同，该方法通过添加@ResponseBody注解实现了数据直接返回。当访问"/tradition"路径时，Spring会自动从请求参数中获取id值并绑定到方法参数，然后将返回的字符串"这是传统方式访问的"直接写入HTTP响应体，而不是像传统MVC那样解析为视图名称进行页面跳转，这种传统访问方式体现了Spring框架在RESTful风格普及前的典型用法，即通过注解组合来实现数据接口的功能，为理解现代@RestController的诞生背景提供了重要的历史参照。

（2）RESTful风格访问方式

例如，http://localhost:8080/xxxx/1。在RESTful风格访问方式中，通过在URL路径中直接嵌入资源标识符（如上述示例中的“1”），实现了对特定资源的直接访问。这种设计使得客户端能够清晰地理解请求的资源以及操作类型。RESTful风格强调资源的定位和操作的明确性，通过简洁的URL设计提高了系统的可读性和可维护性。与传统的访问方式相比，RESTful风格更加符合互联网开放、简洁的设计原则，因此在现代Web开发中得到了广泛应用。

【例6-3】创建一个Spring Boot项目，需要实现一个符合RESTful架构风格的接口，使用@RestController注解和路径参数技术，通过"/rest/{id}"的URL模式接收动态资源标识，将路径中的id参数提取并记录到日志中，最后返回"这是RESTful风格访问方式"的响应内容，以提供符合现代Web开发标准的API服务。

编写RestFulController类，体现了RESTful风格访问下的控制器编写方法。

```
package com.ailsa.springboot.controller;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
/**
 * @Description: RESTful风格访问方式的控制器
 *          使用@RestController注解，该类中所有方法默认返回数据而非视图
 *          采用RESTful架构风格，通过URL路径参数传递数据
 * @Version: 1.0
 */
@RestController // 组合注解，相当于@Controller + @ResponseBody，所有方法返回值直接写入响应体
public class RestFulController {

    /**
     * 处理RESTful风格访问的请求
     * 使用路径变量{id}作为参数，体现RESTful风格的URL设计
     *
     * @param id 路径参数，从URL路径中获取，例如：/rest/123 中的123
     * @return String 直接返回字符串数据作为响应内容
     */
    @RequestMapping("/rest/{id}") // 映射URL模式，{id}为路径占位符
    public String rest(@PathVariable(value = "id") String id){ // @PathVariable注解从URL路径中提取参数值
        // 打印接收到的路径参数值到控制台，用于调试和日志记录
        System.out.println("接收到的路径参数id: " + id);

        // 返回固定的响应字符串，标识此为RESTful风格访问
        return "这是RESTful风格访问方式";
```

```
    }
}
```

综上代码所述，呈现如下效果，如图6-4所示。

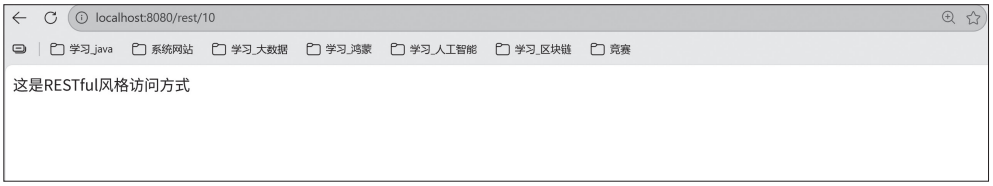


图6-4 RESTful风格访问方式的案例

在上述代码中，一个典型的RESTful风格控制器实现，使用@RestController注解标记类级别，表明所有方法返回值都将直接作为HTTP响应体内容。它通过@RequestMapping("/rest/{id}")定义了一个RESTful风格的接口路径，其中{id}作为路径参数，配合@PathVariable注解实现从URL路径中动态提取参数值。当访问诸如"/rest/123"这样的URL时，Spring会自动将路径中的123绑定到id参数，方法在执行过程中会打印接收到的参数值用于日志记录，最后返回"这是RESTful风格访问方式"的固定文本响应，充分体现了RESTful架构中资源定位和统一接口的设计原则。

3.HTTP请求方式

HTTP/1.0定义了三种请求方法：GET、POST和HEAD。HTTP/1.1协议则扩展至八种方法（有时也称为“动作”），用于指示对Request-URL指定资源的不同操作方式。HTTP/1.1新增了五种请求方法：OPTIONS、PUT、DELETE、TRACE和CONNECT，如表6-2所示。

表6-2 HTTP请求方式

方法	描述
GET	请求指定的页面信息，并返回实体主体
HEAD	类似于GET请求，只不过返回的响应中没有具体的内容，用于获取报头
POST	向指定资源提交数据进行处理请求，数据被包含在请求体中，POST请求可能会导致新的资源的建立或者已有资源的修改
PUT	从客户端向服务器传送的数据取代指定的文档的内容
DELETE	请求服务器删除指定页面
CONNECT	HTTP/1.1协议中预留给能够将连接改为管道方式的代理服务器
OPTIONS	允许客户端查看服务器的性能
TRACE	回显服务器收到的请求，主要用于测试或者诊断

【例6-4】开发一套符合RESTful架构风格的服装商品管理系统，通过标准的HTTP方法提供完整的查询操作接口，支持商品列表查询业务功能，为前端应用和移动端提供统一规范的数据交互服务，若需查看完整代码，请参阅源码。

（1）编写ClothingApiController类，系统对外提供的接口，将内部的服装业务数据以标准化的RESTful API形式暴露给各种客户端使用，是现代Web应用前后端分离架构的关键组成部分。

```
package com.ailsa.springboot.controller;

import com.ailsa.springboot.pojo.Clothing;
import com.ailsa.springboot.service.ClothingService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.web.bind.annotation.*;
import java.util.List;
/**
 * @Description: ClothingApiController RESTful API控制器
 *               提供服装商品的RESTful风格数据接口
 *               使用@RestController注解，所有方法返回值自动转换为JSON格式
 *               专为前端应用、移动端和第三方系统提供数据服务
 * @Version: 1.0
 */
@RestController // 标识为RESTful控制器，组合了@Controller和@ResponseBody功能
@RequestMapping("/api/clothes") // 定义基础请求路径，统一管理服装相关API接口
public class ClothingApiController {

    @Autowired // 自动注入ClothingService业务逻辑组件，实现控制层与服务层的解耦
    private ClothingService clothingService;

    /**
     * 获取所有服装列表 (JSON格式)
     * GET /api/clothes
     * 用途：为前端页面提供完整的服装商品数据，支持商品展示、列表渲染等功能
     *
     * @return List<Clothing> 服装对象列表，自动序列化为JSON格式返回给客户端
     */
    @GetMapping // 映射HTTP GET请求，对应查询操作
    public List<Clothing> getAllClothes() {
        // 记录请求日志，便于调试和监控API调用情况
```

```
        System.out.println("REST API: 获取所有服装列表");
        // 调用服务层获取数据，体现分层架构的设计思想
        return clothingService.getAllClothes();
    }
}
```

（2）编写 ThymeleafController类，负责页面渲染（服务端渲染）。

```
package com.ailsa.springboot.controller;

import com.ailsa.springboot.pojo.Clothing;
import com.ailsa.springboot.service.ClothingService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

import java.util.List;

/**
 * @Description: Thymeleaf页面控制器 – 负责服装商品的页面渲染和展示
 *               使用@Controller注解，专门处理服务器端页面渲染
 *               与RESTful API控制器分离，实现前后端职责分离
 * @Version: 1.0
 */
@Controller // 标识为页面控制器，返回视图名称而非直接数据
@RequestMapping("/clothes") // 定义页面访问的基础路径
public class ThymeleafController {
    @Autowired // 注入服装业务服务，复用相同的业务逻辑
    private ClothingService clothingService;

    /**
     * 服装商品列表页面 – 服务器端渲染的页面接口
     * GET /clothes/list
```

```
     * 用途：提供完整的HTML页面，包含服装商品的展示和布局
     * 特点：服务器端渲染，SEO友好，首屏加载快
     *
     * @param model Spring MVC模型对象，用于向模板传递数据
     * @return String 视图名称，对应Thymeleaf模板路径
     */
    @GetMapping("/list") // 处理GET请求，对应页面访问
    public String clothingList(Model model) {
        // 从服务层获取所有服装数据
        List<Clothing> clothes = clothingService.getAllClothes();
        // 将数据添加到模型，供Thymeleaf模板使用
        model.addAttribute("clothes", clothes); // 服装列表数据
        model.addAttribute("pageTitle", "服装商品列表"); // 页面标题

        // 记录日志，便于调试和监控页面访问情况
        System.out.println("Thymeleaf: 渲染服装列表页面，商品数量：" + clothes.size());

        // 返回Thymeleaf模板名称，实际路径为：templates/list.html
        return "list";
    }
}
```

（3）综上代码所述，呈现如下效果，如图6-5所示。



图6-5　RESTful风格使用的案例

6.4 SpringBoot整合持久层



在“服装商品管理”案例中，Controller层已经能够通过内存Map完成列表查询。然而，Map中的数据在应用重启后会全部丢失，无法满足真实生产环境“数据不丢、可恢复、可扩展”的基本要求。因此，把数据从内存迁移到关系型数据库，是Spring Boot项目从“演示”走向“可用”的第一步。Spring Boot对主流持久层框架均提供了开箱即用的Starter，通过“约定大于配置”的方式，几乎零XML即可完成整合。本节以Spring Data JPA为例，完成“服装表”的建库、建表、CRUD与单元测试，为后续进阶（MyBatis-Plus、多数据源、分页插件等）打下基础。

6.4.1 依赖与准备

在根pom.xml中添加两套starter：

1.spring-boot-starter-data-jpa 提供 ORM 能力与 Repository 机制

2.MySQL数据库驱动。

```
<!-- 1. JPA 启动器 -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- 2. MySQL 驱动 -->
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
</dependency>
```

6.4.2 数据源与 JPA 配置

在Spring Boot开发过程中，当需要对Spring Boot的默认约定进行覆盖时，只需在application.yml文件里追加如下的代码片段，就能够轻松实现对Spring Boot默认约定的覆盖操作：

这里需要给大家提供一些约定提示信息

1.在 Spring Boot 的开发规范中，把相关的配置信息写在 application.yml 文件当中，而不是采用 XML 文件的形式，这种做法是非常符合 Spring Boot 所倡导的“零 XML”开发理念的，它能够让项目的配置更加简洁和高效。

2.在开发或者演示阶段，我们可以将 ddl-auto 的属性值设置为 update，这样能够方便我们对数据库结构进行动态更新。不过需要特别注意的是，当项目正式上线之后，为了保证数据库结构的稳定性和安全性，应该把 ddl-auto 的属性值改为 validate，或者使用 Flyway 工具来进行版本化的数据库迁移操作。

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/clothing_db?serverTimezone=Asia/Shanghai&characterEncoding=utf8
    username: root
    password: 123456
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update        # 第一次启动自动建表；后续 update 不会清空数据
    show-sql: true            # 开发阶段打开，生产关闭
    properties:
      hibernate:
        format_sql: true
```

6.4.3 实体与仓储接口

若要将原本的Clothing POJO进行升级，使其转变为实体，其实操作并不复杂，仅仅需要使用3个注解就可以达成这一目标：

```
package com.ailsa.springboot.pojo;


import javax.persistence.*;


@Entity
@Table(name = "t_clothing")
public class Clothing {
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;          // 主键自增

    private String name;      // 服装名称
    private String category;  // 上衣/裤子/裙子
    private String brand;
    private Double price;
    private Integer stock;
    private String size;
    private String color;
    private String imageUrl;
    private String description;
    /* 省略 getter/setter/构造 */
}
```

写仓储接口只需继承，无需实现类：

```
package com.ailsa.springboot.repository;

import com.ailsa.springboot.pojo.Clothing;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ClothingRepository extends JpaRepository<Clothing, Long> {
    // 根据 Spring Data 命名规则，可一键生成查询
    List<Clothing> findByCategory(String category);
}
```

6.4.4 Service层解耦

在进行代码优化和架构调整的过程中，需要把原先使用内存Map来实现具体功能的ClothingService模块，改为采用面向接口编程的方式。这种改变的核心目的在于仅对其底层的具体实现进行替换，而不会对与之相关联的Controller部分造成任何改动，从而确保系统的稳定性和功能的正常运行：

```
@Service
public class ClothingService {
```

```
    @Autowired
    private ClothingRepository clothingRepository;

    public List<Clothing> getAllClothes() {
        return clothingRepository.findAll();  // 原来 new ArrayList<>(map.values())
    }

    public Clothing saveClothing(Clothing c) {
        return clothingRepository.save(c);    // 原来 map.put(id,c)
    }
}
```

6.4.5 单元测试

在实际的软件开发过程中，常常需要对系统的功能进行全面且细致地测试。这里，利用Spring Boot所提供的切片测试这一强大功能，来对系统中的增加、删除以及查询操作进行严格的验证，以确保这些基础操作的准确性和稳定性。

```
@SpringBootTest
class ClothingRepositoryTests {
    @Autowired
    private ClothingRepository repo;

    @Test
    void testInsertAndQuery() {
        Clothing c = new Clothing();
        c.setName("羊毛大衣");
        c.setBrand("品牌M");
        c.setPrice(899.0);
        c = repo.save(c);

        Optional<Clothing> op = repo.findById(c.getId());
        assertTrue(op.isPresent());
        assertEquals("羊毛大衣", op.get().getName());
```

```
    }
}
```

本章小结

本章梳理了Spring Boot核心知识体系与实践路径。开篇以“约定大于配置”理念为引，指出Spring Boot通过自动配置与起步依赖实现近乎零配置、“开箱即用”，对比J2EE时代痛点，具备了“快速构建、独立运行、云原生”优势。随后，演示JDK 8+、Maven 3.6、IDEA 2024环境搭建，配合截图与命令行验证。项目创建给出Spring Initializr在线生成、IDE向导、Maven手工三种方式，剖析标准目录，让代码存放一目了然。接着聚焦Web开发，详解常用注解，示范静态资源路径映射，引入Thymeleaf模板引擎，实现前后端分离与服务端渲染并行。RESTful部分以服装商品管理为例，覆盖全栈流程，将内存Map迁移到MySQL，借助Spring Data JPA完成CRUD，强调生产环境数据迁移安全。

练习与实践

简答题

（1）Spring Boot 的“约定优于配置”理念具体体现在哪些方面？请至少列举两点并简要说明。

（2）使用 Spring Initializr 在线创建项目时，需要填写的三个核心坐标（GAV）是什么？

（3）简述 Spring Boot 默认提供的四种静态资源目录，并给出其中任意一种的 classpath 写法。

（4）@RestController 与 @Controller 有何区别？什么场景下必须选用 @RestController?

实践题

（1）页面美化：在服装列表案例基础上，使用 Bootstrap 或 Tailwind 重写 list.html，使表格响应式、增加卡片布局、图片懒加载与顶部搜索栏，提交截图与关键代码。

（2）RESTful 扩展：为 ClothingApiController 新增“根据分类查询”与“分页查询”两个接口，要求返回统一 JSON 包装类（含 code、msg、data、total），并写 Postman 测试脚本验证。

第 7 章

项目总体设计



本章导读

本项目致力于打造一个全面智能化、高度兼容的跨平台服务，搭配先进的智能推荐系统。通过深度融合前沿的推荐算法与现代化的Web开发技术，构建一个从用户灵感激发到最终便捷购物的完整闭环体验旨在提升用户的参与感和满意度。

这一创新模式彻底颠覆了传统的“人找货”被动购物方式，升级为“货搭人”的智能服务模式，真正实现个性化、场景化的时尚推荐，为用户带来前所未有的智能化购物体验。

学习目标

1.能独立完成Spring Boot后端RESTful API开发，并基于Vue3实现前端交互逻辑，理解跨域解决方案与接口文档规范。

2.掌握集成支付宝沙箱支付接口，实现从订单生成到支付回调的完整流程，理解支付安全机制（如签名验证）。

3.设计合理的服装SKU数据库模型（如颜色/尺码变体），实现搭配关系表结构与查询优化。

思政目标及设计建议

将服装搭配智能推荐系统与服装专业深度融合，并融入思想政治教育，是极具创新性且契合“课程思政”理念的研究方向。本章将阐述思政目标及设计，建议如表7-1所示。

表7-1　第7章思政目标及设计建议

思政目标	思政元素及融入
培养文化自信与民族自豪感	为重要场合（毕业礼、成人礼、婚礼）提供中华礼仪服饰指导，在商品详情页展示设计灵感来源和工艺特色。
锤炼严谨规范、精益求精的工匠精神	采用Spring Boot的规范分层架构，培养模块化、标准化的开发思维，使用MyBatis进行精细化数据操作管理，培养数据处理的严谨态度
强化团队协作与集体主义精神	建立清晰的提交规范和代码审查流程，强化集体责任意识，建立可视化任务看板，增强团队进度透明度和集体监督

7.1 总体设计



7.1.1 项目需求

本项目为服装搭配智能推荐系统，基于SpringBoot后端框架与VUE3前端框架进行架构设计，并采用MyBatis作为持久层技术方案，成功构建了一个功能完备的B2C电子商务平台。在当前开发阶段，项目主要致力于核心电商功能的设计与实现，通过PC端系统为用户提供完整的购物流程支持，包括首页内容展示、多样化商品浏览、购物车灵活管理、订单生成处理以及安全便捷的支付流程等功能模块。系统内部已建立起结构完善的服装商品信息数据库与分类体系，全面覆盖服装品类及其属性数据，为下一阶段实现智能推荐与搭配算法提供了充分的数据储备与系统架构支持。当前版本的核心目标在于构建一个稳定可靠、用户体验优良的在线服装销售平台，全面满足用户从浏览查看、筛选比较到最终完成交易的基础需求，同时为未来集成基于人工智能的服装推荐与个性化搭配模块奠定了坚实的技术基础与业务框架。

7.1.2 项目需求分析

1.功能需求分析

（1）用户管理模块采用用户表与管理员表分离的数据库设计，通过核心的注册与登录接口提供服务。在业务逻辑上，注册流程负责数据校验与密码加密存储，而登录流程则验证凭证并依据角色字段进行权限划分。在安全方面，综合运用BCrypt算法加密存储密码，并采用JWT令牌方案承载用户身份与权限信息，为后续请求提供安全的认证与授权机制，构建了一个权责清晰、安全可靠的基础用户管理体系。

（2）商品管理模块围绕商品全生命周期及其搭配关系进行设计。在数据库层面，通过商品主表、SKU表、标签表及核心的搭配关系表，构建了涵盖基础属性、多规格库存、多维标签到智能搭配规则的完整数据模型。整体设计实现了商品信息的精细化管理及搭配关系的结构化存储，为个性化推荐与营销策略奠定了坚实的数据基础。

（3）购物车与订单模块构成了电商交易的核心链路。在数据表设计上，通过购物车表、订单主表/明细表及收货地址表，清晰地区分了选购、下单与交付信息。业务逻辑的核心在于处理登录前后的购物车合并，确保用户体验的连贯性；并设计了完整的订单状态机（如待支付、已支付、发货中、已完成），严格管控订单生命周期。在接口层面，提供购物车的增删改查操作，以及订单的创建、查询与取消功能，为前端交易流程提供了完整、可靠的数据与服务支撑。整体设计兼顾了用户体验的流畅性与后台业务状态的严谨性，形成了从选购到履约的闭环管理。

（4）支付模块是确保交易完整性与安全性的核心组件。它通过支付流水表完整记录每一笔交易的详细信息；采用标准化的模拟接口与第三方支付平台（如支付宝、微信支付）进行对接，隔离外部系统的复杂性；并通过严谨的回调处理逻辑，实现对支付结果的可靠接收、验证与业务状态同步，最终保障交易数据的准确性与流程的最终一致性。

2.非功能需求分析

本项目在非功能性需求方面进行了全面而深入的分析，重点关注以下五个核心维度，以确保系统的高质量与可靠性：

（1）在数据安全方面，项目实施了全面的数据保护策略。所有用户密码均采用高强度加密算法进行安全存储，支付信息通过SSL/TLS协议进行端到端加密传输，有效防止数据在传输过程中被窃取或篡改。同时，系统部署了严格的SQL注入防护机制，对用户输入进行有效过滤和参数化查询处理，从多层面保障数据的机密性、完整性和可用性。

（2）在权限控制层面，系统建立了精细化的权限管理体系。通过实现接口级别的身份验证与授权机制，确保每个用户只能访问其权限范围内的资源与操作。此外，系统详细记录所有敏感操作日志，包括操作时间、执行人及具体内容，极大地增强了系统的安全防控能力与事后审计的可追溯性。

（3）在界面友好性方面，项目采用了先进的响应式网页设计技术，使系统能够自适应不同终端设备，为用户提供一致且流畅的视觉与操作体验。界面设计充分结合服装行业的审美特点，运用和谐的色彩搭配与布局，优化视觉层次，重点突出业务关键信息，从而显著提升用户的交互满意度和使用效率。

（4）在代码规范方面，开发团队严格遵循SpringBoot后端与Vue3前端的官方最佳实践与编码标准。采用模块化与组件化的开发模式，强调代码的低耦合与高内聚特性，使得系统结构清晰、易于扩展。同时，代码中编写了大量清晰且规范的注释，极大提高了代码的可读性、可维护性与团队协作效率。

（5）在日志管理方面，系统实现了完备的日志记录机制，全面捕获关键业务操作、系统运行状态与异常错误信息。通过对日志实行分级分类管理，能够快速定位和诊断问题，不仅保障了系统运行的安全性与稳定性，也为后续的系统维护和性能优化提供了坚实的数据支撑。

3.关键业务流程图

（1）购物下单流程：用户从浏览商品或搭配方案开始，可将心仪的商品加入购物车；随后进入结算环节，依次选择收货地址与支付方式，并确认订单信息；完成支付后，即可随时查看订单状态，如图7-1所示。

（2）搭配内容管理流程：管理员登录系统后，进入搭配管理模块，可创建新的搭配方案或编辑现有方案，通过关联具体商品并设置其适用场景，完成方案的配置与优化，随后可执行发布或下架操作，并可随时查看方案的浏览、采纳等相关效果统计，如图7-2所示。

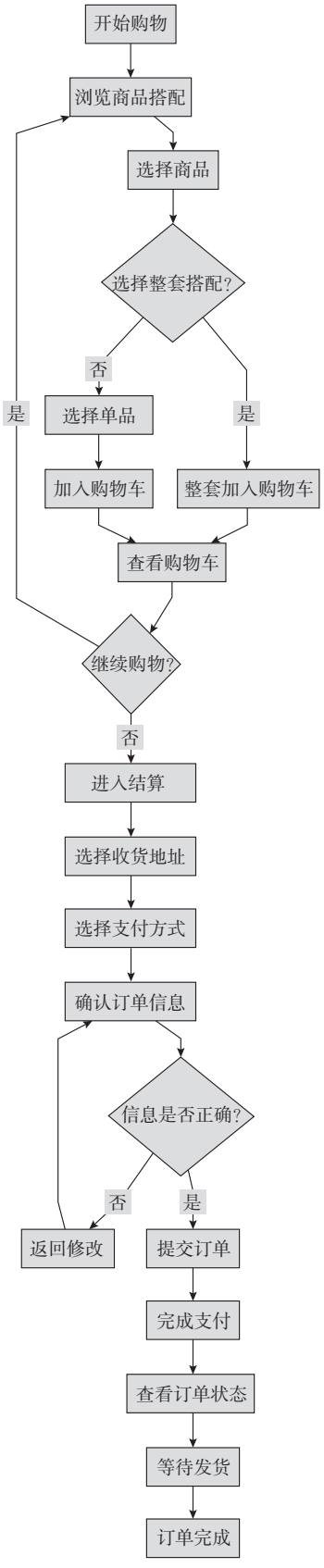


图 7-1　购物车下单流程图

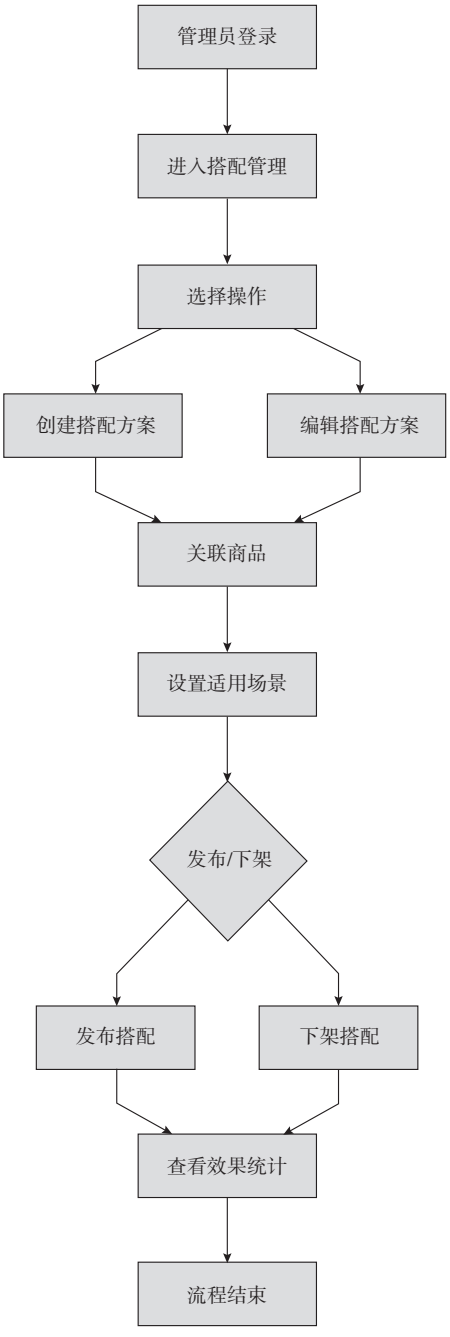


图 7-2　智能搭配推荐流程图

7.2 项目详细功能设计



7.2.1 功能模块详细设计

本项目采用了当前业界主流的SpringBoot后端框架与Vue3前端框架进行整体架构设计，通过前后端分离的开发模式实现高效协同开发。在数据持久化层面，系统选择了轻量且功能强大的MyBatis作为基础ORM框架，同时集成MyBatis Plus增强组件以进一步提升开发效率与数据操作能力。在系统功能架构设计方面，经过细致的业务需求分析与模块化规划，我们将系统划分为六大核心业务模块，每个模块都具有清晰的职责边界和高度内聚的特性，共同支撑起完整的系统功能体系。

1.用户管理模块

用户管理模块负责系统的用户身份认证、授权及个人信息管理，是系统安全的基础。

（1）用户注册：支持通过邮箱或手机号进行账户注册，注册过程中会发送验证码进行身份校验，并设有密码强度检查机制以确保账户安全。

（2）用户登录：提供多种登录方式，包括传统的账号密码登录、便捷的短信验证码登录以及第三方平台授权登录，满足不同用户的使用习惯。

（3）个人信息管理：用户可上传个人头像、修改包括昵称和联系方式在内的基本信息，并可对个人偏好进行设置，如界面风格偏好和主题颜色偏好。

（4）收货地址管理：支持用户对收货地址进行增加、删除、修改和查询操作，同时可以设置常用地址为默认收货地址，提升购物体验。

（5）安全中心：提供密码修改功能，用户可以定期更新密码以增强账户的安全性。

（6）权限管理：采用基于角色的访问控制（RBAC）机制，通过用户－角色－权限三级模型实现精细化的权限分配和管理。

1.商品管理模块

该模块是整个项目的核心，负责服装商品的分类、属性、SKU/SPU管理以及图片管理等。

（1）商品分类管理：支持建立多级商品分类体系，例如从大类（如男装/女装）到子类（如上装）再到具体品类（如T恤），采用树形结构进行可视化展示，并支持通过拖拽操作灵活调整分类顺序与层级关系。

（2）商品属性管理：提供商品通用属性（如材质、产地）与销售属性（如颜色、尺码、款式）的创建、编辑与维护功能，支持根据不同商品类目配置不同的属性组和属性项。

（3）SPU管理：用于管理商品的基本公共信息，包括商品标题、详细描述、品牌归属、所属类目等核心数据，实现标准化商品信息的统一维护与发布。

（4）SKU管理：基于SPU生成具体规格的商品单元，管理不同规格（如不同颜色、尺码）对应的价格、库存数量、条形码等独立销售属性，确保商品的可售状态与库存准确性。

（5）商品搜索与筛选：支持通过商品分类、属性选项、价格区间、品牌等多个维度进行商品检索与条件筛选，提升商品查找效率与精准度。

（6）品牌管理：提供品牌信息的创建、修改、删除与查询功能，支持品牌Logo上传与品牌故事维护，并可关联对应品牌下的所有商品，实现品牌化运营。

（7）商品标签管理：允许为商品添加多种自定义标签（例如“热销商品”“新品上市”“首页推荐”等），用于商品标记、分类推广与个性化展示。

（8）商品图片管理：支持批量上传商品图片，可对图片进行排序、删除操作，设置主展示图与细节展示图，并提供预览功能以确保商品视觉呈现效果。

2.智能推荐模块

该模块是本系统的核心价值所在，融合多种推荐算法，为用户提供个性化服装搭配建议。

热门推荐：综合考量商品的近期销量、用户点击量、收藏次数等热度指标，向用户展示当前受欢迎的服装单品及搭配方案。此类推荐充分利用群体智慧，反映市场趋势和大众偏好。

（1）搭配方案管理：支持用户或管理员对搭配方案进行全生命周期管理，包括创建新的搭配组合、编辑现有方案的细节内容、删除不再适用的方案以及将审核通过的方案发布至平台供用户浏览和使用。

（2）用户画像构建：通过收集和分析用户的浏览历史、购买记录、收藏行为等数据，逐步构建包含风格偏好、颜色倾向、场合需求等标签的立体化用户画像，为个性化推荐提供精准的数据支撑。

3.订单管理模块

该模块是本项目的交易核心，负责处理用户从下单到完成的整个交易流程。

（1）订单创建流程：允许用户从购物车中快速生成订单，支持同时选择多种商品及多个SKU进行合并下单，满足用户批量购买需求。

（2）订单状态管理机制：涵盖完整的订单生命周期，包括从待支付、待发货、待收货直至已完成的各个环节，确保订单流转清晰可控。

（3）订单支付服务：集成主流支付渠道，安全高效地完成支付流程，并实时处理支付结果回调，保障交易状态同步。

（4）订单取消策略：支持用户主动取消订单，系统自动识别超时未支付订单并执行取消，同时商家也可根据实际情况手动取消订单。

（5）订单统计功能：提供基于订单数据的销售统计与分析，深入洞察用户购买行为，为运营决策提供数据支持。

（6）订单查询系统：支持按时间、状态、商品等多维度条件进行订单搜索与筛选，并提供数据导出功能，便于进一步处理与分析。

4.支付管理模块

负责处理用户订单的支付流程，支持多种支付方式，确保交易安全可靠。

（1）支付方式管理功能：全面支持包括支付宝、微信支付以及模拟支付等多种主流支付渠道，满足不同用户的多样化支付需求，并可根据业务场景灵活扩展接入其他支付方式。

（2）支付发起流程：系统能够自动生成安全可靠的支付请求参数，并实现无缝跳转至对应的支付页面，为用户提供流畅、高效的支付体验，同时保障交易信息的准确性和完整性。

（3）支付回调处理机制：系统设计了完善的异步通知和同步返回处理逻辑，确保支付结果能够及时、准确地被接收和处理，有效避免因网络延迟或异常导致的支付状态不一致问题。

（4）支付状态查询服务：提供实时、高效的支付结果查询接口，用户和系统可随时获取最新的支付状态信息，保障交易过程的透明度和可追溯性，便于及时处理异常情况。

（5）支付安全防护措施：采用多重安全策略，包括严格的签名验证机制、防重放攻击方案以及端到端的数据加密传输，全方位保障支付数据的安全性和交易过程的可靠性，有效防范潜在风险。

（6）支付配置管理：支持商户号、密钥等关键支付参数的灵活配置与管理，提供友好的管理界面，方便用户根据实际需求快速完成支付相关的各项设置，确保支付功能的正常运行。

（7）支付日志记录系统：详细记录所有支付操作的关键信息，形成完整的支付日志，便于后续审计、对账和问题排查，为支付流程的优化和故障分析提供坚实的数据支持。

5.购物车管理模块

是用户购物流程中的核心环节，负责临时存储用户选择的商品，支持多种操作和状态管理

（1）购物车商品管理：涵盖了多个核心操作，包括将商品加入购物车、从购物车中移除指定商品，以及灵活调整购物车中已有商品的数量。

（2）购物车状态管理：负责维护商品的不同状态，例如标记哪些商品被用户选中，以及识别并处理因下架或库存不足等原因导致的商品失效情况。

（3）购物车合并处理机制：在用户登录时自动触发，系统会智能地将用户未登录时生成的临时购物车与其账户下的持久购物车进行无缝合并，确保购物记录不丢失。

（4）购物车上限控制：通过设定商品数量或总金额阈值，有效防止用户或恶意程序过量添加商品，保障系统资源合理使用与业务安全。

（5）购物车批量操作：支持用户一次性对多个商品执行统一动作，例如全选或反选、批量删除不需要的商品，以及同时修改多个条目的购买数量，显著提升操作效率。

各模块之间通过设计良好的RESTful API接口实现松耦合的交互方式，各服务保持独立性和可扩展性。系统采用共享的MySQL关系型数据库作为核心数据存储方案，通过事务机制和数据约束有效保障整

个系统中的数据一致性与完整性。同时，针对高频访问的数据查询场景，引入Redis作为高速缓存层，显著提升数据的读取与写入效率，降低数据库的直接访问压力。这些技术组件协同工作，彼此配合，最终构建出一个功能完备、结构清晰、运行稳定的单体式应用系统。

7.2.2 项目概要设计

1.物理部署架构图与说明

这一设计架构充分体现了当前系统设计的核心理念，即采用前后端分离与职责解耦的模式。在这一架构中，前端与后端各自独立部署运行，分别专注于用户交互体验与业务逻辑处理这两个关键领域。通过这种明确的分工方式，不仅有效降低了系统的整体复杂性，而且显著提升了团队协作效率和开发速度，同时也有利于系统的可维护性和可扩展性，如图7-3所示。

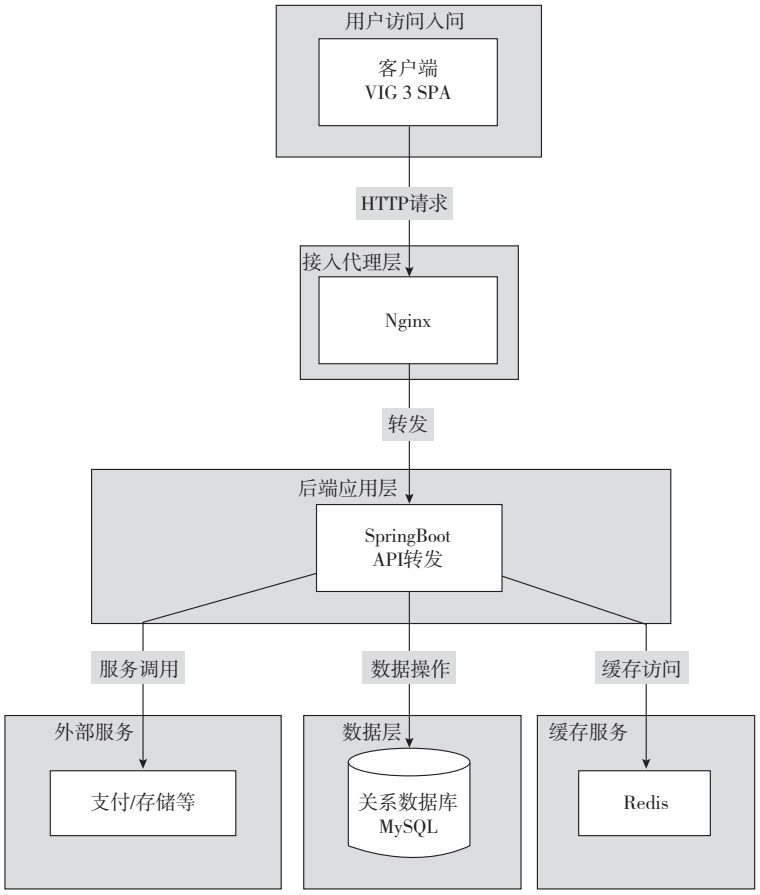


图7-3 物理部署架构图

（1）客户端层：用户主要通过PC端的浏览器来访问和使用系统，其前端部分采用Vue 3框架构建，实现现代化的单页面应用，能够提供流畅且响应迅速的用户交互体验。

（2）接入层：部署Nginx作为反向代理服务器，承担多类关键功能，包括提供静态资源服务（如CSS样式文件、JavaScript脚本、各类图片资源等），将来自前端的API请求按规则转发至相应的后端应用服

务器，实现请求的负载均衡（支持配置多台应用服务器以提升系统处理能力），以及完成SSL/TLS连接的终止处理，从而全面支持HTTPS协议以保障通信安全。

（3）应用层：运行业务逻辑的核心部分采用Spring Boot框架开发，部署于Tomcat容器之中，负责接收、处理各类用户请求，并实现系统全部业务功能与流程控制。

（4）数据层：由以下多个组件共同构建而成MySQL 8.0作为主关系型数据库，采用主从复制架构以提升数据的可用性与可靠性；Redis充当高速缓存层，用于存储用户会话信息、热点访问数据及个性化推荐结果等；文件存储则依托MinIO（可自建部署）或阿里云OSS对象存储服务，主要用于保存商品图片等大型静态文件资源。

2.逻辑分层架构图与说明

本系统采用前后端分离的架构设计，前端部分使用Vue 3框架构建高效的单页应用，主要负责用户交互与界面渲染，并通过Nginx服务器托管静态资源，同时实现反向代理功能，将API请求转发至后端服务，后端以SpringBoot框架为核心，通过RESTful API接口提供业务逻辑处理，并采用MyBatis作为数据访问层框架，操作MySQL数据库以实现核心业务数据的持久化存储。此外，系统引入Redis作为缓存中间件，用于缓存用户会话信息、热门商品数据以及推荐结果等高频访问数据，从而显著提升系统整体性能，同时，系统还集成了第三方支付网关和对象存储服务等外部系统，以完成在线支付与文件存储等关键功能，如图7-4所示。

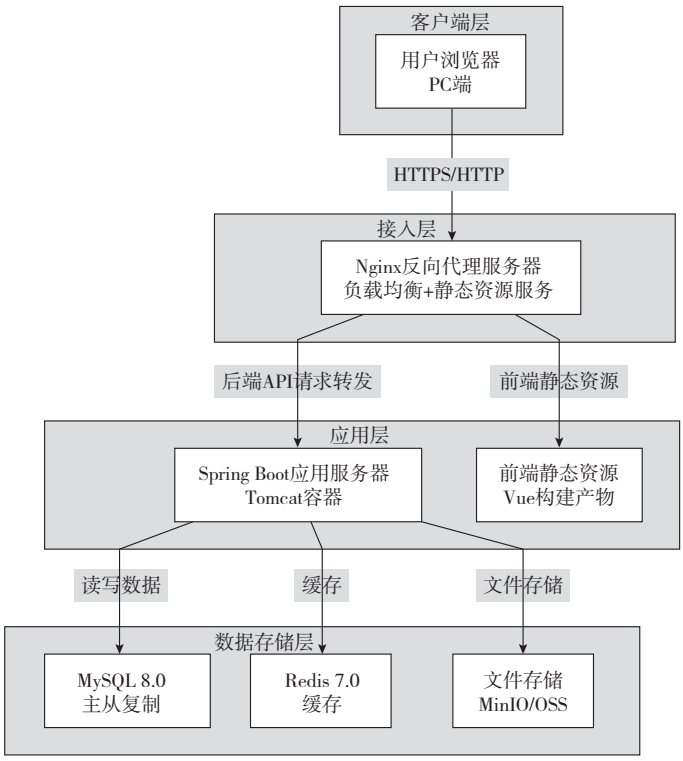


图7-4 系统技术架构图

3.系统功能结构图

包含用户管理、商品管理、购物车管理、订单管理、智能推荐管理和支付管理这六大核心功能模块，每个模块均承担特定的职责，共同构建了一个完整、高效且安全的电子商务平台运行体系，这些模块协同工作，为用户提供流畅、可靠的在线购物体验，如图7-5所示。

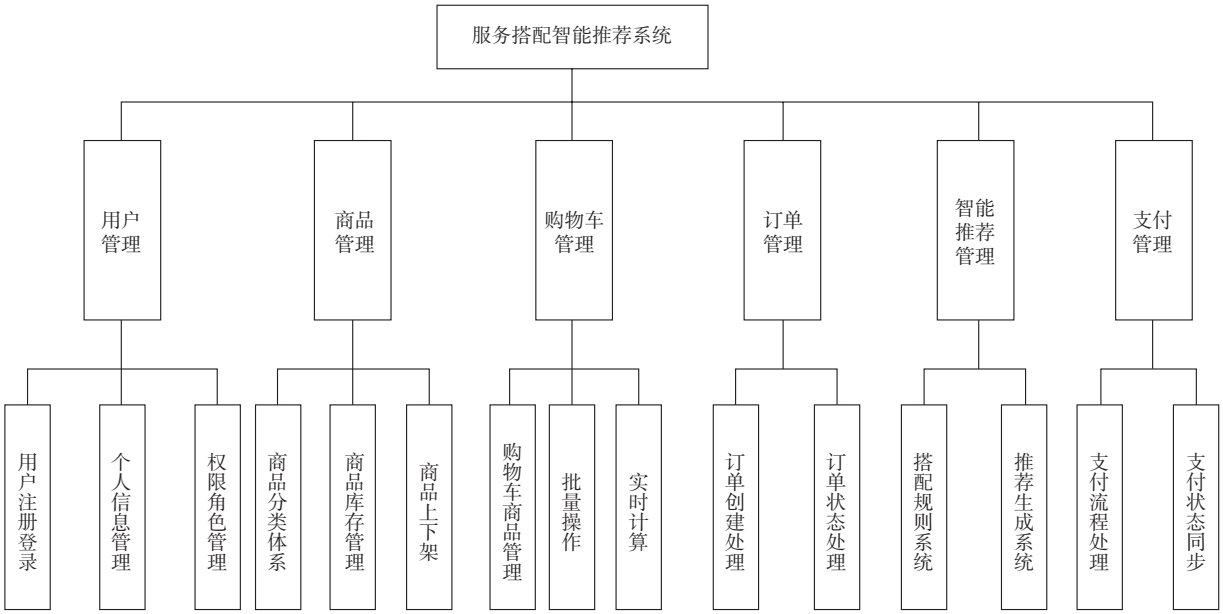


图7-5 系统功能结构图

4.技术选型分析

本项目全面覆盖了前端、后端及移动端三大核心领域的技术体系。前端方面，我们将涉及HTML、CSS、JavaScript、Vue3等主流框架和工具；后端方面，将包含Java、Node.js等语言以及Spring Boot等框架的应用，通过系统化的技术整合，本项目致力于构建一个功能完备、技术先进、扩展性强的全栈开发解决方案，如表7-2所示。

表7-2 技术选型分析表

技术选型	技术名称	说明
后端框架	Spring Boot2.5+	简化Spring应用初始搭建和开发过程，提供自动配置、嵌入式Web服务器等特性
前端框架	Vue3和Element Plus、Axios	渐进式JavaScript框架，采用Composition API；基于Vue 3的桌面端组件库；基于Promise的HTTP客户端
持久层	MyBatis框架、MyBatis Plus框架	MyBatis的增强工具，简化CRUD操作
数据库	MySQL8.0+	存储数据，性能优越，功能丰富
缓存	Redis缓存	将临时数据存储在Redis缓存，提高查询的效率
消息队列	RabbitMQ消息队列	消息代理，用于异步处理
支付接口	支付宝沙箱环境	使用沙箱支付
安全框架	Spring Security、JWT	安全框架，用于认证和授权控制。

5.设计原则

（1）前后端分离架构模式采用了现代化的技术栈。前端采用Vue3框架进行开发，借助其响应式特性和组合式API，实现了高效的界面交互与状态管理；后端基于SpringBoot搭建，利用其自动配置和快速开发能力，构建了稳定可靠的业务逻辑处理层。

（2）在系统设计层面，秉承模块化设计理念，将系统划分为多个功能模块，每个模块职责明确、边界清晰，达到了高内聚低耦合的架构目标，从而提升了代码的可维护性和可测试性。

（3）系统具备良好的可扩展性，架构设计充分考虑了后续功能扩展的需求，支持灵活的功能模块添加和系统能力增强。同时，技术选型注重前瞻性和兼容性，为未来的技术升级迭代奠定了坚实基础。

（4）在安全性方面，系统建立了完善的用户认证机制和细粒度的权限管理体系，通过多层次的安全防护策略，确保数据和业务操作的安全性，有效防范未授权访问和恶意攻击。

（5）针对性能优化，实施了合理的数据库表结构设计和索引优化策略，并结合Redis等缓存技术，制定了高效的数据缓存方案，显著提升了系统响应速度和处理能力，保障了系统在高并发场景下的稳定运行。

7.2.3 数据库设计原则

1.在数据库设计中，范式设计原则是构建高效、可靠数据库系统的基础。该原则强调在确保数据一致性与完整性的前提下，可以根据实际应用场景，适当采用反范式化的策略来提升系统的查询性能和整体运行效率。具体来说，通过合理减少表之间的关联和冗余，可以在不牺牲数据准确性的情况下，优化数据读写操作，从而更好地满足高并发与复杂业务需求。

（1）第一范式（1NF）的原子性是指数据库表中的每一列都必须是不可再分的最小数据单元，即每个字段中存储的数据都应当是单一、独立且无法继续分解的基本元素，这一特性有助于避免数据冗余和保证数据的结构清晰。

（2）第二范式（2NF）的完全依赖性要求非主键列必须完全依赖于整个主键，而不能仅依赖于主键的一部分，也就是说，当主键由多个字段组成时，所有非主键属性都需要与整个复合主键相关，而不仅仅是其中的某个子集，以此确保数据的逻辑一致性和完整性。

（3）第三范式（3NF）的传递依赖性消除是指非主键列之间不能存在间接的依赖关系，即任何非主键属性都不应依赖于其他非主键属性，而应直接依赖于主键，这有助于进一步减少数据冗余并提升表的规范程度。

（4）反范式化设计（性能优化）：在数据库设计过程中，为了提升系统性能，特定场景下可适当采用反范式化策略。该设计方式通过减少表连接操作和增加数据冗余，以优化查询效率，特别适用于对读取性能要求较高的应用场景。需要注意的是，反范式化需在数据一致性与性能之间进行权衡，合理运用方能达到最佳效果。

2.命名设计原则的核心在于制定并遵循一套统一的命名规范，这种规范能够显著提升代码、文件或项目的可读性与可维护性。通过采用清晰、一致且具有描述性的命名方式，团队成员可以更轻松地理解各个组成部分的功能与用途，从而减少误解和沟通成本。同时，良好的命名习惯也有助于长期的项目维护，使得后续修改、扩展或调试工作变得更加高效和准确。因此，统一的命名规范不仅是编程中的基本最佳实践，也是保障软件质量和团队协作效率的重要基石。

（1）在数据库的各类命令规范中，统一要求采用小写字母结合下划线的命名方式，并且必须使用项目前缀，以增强可读性并确保命名具备明确的语义表达。

（2）对于表的命名，同样遵循小写字母与下划线组合的形式，规定表名需使用复数格式，以便直观体现其存储数据的集合性质，同时命名应做到见名知意。

（3）在数据库设计与开发过程中，严格遵循统一的命名规范对提高代码的可读性和维护性至关重要。具体而言，对于字段的命名，应采用小写英文单词结合下划线的格式，并且要求字段名称能够直观地表达其含义，做到见名知意。例如，使用“user_name”而非“un”来明确表示用户名字段，确保其他开发人员能够快速理解其用途。

（4）关于索引命令的规范，主键索引的命名建议直接使用数据库默认的“PRIMARY”关键字，无需额外设置，以保持简洁和一致性。而对于其他非主键索引，命名时应采用前缀、下划线及表名的组合方式。例如，如果为“user”表的“email”字段创建索引，可以命名为“idx_user_email”，这样既明确了索引类型，也关联了相关表与字段，便于后续管理与优化。

（5）在约束命名方面，特别是检查约束和默认约束，命名规则应采用前缀、下划线以及对应字段名称的组合形式。举例来说，若为“age”字段设置检查约束确保年龄大于零，可命名为“chk_age_positive”；若为“create_time”字段设置默认值为当前时间，则可命名为“def_create_time_current”。这种命名方式不仅增强了约束的可识别性，也有助于在复杂数据库结构中快速定位和处理相关约束。

3.在数据库设计过程中，表结构之间的关联关系通常借助实体关系图（ER图）进行可视化呈现，ER图作为一种高效的建模工具，能够清晰地展示不同数据表之间的连接方式、主外键约束以及实体间的对应关系，通过ER图，开发人员能够直观地理解数据模型的结构，从而有效支持数据库的设计、优化及维护工作，如图7-6所示。

7.2.4 数据库详细设计

在本项目中，User用户表、订单表、订单详细表和购物车表构成了核心的业务数据模型，它们相互协作，共同支撑起完整的购物流程。以下将详细介绍这些核心表的结构。

1.用户表（User）

是系统中存储用户核心信息的关键数据表，该表以用户ID（id）作为主键，唯一标识每个用户记录，并包含用户登录所需的账户名（user_name）和加密密码（password）等必备字段。同时，它记录

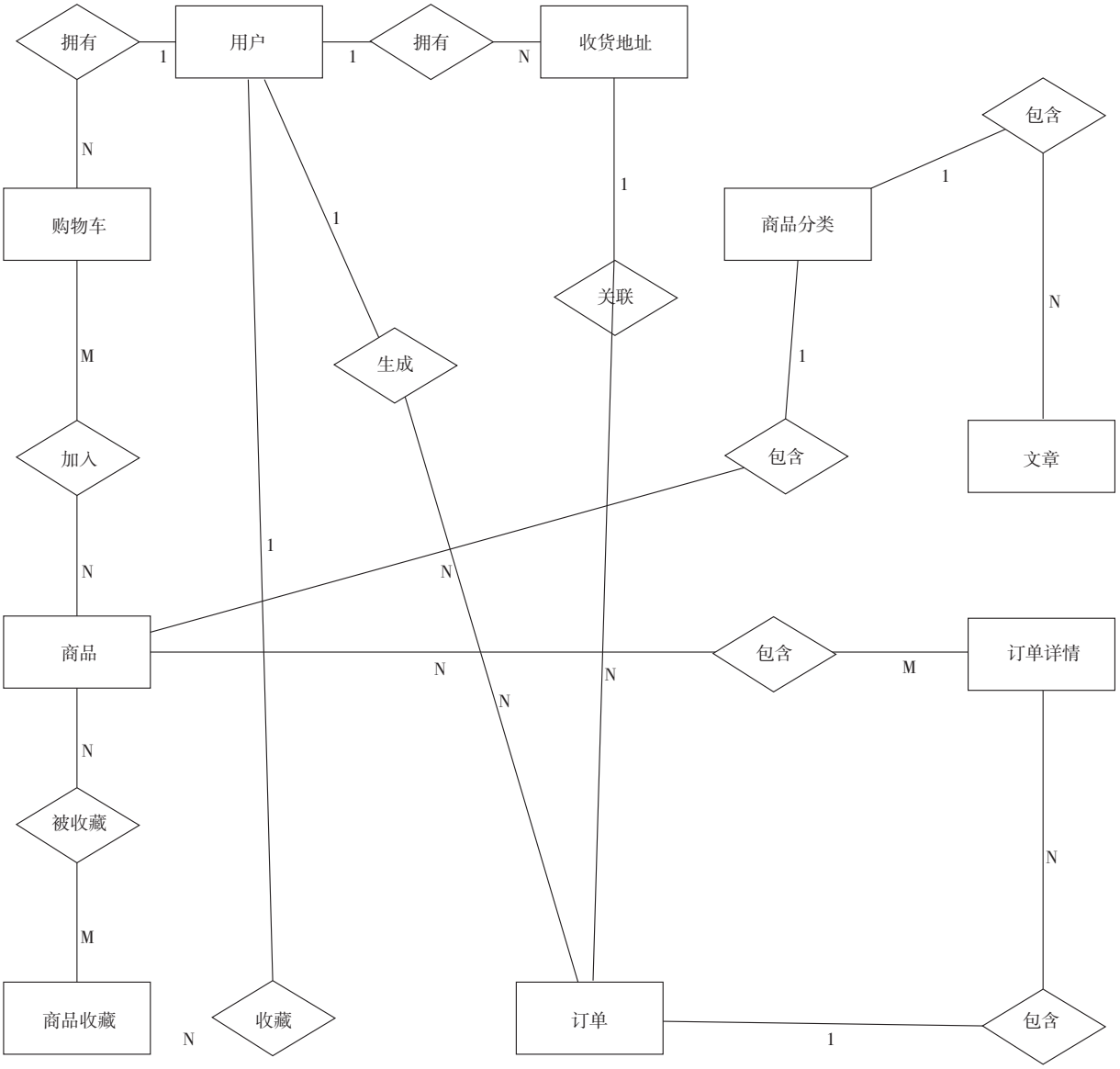


图7-6 ER图

了用户的个性化信息，如昵称（nick_name）、性别（gender）、联系方式（phone）、电子邮箱（email）和头像地址（avatar），还支持以JSON格式存储兴趣偏好（interest），增强了数据的灵活性，如表7-3所示。

表7-3 users用户表

字段	类型	字段说明	字段大小	备注
id	int	用户ID	0	主键
user_name	varchar	用户名称	60	不为空
password	varchar	密码	128	不为空
nick_name	varchar	昵称	30	
gender	varchar	性别	5	

续表

字段	类型	字段说明	字段大小	备注
phone	varchar	电话	20	
email	varchar	电子邮箱	100	
avatar	varchar	头像	200	
interest	json		0	
status	tinyint	状态	0	
deleted	tinyint		0	
create_by	int		0	
create_time	datetime	创建时间	0	
upate_by	int		0	
update_time	datetime	更新时间	0	

2.订单表（Orderform）

是本项目中用户购买行为与订单核心信息的关键数据表，该表以订单ID（id）为主键，通过用户ID（user_id）关联用户表，确保了每个订单都有明确的归属用户。其核心业务字段包括唯一订单号（orderform_no）、收货地址ID（address_id）、订单总金额（total）以及商品数量（amount），共同构成了订单的财务与物流基础，如表7-4所示。

表7-4 orderform订单表

字段	类型	字段说明	字段大小	备注
id	int	订单ID	0	主键
user_id	int	用户ID	0	不为空
orderform_no	varchar	订单号	30	不为空
address_id	int	地址ID	0	不为空
total	int	总计	0	
amount	float	数量	10	小数点：2位
invoice_type	varchar		30	
invoice_title	varchar		100	
status	tinyint	状态	0	
create_time	datetime	创建时间	0	

3.购物车表（Mycart）

是本项目中用于临时存储用户选购商品的核心数据表，它记录了用户在最终下单前对商品的选择与意向，该表以购物车ID（id）作为主键，并通过用户ID（user_id）和产品ID（product_id）分别关联用户表和产品表，确保了数据的完整性与归属清晰，如表7-5所示。

表7-5 mycart购物车表

字段	类型	字段说明	字段大小	备注
id	int	购物车ID	0	主键
user_id	int	用户ID	0	不为空
product_id	int	产品id	0	不为空
title	varchar	标题	255	不为空
thumb_url	varchar	图片地址	255	不为空
price	float	价格	10	小数点：2位；不为空
total	int	总计	0	不为空
create_time	datetime	更新时间	0	

4.订单详情表（Orderform_detail）

是订单管理的核心扩展表，用于记录和分解一个主订单中所包含的具体商品明细，该表以自增ID（id）为主键，并通过订单号（orderform_no）和产品ID（product_id）分别与订单主表及产品表关联，确保每条记录都明确归属于某一笔具体订单，如表7-6所示

表7-6 orderform_detail订单详情表

字段	类型	字段说明	字段大小	备注
id	int		0	主键
orderform_no	varchar	订单号	30	不为空
product_id	int	产品id	0	不为空
title	varchar	标题	255	
thumb_url	varchar	图片地址	255	
price	float	价格	10	小数点：2位；不为空
total	int	总计	0	
create_time	datetime	更新时间	0	

7.2.5 接口设计规范

1.RESTful API规范

（1）统一响应格式设计规范，旨在确保系统各模块间的数据交互严格遵循一致的协议标准，从而显著提升接口的可读性与可维护性。通过建立明确的格式约束和通信规则，各模块之间的协作变得更加高效和可靠，同时有效减少了因格式不统一而引发的潜在错误。

（2）状态码定义采用分层结构，涵盖全局通用状态码和业务特定状态码两个层级。通过数字编码方式，精确标识各类操作结果类型，如操作成功、参数错误、权限异常等常见情况。这种分层设计不仅增强了状态信息的表达力，还提升了系统对各类业务场景的适应能力，使开发人员能够更直观地理解和处理不同情境下的状态反馈。

（3）错误处理机制全面覆盖异常捕获、日志记录、用户提示及错误码映射等关键环节，实现了从底层异常到前端用户展示的全链路统一处理。借助完善的错误码映射机制，系统能够准确识别问题来源，并及时生成清晰的用户提示信息，从而有效保障系统的稳定性与用户体验，避免因错误信息不明确而导致的用户困惑或操作中断。

2.API设计规范

API设计规范是构建高效、可维护和可扩展Web服务的重要基础，其中GET、POST、PUT和DELETE这四种HTTP方法构成了对资源进行基本操作的核心语义。这些方法分别对应资源的查询、创建、更新和删除操作，通过明确且一致的语义，不仅能够提高接口的可读性和可理解性，快速掌握和使用API，在实际应用中，遵循这些核心语义可以确保API的行为符合预期，同时增强系统的稳定性和安全性，是设计高质量API不可或缺的关键要素，如表7-7所示。

表7-7 核心接口列表

HTTP方法	概述
GET	请求指定的页面信息，并返回实体主体
POST	类似于GET请求，返回的响应中没有具体的内容，用于获取报头
PUT	从客户端向服务器传送的数据取代指定的文档的内容
DELETE	请求服务器删除指定的页面

7.2.6 接口详细设计

本项目的接口文档详细阐述了系统的核心功能模块，具体包括首页展示、用户管理、购物车管理、订单管理、支付管理以及智能推荐这六大核心组成部分，整个接口设计逻辑清晰、层次分明，全面覆盖了用户从浏览商品、账户登录、添加购物车、生成订单、完成支付到接收个性化推荐的全流程购物体验，如表7-8所示。

表7-8 接口列表

模块	接口名称	方法	路径	说明
首页	获取首页	GET	/web	获取首页的显示
用户管理	注册用户信息	POST	/web/api/register	注册用户的信息
用户管理	登录	POST	/web/api/login	登录
购物车管理	加入购物车	POST	/web/api/user/mycart/add	添加购物车
订单管理	创建订单信息	GET	/web/user/orders	添加订单信息
支付管理	结算	GET	/alipay/pay	物品结算
智能推荐模块	获取搭配推荐	GET	/web/recommend	用户画像推荐

本章小结

在明确服装搭配智能推荐系统的项目背景与市场需求的基础上，系统性地梳理了项目的功能性需求与非功能性需求，并通过深入的需求分析，将这些需求进一步转化为具体的设计目标。随后，通过概要设计，构建了系统的整体架构及其核心模块的划分。在详细设计阶段，详细阐述了各功能模块的具体流程、数据库表结构的设计与关系模型，以及系统内外部的接口规范与数据交互方式，从而为后续的系统开发与实现提供了全面且清晰的技术蓝图。

练习与实践

简答题

（1）本项目为何强调“前后端分离”？请用一句话概括其核心价值。

（2）智能推荐引擎采用“混合推荐策略”，请写出其中两种算法名称。

（3）数据库设计中，反范式化可能带来哪些收益与风险？各写一条。

（4）JWT 令牌在本系统主要解决哪两类安全问题？

（5）请写出订单状态表中任意三个状态。

（6）支付回调处理为何必须“签名验证”？一句话说明。

（7）Redis 在本系统三类典型缓存场景是什么？任写两点即可。

实践题

（1）基于 Vue3 + Element Plus 实现“商品卡片”组件：支持鼠标 hover 放大图片、显示价格、库存，并能在无库存时自动置灰。

（2）在 SpringBoot 端编写一个 RESTful 接口：接收订单号，返回该订单的支付状态（已支付/未支付/支付失败），要求加本地缓存（Redis）降低数据库查询，并给出 Swagger 注解。

（3）使用支付宝沙箱完成“单笔订单支付”最小闭环：从创建订单、生成支付二维码、轮询查询支付结果到更新订单状态，用 Postman 或前端页面截图展示全流程，并贴出关键代码（含签名验证）。

第 8 章

系统功能的实现



本章导读

本项目成功构建了一个基于SpringBoot后端、Vue前端及MyBatis数据持久层的全栈式服装搭配智能推荐系统。在功能实现方面：用户管理模块通过JWT令牌实现了身份认证与精细化权限控制；商品管理模块利用MyBatis动态SQL高效处理服装信息的增删改查（CRUD）操作及多维度分类；购物车管理模块通过前后端状态同步，实现了实时增删改功能；订单管理模块采用状态机驱动模式，确保订单全生命周期的顺畅流转；支付管理模块安全集成了沙箱支付接口，完成了从下单到回调的完整闭环操作。

学习目标

1.能够实现完整的用户认证与授权体系，包括用户注册登录、JWT令牌管理、角色权限控制。

2.掌握基于SpringBoot与Vue的前后端分离架构设计协作流程，能够实现RESTful API接口规范，并通过MyBatis完成数据持久化层的配置与优化。

3.能够实现订单管理功能的全面实施与操作，包括订单的创建、处理、跟踪和归档等各个环节，确保订单流程的顺畅运行和数据的准确记录。

思政目标及设计建议

将系统功能的实现与服装类专业深度融合，并融入思想政治教育，是极具创新性且契合“课程思政”理念的研究方向。本章将阐述思政目标及设计，建议如表8-1所示。

表8-1　第8章思政目标及设计建议

思政目标	思政元素及融入
培养社会责任与诚信意识	通过用户管理模块，强调用户信息保护与隐私安全，引导学生遵守数据伦理，树立网络安全责任感
倡导勤俭节约与理性消费	在购物车和订单管理中，设计“需求合理性提示”功能，鼓励用户理性规划消费，避免过度购买
树立团队协作与工匠精神	通过系统开发实践，强调代码规范、文档完整性和测试严谨性，培养精益求精的工程素养

8.1 PC端的功能实现



8.1.1 首页功能实现

在首页功能的实现上，前端采用VUE组件化开发，结合Element UI等库构建了响应式布局，设计了包括轮播图展示热门搭配、个性化推荐卡片流、快捷分类筛选及动态资讯模块在内的可视化界面。后端通过SpringBoot提供RESTful API支持，由MyBatis持久层高效查询和聚合各类数据，通过算法模型与用户行为分析生成“猜你喜欢”的个性化推荐，并动态展示“热销”“新品”“精品”等分类商品；同时针对“清仓”与“断码”等促销场景，设置独立数据查询与缓存策略。系统还利用Redis缓存热门数据，有效提升首页模块的加载速度与用户体验。

以下将展示首页功能实现的核心代码，若需查看完整代码，请参阅源码。

```
@GetMapping()
public String index(
        ModelMap model,
        HttpSession session,
        @RequestParam(value = "scpickedId", defaultValue = "1") Integer scpickedId,
        @RequestParam(value = "followId", defaultValue = "0") Integer followId,
        @RequestParam(value = "viewedId", defaultValue = "0") Integer viewedId
){
    // ==================== 1. 商品类别导航 ====================
    // 获取所有商品分类，以树形结构展示
    model.put("categorys", categoryController.getCategoryTree());

    // ==================== 2. 轮播图数据 ====================
    // 查询状态为启用(2)的轮播图，按order_id升序排序
    QueryWrapper<Slide> wrapper = new QueryWrapper<>();
    wrapper.eq("status", 2).orderByAsc("order_id");
    List<Slide> slides = slideMapper.selectList(wrapper);
    model.put("slides", slides);
```

```
// ==================== 3. 思诚挑选商品板块 ====================
// 3.1 导航标签：热销、新品、精品、清仓、断码
model.put("scpickedNav", Arrays.asList("热销","新品","精品","清仓","断码"));
model.put("scpickedId", scpickedId); // 当前选中的标签ID

// 3.2 根据标签ID查询对应商品，按创建时间倒序，取16条
QueryWrapper<Product> wrapper1 = new QueryWrapper<>();
wrapper1.eq("flag", scpickedId)
        .orderByDesc("create_time")
        .last("limit 16");
List<Product> scpickedList = productMapper.selectList(wrapper1);
model.put("scpickedList", scpickedList);

// ==================== 4. 基于兴趣推荐的商品板块 ====================
// 4.1 从session中获取用户兴趣标签数据（JSON格式）
ArrayList<Map<String, Object>> followNav = new ArrayList<>();
Map<String, List<Product>> followLists = new HashMap<>();
List<Category> tagList = new ArrayList<>();
String jsonString = (String) session.getAttribute("interest");
if(jsonString != null) {
    ObjectMapper objectMapper = new ObjectMapper();
    try {
        // 解析兴趣数据：格式为 "id-name": ["category_id1", "category_id2"]
        Map<String, List<String>> data = objectMapper.readValue(jsonString,
            new TypeReference<LinkedHashMap<String, List<String>>>() {});

        // 4.2 遍历兴趣数据，生成导航菜单和商品列表
        int index = 0;
        for (Map.Entry<String, List<String>> entry : data.entrySet()) {
            // 解析键值：格式 "1-服装"
            String[] keys = entry.getKey().split("-");
```

```
            int id = Integer.parseInt(keys[0]);  // 分类 ID
            String name = keys[1];               // 分类名称
            // 创建导航项
            Map<String, Object> itemNav = Map.of("id", id, "name", name);
            // 设置默认选中第一个标签
            if (followId == 0 && index == 0) followId = id;
            followNav.add(itemNav);
            // 4.3 查询该兴趣分类下的商品
            QueryWrapper<Product> wrapper2 = new QueryWrapper<>();
            wrapper2.in("category_id", entry.getValue())  // 查询多个分类 ID
                    .orderByDesc("create_time")
                    .last("limit 16");
            // 将商品列表存入 Map，key 格式：follow_{id}
            followLists.put("follow_" + id, productMapper.selectList(wrapper2));

            // 获取选中标签的子分类
            tagList = categoryController.getAllByParentId(followId);
            index++;
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
// 4.4 将兴趣推荐数据放入模型
model.put("followId", followId);              // 当前选中的兴趣标签 ID
model.put("followNav", followNav);            // 兴趣导航菜单
// 获取当前选中标签的商品列表，如果不存在则返回空列表
List<Product> followList = followLists.computeIfAbsent("follow_" + followId,
    k -> Collections.emptyList());
model.put("followList", followList);          // 当前兴趣标签的商品列表
model.put("tagList", tagList);                // 当前标签的子分类
```

```
    // ==================== 5. 浏览历史商品板块 ====================
    // 5.1 检查用户是否登录
    Integer userId = (Integer) session.getAttribute("userId");
    List<Category> viewedNav = new ArrayList<>();
    List<Product> viewedList = new ArrayList<>();
    if (userId != null) {
        // 5.2 获取用户浏览过的商品分类（去重，最多7个）
        viewedNav = Util.filterUniqueCategoriesByName(
            historyMapper.getViewedCategory(userId), 7);
        // 5.3 设置默认选中的浏览历史分类
        if(viewedId == 0 && viewedNav.size() > 0) {
            viewedId = viewedNav.get(0).getId();
        }
        // 5.4 查询用户在该分类下浏览过的商品
        viewedList = productMapper.getViewedProduct(userId, viewedId);
    }
    // 5.5 将浏览历史数据放入模型
    model.put("viewedId", viewedId);   // 当前选中的浏览历史分类ID
    model.put("viewedNav", viewedNav); // 浏览历史分类导航
    model.put("viewedList", viewedList);// 浏览过的商品列表
    // ==================== 6. 返回视图 ====================
    return "web/index";
}
```

综上代码所述，呈现如下效果，如图8-1所示。

8.1.2 用户管理功能实现

前端VUE组件负责收集用户基本信息（如用户名、邮箱、密码等），并通过加密方式传输至SpringBoot后端；后端利用MyBatis框架操作数据库，实现用户数据的持久化存储，并对密码进行BCrypt加密处理。同时，业务层对数据进行唯一性验证，最终将注册结果返回给用户。用户提交凭证后，SpringBoot的安全模块进行认证校验，并通过JWT生成访问令牌。MyBatis从数据库中查询并验证用户信息，登录状态则通过Vuex在前端进行管理，确保会话的安全性。

1.以下将展示注册功能实现的核心代码，若需查看完整代码，请参阅源码。

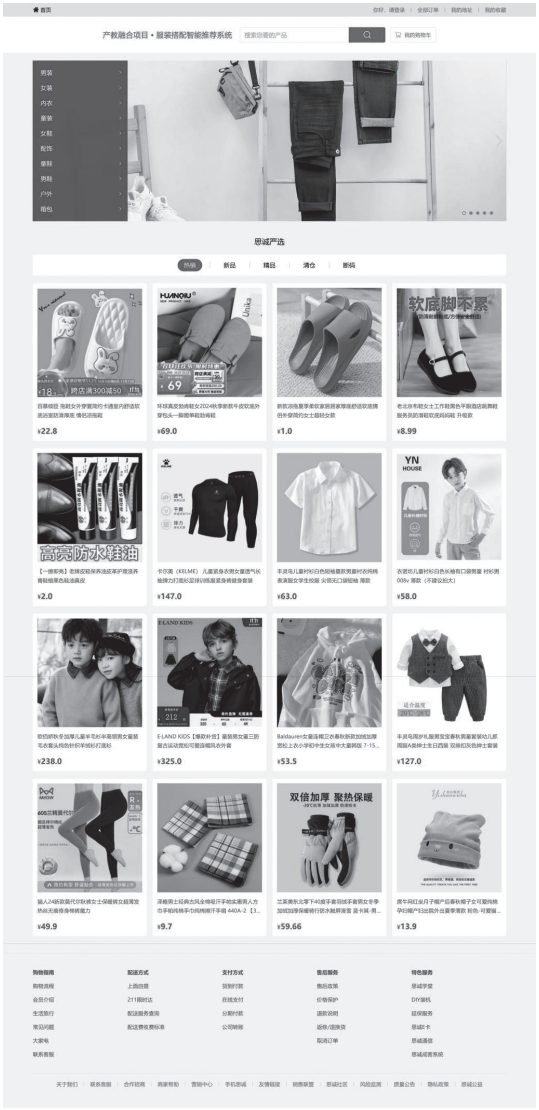


图8-1　首页

```
/**
     * 注册的方法
     * @param formData
     * @param model
     * @return
     */
    @PostMapping(value = "/api/register", consumes = MediaType.APPLICATION_FORM_URLENCODED_
VALUE)
    public String apiRegister(@RequestParam Map<String, String> formData, ModelMap model){
       // 创建用户实体对象
```

```
        User user = new User();

        // 从表单数据中提取并设置用户属性
        user.setUserName(formData.get("userName"));
        // 使用MD5加密密码（注意：MD5已不推荐用于密码存储，建议使用BCrypt等更安全的哈希算法）
        user.setPassword(EncryptUtil.md5(formData.get("password")));
        user.setPhone(formData.get("phone"));
        // 设置用户兴趣偏好，用于后续服装搭配推荐算法
        user.setInterest(formData.get("interest"));

        // 检查用户名是否已存在
        if(checkUserNameIsExist(user.getUserName())){
            // 用户名已存在，返回错误信息
                model.put("message", "用户名已经存在，请换个用户名！，<a href=\"javascript:history.back();\"
class=\"text-red-600\">返回重试</a>");
            return "web/fail"; // 返回失败页面视图
        }

        // 检查手机号是否已存在
        if(checkPhoneIsExist(user.getPhone())){
            // 手机号已存在，返回错误信息
                model.put("message", "手机号已经存在，请换个手机号！，<a href=\"javascript:history.back();\"
class=\"text-red-600\">返回重试</a>");
            return "web/fail"; // 返回失败页面视图
        }

        // 执行数据库插入操作
        int result = userMapper.insert(user);

        // 根据插入结果判断注册是否成功
        if(result == 1){
            // 插入成功（影响行数为1），注册成功
```

```
            model.put("message", "恭喜注册成功，<a href=\"/web/login\" id=\"goLink\" class=\"text-red-600\">请开始登录<b></b></a>");
            return "web/success"; // 返回成功页面视图
        }else{
            // 插入失败，返回服务器错误信息
            model.put("message", "服务器出错，注册失败!，<a href=\"javascript:history.back();\" class=\"text-red-600\">返回重试</a>");
            return "web/fail"; // 返回失败页面视图
        }
    }
```

综上代码所述，呈现如下效果，如图8-2所示。

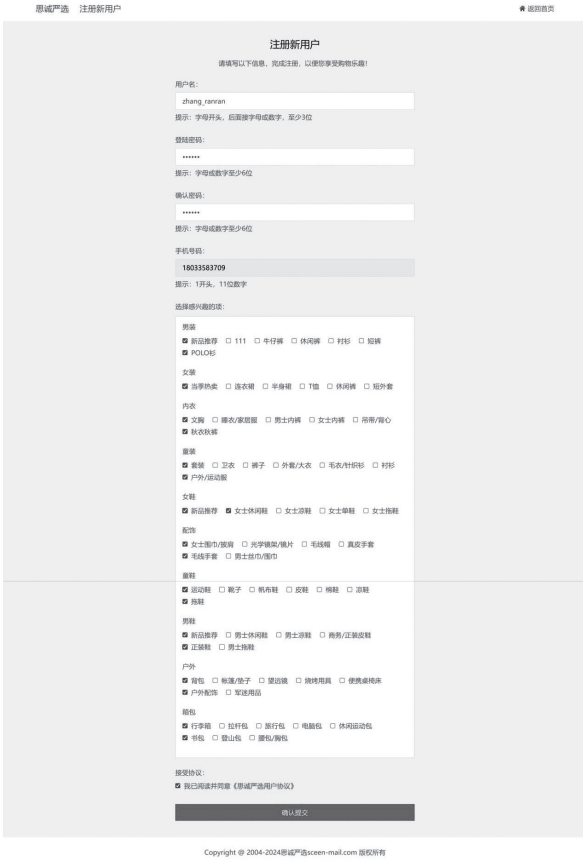


图8-2　注册页面

2.以下将展示登录功能实现的核心代码，若需查看完整代码，请参阅源码。

```
/**
 *登录的方法
```

```
 */
// 处理前端登录请求
@PostMapping(value = "/api/login", consumes = MediaType.APPLICATION_FORM_URLENCODED_VALUE)
public String apiLogin(@RequestParam Map<String, String> formData, ModelMap model, HttpSession session){

    // 创建MyBatis-Plus查询条件包装器，用于构建SQL查询条件
    QueryWrapper<User> wrapper= new QueryWrapper<>();
    // 添加查询条件：用户名字段等于表单提交的用户名
    wrapper.eq("user_name", formData.get("userName"));
    // 执行查询，获取用户信息
    User user = userMapper.selectOne(wrapper);

    // 判断用户是否存在
    if(user != null){
        // 用户存在，验证密码
        // 将前端传来的密码进行MD5加密后与数据库中存储的加密密码比较
        if(user.getPassword().equals(EncryptUtil.md5(formData.get("password")))){
            // 密码验证成功，将用户信息存入Session

            // 存储用户ID，用于后续业务逻辑中识别用户身份
            session.setAttribute("userId", user.getId());
            // 存储用户名，用于显示和验证
            session.setAttribute("userName", user.getUserName());
            // 存储昵称，用于界面显示
            session.setAttribute("nickName", user.getNickName());
            // 存储用户头像URL，用于界面展示
            session.setAttribute("avater", user.getAvatar());
            // 存储用户兴趣偏好，用于服装搭配推荐功能
            session.setAttribute("interest", user.getInterest());

            // 设置成功消息，包含跳转到会员中心的链接
                model.put("message", "恭喜，登录成功!<a href=\"/web/user/profile\" id=\"goLink\" class=\"text-
```

```
red-600\">进入会员中心<b></b></a>");
                return "web/success"; // 返回登录成功页面
            }else{
                // 密码不正确，返回错误提示
                    model.put("message", "您输入的密码不正确!<a href=\"javascript:history.back();\" class=\"text-
red-600\">返回重试</a>");
                return "web/fail"; // 返回登录失败页面
            }
        }else{
            // 用户不存在，返回错误提示
            model.put("message", "您输入的用户名不存在!<a href=\"javascript:history.back();\" class=\"text-
red-600\">返回重试</a>");
            return "web/fail"; // 返回登录失败页面
        }

    }
```

综上代码所述，呈现如下效果，如图8-3所示。



图8-3　登录的页面

8.1.3 详情页功能实现

首先，商品介绍模块是详情页的基础，它通过VUE组件动态渲染从SpringBoot后端获取、由MyBatis查询的详细数据，包括高清轮播图、设计亮点，旨在建立商品认知与情感连接。紧随其后的规格与包装及售后保障模块，则提供了关键的决策硬信息，如尺码表、具体材质成分、洗涤建议以及退换货政

策，这些结构化的信息由后端统一管理并通过API精确送达前端，极大增强了购买信心，而核心的加入收藏功能，则是连接用户行为与智能推荐系统的关键交互点。用户点击收藏时，VUE会通过Axios将请求发送至SpringBoot后端，MyBatis随之将用户－商品关系持久化至数据库。

以下将展示详情页功能实现的核心代码，若需查看完整代码，请参阅源码。

```
@RequestMapping("/product/detail")
public String productDetail(
    ModelMap model,                  // 用于向视图传递数据的模型对象
    HttpSession session,             // HTTP 会话对象，用于获取用户会话信息
    @RequestParam(value = "id", defaultValue = "0") Integer id  // 商品 ID 参数，默认值为 0
) {
    // 根据商品 ID 查询商品详细信息
    Product productDetail = productMapper.selectById(id);

    // 通过商品详情中的分类 ID，查找对应的分类对象
    // 这里查找的是商品直接所属的分类
    Category category = categoryMapper.selectById(productDetail.getCategoryId());

    // ==================== 用户浏览记录统计逻辑 ====================
    // 从会话中获取当前登录用户的 ID
    Integer userId = (Integer) session.getAttribute("userId");
    // 只有已登录用户才记录浏览历史
    if (userId != null) {
        // 构建查询条件：查找该用户对该商品的浏览记录
        // 使用 QueryWrapper 构建查询条件，查询用户 ID 和商品 ID 都匹配的记录
        History historyExist = historyMapper.selectOne(
            new QueryWrapper<History>()
                .eq("product_id", id)       // 商品 ID 条件
                .eq("user_id", userId)      // 用户 ID 条件
        );

        if (historyExist != null) {
            // 情况 1：浏览记录已存在
```

```
            // 增加该商品的浏览次数（viewTotal加1）
            historyExist.setViewTotal(historyExist.getViewTotal() + 1);
            // 更新数据库中的浏览记录
            historyMapper.updateById(historyExist);
        } else {
            // 情况2：浏览记录不存在，创建新的浏览记录
            History history = new History();
            history.setUserId(userId);          // 设置用户ID
            history.setProductId(id);           // 设置商品ID

            // 如果分类对象不为空，设置分类ID为父分类ID
            // 这里使用父分类ID而非商品直接分类ID，可能是为了更高级别的分类统计
            if (category != null) {
                history.setCategoryId(category.getParentId());
            }
            history.setViewTotal(1);            // 初始化浏览次数为1
            historyMapper.insert(history);      // 插入新的浏览记录
        }
    }
    // ==================== 浏览记录统计结束 ====================

    // 将商品所属分类放入模型，供前端页面使用
    model.put("category", categoryMapper.selectById(productDetail.getCategoryId()));
    // 将商品详情信息放入模型，供前端页面使用
    model.put("detail", productDetail);
    // 返回商品详情页的视图名称
    return "web/product-detail";
}
```

综上代码所述，呈现如下效果，如图8-4所示。

8.1.4 购物车管理功能实现

购物车功能构建了一个完整且高效的数据闭环。用户在前端通过VUE组件触发“加入购物车”操

作后，商品信息通过Axios以JSON格式提交至后端，SpringBoot控制器进行身份校验并调用MyBatis Mapper执行智能逻辑：若商品已存在则更新数量，否则新增记录。查看购物车时，VUE自动发起请求，后端通过MyBatis多表关联查询返回包含详情的完整列表，前端利用响应式特性实时渲染并计算总价。对于清空或删除操作，前端发送对应请求，后端执行精确的SQL删除并同步更新视图。

最终的结算流程则是一个在事务保障下的关键转化：前端先进行预检核验，用户确认后提交订单数据，后端在原子性事务中依次完成库存扣减、订单生成及购物车清理，确保数据一致性。整个模块从前端交互、后端逻辑到数据持久化，各层职责清晰，协同实现了从商品暂存、管理到订单生成的无缝流转。

1.以下将展示添加购物车的核心代码，若需查看完整代码，请参阅源码。

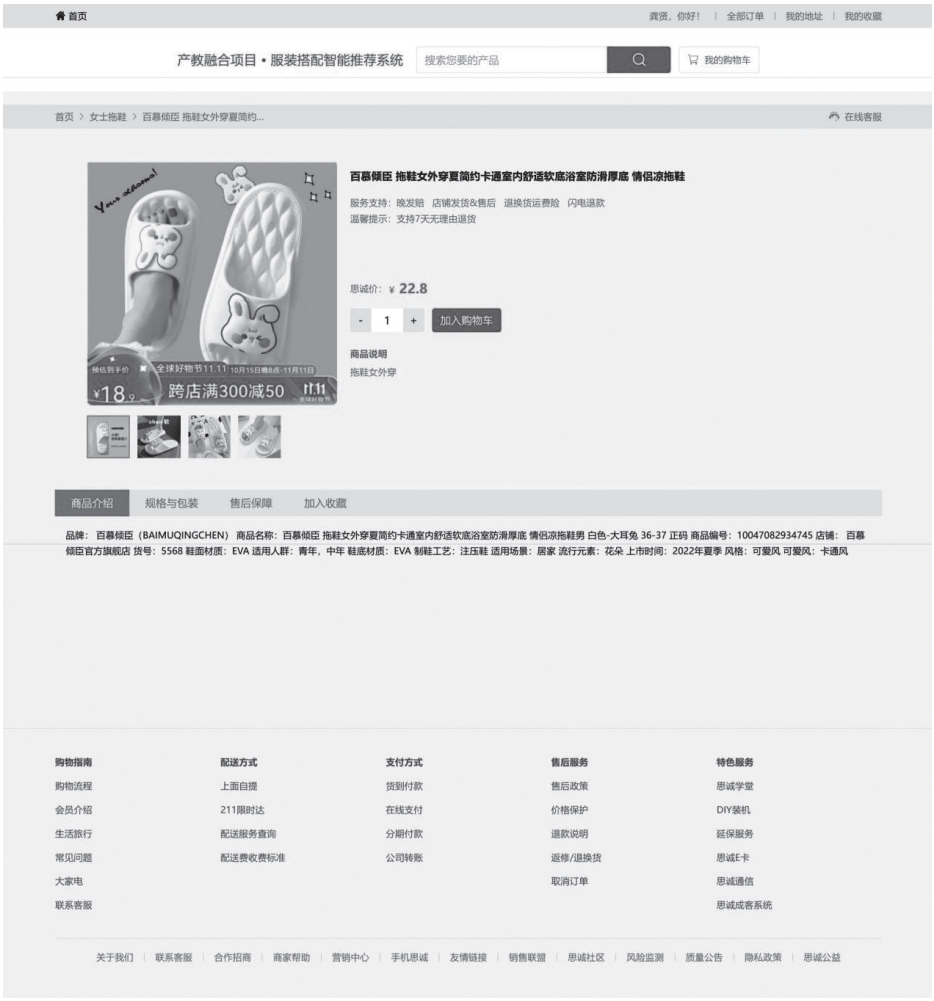


图8-4　详情页页面

```
/**
 * 添加购物车的方法
 */
@PostMapping("/api/user/mycart/add")
```

```
public String apiMycarteAdd(
    @RequestParam Map<String, String> formData, // 接收表单提交的所有参数，转换为 Map
    ModelMap model,                             // Spring MVC 模型，用于传递数据到视图
    HttpSession session                         // HTTP 会话对象，获取用户会话信息
) {
    // 从会话中获取当前登录用户的 ID
    Integer userId = (Integer) session.getAttribute("userId");

    // 检查用户是否已登录
    if (userId != null) {
        // 从表单数据中提取商品 ID 和购买数量
        Integer productId = Integer.parseInt(formData.get("productId"));
        Integer total = Integer.parseInt(formData.get("total"));
        // 构建查询条件：查找该用户是否已经将该商品加入购物车
        QueryWrapper<Mycart> wrapper = new QueryWrapper<>();
        wrapper.eq("user_id", userId)      // 用户 ID 匹配条件
               .eq("product_id", productId); // 商品 ID 匹配条件
        // 查询购物车中是否已存在该商品
        Mycart mycart1 = mycartMapper.selectOne(wrapper);
        int result = 0; // 数据库操作结果，1 表示成功，0 表示失败
        // ==================== 情况 1：商品已存在于购物车 ====================
        if (mycart1 != null) {
            // 商品已在购物车中，增加购买数量
            // 将原有数量加上新添加的数量
            mycart1.setTotal(mycart1.getTotal() + total);
            // 更新数据库中的购物车记录
            result = mycartMapper.updateById(mycart1);
        }
        // ==================== 情况 2：商品未在购物车中 ====================
        else {
            // 创建新的购物车记录
            Mycart mycart2 = new Mycart();
```

```
            // 设置用户 ID
            mycart2.setUserId(userId);
            // 设置商品 ID
            mycart2.setProductId(productId);
            // 设置商品缩略图 URL
            mycart2.setThumbUrl(formData.get("thumbUrl"));
            // 设置商品标题
            mycart2.setTitle(formData.get("title"));
            // 设置商品价格（从字符串转换为 Double 类型）
            mycart2.setPrice(Double.parseDouble(formData.get("price")));
            // 设置购买数量
            mycart2.setTotal(total);
            // 插入新的购物车记录到数据库
            result = mycartMapper.insert(mycart2);
        }
        // ==================== 处理数据库操作结果 ====================
        if (result == 1) {
            // 操作成功：向模型添加成功消息和返回链接
            model.put("message", "加入购物车成功！，<a href=\"/web/product/detail?id=" + productId +
                    "\" id=\"goLink\" class=\"text-red-600\">返回产品详情 <b></b></a>");
            // 返回成功页面
            return "web/success";
        } else {
            // 操作失败：向模型添加失败消息和返回链接
            model.put("message", "加入购物车失败！，<a href=\"javascript:history.back();\" " +
                    "class=\"text-red-600\">返回上页 </a>!");
            // 返回失败页面
            return "web/fail";
        }
    }
    // ==================== 用户未登录情况 ====================
    else {
```

```
        // 用户未登录：提示登录信息
        model.put("message", "您还没有登录，<a href=\"/web/login\" " +
                "class=\"text-red-600\">请先登录</a>");
        // 返回失败页面
        return "web/fail";
    }
}
```

综上代码所述，呈现如下效果，如图8-5所示。



图8-5　添加购物车页面

2.以下将展示查看购物车的核心代码，若需查看完整代码，请参阅源码。

```
/**
 * 获取用户购物车列表页面
 * 显示当前登录用户的购物车中所有商品，并计算总金额
 */
@GetMapping("/user/mycart")
public String mycart(ModelMap model, HttpSession session){
    // 从HTTP会话中获取当前登录用户的ID
    Integer userId = (Integer) session.getAttribute("userId");
    // 检查用户是否已登录
    if (userId != null) {
        // 构建查询条件，获取当前用户的购物车数据
```

```
        QueryWrapper<Mycart> wrapper = new QueryWrapper<>();
        // 条件1：用户ID匹配，只查询当前用户的购物车记录
        wrapper.eq("user_id", userId);
        // 条件2：按创建时间倒序排序，最新添加的商品显示在最前面
        wrapper.orderByDesc("create_time");
        // 执行查询，获取用户的所有购物车记录
        List<Mycart> mycarts = mycartMapper.selectList(wrapper);
        // 初始化购物车总金额变量
        double amount = 0;
        // 遍历购物车中的每个商品，计算总金额
        for (Mycart mycart : mycarts) {
            // 累加：商品单价 × 购买数量
            amount += mycart.getPrice() * mycart.getTotal();
        }
        // 将数据放入模型，供前端视图使用：
        // 1. 购物车中商品的总种类数（非总件数）
        model.put("total", mycarts.size());
        // 2. 购物车中所有商品的详细信息列表
        model.put("mycarts", mycarts);
        // 3. 购物车中所有商品的总金额
        model.put("amount", amount);
        // 返回购物车页面视图
        return "web/user-mycart";
    } else {
        // 用户未登录的情况处理
        // 向模型添加错误提示信息，包含登录链接
        model.put("message", "您还没有登录，<a href=\"/web/login\" class=\"text-red-600\">请先登录</a>");
        // 返回失败页面视图
        return "web/fail";
    }
}
```

综上代码所述，呈现如下效果，如图8-6所示。



图8-6　查看购物车页面

3.以下将展示清空和删除购物车商品的核心代码，若需查看完整代码，请参阅源码。

```
/**
 * 批量删除购物车商品接口
 * 删除用户购物车中指定的多个商品
 */
@GetMapping("/api/user/mycart/delete")
public String apiDeleteMycart(ModelMap model, HttpSession session, @RequestParam() String ids){
    // 从HTTP会话中获取当前登录用户的ID
    Integer userId = (Integer) session.getAttribute("userId");
    // 检查用户是否已登录
    if (userId != null) {
        // 将逗号分隔的ID字符串转换为整数列表
        // 例如："1,2,3" → [1, 2, 3]
        List<Integer> delIds = Util.str2List(ids);
        // 使用MyBatis Plus的批量删除方法，根据ID列表删除购物车记录
        // deleteBatchIds方法会删除id字段在delIds列表中的所有记录
        int i = mycartMapper.deleteBatchIds(delIds);
        // 检查删除操作是否成功
        if( i > 0 ){
            // 删除成功，重定向到购物车页面
            // 使用redirect实现页面跳转，浏览器地址栏会发生变化
            return "redirect:/web/user/mycart";
        } else {
            // 删除失败（可能ID不存在或数据库操作失败）
            // 向模型添加错误提示信息，包含返回上一页的链接
             model.put("message", "删除商品失败！<a href=\"javascript:history.back();\" class=\"text-red-600\">
```

```
返回上页</a>");
                // 返回失败页面
                return "web/fail";
            }
        } else {
            // 用户未登录的情况处理
            // 向模型添加登录提示信息
            model.put("message", "您还没有登录，<a href=\"/web/login\" class=\"text-red-600\">请先登录</a>");
            // 返回失败页面
            return "web/fail";
        }
    }

    /**
     * 清空购物车接口
     * 删除当前用户购物车中的所有商品
     */
    @GetMapping("/api/user/mycart/clear")
    public String apiDeleteMycart(ModelMap model, HttpSession session){  // 注意：方法名冲突！
        // 从HTTP会话中获取当前登录用户的ID
        Integer userId = (Integer) session.getAttribute("userId");
        // 检查用户是否已登录
        if (userId != null) {
            // 创建删除条件：根据用户ID删除所有购物车记录
            // Collections.singletonMap 创建只包含一个键值对的Map
            // 条件：user_id = userId
            int i = mycartMapper.deleteByMap(Collections.singletonMap("user_id", userId));
            // 检查删除操作是否成功
            if( i > 0 ){
                // 清空成功，重定向到购物车页面
                return "redirect:/web/user/mycart";
            } else {
```

```
            // 清空失败（可能购物车本来就是空的）
            // 向模型添加错误提示信息
                model.put("message", "清空购物车失败！<a href=\"javascript:history.back();\" class=\"text-red-600\">返回上页</a>");
            // 返回失败页面
            return "web/fail";
        }
    } else {
        // 用户未登录的情况处理
        model.put("message", "您还没有登录，<a href=\"/web/login\" class=\"text-red-600\">请先登录</a>");
        return "web/fail";
    }
}
```

综上代码所述，呈现如下效果，如图8-7所示。



图8-7　清空购物车与删除商品页面

4.以下将展示结算功能的核心代码，若需查看完整代码，请参阅源码。

```
/**
 * 结算页面接口
 * 显示用户选中的购物车商品，用于订单确认和结算
 */
@GetMapping("/user/checkout")
public String checkout(
```

```
    ModelMap model,
    HttpSession session,
    @RequestParam() String ids  // 必须参数：要结算的购物车商品ID列表，如 "1,3,5"
) {
    // 从HTTP会话中获取当前登录用户的ID
    Integer userId = (Integer) session.getAttribute("userId");

    // 检查用户是否已登录
    if (userId != null) {
        // ==================== 1. 获取用户收货地址 ====================
        QueryWrapper<Address> wrapper1 = new QueryWrapper<>();
        // 条件1：只查询当前用户的收货地址
        wrapper1.eq("user_id", userId);
        // 条件2：按默认地址优先排序(is_default DESC)，再按创建时间倒序(id DESC)
        // 默认地址（is_default=1）会显示在最前面
        wrapper1.orderByDesc("is_default", "id");

        // 执行查询，获取用户的收货地址列表
        List<Address> addresses = addressMapper.selectList(wrapper1);
        // 将地址列表放入模型，供前端选择使用
        model.put("addresses", addresses);
        // ==================== 2. 获取选中的购物车商品 ====================
        // 将逗号分隔的ID字符串转换为整数列表
        // 例如："1,3,5" → [1, 3, 5]
        List<Integer> pickIds = Util.str2List(ids);

        // 构建购物车商品查询条件
        QueryWrapper<Mycart> wrapper2 = new QueryWrapper<>();
        // 条件1：ID在用户选中的ID列表中
        wrapper2.in("id", pickIds);
        // 条件2：按创建时间倒序排序，最新添加的商品显示在前面
        wrapper2.orderByDesc("create_time");
```

```
        // 执行查询，获取用户选中的购物车商品列表
        List<Mycart> mycarts = mycartMapper.selectList(wrapper2);
        // ==================== 3. 计算订单总金额 ====================
        // 初始化总金额为0
        double amount = 0;
        // 遍历选中的每个商品，累加金额
        for (Mycart mycart : mycarts) {
            // 计算公式：商品单价 × 购买数量
            amount += mycart.getPrice() * mycart.getTotal();
        }
        // ==================== 4. 将数据放入模型 ====================
        // 原始ID字符串，可能用于后续表单提交
        model.put("ids", ids);
        // 选中的商品种类数（非总件数）
        model.put("total", mycarts.size());
        // 选中的商品详细信息列表
        model.put("mycarts", mycarts);
        // 订单总金额
        model.put("amount", amount);
        // 返回结算页面视图
        return "web/user-checkout";

    } else {
        // 用户未登录的情况处理
        // 向模型添加登录提示信息
        model.put("message", "您还没有登录，<a href=\"/web/login\" class=\"text-red-600\">请先登录</a>");
        // 返回失败页面
        return "web/fail";
    }
}
```

综上代码所述，呈现如下效果，如图8-8所示。



图8-8　结算页面

8.1.5 订单管理功能实现

订单功能通过前后端协同高效实现。用户在结算时，前端将订单数据提交至后端，后端在事务控制下原子化完成库存锁定、订单主从表插入、库存扣减及购物车清理等操作，生成订单后跳转至支付。订单取消时，系统校验状态后更新订单信息并恢复库存，确保数据一致性。整个流程通过严谨的事务管理和状态控制，保障了订单生命周期的可靠性与业务正确性。

1.以下将展示创建订单的核心代码，若需查看完整代码，请参阅源码。

```
    /**
     * 提交订单接口
     * 处理用户结算页面的订单提交，创建订单并清理购物车
     *
     */
    @PostMapping(value = "/api/user/checkout/submit", consumes = MediaType.APPLICATION_FORM_
URLENCODED_VALUE)
    public String apiCheckoutSubmit(
        @RequestParam Map<String, String> formData, // 接收所有表单参数为键值对
        ModelMap model,
        HttpSession session
    ) {
        // 从HTTP会话中获取当前登录用户的ID
        Integer userId = (Integer) session.getAttribute("userId");

        // 检查用户是否已登录
        if (userId != null) {
```

```
// ==================== 1. 解析选中的购物车商品 ID ====================
// 从表单数据中获取 ids 参数，并转换为整数列表
// 格式：ids="1,2,3" → [1, 2, 3]
List<Integer> pikeIds = Util.str2List(formData.get("ids"));

// ==================== 2. 生成唯一订单号 ====================
// 使用工具类生成订单号，通常包含时间戳、随机数等
String orderformNo = Util.getOrderNumber();

// ==================== 3. 查询选中的购物车商品 ====================
// 构建查询条件：查询用户选中的购物车商品
QueryWrapper<Mycart> wrapper = new QueryWrapper<>();
wrapper.in("id", pikeIds);  // ID 在选中的 ID 列表中

// 执行查询，获取购物车中选中的商品列表
List<Mycart> mycarts = mycartMapper.selectList(wrapper);

// ==================== 4. 批量插入订单详情（订单项）====================
// 遍历购物车中的每个商品，创建订单详情记录
mycarts.forEach(item -> {
    OrderformDetail orderformDetail = new OrderformDetail();
    // 设置订单编号，所有订单项共享同一个订单号
    orderformDetail.setOrderformNo(orderformNo);
    // 复制商品信息到订单详情
    orderformDetail.setThumbUrl(item.getThumbUrl());  // 商品缩略图
    orderformDetail.setProductId(item.getProductId()); // 商品 ID
    orderformDetail.setTitle(item.getTitle());        // 商品标题
    orderformDetail.setPrice(item.getPrice());        // 商品单价
    orderformDetail.setTotal(item.getTotal());        // 购买数量

    // 插入订单详情到数据库
```

```
            orderformDetailMapper.insert(orderformDetail);
        });

        // ==================== 5. 清理购物车 ====================
        // 删除用户已经下单的购物车商品
        // 注意：这里没有验证这些商品是否属于当前用户，存在安全隐患
        mycartMapper.deleteBatchIds(pikeIds);

        // ==================== 6. 创建主订单记录 ====================
        Orderform orderform = new Orderform();
        orderform.setUserId(userId);                    // 用户 ID
        orderform.setOrderformNo(orderformNo);          // 订单编号
        orderform.setAddressId(Integer.parseInt(formData.get("addressId"))); // 收货地址 ID
        orderform.setTotal(Integer.parseInt(formData.get("total")));     // 商品种类数
        orderform.setAmount(Double.parseDouble(formData.get("amount")));  // 订单总金额

        // 发票信息
        orderform.setInvoiceTitle(formData.get("invoiceType")); // 注意：这里有错误，设置了两遍 invoiceTitle
            orderform.setInvoiceTitle(formData.get("invoiceTitle")); // 第二遍覆盖了第一遍，可能是
setInvoiceContent 或 setInvoiceType

        orderform.setStatus(1); // 订单状态：1 可能表示 "待支付" 或 "已提交"

        // 插入主订单到数据库
        int result = orderformMapper.insert(orderform);

        // ==================== 7. 处理订单创建结果 ====================
        if (result == 1) {
            // 订单创建成功
                model.put("message", "订单已经提交!<a href=\"/web/user/orders\" id=\"goLink\" class=\"text-
red-600\">返回我的订单<b></b></a>");
            return "web/success";
```

```
            } else {
                // 订单创建失败
                 model.put("message", "订单提交失败!<a href=\"/web/user/mycart\" class=\"text-red-600\">我的购物车</a>");
                return "web/fail";
            }

        } else {
            // 用户未登录的情况
            model.put("message", "您还没有登录，<a href=\"/web/login\" class=\"text-red-600\">请先登录</a>");
            return "web/fail";
        }
    }
```

综上代码所述，呈现如下效果，如图8-9所示。



图8-9　创建订单的页面

2.以下将展示查看订单的核心代码，若需查看完整代码，请参阅源码。

```
/**
```

```
 * 用户订单列表页面接口
 * 查询并显示当前用户的订单列表，支持按状态筛选和分页显示
 */
@GetMapping("/user/orders")
public String orders(
    ModelMap model,
    HttpSession session,
    @RequestParam(value = "status", defaultValue = "0") int status,
    @RequestParam(value = "pageNo", defaultValue = "1") int pageNo,
    @RequestParam(value = "pageSize", defaultValue = "10") int pageSize
) {
    // 从HTTP会话中获取当前登录用户的ID
    Integer userId = (Integer) session.getAttribute("userId");

    // 检查用户是否已登录
    if (userId != null) {

        // ==================== 1. 构建联表查询条件 ====================
        // 使用MyBatis Plus Join的LambdaWrapper进行联表查询
        MPJLambdaWrapper<Orderform> wrapper = new MPJLambdaWrapper<>();

        // 选择订单表的所有字段
        wrapper.selectAll(Orderform.class);
        // 选择地址表的三个字段：收货人姓名、电话、详细地址
        wrapper.select(Address::getName, Address::getPhone, Address::getAddress);
        // 左连接地址表，通过address_id关联
        wrapper.leftJoin(Address.class, Address::getId, Orderform::getAddressId);
        // 条件：只查询当前用户的订单
        wrapper.eq(Orderform::getUserId, userId);
        // 排序：按订单创建时间倒序排列，最新订单显示在最前面
        wrapper.orderByDesc(Orderform::getCreateTime);
```

```
// 如果状态参数不为0，添加状态筛选条件
if (status != 0) {
    wrapper.eq(Orderform::getStatus, status);
}

// ==================== 2. 执行分页查询 ====================
// 创建分页对象，设置当前页码和每页大小
IPage<Orderform> orderforms = orderformMapper.selectJoinPage(
    new Page<>(pageNo, pageSize),  // 分页参数
    Orderform.class,               // 返回的主实体类型
    wrapper                        // 查询条件
);
// ==================== 3. 设置分页相关参数 ====================
model.put("status", status);              // 当前筛选状态
model.put("pageNo", pageNo);              // 当前页码
model.put("pageSize", pageSize);          // 每页显示条数
Long total = orderforms.getTotal();       // 订单总数
model.put("total", total);                // 总记录数
// 计算总页数：向上取整(总记录数 / 每页大小)
model.put("pageCount", Math.ceil((double) total / pageSize));
// ==================== 4. 查询每个订单的详情（订单项）====================
// 遍历查询结果中的每个订单
orderforms.getRecords().forEach(orderform -> {
    // 创建查询条件：通过订单号查询对应的订单详情
    Map<String, Object> columnMap = new HashMap<>();
    columnMap.put("orderform_no", orderform.getOrderformNo());
    // 执行查询，获取该订单的所有商品详情
    List<OrderformDetail> details = orderformDetailMapper.selectByMap(columnMap);
    // 将详情列表设置到订单对象中（假设Orderform类有setDetails方法）
    orderform.setDetails(details);
});
// ==================== 5. 将数据放入模型 ====================
```

```
        // 将处理好的订单记录列表放入模型，供前端显示
        model.put("records", orderforms.getRecords());

        // 返回订单列表页面
        return "web/user-orders";
    } else {
        // 用户未登录的情况处理
        model.put("message", "您还没有登录，<a href=\"/web/login\" class=\"text-red-600\">请先登录</a>!");
        return "web/fail";
    }
}
```

综上代码所述，呈现如下效果，如图8-10所示。



图8-10　查看订单页面

3. 以下将展示移除订单的核心代码，若需查看完整代码，请参阅源码。

```
/**
 * 删除订单接口，处理用户删除单个订单的请求
*/
@GetMapping("/api/user/orders/delete")
public String apiDeleteOrders(
    ModelMap model,                  // 用于向视图传递消息和重定向链接
    HttpSession session,             // 获取当前用户会话信息
    @RequestParam() Integer id       // 必须参数：要删除的订单 ID
) {
    // 从 HTTP 会话中获取当前登录用户的 ID
    Integer userId = (Integer) session.getAttribute("userId");
    // 检查用户是否已登录
    if (userId != null) {
        // ==================== 1. 执行订单删除操作 ====================
        // 问题：直接根据 ID 删除，没有验证订单是否属于当前用户
        // 风险：用户 A 可以删除用户 B 的订单，如果知道订单 ID
        int i = orderformMapper.deleteById(id);
        // ==================== 2. 处理删除结果 ====================
        if (i > 0) {
            // 删除成功：返回成功页面，提供返回订单列表的链接
            model.put("message", "删除成功！<a href=\"/web/user/orders\" id=\"goLink\" " +
                    "class=\"text-red-600\">返回订单列表<b></b></a>!");
            return "web/success";
        } else {
            // 删除失败：可能订单不存在或数据库操作失败
            model.put("message", "删除失败！<a href=\"javascript:history.back();\" " +
                    "class=\"text-red-600\">返回上页</a>!");
            return "web/fail";
        }
    } else {
        // 用户未登录的情况处理
```

```
        model.put("message", "您还没有登录，<a href=\"/web/login\" " +
                "class=\"text-red-600\">请先登录</a>!");
        return "web/fail";
    }
}
```

综上代码所述，呈现如下效果，如图8-11所示。



图8-11　移除订单的页面

8.1.6 支付管理功能实现

支付宝支付功能（含沙箱环境）采用标准的安全对接流程。在开发阶段，系统接入支付宝沙箱环境进行测试：前端提交支付请求后，后端调用支付宝沙箱SDK，使用沙箱专属密钥与API网关生成支付参数并返回沙箱支付页面；用户完成沙箱模拟支付后，系统通过支付宝沙箱的异步回调进行签名验证，并调用MyBatis更新测试订单的支付状态，完整模拟真实支付流程。这一机制确保在开发调试阶段能安全验证支付全链路，待测试通过后，仅需切换为正式环境配置即可上线，实现了从开发到生产环境的安全可靠支付闭环。

以下将展示支付订单的核心代码，若需查看完整代码，请参阅源码。

```
/**
 * 支付宝支付接口
 * 处理用户订单支付请求，调用支付宝接口生成支付页面
 */
@GetMapping("/pay")
public void pay(
    String orderformNo,              // 订单编号参数，从URL中获取
```

```
    HttpServletResponse httpResponse   // HTTP响应对象，用于输出支付页面
) throws Exception {
    // ==================== 1. 根据订单号查询订单信息 ====================
    QueryWrapper<Orderform> wrapper = new QueryWrapper<>();
    wrapper.eq("orderform_no", orderformNo); // 通过订单编号查询
    Orderform orderform = orderformMapper.selectOne(wrapper);
    // 调试输出：打印签名类型（可能用于调试）
    System.out.println("signType ===>" + signType);
    // ==================== 2. 订单存在性检查 ====================
    if (orderform != null) {
        // ==================== 3. 初始化支付宝客户端 ====================
        // 创建支付宝SDK客户端实例，配置商户信息
        AlipayClient alipayClient = new DefaultAlipayClient(
            gatewayUrl,        // 支付宝网关地址（沙箱/生产环境）
            appId,             // 商户应用APP_ID
            appPrivateKey,     // 商户私钥
            format,            // 数据格式，如：JSON
            charset,           // 字符编码，如：UTF-8
            aliPayPublicKey,   // 支付宝公钥
            signType           // 签名算法类型，如：RSA2
        );

        // ==================== 4. 创建支付请求对象 ====================
        AlipayTradePagePayRequest request = new AlipayTradePagePayRequest();
        // 设置异步通知地址（支付成功后支付宝服务器回调的地址）
        request.setNotifyUrl(notifyUrl);

        // ==================== 5. 构建业务参数 ====================
        JSONObject bizContent = new JSONObject();
        bizContent.put("out_trade_no", orderform.getOrderformNo());  // 商户订单号
        bizContent.put("product_code", "FAST_INSTANT_TRADE_PAY");    // 产品码：电脑网站支付
        bizContent.put("total_amount", orderform.getAmount());       // 订单总金额（单位：元）
```

```
        bizContent.put("subject", "订单支付");                    // 订单标题
        // 设置业务参数到请求对象
        request.setBizContent(bizContent.toString());
        // 设置同步返回地址（支付完成后用户浏览器跳转的地址）
        request.setReturnUrl(returnUrl);
        // ====================== 6. 调用支付宝接口生成支付表单 ======================
        String form = "";
        try {
            // 调用支付宝SDK，生成支付页面的HTML表单代码
            form = alipayClient.pageExecute(request).getBody();
        } catch (Exception e) {
            e.printStackTrace(); // 异常处理简单打印，生产环境需要更完善处理
        }

        // ====================== 7. 将支付表单返回给客户端 ======================
        // 设置响应内容类型为HTML
        httpResponse.setContentType("text/html;charset=" + charset);
        // 将支付宝返回的HTML表单写入响应
        httpResponse.getWriter().write(form);
        httpResponse.getWriter().flush();
        httpResponse.getWriter().close();
    }
}
```

综上代码所述，呈现如下效果，如图8-12所示。



图8-12 支付宝支付页面

8.2 服务端的功能实现



8.2.1 用户管理功能实现

提供管理员信息的全量查询和按ID查询功能，其中查询操作经过安全设计，通过字段选择器避免返回密码等敏感信息。支持管理员的增删改操作，新增和修改功能中包含密码的MD5加密处理。此外，提供了管理员状态更新接口，采用PATCH方法实现部分更新。还包括获取当前登录管理员信息的接口。整体实现了对管理员账号的基础CRUD操作和状态管理，并充分考虑了数据安全性。

以下将展示用户管理功能核心代码，若需查看完整代码，请参阅源码。

```
/**
 * 获取所有管理员列表
 * @return 包含所有管理员列表的响应结果，按ID倒序排列（最新的在前）
 */
@GetMapping("/admin")
public ResultUtil findAll(){
    QueryWrapper<Admin> wrapper = new QueryWrapper<>();
    wrapper.orderByDesc("id");
    return ResultUtil.isSuccess(gxAdminMapper.selectList(wrapper));
}

/**
 * 根据ID查询管理员信息（部分字段）
 * @param id 管理员ID（路径参数）
 * @return 包含指定管理员信息的响应结果，只返回选择的字段
 * @apiNote 该方法通过字段选择器限制返回的字段，避免返回密码等敏感信息
 */
@GetMapping("/admin/{id}")
public ResultUtil findOne(@PathVariable Integer id){
    QueryWrapper<Admin> queryWrapper = new QueryWrapper<>();
```

```
        queryWrapper.select("id", "user_name","nick_name","gender","phone", "email","avatar", "status"); // 只选择需要的字段
        queryWrapper.eq("id", id);
        return ResultUtil.isSuccess(gxAdminMapper.selectOne(queryWrapper));
    }

    /**
     * 获取当前登录管理员信息
     * @param token 认证令牌（请求头 Authorization）
     * @return 包含当前管理员信息的响应结果
     * @apiNote 注意：此处为简化示例，实际应解析token获取用户ID而非硬编码为1
     */
    @GetMapping("/admin/current")
    public ResultUtil current(@RequestHeader("Authorization") String token){
        return ResultUtil.isSuccess(gxAdminMapper.selectById(1));
    }

    /**
     * 更新管理员状态
     * @param id 管理员ID（路径参数）
     * @param status 要更新的状态值（路径参数）
     * @return 包含更新操作结果的响应
     * @apiNote 使用PATCH方法进行部分更新，仅更新状态字段
     */
    @PatchMapping("/admin/{id}/{status}")
    public ResultUtil status(@PathVariable Integer id,@PathVariable Integer status){
        Admin admin = new Admin();
        admin.setId(id);
        admin.setStatus(status);
        return ResultUtil.isSuccess(gxAdminMapper.updateById(admin));
    }
```

```
/**
 * 新增管理员
 * @param admin 管理员信息（请求体，JSON格式）
 * @return 包含新增操作结果的响应
 * @apiNote 密码会使用MD5进行加密存储
 */
@PostMapping("/admin")
public ResultUtil add(@RequestBody Admin admin){
    admin.setPassword(EncryptUtil.md5(admin.getPassword()));
    return ResultUtil.isSuccess(gxAdminMapper.insert(admin));
}

/**
 * 修改管理员信息
 * @param admin 待更新的管理员信息（请求体，JSON格式）
 * @return 包含更新操作结果的响应
 * @apiNote 1. 如果密码不为空，则会对密码进行MD5加密<br>
 *          2. 如果密码为空，则不更新密码字段<br>
 *          3. 管理员ID必须包含在admin对象中
 */
@PutMapping("/admin")
public ResultUtil edit(@RequestBody Admin admin){
    if(admin.getPassword() != null && !admin.getPassword().isEmpty()){
        admin.setPassword(EncryptUtil.md5(admin.getPassword()));
    }
    return ResultUtil.isSuccess(gxAdminMapper.updateById(admin));
}

/**
 * 根据ID删除管理员
 * @param id 管理员ID（路径参数）
 * @return 包含删除操作结果的响应
```

```
 */
@DeleteMapping("/admin/{id}")
public ResultUtil delete(@PathVariable Integer id){
    return ResultUtil.isSuccess(gxAdminMapper.deleteById(id));
}
```

综上代码所述，呈现如下效果，如图8-13所示。

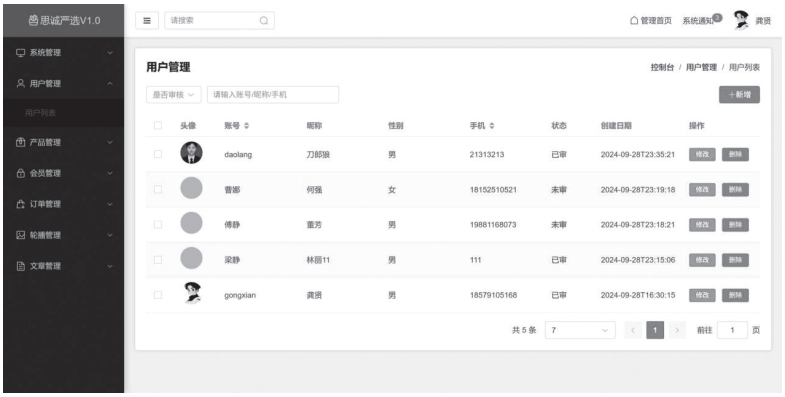


图8-13 用户管理的页面

8.2.2 产品管理功能实现

产品管理模块全面支持基础的CRUD操作，并具备高效的分页条件查询功能。具体而言，分页查询接口允许用户根据商品状态、预设价格区间以及关键词等多个条件进行灵活的动态筛选，且默认按商品创建时间降序排列，确保最新添加的商品优先展示。在基础功能方面，该模块提供了查询所有商品列表、根据商品唯一ID精确查询、新增商品记录、修改现有商品信息以及删除指定商品等核心操作。所有数据持久化操作均通过MyBatis-Plus框架高效实现，保障了数据访问层的简洁性与性能优化。此外，系统采用统一的ResultUtil工具类对响应结果进行标准化封装，确保返回数据格式的一致性及前端处理的便利性。

以下将展示产品管理核心代码，若需查看完整代码，请参阅源码。

```
/**
 * 商品管理控制器
 * 提供商品信息的增删改查及相关管理功能
 * 访问路径前缀：/api/manage
 */
@RestController
@RequestMapping("/api/manage")
public class GxProductController {
```

```
// 商品数据访问接口
@Resource
private GxProductMapper gxProductMapper;

/**
 * 获取所有商品列表
 * @return 包含所有商品列表的响应结果
 * @apiNote 不推荐在生产环境使用，数据量大时可能存在性能问题
 */
@GetMapping("/product")
public ResultUtil findAll(){
    return ResultUtil.isSuccess(gxProductMapper.selectList(null));
}

/**
 * 分页查询商品列表（支持多条件筛选）
 *
 * @param status 商品状态筛选条件（可选）
 * @param minPrice 最低价格筛选条件（可选）
 * @param maxPrice 最高价格筛选条件（可选）
 * @param keyword 关键词搜索条件（可选），匹配商品标题或摘要
 * @param pageNo 页码，默认值为1（可选）
 * @param pageSize 每页记录数，默认值为10（可选）
 * @return 包含分页商品数据的响应结果
 */
@GetMapping("/product/page")
public ResultUtil findPage(
    @RequestParam(required = false) Integer status,
    @RequestParam(required = false) Float minPrice,
    @RequestParam(required = false) Float maxPrice,
    @RequestParam(required = false) String keyword,
    @RequestParam(required = false, defaultValue = "1") Integer pageNo,
```

```
    @RequestParam(required = false, defaultValue = "10") Integer pageSize
){
    // 创建分页对象
    Page<Product> page = new Page<>(pageNo, pageSize);
    // 创建查询条件包装器
    QueryWrapper<Product> wrapper=new QueryWrapper<>();
    // 商品状态筛选
    if(status !=null){
        wrapper.eq("status", status);
    }
    // 最低价格筛选（大于等于）
    if (minPrice != null) {
        wrapper.ge("price", minPrice);
    }
    // 最高价格筛选（小于等于）
    if (maxPrice != null) {
        wrapper.le("price", maxPrice);
    }

    // 关键词搜索（在标题或摘要中模糊匹配）
    if(keyword !=null && !keyword.isEmpty() ){
        wrapper.like("title", keyword).or().like("summary", keyword);
    }
    // 按创建时间倒序排列（最新商品在前）
    wrapper.orderByDesc("create_time");
    // 执行分页查询
    List<Product> products = gxProductMapper.selectList(page, wrapper);
    // 构建分页响应结果
    PageResult<Product> pageResult = new PageResult<>();
    pageResult.setPage(pageNo);
    pageResult.setPageSize(pageSize);
    pageResult.setTotal(page.getTotal()); // 总记录数
```

```
        pageResult.setRecords(products); // 当前页数据
        return ResultUtil.isSuccess(pageResult);
    }
    /**
     * 根据ID查询单个商品详情
     * @param id 商品唯一标识符（路径参数）
     * @return 包含指定商品详情的响应结果
     */
    @GetMapping("/product/{id}")
    public ResultUtil findOne(@PathVariable Integer id){
        return ResultUtil.isSuccess(gxProductMapper.selectById(id));
    }

    /**
     * 新增商品
     * @param product 商品信息（请求体，JSON格式）
     * @return 包含新增操作结果的响应
     */
    @PostMapping("/product")
    public ResultUtil add(@RequestBody Product product){
        return ResultUtil.isSuccess(gxProductMapper.insert(product));
    }
    /**
     * 修改商品信息
     * @param product 待更新的商品信息（请求体，JSON格式）
     * @return 包含更新操作结果的响应
     * @apiNote 商品ID必须包含在product对象中
     */
    @PutMapping("/product")
    public ResultUtil edit(@RequestBody Product product){
        return ResultUtil.isSuccess(gxProductMapper.updateById(product));
    }
```

```
    /**
     * 根据ID删除商品
     * @param id 商品唯一标识符（路径参数）
     * @return 包含删除操作结果的响应
     */
    @DeleteMapping("/product/{id}")
    public ResultUtil delete(@PathVariable Integer id){
        return ResultUtil.isSuccess(gxProductMapper.deleteById(id));
    }

}
```

综上代码所述，呈现如下效果，如图8-14所示。



图8-14 产品管理的页面

8.2.3 会员管理功能实现

服务端会员管理功能主要描述了普通用户功能的实现。后端采用SpringBoot框架构建RESTful API，并结合MyBatis Plus的强大CRUD能力，利用其内置的通用Mapper和Service类，仅需少量代码即可快速实现用户的增删改查操作，大幅提升开发效率。分页查询功能通过MyBatis Plus的分页插件与Page对象轻松实现，支持多条件动态查询及自动统计总数。前端则采用VUE3的组合式API搭配Element Plus组件库，构建响应式的用户管理界面，并通过Axios与后端API高效交互，实现用户数据的表格分页展示、表单验证提交及批量操作。系统通过JWT技术实现接口权限控制，确保用户数据操作的安全性，从而形成了一套完整、高效且安全的用户管理体系。

以下将展示会员管理核心代码，若需查看完整代码，请参阅源码。

```
/**
 * 查询所有的用户信息
 * @return
 */
@GetMapping("/user")
public ResultUtil findAll(){
    QueryWrapper<User> wrapper = new QueryWrapper<>();
    wrapper.orderByDesc("id");
    return ResultUtil.isSuccess(gxUserMapper.selectList(wrapper));
}
/**
     * 添加用户的信息
     * @param user
     * @return
     */
    @PostMapping("/user")
    public ResultUtil add(@RequestBody User user){
        user.setPassword(EncryptUtil.md5(user.getPassword()));
        return ResultUtil.isSuccess(gxUserMapper.insert(user));
}
 /**
     * 修改用户的信息
     * @param user
     * @return
     */
    @PutMapping("/user")
    public ResultUtil edit(@RequestBody User user){
        if(user.getPassword() != null && !user.getPassword().isEmpty()){
            user.setPassword(EncryptUtil.md5(user.getPassword()));
        }
        return ResultUtil.isSuccess(gxUserMapper.updateById(user));
    }
```

```
/**
 * 删除用户的信息
 * @param id
 * @return
 */
@DeleteMapping("/user/{id}")
public ResultUtil delete(@PathVariable Integer id){
    return ResultUtil.isSuccess(gxUserMapper.deleteById(id));
}
```

综上代码所述，呈现如下效果，如图8-15所示。

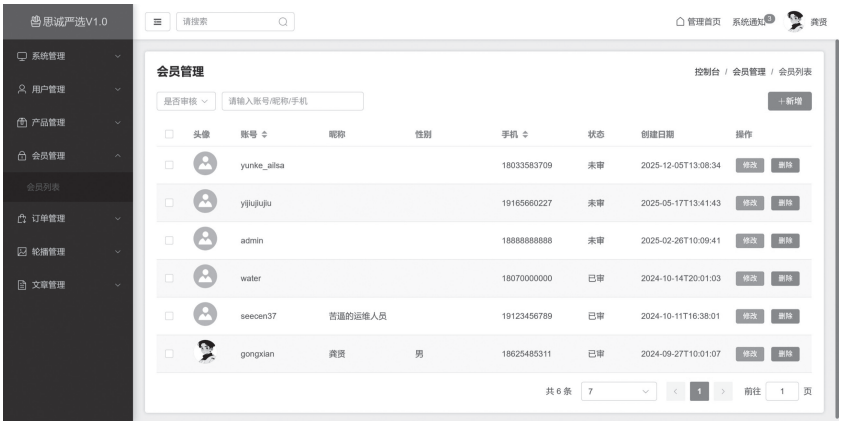


图8-15 会员列表的页面

8.2.4 订单管理功能实现

订单管理功能实现了全面的订单生命周期管控。后端采用SpringBoot构建RESTful API，并结合MyBatis Plus的内置CRUD接口和Lambda查询链式语法，高效地实现了订单的多条件复合查询、订单信息修改及特定条件下的删除功能。系统通过@TableField注解的状态字段管理订单状态（如待支付、已支付、发货中、已完成、已取消等）。前端则使用VUE3的Composition API配合Element Plus组件，构建响应式订单管理界面，并通过Axios与后端进行状态同步，实现订单状态的实时跟踪与可视化操作。

以下将展示订单管理核心代码，若需查看完整代码，请参阅源码。

```
@RestController
@RequestMapping("/api/manage")
public class GxOrderformController {
    // 订单数据访问接口
    @Resource
```

```
private GxOrderformMapper gxOrderformMapper;
// 订单详情数据访问接口
@Resource
private GxOrderformDetailMapper gxOrderformDetailMapper;
// 地址数据访问接口
@Resource
private GxAddressMapper gxAddressMapper;
/**
 * 订单分页查询（支持多条件筛选和关联查询）
 *
 */
@GetMapping("/orderform/page")
public ResultUtil findAll(
    @RequestParam(required = false) Integer status,
    @RequestParam(required = false) LocalDateTime minDate,
    @RequestParam(required = false) LocalDateTime maxDate,
    @RequestParam(required = false) String keyword,
    @RequestParam(required = false, defaultValue = "1") Integer pageNo,
    @RequestParam(required = false, defaultValue = "10") Integer pageSize
){
    // 创建 MPJLambdaWrapper 用于多表关联查询
    MPJLambdaWrapper<Orderform> wrapper = new MPJLambdaWrapper<>();
    // 选择订单表所有字段
    wrapper.selectAll(Orderform.class);
    // 选择地址表相关字段（收货人信息）
    wrapper.select(Address::getName, Address::getPhone, Address::getAddress);
    // 左连接地址表，通过 address_id 关联
    wrapper.leftJoin(Address.class, Address::getId, Orderform::getAddressId);
    // 订单状态筛选
    if(status != null){
        wrapper.eq(Orderform::getStatus, status);
    }
```

```
        // 创建时间范围筛选（大于等于）
        if(minDate != null){
            wrapper.ge(Orderform::getCreateTime, minDate);
        }
        // 创建时间范围筛选（小于等于）
        if(maxDate != null){
            wrapper.le(Orderform::getCreateTime, maxDate);
        }
        // 关键词搜索（订单号、收货人姓名、收货人电话）
        if(keyword != null && !keyword.isEmpty()){
            wrapper.like(Orderform::getOrderformNo, keyword)
                    .or(i -> i.like(Address::getName, keyword))
                    .or(i -> i.like(Address::getPhone, keyword));
        }
        // 执行分页查询，获取关联查询结果
          IPage<Orderform> orderforms = gxOrderformMapper.selectJoinPage(new Page<>(pageNo, pageSize),
Orderform.class, wrapper);
        // 为每个订单加载订单详情（商品条目信息）
        orderforms.getRecords().forEach(orderform -> {
            Map<String, Object> columnMap = new HashMap<>();
            // 通过订单号查询对应的订单详情
            columnMap.put("orderform_no", orderform.getOrderformNo());
            List<OrderformDetail> details = gxOrderformDetailMapper.selectByMap(columnMap);
            // 将详情列表设置到订单对象中
            orderform.setDetails(details);
        });
        return ResultUtil.isSuccess(orderforms);
    }
    /**
     * 更新订单状态
     *
     * @param id 订单ID（路径参数）
```

```
     * @param status 要更新的状态值（路径参数）
     * @return 包含更新操作结果的响应
     *
     * @apiNote 使用PATCH方法进行部分更新，仅更新状态字段
     *          此操作通常用于订单状态流转，如：待付款→已付款→已发货→已完成等
     */
    @PatchMapping("/orderform/{id}/{status}")
    public ResultUtil status(@PathVariable Integer id,@PathVariable Integer status){
        Orderform orderform = new Orderform();
        orderform.setId(id);
        orderform.setStatus(status);
        return ResultUtil.isSuccess(gxOrderformMapper.updateById(orderform));
    }
}
```

综上代码所述，呈现如下效果，如图8-16所示。

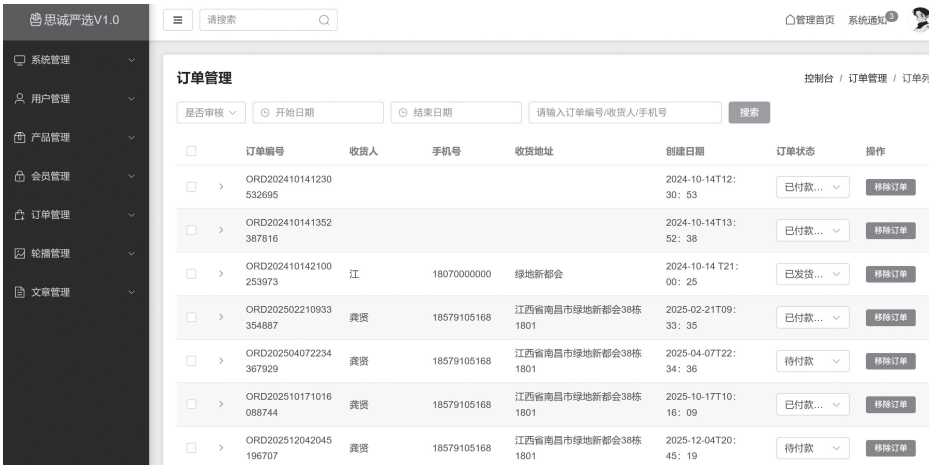


图8-16　订单列表的页面

本章小结

本章围绕“系统功能的实现”这一核心主题，系统阐述了服装搭配智能推荐系统的完整开发过程与技术实现细节。该系统基于SpringBoot后端、Vue前端及MyBatis数据持久层构建，实现了从前端用户交互

到后端业务逻辑的全栈功能闭环。

在PC端，系统完成了包括响应式首页、用户注册登录、商品详情展示、购物车管理、订单处理及支付宝支付在内的完整电商流程；在服务端，则实现了面向管理员的产品管理、会员管理、订单管理等后台管控功能。

整个实现过程不仅体现了模块化、组件化的工程思想，更通过功能实现与思政目标的有机融合，在培养技术能力的同时，强化了数据伦理、理性消费与工程规范的价值引导，完成了从技术实现到育人目标的协同推进。

练习与实践

简答题

（1）请简述本系统中用户管理模块如何实现“身份认证”与“权限控制”，并说明JWT在其中的作用。

（2）本系统首页模块中使用了Redis缓存，请说明Redis在本系统中主要用于缓存哪些数据，并分析其对系统性能的影响。

（3）购物车管理模块中，如何处理“商品已存在”与“商品未存在”两种情况下的添加逻辑？请结合代码说明。

（4）在支付管理模块中，系统为何使用“支付宝沙箱环境”进行开发测试？请说明沙箱环境与正式环境的区别及其在开发中的作用。

实践题

（1）购物车批量操作功能实现

①需求说明：目前购物车仅支持单个商品添加和删除，现需增加批量加入购物车和批量修改数量功能。

②任务要求：

1)设计前端页面组件，支持勾选多个商品后批量加入购物车

2)实现后端API接口：POST /api/user/mycart/batch-add 批量添加商品到购物车、PUT /api/user/mycart/batch-update 批量修改购物车商品数量

（2）编写完整的Java实现代码（基于MyBatis Plus），提交截图和源码

第 9 章

项目构建与部署



本章导读

本项目旨在构建一个与服装行业深度融合的智能搭配推荐系统。后端核心服务框架采用SpringBoot，前端用户界面则利用Vue3打造现代化、交互流畅的体验。通过MyBatis，系统实现了与各类数据库的高效、稳定数据交互。此外，系统将集成先进的推荐算法模型，提供个性化、精准化的搭配建议，涵盖从用户需求分析、数据处理到推荐结果生成的全流程。最终，项目将完成从算法融合、前后端开发到系统部署的一体化解决方案，确保系统在云端的高可用性和可扩展性。

学习目标

1.掌握SpringBoot多环境配置与构建：学习配置application-dev/prod.yml，并能通过Maven/Gradle命令，将SpringBoot后端项目分别打包为可直接运行的JAR包和适用于传统Servlet容器的WAR包。

2.掌握Vue3前端项目的工程化构建：学会配置Vue3项目的生产环境变量、进行代码压缩与优化，并执行npm run build生成静态资源文件，为部署做好准备。

思政目标及设计建议

将服装搭配智能推荐系统构建与部署与服装专业深度融合，并融入思想政治教育，是极具创新性且契合“课程思政”理念的研究方向。本章将阐述思政目标及设计，建议如表9-1所示。

表9-1　第9章思政目标及设计建议

思政目标	思政元素及融入
服务意识与社会责任	在阐述为何需区分开发与生产环境、为何需确保服务高可用性的重要性时，结合系统上线后将为服装企业、消费者及整个行业带来的实际影响
锤炼严谨规范、精益求精的工匠精神	通过对JAR/WAR两种部署包的规范构建、多环境严谨配置及自动化脚本编写的严格要求，引导学生树立对技术细节精益求精、对生产环境高度负责的职业态度
强化团队协作与集体主义精神	在模拟团队协作部署流程中，我们着重强调部署文档的规范性和操作的可追溯性，以及对线上服务影响的深刻敬畏，旨在培养学生的团队协作精神和工程伦理意识

9.1 环境准备



1.JDK 作为 Java 开发环境的最新稳定版本，提供了更高效的内存管理、增强的性能优化以及更丰富的语言特性支持，是当前企业级应用开发的首选运行环境。

2.Maven3.6.x 作为项目构建与依赖管理的核心工具，不仅能够自动化管理项目构建的生命周期，还能通过其强大的插件机制和依赖解析功能，显著提升开发效率并确保依赖版本的一致性。

3.IDEA 开发工具作为业界领先的集成开发环境，提供了智能代码提示、实时错误检测、强大的调试功能以及无缝的版本控制集成，极大地优化了开发者的编码体验与生产力。

4. 除了上述核心工具外，环境准备阶段还需配置 Git 版本控制系统，用于项目源码的协同管理与版本追溯。通过 Git 的分支管理策略（如 feature 分支开发、release 分支发布），可在团队协作中清晰记录每一次代码变更，确保部署前的源码版本与开发阶段的最终迭代保持一致，避免因版本混乱导致的部署失败。

5. 需提前在本地搭建 MySQL 8.0 及以上版本的数据库环境，创建与项目匹配的数据库实例并初始化基础表结构——这一过程需严格参照项目设计文档中的数据字典，确保字段类型、约束条件与业务逻辑完全对应，为后续 JAR 包运行时的数据交互奠定稳定基础。

6. 需安装 Postman 接口测试工具，用于在本地运行 JAR 包后快速验证接口的可用性：学生可通过 Postman 模拟前端请求，检查接口返回的状态码、响应数据格式是否符合设计要求，提前排查潜在的接口逻辑问题，减少部署至生产环境后的调试成本。

9.2 JAR 部署



9.2.1 获取 JAR 包

1. 使用 IDEA 工具进行 JAR 包打包

（1）将 scpicked 项目成功导入到 IntelliJ IDEA 开发环境中，可以按照以下步骤进行操作：打开 IntelliJ IDEA，在主界面选择“File”菜单，然后点击“Open”选项，在弹出的文件浏览对话框中找到并选中 scpicked 项目的根目录，确认选择后 IDEA 会自动识别项目结构并进行导入，如图 9-1 所示。

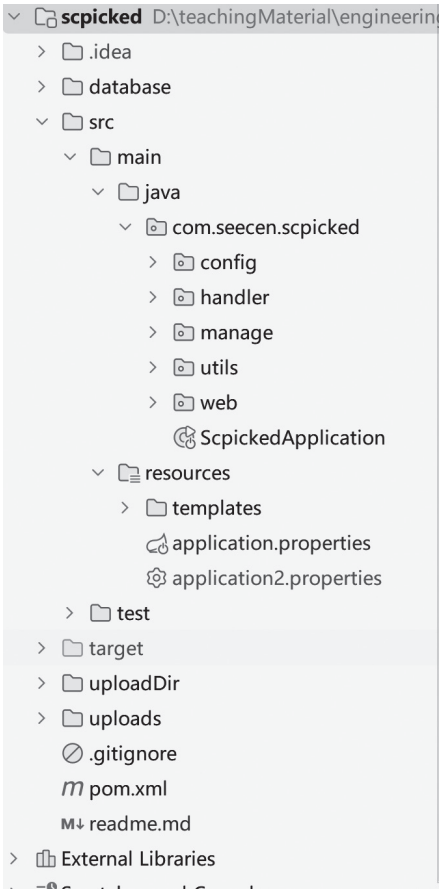


图9-1　项目导入到IDEA

（2）在Maven的图形界面窗口中，找到scpicked项目对应的模块，然后通过鼠标双击操作来触发该项目的打包流程，执行package命令以生成最终的可部署包，如图9-2所示。

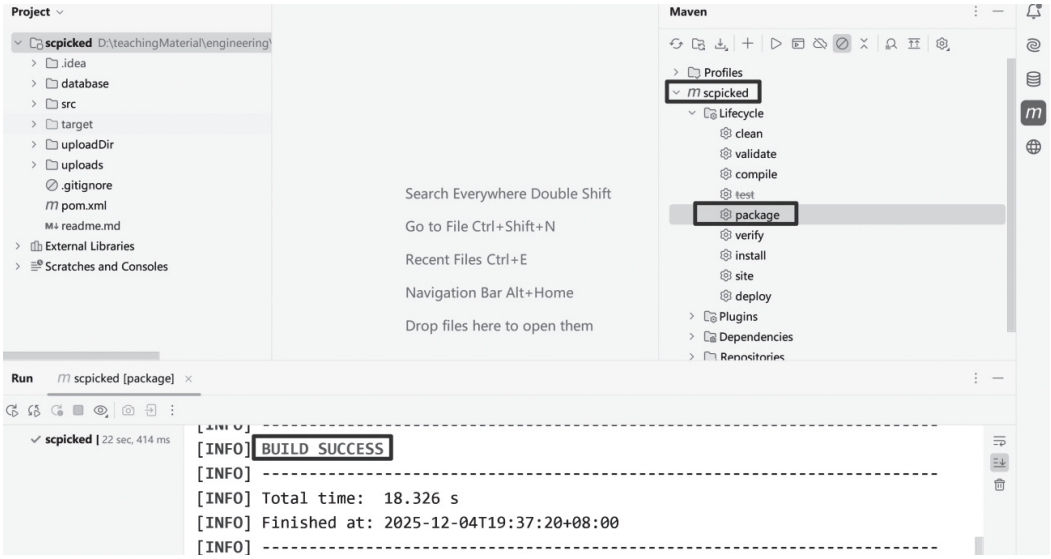


图9-2　项目打包

（3）在scpicked项目的target目录中，找到并查看scpicked-0.0.1-SNAPSHOT.jar这个文件，该文件通常位于项目的构建输出路径下，是项目编译打包后生成的可执行jar包，如图9-3所示。

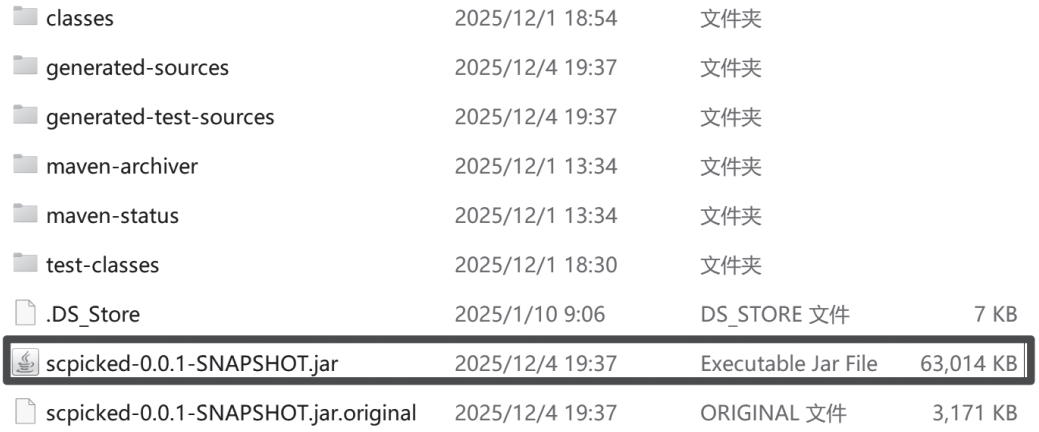


图9-3　所示 查看scpicked-0.0.1-SNAPSHOT.jar包

2.通过源码编译获取JAR包

（1）进入scpicked项目的根目录，输入cmd命令即可直接进入DOS窗口，相关代码示例如下：

```
D:\teachingMaterial\engineering\code\SpringBootProject\scpicked>mvn clean package -DskipTests
```

（2）将JAR包复制至 ` scpicked/target ` 目录中，相关代码示例如下：

```
D:\teachingMaterial\engineering\code\SpringBootProject\scpicked>copy target\scpicked-*.jar C:\scpicked\scpicked.jar
```

9.2.2 本地运行JAR包

1.基础命令运行

（1）进入 `scpicked/target` 目录，输入 `cmd` 命令即可直接打开 DOS 窗口，相关代码示例如下：

```
D:\teachingMaterial\engineering\code\SpringBootProject\scpicked\target>java -jar scpicked-0.0.1-SNAPSHOT.jar
```

（2）直接进入DOS窗口，相关代码示例如下：

```
C:\Users\ZephyrusG14>java -jar D:\teachingMaterial\engineering\code\SpringBootProject\scpicked\target\scpicked-0.0.1-SNAPSHOT.jar
```

（3）查看访问项目的首页，如图9-4所示。

2.高级命令运行

（1）运行带有指定端口参数的命令，相关代码示例如下：

```
D:\teachingMaterial\engineering\code\SpringBootProject\scpicked\target>java -jar scpicked-0.0.1-SNAPSHOT.jar --server.port=8585
```

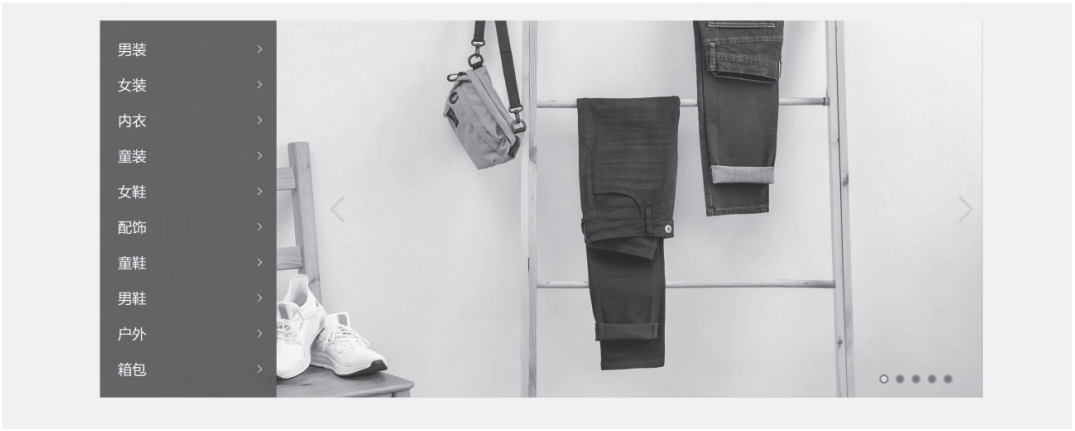


图9-4 项目的首页

（2）指定激活配置文件，相关代码示例如下：

```
D:\teachingMaterial\engineering\code\SpringBootProject\scpicked\target>java -jar scpicked-0.0.1-SNAPSHOT.jar --spring.profiles.active=prod
```